

CATEGORICAL FOUNDATIONS OF FIRST-ORDER ABSTRACT SYNTAX

Lawrence Henry Dunn III

A DISSERTATION

in

Computer and Information Science

Presented to the Faculties of the University of Pennsylvania

in

Partial Fulfillment of the Requirements for the

Degree of Doctor of Philosophy

2025

Supervisor of Dissertation

Val B. Tannen

Professor Computer and Information
Science

Co-Supervisor of Dissertation

Steve Zdancewic

Schlein Family President's
Distinguished Professor of Computer
and Information Science

Graduate Group Chairperson

Anindya De, Associate Professor Computer and Information Science

Dissertation Committee

Stephanie Weirich, ENIAC President's Distinguished Professor of Computer and Information
Science, University of Pennsylvania

Benjamin Pierce, Henry Salvatori Professor Department of Computer and Information Science,
University of Pennsylvania

Conor McBride, Lecturer, Department of Computer and Information Sciences, University of
Strathclyde

CATEGORICAL FOUNDATIONS OF FIRST-ORDER ABSTRACT SYNTAX

COPYRIGHT

2025

Lawrence Dunn III

For Jane—many works have been dedicated to a Jane, but this one is for you.

ABSTRACT

CATEGORICAL FOUNDATIONS OF FIRST-ORDER ABSTRACT SYNTAX

Lawrence Henry Dunn III

Val B. Tannen

Steve Zdancewic

The representation of syntactic datatypes is a foundational topic in programming languages and formal verification. Existing work has typically prioritized encodings that facilitate automated manipulation of formulas, ease of implementation, or theoretical frameworks where only well-scoped, well-typed syntax can be represented. However, these approaches abstract away from the first-order nominal treatment of syntax found in textbooks, where substitution requires systematic renaming of bound variables to avoid variable capture. Despite its practical relevance—being the form closest to what real compilers manipulate in the frontend—this representation remains notoriously difficult to formalize in proof assistants. Moreover, converting this form into a more convenient internal representation, even one with well-understood correctness properties, does not eliminate the need to reason about nominal variable binding but shifts the burden to verifying that the translation itself is correct. As a result, formalizing nominal substitution is not just a theoretical concern but a crucial aspect of compiler correctness. Addressing this gap requires a rigorous mathematical treatment of concrete first-order representations of syntax.

This work defines a novel categorical structure—decorated traversable monads (DTMs)—as a canonical foundation for extrinsically-scoped, extrinsically-typed first-order abstract syntax. A free monad forms a DTM whenever its signature can be represented as a decorated-traversable functor, a class with strong compositional properties. Accompanying this work is a Coq library, *Tealeaves*, that formalizes the core theoretical results and demonstrates their practical applicability. *Tealeaves* reproduces the core functionality of other well-known tools used with Coq while extending them in several respects. Our core abstraction handles variadic and mutually-recursive binders without modification, and supports certified translations between different representations of variables within a common framework. To our knowledge, *Tealeaves* provides the first datatype-generic formalization of α -equivalence that both (i) matches its conventional definition (ii) comes with an executable proof α -equivalence classes are in bijection with well-formed locally nameless terms, which in turn correspond to de Bruijn terms in a well-formed environment. This result enables new forms of modular, reusable, and mechanically verified reasoning about capture-avoiding substitution.

TABLE OF CONTENTS

ABSTRACT	iv
LIST OF TABLES	viii
LIST OF ILLUSTRATIONS	ix
CHAPTER 1: INTRODUCTION	1
1.1 Abstract Syntax	1
1.2 Monadic Syntax	3
1.3 Contributions	7
1.4 Decorated Traversable Monads	8
1.5 Navigation	9
CHAPTER 2: FIRST ORDER ABSTRACT SYNTAX	12
2.1 Fully Nominal Binding	12
2.2 de Bruijn Indices	16
2.3 Locally Nameless	21
CHAPTER 3: TEALEAVES	25
3.1 Instantiating Tealeaves	27
3.2 De Bruijn Indices Backend	29
3.3 Locally Nameless Backend	31
3.4 Translating Representations	34
3.5 Variadic Binders	35
3.6 Nominal Backend	38
3.7 Multisorted Tealeaves	40
CHAPTER 4: DECORATED MONADS	42
4.1 Introduction to Diagrammatic Reasoning	42
4.2 Monads and Algebras	63
4.3 The Category of Decorated Functors	79
4.4 de Bruijn Substitution	99
4.5 Locally Nameless Substitution	106
4.6 Nominal Substitution	108
CHAPTER 5: DECORATED TRAVERSABLE MONADS	129
5.1 Monoidal Folds	129

5.2	Locally Nameless Monoidal Folds	150
5.3	The Category of Traversable Functors	156
5.4	The Category of Decorated Traversable Functors	172
5.5	Locally Nameless Metatheory	187
5.6	Translating Variable Binding Representations	190
5.7	Traversing Binders	197
CHAPTER 6: COALGEBRAIC DECORATED TRAVERSABLE MONADS		199
6.1	Coalgebraic Traversable Functors	199
6.2	Variadic Binders	219
6.3	Coalgebraic Decorated Traversable Monads	223
6.4	Mutually Recursive Binders	228
6.5	Alpha Equivalence	230
CHAPTER 7: EVALUATION		234
7.1	Expressiveness of Backends	234
7.2	Expressiveness of DTMs	238
7.3	Cost of Instantiation	239
7.4	Limitations of the Current Framework	240
CHAPTER 8: RELATED WORK		241
8.1	Binding, Substitution, and Recursion	241
8.2	Monadic Syntax	242
8.3	Well Typed Syntax	244
8.4	Presheaf Theoretic Syntax	245
8.5	Nominal Logic	246
CHAPTER 9: Future Work		248
9.1	Automatic Instantiation	248
9.2	Intrinsically Scoped DTMs	248
CHAPTER 10: CONCLUDING REMARKS		253
CHAPTER A: Proof of DTM Instance for lam		254
A.1	Extended Applicative Functor Laws	254
A.2	Preincrement Laws	257
A.3	Make Function Definitional Equalities	257
A.4	DTM Proofs for Lambda Terms	258
A.5	DTM Proofs for Variadic Syntax	264

APPENDIX B: Monoidal Category Instances	268
APPENDIX C: Multisorted Tealeaves	274
BIBLIOGRAPHY	280

LIST OF TABLES

LIST OF ILLUSTRATIONS

FIGURE 1.1	Section Dependencies	10
FIGURE 2.1	Some lemmas generated by LNgen	24
FIGURE 2.2	Snippet of an Ott specification for lambda calculus for input to LNgen	24
FIGURE 3.1	General structure of a Tealeaves development	26
FIGURE 3.2	Locally nameless operations defined by Tealeaves	32
FIGURE 3.3	Some locally nameless lemmas provided by Tealeaves	32
FIGURE 3.4	Locally nameless and de Bruijn representation translation functions	34
FIGURE 3.5	LN.level applied to locally nameless terms under different readings of letin . .	37
FIGURE 4.1	Tealeaves nominal substitution demo	127
FIGURE 4.2	Conventional nominal substitution Demo	128
FIGURE 5.1	The five constituent operations of a DTM	177
FIGURE 5.2	DTM axioms	178
FIGURE 8.1	Intrinsically scoped definition of lam	244
FIGURE 9.1	Intrinsically scoped definition of lam in the style of Tealeaves	249

INTRODUCTION

Since the early 1960's, computer scientists have wrestled with a fundamental challenge: how can we be sure that a program actually does what we intend for it to do? The predominant approach remains simple: testing programs “on cases one hopes are typical, until it seems to work” [56]. This method is accessible to non-specialists, well supported, and certainly helpful, but inherently limited: the sheer magnitude of possible program states vastly exceeds what any test suite can cover, and even a single, rare corner case, if mishandled, can lead to catastrophic failures in critical systems [71], sometimes resulting in serious injury or death [50]. Such failures underscore why formal methods—the use of mathematical and logical techniques to rigorously validate system correctness—have become the highest standard for ensuring reliability in critical systems. Such efforts align with a broader movement towards setting the science of computation on a rigorous mathematical foundation [57].

One particularly rich domain within this field, considered in some of the earliest work [58], is concerned with the correctness of programming language compilers. Programming languages are the means of expressing computational intent, and subtle mistakes in compilers can silently undermine an otherwise correct source program. Modern developments such as CompCert [48] have demonstrated the feasibility and benefits of verified compilers, but such efforts still require both specialized expertise and substantial human effort. Making such efforts more accessible, scalable, and mechanizable remains a crucial open challenge.

1.1 ABSTRACT SYNTAX

The POPLmark challenge [11] introduced a suite of benchmarks in 2005 to assess the state of the art in formal reasoning about programming languages using proof assistants. A central focus of this challenge, which remains a major topic in the field today, is the representation and manipulation of syntactic data structures. Most formal systems of interest—including lambda calculi, programming languages, type theories, specification languages, query languages, and mathematical logics—incorporate variable binding constructors in their syntax, such as $\lambda, \forall, \int, \Pi, \mu, \text{let}$, etc. Variable binding introduces complexities absent from simple algebraic structures such as lists and trees, and incorrect handling of bound variables is a frequent source of subtle errors in both theoretical treatments and practical implementations.

The challenges of variable binding are easily overlooked on paper, particularly by mathematicians and logicians studying formal systems at a metatheoretic level rather than implementing them. However, computer science is “that great manipulator of syntax” [37], and the demand for a correct computational implementation of formal systems calls for thorough study of syntactic datatypes.

To illustrate these issues, this manuscript takes the lambda calculus, introduced by Church as a formalism for expressing variable binding [24], as our motivating example (for an introduction to untyped lambda calculus see [30]). Its syntax is defined by the following grammar:

$$t ::= v \mid \lambda v. t \mid (t t)$$

Here, $v \in V$ is drawn from a variable set V . The set of terms generated by a particular choice of V will be denoted $T V$. Terms of the form v where $v \in V$ are *atomic*, while $\lambda v. t$ denotes an *abstraction* with *label* b and *body* t . The λ constructor is said to be a *binder*. Finally $(t_1 t_2)$ denotes the *application* of t_1 to t_2 .

There is a fundamental distinction between free and bound occurrences of variables.¹ An occurrence of v is *bound* if it occurs in the body of an enclosing abstraction λv that shares the same name; it is bound *to* the innermost such abstraction. Otherwise, it is *free*. The names of bound variables are semantically insignificant: they merely serve to indicate which occurrences are bound to which λ -abstractions by using the same name at all locations. For instance, the terms $\lambda x. x$ and $\lambda y. y$ both convey the operation that accepts an input and returns the same as output, i.e. the identity function. Terms that differ only in the names of corresponding bound variables are said to be α -*equivalent*. Ensuring that all syntactic operations respect this congruence is a fundamental requirement for implementations. Yet, formal definitions of α -equivalence vary in the literature—many correct and equivalent, some with subtle flaws [26].

The fundamental operation on terms is substitution, which replaces free occurrences of a variable with a given term. This operation underlies the β -reduction rule of the calculus, which defines the semantics of applying a procedure to an argument:

$$(\lambda x. t)u \rightarrow_{\beta} t[u/x]$$

Here, $t[u/x]$ denotes the *capture-avoiding* substitution of u for free occurrences of x in t . Implementing this function is not entirely trivial. Since u may itself include free occurrences, carelessly inserting it in t may place those occurrences into a context where the same name happens to belong to an enclosing abstraction, to which the occurrence becomes inadvertently bound. To avoid this, bound variables must be systematically renamed during substitution to avoid conflicts, but the bookkeeping required to track potential conflicts can be surprisingly intricate. As a result, simple reasoning about lambda terms quickly devolves into meticulous reasoning about tree structures rather than the calculus.

These complications are, in a sense, just a technical nuisance. Informal treatments can easily sidestep them using the so-called Barendregt convention [30], which assumes that no bound vari-

¹Note that a variable is an element $v \in V$, while an *occurrence* is a variable at a particular leaf of an abstract syntax tree. We will not be overly pedantic about the distinction when the meaning is clear.

ables share the same name as any free variable. However, this informal assumption is untenable in formal proofs and implementations, becoming a pronounced challenge in proof assistants, where operations must be fully defined and every proof must account for even the most obscure corner case.

In response to these issues, the literature has proposed a bewildering array of approaches to representing syntax with variable binding. As one observer remarked, “One of the most curious things about formal treatments of type theory is the attention paid to dummy variables” [66]. One reason for such attention is that the cost of dealing with these issues is paid not just once, but once *per implementation*—formalized metatheory for one syntax is not easily repurposed for the syntax of another language. This is because operations on terms are defined by recursion on their structure, while proofs are given by induction on the same. This structure reflects the grammar of the language and, consequently, even a minor alteration in the formal language may necessitate rewriting large portions of definitions and proofs, leading to significant maintenance costs.

This situation is deeply unsatisfying: *substitution should be not datatype-specific*. Rather, it is a fundamental operation appearing in many guises, “one of many problems in computer science that, once understood in one context, is understood in all contexts” [14]. The challenge is to capture this generality in a way that abstracts over the grammar of a specific syntax.

1.2 MONADIC SYNTAX

The desire to formalize the metatheory of syntactic datatypes is not new. Like much related work, I begin with the observation that syntax is monadic, an observation arising in the category-theoretic understanding of universal algebra (one of two such understandings [41]). We provide an abridged summary of the categorical approach to single-sorted algebraic languages for readers that have a basic familiarity with category theory (for an introduction to category theory see [8]).

Associated to a formal language is a *ranked alphabet* Σ of primitive symbols, meaning each symbol $\sigma \in \Sigma$ is paired with a natural-number arity $\text{ar}(\sigma) \in \mathbb{N}$. For example, the language of monoids includes a nullary symbol $e \in \Sigma$ and a binary function symbol $m \in \Sigma$. An interpretation of these symbols into a set X consists of an element $\llbracket e \rrbracket \in X$ and a binary function $\llbracket m \rrbracket : X \times X \rightarrow X$.

Rather than treating Σ as a *set*, the categorical story encodes the signature into an endofunctor $\Sigma : \mathbf{SET} \rightarrow \mathbf{SET}$ on the category of sets. For our monoid example, this functor acts as follows:

$$\Sigma(X) \equiv \mathbf{1} + X \times X$$

Note that parentheses around functor arguments (e.g., X in $\Sigma(X)$) will generally be omitted when there is no risk of ambiguity. In this example, $\mathbf{1}$ is the singleton set, $+$ denotes disjoint union, and $\times$ denotes the Cartesian product.² An interpretation of this signature is encoded into a single

²We will assume a particular choice of these objects, which are specified up to a unique isomorphism.

function called a Σ -algebra:

$$\Sigma X \rightarrow X$$

A fundamental object to consider is the set of all *terms* generated by Σ , augmented with a set V of symbols to serve as *variables*. These terms correspond to *derived operations* expressible in the language of Σ . The standard construction follows an iterative process: starting with variable symbols, then adding expressions (well-)formed by applying function symbols to these, repeating until the set is *closed* under application of the function symbols. The limit of this process yields the *term algebra* generated by Σ from V , denoted TV and defined as a least fixpoint:

$$\begin{aligned} TV &\equiv V + \Sigma(V) + \Sigma(V + \Sigma(V)) + \dots \\ &\equiv \mu X. (V + \Sigma(X)) \end{aligned}$$

By Lambek's lemma [46], T itself forms a functor satisfying the following natural isomorphism:

$$TV \simeq V + \Sigma(TV)$$

The left-to-right direction states that any term $t \in TV$ can be decomposed into either a variable or one of the function symbols in Σ applied to terms recursively drawn from TV . This represents *elimination* or *pattern matching*, which the literature has called *analytic syntax* [56]. The right-to-left direction states that a term can be built up from either a variable or a function symbol applied to terms. This encodes the constructors of the language, which the literature has called *synthetic syntax* [56].

Rather than explicit construction, TV can be characterized up to a unique isomorphism as the *initial algebra* of the functor $V + \Sigma(-)$. As such, any other algebra of this functor induces a unique homomorphism from TV into itself. This specification can be encoded into the following higher-order function, often called the *fold* combinator for T :

$$\text{fold}_X^T: (V \rightarrow X) \rightarrow (\Sigma X \rightarrow X) \rightarrow TV \rightarrow X$$

Here, X represents an arbitrary type argument to fold^T , indicated in this work with a subscript. This function defines operations on T by *structural recursion*: variables are mapped using the first argument, which is the *base case* of the recursion, while the second operation encodes the *recursive case*, which is applied after recursively folding over the arguments to a non-atomic term. Operations of the form $\text{fold}^T f g$ are often called *catamorphisms* or *initial homomorphisms*.

Unfortunately, operations defined by structural recursion have a major limitation: they are tied to the specific signature Σ in the form of specifying the recursive case $\Sigma X \rightarrow X$. Therefore, an operation defined on the syntax trees of one language (e.g. monoids) does not automatically transfer to another (e.g. rings). The intuition that substitution is not datatype-specific calls for

a datatype-generic [35] approach to defining syntactic operations independently of Σ . Such an approach would involve handling the recursive cases in a *formulaic* manner, leading to a class of functions less expressive than arbitrary catamorphisms, but (perhaps) satisfying desirable properties by virtue of being so-defined.

To abstract over Σ , we observe that T forms a *monad*, specifically the *free* monad generated by Σ .³ Furthermore, for any V , TV is the *free algebra* of T over V . This fact yields (among other things) the polymorphic *bind* combinator, where A and B are arbitrary type arguments:

$$\text{bind}_{A,B}^T: (A \rightarrow TB) \rightarrow (TA \rightarrow TB)$$

Conceptually, *bind* represents *substitution*. Specializing A and B to the variable set V , observe that *bind* accepts a function σ from variables to terms and lifts this function over a term by applying σ to each occurrence. For instance, a simple single-variable substitution replacing x with u can be encoded into the following function:

$$\text{subst}_{\text{loc}} x u v = \begin{cases} u & \text{if } v = x \\ v & \text{otherwise} \end{cases}$$

where the subscript indicates the “localized” nature of this operation, in the sense that it encodes the logic of substitution on individual occurrences. Then, substitution can be formally defined as follows:

$$t[u/x] = \text{bind}^T (\text{subst}_{\text{loc}} x u) t$$

Importantly, this definition does not depend on Σ , but only on the fact that T is a monad. Furthermore, operations defined from bind^T satisfy the monad laws (presented in Section 4.2), which provides a basis for reasoning about substitution.

Unfortunately, the substitution provided by the monad is the *naïve* kind: it replaces all occurrences of x , free or bound, with u . However, our intention is to replace only the free occurrences while simultaneously renaming bound variables as-needed to avoid capture. What we have defined is the substitution of equational logic, which is appropriate for equational algebraic theories (monoids, groups, etc.) but inadequate as-is for syntactic structures involving variable binding. This presents a dilemma:

1. Defining operations assuming only that T is a monad fails to capture the “capture-avoiding” aspect of substitution.
2. Defining operations using the initial algebra structure ties them to a particular Σ , preventing reuse.

³*Free* relative to the forgetful functor from monads to functors. Nearly all of the results in this work are stated for an arbitrary decorated traversable monad and do not require T to be freely generated.

Much prior work on substitution, beginning with Bellegarde and Hooke [14], is grounded in these observations and has sought to modify the monad to account for bound variables. Commonly, this involves refining the parameter V based on the intuition that variables are not drawn not from a flat set, but a *set in context*, the context somehow reflecting the binders in scope at each occurrence. This intuition has led to the use of nested inductive datatypes [16, 5] to define T , or described T not as a monad on the category of sets but on a different category, such as a category of function sequences [14] or presheaves [31]. This conceptual approach naturally restricts the definition of “syntax” so that only variables well-scoped in their context can be represented. More broadly, one is led to consider view syntax as a type- and context-indexed family of sets, e.g. rather than the set T of all terms, one has the set $T \Gamma \tau$ of terms with type τ in context Γ . Such an approach, which is known as *intrinsic* syntax, has certain benefits. For example, the type assigned to operations like substitution essentially states the correctness of the operation, without requiring a separate proof. For the formal metatheorist, defining such an operation essentially amounts to computing the proof of correctness directly inside the definition of the operation—a style of programming whose convenience varies widely across proof assistants.

This work takes a different approach by emphasizing *raw* syntax: the sort of abstract syntax tree (AST) computed by a compiler parsing human-written code. A raw AST, by itself, does not guarantee that terms are well-formed, well-scoped, or well-typed in any sense. For example, a programmer can mistakenly refer to a variable that is not in the lexical scope at a particular location. In fact, one of the very first operations we might perform on an AST is to *compute* the lexical scope of each occurrence to *decide* whether a term is well-formed in this way. This is often known as *extrinsic* syntax.

There are two fundamental reasons to consider raw syntax. First, the workflow of a formal metatheorist working with this kind of syntax resembles treatments found in conventional textbooks about the lambda calculus: operations such as capture-avoiding substitution are defined first, and their correctness is reasoned about afterward. For some purposes, this is more natural than the intrinsic approach. My own experience working with Coq suggests this approach is more convenient in that environment, while the heavy dependent type machinery of an intrinsic approach can be burdensome. I do not find it surprising that recent work [4, 33] emphasizing well-typed syntax been formalized in Agda [20], while the utilities Autosubst [64] and LNgen [9], both used with Coq, favor an extrinsically scoped, extrinsically typed approach.

The more fundamental reason to consider raw syntax is simply that it is the kind of syntax processed by compilers, at least immediately after parsing. To be sure, there are many advantages of translating an AST into a more structured internal representation after parsing, perhaps using a well-scoped or well-typed representation. However, changing representations raises a troublesome question: is the translation itself correct, i.e., does the output of this process represent the same term we started with? Correctness can be defined by showing (at least) that the translation

respects α -equivalence of source programs and commutes with capture-avoiding substitution in the raw syntax. For the reasons described above, this approach can only be scaled to realistic compilers if a formally verified translation is developed generically over Σ . Such a task requires a rigorous mathematical foundation for raw, extrinsic syntax.

For this purpose, I have proceeded based on the assumption that there is only a single, flat set of variables in the syntax, and any variable may occur at any location in a syntax tree. In particular, $T : \text{SET} \rightarrow \text{SET}$ remains a monad on the category of sets, and the bind^T combinator used to define naïve substitution, and the equational laws it satisfies, are also unchanged. Clearly this is not enough, which leads to the fundamental question of my research:

What extension of the theory of monads in SET is required to define and prove the correctness of capture-avoiding substitution for raw syntactic datatypes generically?

“Extension” here means *conservative extension* as an equational theory: the theory includes bind^T and the monad laws, and these are augmented with additional operations and equations. The phrasing of the question suggests there is a single, clear answer. I claim the abstraction presented in this work is canonical, rather than cobbled together ad-hoc. The justifications for this claim are indicated below.

1.3 CONTRIBUTIONS

The contributions of this work are three-fold. First, I develop a novel abstraction applicable to raw syntax: decorated traversable monads (DTMs, Definition 5.4.17). By themselves, DTMs suffice to reason about de Bruijn and locally nameless representations of binding. To reason about the conventional nominal representation, DTMs are extended to form *parametrically* decorated traversable monads, which support operations that rename binders. One indication of the canonicity of DTMs is that they have equivalent characterizations as monoids in the category of decorated traversable endofunctors (defined in this work) or in terms of a single binddt combinator subject to a set of axioms I believe experienced Haskell programmers would find intuitive (Theorem 5.4.22).

The second contribution of this work is a Coq implementation, Tealeaves, which both formalizes the core theoretical results discussed here and demonstrates their practical applicability. To demonstrate practicality, Tealeaves reproduces the core functionality of Autosubst [64] (now called Autosubst 1) and LNgen [9], two utilities commonly used with Coq to reason about first order syntax, bringing them into a common theoretical framework with a modular architecture akin to that of GMeta [47]. Tealeaves goes much further than these tools in a few respects. First, Tealeaves also formalizes a nominal representation of syntax, implementing both capture-avoiding substitution and α -equivalence. Second, by working in a common framework, I am able to formalize certified translations between de Bruijn indices, locally nameless representations, and nominal terms. Finally, I demonstrate that Tealeaves can be used with syntax involving

variadic binders, which bind a dynamic number of variables in their arguments, without modifying the underlying theory. Tealeaves is a step towards a provably correct generic mechanism for realistic compilers to parse raw syntax trees into representations more convenient for internal manipulation.

The third contribution of this work is a representation theorem for DTMs, which extends the representation theorem for traversable functors [17, 43]. This offers a third, equivalent presentation of DTMs as a particular type of coalgebra. The representation theorem characterizes the operations definable for an arbitrary DTM: they are the ones that manipulate data in the tree *concretely*, but manipulate the shape of a syntax tree only *parametrically*—exactly what would be expected of a theory that treats the complexities of variable binding for a concrete representation of variables but remains abstract over the underlying signature endofunctor. The representation theorem leads to a theory of lifting context-sensitive relations over first-order syntactic datatypes, which forms the basis of our treatment of α -equivalence: terms are α -equivalent when they have the same shape and each occurrence in one tree resolves to the same abstraction as the occurrence at the corresponding location in the other tree. Although the coalgebraic characterization of DTMs is equivalent to the other two, it is the *sine qua non* of practical reasoning about lifted relations, and likely has applications beyond purely syntactic reasoning.

1.4 DECORATED TRAVERSABLE MONADS

Decorated traversable monads, or DTMs, are highly monoidal creatures in several respects. One formulation of DTMs is as the coherent combination of three independent notions of functors equipped with additional structure, each somehow defined in terms of generalized (co)monoids:

Decoration A functor *decorated by* a set E (Def. 4.3.14) generalizes the concept of a comonoid co-acting on a set (Def. 4.1.30). In the case of decorated *monads* (Def. 4.3.37), the underlying set W of the comonoid is also assumed to form a monoid, giving rise to the monoidal category DEC_W (Def. 4.3.34).

Traversability Monoids in SET are preserved by functors with the right structure—the so-called *lax monoidal* (Def. 4.3.6) or *applicative* functors (Def. 5.3.5). A traversable functor [36, 44] (Def. 5.3.21) distributes over these, generalizing the concept of folding a functor over a monoid (Sec. 5.1). This leads to a proof technique by which lemmas about first-order syntax are proved essentially by induction on lists, which are the free (read “most prototypical”) monoid. See, for instance, Corollary 5.1.27.

Monad As the apocryphal saying goes: “Monads are just monoids in the category of endofunctors, what’s the problem?”⁴

⁴A quote fictionally attributed to Phil Wadler in a blog post by James Iry [42]

In short, this work observes that for the sorts of languages under consideration, the signature endofunctor Σ is naturally associated with the structure of a decorated traversable functor (Def. 5.4.3). This class of functors has excellent compositional properties and gives rise to a monoidal category, DECTRAV_W , where W is a choice of monoid corresponding to variable binding contexts. Any monad freely generated from a decorated traversable functor forms a DTM, and the combination of the three aspects above provides the required expressiveness to reason generically about first-order abstract syntax, including context-sensitive substitution and α -equivalence.

1.5 NAVIGATION

Figure 1.1 presents a diagram of the dependencies among the core sections of this document, with hyperlinks for navigation. We provide a short summary of each chapter below.

Chapter 2 summarizes three common representations of variable binding for raw syntax: the fully nominal approach (i.e. the textbook definition), the de Bruijn representation, and locally nameless. I explain the fundamental operations of each, some of the properties these should satisfy, and the utilities `Autosubst` and `LNgen` which support these representations in practice in Coq.

Chapter 3 discusses the use of `Tealeaves`. I demonstrate how the library is instantiated with the untyped lambda calculus and how a `Tealeaves` user invokes backend libraries providing functionality like that of `Autosubst` and `LNgen`. I also discuss how the library is used with nominal syntax, syntax with mutually recursive binders, and multisorted syntax. Such instantiation is not entirely automatic, and some of these cases currently require the user to study the examples discussed here and apply deep proof techniques developed in Chapter 6 to prove that their syntax forms a DTM.

Chapter 4 presents background material on string diagrams for monoidal categories and gives a diagrammatic account of monads. Decorated functors, and then decorated monads, are introduced as generalizations of comonoidal coactions, giving rise to the monoidal category of decorated functors. The final sections of this chapter define substitution for de Bruijn indices and locally nameless. In the former case, I derive the de Bruijn algebra from the decorated monad axioms, but reasoning about locally nameless requires material developed in Chapter 5. The concluding section discusses how nominal substitution is handled by extending the abstraction to *parameterized* decorated monads, which incorporate new operations and axioms.

Chapter 5 incorporates traversable functors, motivating these by way of considering a weaker structure first: functors equipped with a transformation to the list monad. Functors equipped with both a decoration and a traversal are required to satisfy a novel coherence law presented in this chapter. By extending decorated monads with a compatible traversal, this chapter defines all of the outstanding operations that could not be defined in Chapter 4 and proves some, but not all, of the outstanding lemmas. I also examine the translation between the de Bruijn and

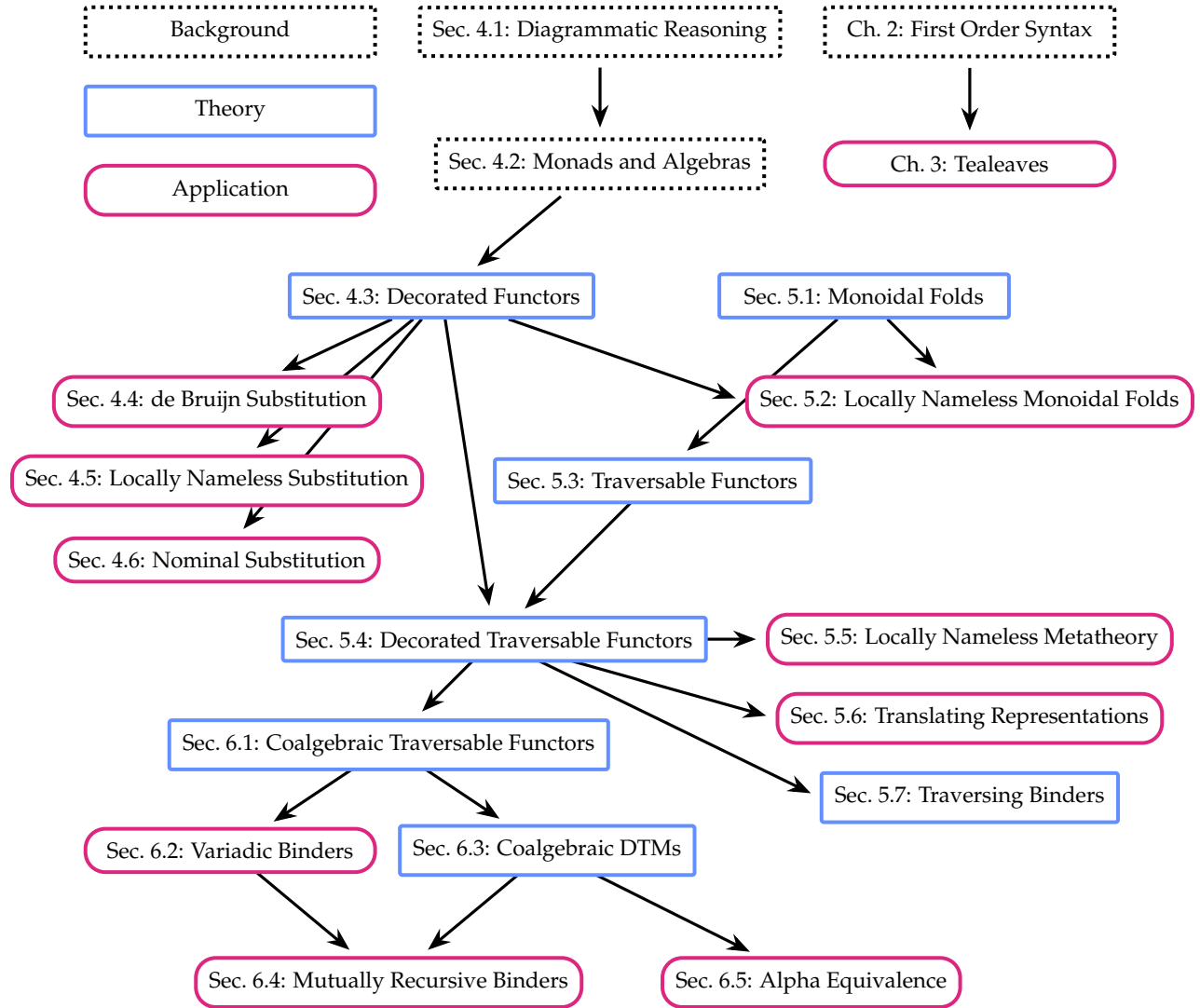


Figure 1.1: Dependencies among sections of this manuscript, with hyperlinks

locally nameless. The conclusion discusses the incorporation of the traversal structure to define parameterized DTMs, forming the basis for reasoning about nominal substitution.

Chapter 6 presents DTMs through a radically different lens emphasizing the *traversal* as the primary structure of interest and incorporating the decoration and monad into it. This chapter builds on previous work on the representation theorem for traversable functors [17, 43] which essentially defines traversable functors as the *coalgebras* of a particular comonad. This chapter extends the result to cover DTMs. The representation theorem provides a powerful mechanism for reasoning about traversable functors and DTMs, culminating in a theory of lifting context-sensitive relations over trees of the same shape. This theory provides the full power to prove the outstanding lemmas of locally nameless, to define alpha-equivalence for nominal syntax, and to reason about a bijection between locally nameless and nominal terms.

1.5.1 Publications

The proof of equivalence between decorated traversable monads presented as an extension system (Definition 5.4.19) and as monoids in the category of DECTRAV_W (Definition 5.4.17) was presented at the 6th International Conference on Applied Category Theory (ACT 2023) and appeared in *Electronic Proceedings in Theoretical Computer Science* [28]. The Tealeaves formalization of locally nameless was presented at the 14th Conference on Interactive Theorem Proving (ITP 2023) and published in *Leibniz International Proceedings in Informatics* (LIPIcs) [29]. An extended version of [29] is under consideration for publication in the Journal of Automated Reasoning. This version includes the de Bruijn indices backend and its translation with locally nameless, as well as the instantiation of Tealeaves with variadic binders. The content related to the nominal Tealeaves backend and its translation with locally nameless, the proof of Theorem 5.4.18, and the coalgebraic presentation of DTMs (Chapter 6) is novel.

FIRST ORDER ABSTRACT SYNTAX

In this chapter, we review the challenges of implementing substitution and survey two sophisticated strategies, along with their associated utilities in Coq, that address these issues in practice. The underlying logic of each of these strategies is important to review carefully because our goal is to abstract this logic away from any specific grammar of the underlying syntax. While a casual reader is not expected to memorize all of the operations defined here, they are given explicitly for comparison to our generic implementation of these operations, which are given in Chapters 4 and 5.

2.1 FULLY NOMINAL BINDING

The lambda calculus provides the simplest example of a language with variable binding—indeed, it was formulated for such a purpose. To facilitate discussion, we define the syntax of the untyped lambda calculus with explicit names given to each of its constructors:

$$t ::= \text{var } v \mid \text{abs } b \, t \mid \text{app } t \, t$$

The `var` constructor forms an atomic term from a variable $v \in V$, while `app` constructs an application. The `abs` constructor is special: it takes a *binder label* $b \in B$ as its first argument, and a term as its second. As a result, the set of terms nominal lambda calculus terms, denoted $\text{nlam } B \, V$, is parameterized by two different types. Categorically, this corresponds to a *bifunctor*, rather than a functor as described in the introduction. While this additional parameter may seem like an unnecessary complication at the moment, it serves both practical and theoretical purposes:

1. In most of this work, we consider representations of variable binding where B is held constant and equal to the unit set $\mathbf{1}$. This yields a decorated traversable monad—the central abstraction of this work. By itself, this suffices to develop the entire metatheory of de Bruijn indices (Section 2.2) and locally nameless (Section 2.3).
2. Theoretically, the functorial behavior of nlam differs substantially in its two arguments. When V is held constant, the functor $\text{nlam } (-) \, V$ exhibits a richer structure explored in Sections 4.6 and 5.7. Although this extra structure is a topic of ongoing research, only a small fragment of it is necessary to reason about nominal substitution.

While shall return to this topic later. For now, let us assume $B = V = \text{atom}$ and consider the set nlam atom atom . The first operation we define accepts an nlam atom atom and computes the set

of free variables, proceeding by structural recursion:

$$\begin{aligned}\mathbf{fv}(\text{var } v) &\equiv \{v\} \\ \mathbf{fv}(\text{abs } b \ t) &\equiv \mathbf{fv} \ t \setminus \{b\} \\ \mathbf{fv}(\text{app } t_1 \ t_2) &\equiv \mathbf{fv} \ t_1 \cup \mathbf{fv} \ t_2\end{aligned}$$

More precisely, we should speak about free or bound *occurrences*, as the same variable may occur free or bound in different places. The context-sensitive nature of bound variables requires care when defining the substitution of u for free occurrences of x in t . A textbook presentation is given by the following recursive function:

$$\begin{aligned}\text{subst } x \ u \ (\text{var } y) &\equiv \begin{cases} u & \text{if } y = x \\ \text{var } y & \text{if } y \neq x \end{cases} \\ \text{subst } x \ u \ (\text{abs } b \ t) &\equiv \begin{cases} \text{abs } b \ t & \text{if } b = x \\ \text{abs } b \ (\text{subst } x \ u \ t) & \text{if } b \neq x \text{ and } b \notin \mathbf{fv}(u) \\ \text{abs } z \ (\text{subst } x \ u \ (\text{subst } b \ (\text{var } z) \ t)) & \text{if } b \neq x, b \in \mathbf{fv}(u) \text{ and } z \text{ is fresh} \end{cases} \\ \text{subst } x \ u \ (\text{app } t_1 \ t_2) &\equiv \text{app}(\text{subst } x \ u \ t_1) (\text{subst } x \ u \ t_2)\end{aligned}$$

The atomic and application cases are simple. Let us scrutinize the logic of the `abs` case by considering each scenario:

Case 1: `abs  $x$   $t$`

Recursion stops immediately. Any occurrence of x in t must already be bound (either to this `abs  $x$` or a deeper one), so there is nothing to substitute.

Case 2: `abs  $b$   $t$` , **where $b \neq x$ and $b \notin \mathbf{fv}(u)$**

Recursion proceeds normally. Any free variables in u introduced in t by substitution remain free in `abs  $b$   $t$` .

Case 3: `abs  $b$   $t$` , **where $b \neq x$ and $b \in \mathbf{fv}(u)$**

Here, caution is required. Inserting u under `abs  $b$` would lead to the capture of free occurrences of b , so b must first be renamed to a fresh variable z that does not present such a hazard. All free occurrences of b in t , which are the ones bound to this `abs`, must be replaced with z before applying `subst  $x$   $u$` to the body.

This definition has two key issues. First, in the final subcase, `subst  $x$   $u$` is applied to the term `(subst  $b$  ( $\text{var } z$ )  $t$ )`, which is not a subterm of `abs  $b$   $t$` . As a result, the termination of this function is not self-apparent to the proof assistant; it does terminate, but proving this requires a proof that `(subst  $b$  ( $\text{var } z$ )  $t$ )` has the same tree depth as t . In systems like Coq, totality of all functions is

required to enjoy decidable typechecking, and thus proof checking, so this definition is rejected as written. The user must reach for a mechanism like well-founded recursion to express the definition in a way that is structurally recursive. This introduces practical challenges, as reflected in the proliferation of different techniques for handling such cases in Coq: `Fix`, `Function`, `Program`, `Fixpoint`, `Equations`, and explicit recursion on a proof of accessibility [49].

The second deficiency is that the definition is underspecified: we have not explained how a fresh variable is generated and by what standard a variable is considered fresh or unfresh, which is the most subtle point. Suppose `subst x u` is applied to `abs b t` where $b \neq x$ and $b \in \mathbf{fv}(u)$, and consider how to rename b :

- First, b clearly cannot be renamed to anything in $\mathbf{fv}(u)$, since that is what we are trying to avoid in the first place.
- Second, it cannot be renamed to anything in $\mathbf{fv}(t)$ either, since doing so may lead to the capture of free variables in its own body.
- More subtle is that b cannot be renamed to x : if the new name chosen for b is x , the inner “deconflicting” operation `subst b (var x) t` may introduce free occurrences of x in t , which would immediately be replaced with u by the subsequent operation.

To faithfully enforce the side condition stating that z is fresh, a naïve implementation scans the body t to compute $\mathbf{fv}(t)$. This is correct in principle, but computationally inefficient in practice. While scanning the body of abstraction may be acceptable to do once for an input of the form `abs b t`, this is not what happens. Instead, each recursive call to `subst` will *also* scan over the body of any abstraction it encounters as a subterm of t . Hence, this runtime of operation is dominated by repeatedly traversing terms and subterms to compute their free variables. Importantly, note that subterms deep inside a term t might contain free occurrences that are not elements of $\mathbf{fv}(t)$, because those same occurrences are bound in t but free in the subterm.

Let us consider a more efficient way to handle the `abs b t` case. Suppose we decide to rename b to z (perhaps $z = b$). After doing so, we must ensure that no free variables in t are renamed to z , since doing so would cause them to become captured. Hence, we must ensure that z is removed as a candidate for subsequent renaming operations. This line of thinking suggests a more efficient implementation that maintains an “avoid set” Γ of names to be excluded as candidates for fresh names. Initially, for a call to `subst x u t`, this value is given by the following union, which we only need to compute *once* at the top level:

$$\Gamma_0 \equiv \{x\} \cup \mathbf{fv}(t) \cup \mathbf{fv}(u)$$

As we recurse down the tree t , we will ensure that all bound variables have names that do not occur in Γ . Then, we will *add* that variable’s name to Γ . This leads to the following more efficient

definition:

$$\text{subst } x \ u \ t = \text{substAvoid } \Gamma_0 \ x \ u \ t$$

where substAvoid is defined recursively as follows:

$$\begin{aligned} \text{substAvoid } \Gamma \ x \ u \ (\text{var } y) &\equiv \begin{cases} u & \text{if } y = x, \\ \text{var } y & \text{if } y \neq x, \end{cases} \\ \text{substAvoid } \Gamma \ x \ u \ (\text{abs } b \ t) &\equiv \begin{cases} \text{abs } b \ t & \text{if } b = x, \\ \text{abs } z \ (\text{substAvoid } (\Gamma \cup \{z\}) \ x \ u \ t') & \text{if } b \neq x \end{cases} \\ &\quad \text{where} \\ &\quad \quad z := \text{freshen } \Gamma \ b \\ &\quad \quad t' := \text{substAvoid } (\Gamma \cup \{z\}) \ b \ (\text{var } z) \ t \\ \text{substAvoid } \Gamma \ x \ u \ (\text{app } t_1 \ t_2) &\equiv \text{app } (\text{substAvoid } \Gamma \ x \ u \ t_1) \ (\text{substAvoid } \Gamma \ x \ u \ t_2), \end{aligned}$$

In the last clause of the abs case, $\text{freshen } \Gamma \ b$ returns b if $b \notin \Gamma$; otherwise it returns a new variable z where $z \notin \Gamma$. Note this definition only differs from the previous one in how it handles the abs case when the abstraction binds a variable whose name is not x . Rather than scanning the body to compute the free variables, we maintain an invariant all the free variables in each subterm are elements of the avoid set. By maintaining this invariant, the specification of freshen ensures we never rename a bound variable in a way that can capture free occurrences.

Note that two implementations of substitution we have described—one that explicitly computes free variables in the body of abstractions, and one that statefully maintains a superset of such variables—are not equal as functions, but equal up to α -equivalence of their outputs. That is, they might make different choices about how bound variables are renamed, though they ensure the binding structure is maintained. In fact, freshen is itself only specified up to satisfying a freshness condition. Thus, capture-avoiding substitution has an inherent non-determinism insofar as it is defined only up to a non-trivial equivalence relation. In a proof assistant, a concrete implementation must define all of these operations fully explicitly and then show that α -equivalent inputs yield α -equivalent outputs. This quickly leads to a situation colloquially known as “setoid hell,” as it is a painfully explicit way to handle this sort of this relation-preservation property.

The nominal representation is impractical for rapid prototyping and not commonly used in formalizations of systems in proof assistants—though this work takes a step towards solving that challenge. With this in mind, we turn to two alternate representations designed to mitigate these difficulties.

2.2 DE BRUIJN INDICES

The de Bruijn and locally nameless representations avoid assigning arbitrary names to bound variables. Instead, they are represented numerically based on the number of abstractions a bound occurrence is underneath. This allows us to reformulate lambda calculus syntax without explicit binder labels:

$$t ::= \text{var } v \mid \text{abs } t \mid \text{app } t t$$

The set of terms with V -valued variables is simply $\text{lam } V$, without the B parameter. Note that $\text{lam } V$ is isomorphic to $\text{nlam } \mathbf{1} V$ where $\mathbf{1}$ is the singleton set.

In the de Bruijn representation, variables are encoded as numbers $n \in \mathbb{N}$, making terms elements of $\text{lam } \mathbb{N}$. The meaning of an occurrence of index n depends on its *depth* $d \in \mathbb{N}$, defined as the number of abstractions on the unique path from the root node of the tree to the individual occurrence:

Bound variables: If $n < d$, n is a bound variable indexing into the sequence of enclosing abstractions, with the innermost binder at index 0.

Free variables: If $n \geq d$, then the occurrence represents the $(n - d)^{\text{th}}$ free variable, which is presumed to index an externally-provided ordered context declaring a set of free variables.

For terms without free variables, each α -equivalence class of nominal terms has a unique de Bruijn representation. For instance, $\lambda x.x$ and $\lambda y.y$ are uniquely represented by $\text{abs } (\text{var } 0)$. This uniqueness extends to terms with free variables as well, provided some ordering among the free occurrences. Note that occurrences of the same logical variable may be represented with distinct values: for example, $\text{var } 0$ and $\text{abs } (\text{var } 1)$ each reference the first free variable in different contexts.

Capture-avoiding substitution for de Bruijn terms is somewhat indirect in its formulation. Following Schäfer et al. [64], we adopt the perspective of *parallel substitutions*, where substitutions replace any number of different free variables simultaneously. Formally, a *substitution* is a function

$$\sigma: \mathbb{N} \rightarrow \text{lam } \mathbb{N}$$

where $\sigma(0)$ replaces the first free variable, $\sigma(1)$ the second, and so on. Substitutions are lifted over terms by the following operation.

Definition 2.2.1. *Let σ be a substitution. The instantiation $t[\sigma]$ of t by σ is the term computed by applying σ to the free variables in t , defined as follows:*

$$(\text{var } n) [\sigma] \equiv \sigma(n) \tag{2.1}$$

$$(\text{abs } t) [\sigma] \equiv \text{abs } (t[\uparrow\sigma]) \tag{2.2}$$

$$(\text{app } t_1 t_2) [\sigma] \equiv \text{app } (t_1[\sigma]) (t_2[\sigma]) \tag{2.3}$$

The key step occurs in the *abs* case, where we modify σ before making the recursive subcall. This is necessary because inside an abstraction, the indices of free variables shift. At the top level (i.e., not under *abs*), $\text{var } 0$ represents the first free variable and should be replaced with $\sigma(0)$, but underneath *abs*, the first free variable is represented with $\text{var } 1$. Additionally, $\sigma(0)$ cannot be inserted as-is in place of $\text{var } 1$, for the same reason: the first free variable at the top level of $\sigma(0)$ is represented by $\text{var } 0$, which must be replaced with $\text{var } 1$ to account for binder we are underneath. In general, each free variable in t must be dereferenced by subtracting the number of enclosing abstractions before applying σ , and each of the free variables in the newly inserted subterm must be incremented by the same value.

The higher-order function $(\uparrow)$ adjusts σ to accounts for the semantics of substitution in the body of an abstraction. This operation is defined from a handful of more primitive operations, as follows:

$$\uparrow \sigma \equiv \text{var } 0 \bullet (\sigma * \uparrow) \quad (2.4)$$

This definition is composed of several primitives identified by Abadi et al. [1] as fundamental:

Prepending/Consing: Let $f: \mathbb{N} \rightarrow A$ be any function, which can be thought of as a list of A values, and let $a: A$. Then $a \bullet f: \mathbb{N} \rightarrow A$ is defined

$$(a \bullet f) n = \begin{cases} a & \text{if } n = 0 \\ f(n - 1) & \text{if } n > 0 \end{cases}$$

This represents the list formed by prepending a to f .

Substitution Composition: The second key operation is the composition $\sigma * \tau$ of substitutions σ and τ :

$$(\sigma * \tau) (n) \equiv (\sigma(n)) [\tau]. \quad (2.5)$$

Note that this notation applies the left side first, to match the postfix notation for instantiation ($t[\sigma * \tau] = t[\sigma][\tau]$ is a derivable rule). We denote composition with $(*)$ where other authors have used $(\circ)$ to reserve the latter for ordinary function composition.

Identity Substitution: The *identity substitution* coincides with the *var* constructor, which lifts a variable to an atomic expression.

Shift Substitution: The *shift* substitution $(\uparrow)$ maps an index to the atomic subterm formed by its successor:

$$\uparrow n \equiv \text{var } (n + 1). \quad (2.6)$$

Therefore $\uparrow \sigma \equiv \text{var } 0 \bullet (\sigma * \uparrow)$ replaces index 0 with $\text{var } 0$ and maps all other variables $n + 1$ to $\sigma(n)[\uparrow]$, where the shift increments every free variable in $\sigma(n)$ by 1. This captures the intended

behavior of substitution directly underneath one binder.

Notably, Definition 2.2.1 exhibits a circular dependency: instantiation $t[\sigma]$ is defined in terms of $(\uparrow)$, which relies on substitution composition $(*)$, which in turn depends on instantiation. Thus, just as with nominal substitution, a naïve formulation of these operations is not structurally recursive and, consequently, not accepted by Coq. To resolve this issue, we can break the circular dependency by giving a two-level definition where renaming—a special case of instantiation—is defined first [63].

Definition 2.2.2. *Let $\rho: \mathbb{N} \rightarrow \mathbb{N}$ be a renaming operation where $\rho(n)$ is the new name of the n^{th} free variable. Then $\text{rename } \rho \ t$ is defined as follows, where $\uparrow_{\text{ren}} \rho \equiv 0 \bullet ((+1) \circ \rho)$ maps 0 to 0 and maps any value $n + 1$ to $\rho(n) + 1$:*

$$\text{rename } \rho \ (\text{var } n) \quad \equiv \quad \text{var } (\rho \ n) \quad (2.7)$$

$$\text{rename } \rho \ (\text{abs } t) \quad \equiv \quad \text{abs } (\text{rename } (\uparrow_{\text{ren}} \rho) \ t) \quad (2.8)$$

$$\text{rename } \rho \ (\text{app } t_1 \ t_2) \quad \equiv \quad \text{app } (\text{rename } \rho \ t_1) \ (\text{rename } \rho \ t_2) \quad (2.9)$$

This operation is redundant in that the following equation holds (where $\circ$ is ordinary function composition):

$$\text{rename } \rho \ t = t[\text{var } \circ \rho] \quad (2.10)$$

However, it breaks the problematic circularity if $(\uparrow)$ is redefined to mean

$$\uparrow \sigma \equiv \text{var } 0 \bullet (\text{rename } (n \mapsto n + 1) \circ \sigma), \quad (2.11)$$

after which (2.4) is proved as a corollary.

The β -reduction rule for the lambda calculus has a simple definition using de Bruijn indices:

$$\text{app } (\text{abs } t) \ u \rightarrow_{\beta} t[u \bullet \text{var}]$$

The advantage of this representation become apparent in the context of manipulating formulas automatically (discussed below). However, de Bruijn terms are difficult for humans to read and write, introducing a gap between informal and formal practice. Because free variables index into an ordered environment, when formalizing a system such as a logic, structural rules like weakening that modify the environment must also modify the underlying terms. For instance, a rule like right-sided weakening has the following presentation, which increments the free variables in t to “skip over” the new type declaration:

$$\frac{\Gamma \vdash t : \tau}{\Gamma, \tau' \vdash t[\uparrow] : \tau} \text{ (WEAK-R)}$$

Such concerns complicate the presentation of inference rules.

A typical language compiler implementation strategy is to parse nominal terms, easy for programmers to read and write, into de Bruijn terms, easy to manipulate internally. Of course, such an implementation raises the question of whether the translation is correct. Hindley and Seldin note the following [40]:

“For [humans], the details of α -conversion would not be avoided by de Bruijn’s notation, but would simply be moved from the stage of manipulating terms to that of translating between the two notations.”

2.2.1 Autosubst

A common utility for de Bruijn terms is Autosubst [64, 69], which implements a complete decision procedure [63], based on the axioms of the explicit substitution calculus [1], for a delineated class of expressions involving terms and substitutions. The de Bruijn backend implemented by Tealeaves is closely modeled on Autosubst, so we discuss its usage. A user instantiates the library with the following definition of terms (`bind term` is a notation used by Autosubst to indicate that `abs` binds a variable):

```
Inductive term: Type :=
| var (x: nat)
| app (t1 t2: term)
| abs (t: {bind term}).
```

We must provide definitions for renaming, instantiation, and the identity substitution. Autosubst provides a tactic `derive` that can instantiate these operations automatically, using the `bind term` notation to deduce which constructors are binders:

```
Instance Ids_term: Ids term. Proof. derive. Defined. (* = var *)
Instance Rename_term: Rename term. Proof. derive. Defined.
Instance Subst_term: Subst term. Proof. derive. Defined.
```

Finally, four equations must be proved. These consist of (2.10) and the following three equations:

$$t[\text{var}] = t \tag{2.12}$$

$$(\text{var } i) [\sigma] = \sigma(i) \tag{2.13}$$

$$t[\sigma][\tau] = t[\sigma * \tau] \tag{2.14}$$

These equations are bundled into a typeclass, `SubstLemmas`. The `derive` tactic can deduce these equations automatically:

`Instance SubstLemmas_term: SubstLemmas term. derive. Qed.`

These four laws suffice to derive a range of other rules for a confluent rewriting system. These include the following identity and associativity laws for composition of substitutions:

$$\text{var} * \sigma = \sigma \quad (2.15)$$

$$\sigma * \text{var} = \sigma \quad (2.16)$$

$$(\sigma * \tau) * \phi = \sigma * (\tau * \phi) \quad (2.17)$$

The following additional equations are also derivable:

$$(\text{var } 0) [t \bullet \sigma] = t \quad (2.18)$$

$$\uparrow * (t \bullet \sigma) = \sigma \quad (2.19)$$

$$(\text{var } 0) [\sigma] \bullet (\uparrow * \sigma) = \sigma \quad (2.20)$$

$$(\text{var } 0) \bullet \uparrow = \text{var} \quad (2.21)$$

$$(t \bullet \sigma) * \tau = t[\tau] \bullet (\sigma * \tau) \quad (2.22)$$

Taken together, (2.15)–(2.22), augmented with (2.12), (2.14), and the definitional equalities (2.2) and (2.3), form the convergent rewriting system depicted in Figure 1 of [64]. This rewriting system is complete for the de Bruijn algebra as defined in [63] and therefore yields a decision procedure for equivalence of a class of expressions denoting de Bruijn terms and de Bruijn substitutions. As noted in [64], the four equations (2.10) and (2.12)–(2.14) suffice to derive all of these (assuming functional extensionality, as both `Autosubst` and `Tealeaves` do). `Autosubst` provides the tactics `asimpl` and `autosubst` that use this rewriting procedure to simplify substitution expressions to a normal form and solve some classes of equational goals automatically, using the σ -calculus as a rewriting system. For instance, the following equational goal, a keystone lemma used in a proof of confluence for the lambda calculus with de Bruijn terms [64], can be solved in one step by invoking the `autosubst` tactic:

$$t[\uparrow \sigma][s[\sigma] \bullet \text{var}] = t[s \bullet \text{var}][\sigma]$$

Though de Bruijn indices are difficult to read, the automated convenience of this approach is more usable than the intricate manual reasoning about operations on trees from Section 2.1.

2.3 LOCALLY NAMELESS

The complexities of a de Bruijn representation primarily stem from the treatment of free variables, which must be shifted during substitution. Conversely, the complexities of nominal variables primarily arise from the arbitrary names of bound variables. The locally nameless approach is a hybrid that has been argued to combine the advantages of both [61]: bound variables are represented as indices, while free variables retain names. The domain of variables, written LN , is therefore the (tagged, disjoint) union of indices and names:

$$\text{LN} \equiv \text{Bd } n \mid \text{Fr } x.$$

Aydemir et al. [10] have advocated a mechanized metatheory methodology that combines the locally nameless representation with the use of cofinite quantification in inference rules involving binders. This approach reduces some of the syntactic boilerplate involved in language formalizations. Notably, locally nameless introduces more operations than de Bruijn. Charguéraud [22] offers a detailed treatment of the locally nameless approach, which we closely follow.

Because named occurrences are free by definition, computing the set of free variables is straightforward and does not require special handling of the abs case:

$$\begin{aligned} \mathbf{fv}(\text{var}(\text{Bd } n)) &\equiv \emptyset \\ \mathbf{fv}(\text{var}(\text{Fr } x)) &\equiv \{x\} \\ \mathbf{fv}(\text{abs } t) &\equiv \mathbf{fv } t \\ \mathbf{fv}(\text{app } t_1 t_2) &\equiv \mathbf{fv } t_1 \cup \mathbf{fv } t_2 \end{aligned}$$

Importantly, there is no possibility for a free variable to become inadvertently bound. This makes the substitution of u for free occurrences of x in t , written $t[x \mapsto u]$, particularly simple:

$$\begin{aligned} (\text{var}(\text{Bd } n))[x \mapsto u] &\equiv \text{var}(\text{Bd } n) \\ (\text{var}(\text{Fr } y))[x \mapsto u] &\equiv u && \text{if } y = x \\ (\text{var}(\text{Fr } y))[x \mapsto u] &\equiv \text{var}(\text{Fr } y) && \text{if } y \neq x \\ (\text{abs } t)[x \mapsto u] &\equiv \text{abs}(t[x \mapsto u]) \\ (\text{app } t_1 t_2)[x \mapsto u] &\equiv \text{app}(t_1[x \mapsto u])(t_2[x \mapsto u]) \end{aligned}$$

This convenience comes at a cost. The substitution operation above is not the operation used to define β -reduction. Instead, reduction is defined in terms of *opening* a term t by a term u , denoted t^u . The β -reduction rule takes this form:

$$\text{app}(\text{abs } t) u \rightarrow_{\beta} t^u$$

In a term of the form $\text{abs } t$, the occurrences bound to the outer abstraction are not of the form $\text{Fr } x$. Rather, they are de Bruijn indices $\text{Bd } n$ where n is exactly equal to the number of abstractions within whose scope they fall as occurrences within t . For example, at the top level of t , these are occurrences of $\text{Bd } 0$. The operation that replaces such occurrences is defined as follows, and requires tracking the number of binders traversed under during recursion:

$$\begin{aligned}
 t^u &\equiv \{0 \rightarrow u\} t \quad \text{where} \\
 \{d \rightarrow u\} (\text{var } (\text{Bd } n)) &\equiv \text{var } (\text{Bd } n) && \text{when } n < d \\
 \{d \rightarrow u\} (\text{var } (\text{Bd } n)) &\equiv u && \text{when } n = d \\
 \{d \rightarrow u\} (\text{var } (\text{Bd } n)) &\equiv \text{var } (\text{Bd } (n - 1)) && \text{when } n > d \\
 \{d \rightarrow u\} (\text{var } (\text{Fr } x)) &\equiv \text{var } (\text{Fr } x) \\
 \{d \rightarrow u\} (\text{abs } t) &\equiv \text{abs } (\{(d + 1) \rightarrow u\} t) \\
 \{d \rightarrow u\} (\text{app } t_1 t_2) &\equiv \text{app } (\{d \rightarrow u\} t_1) (\{d \rightarrow u\} t_2)
 \end{aligned}$$

The approach described by Aydemir et al. makes use of several other operations as well. Closing t by the free variable $\text{Fr } x$, written $\backslash^x t$, acts as a kind of inverse to opening t by the term $\text{var } (\text{Fr } x)$.

$$\begin{aligned}
 \backslash^x t &\equiv \text{close } x t \equiv \text{close}_{\text{at}} 0 x t \quad \text{where} \\
 \text{close}_{\text{at}} d x (\text{var } (\text{Bd } n)) &\equiv \text{var } (\text{Bd } n) && \text{when } n < d \\
 \text{close}_{\text{at}} d x (\text{var } (\text{Bd } n)) &\equiv \text{var } (\text{Bd } (n + 1)) && \text{when } n \geq d \\
 \text{close}_{\text{at}} d x (\text{var } (\text{Fr } y)) &\equiv \text{var } (\text{Bd } d) && \text{if } y = x \\
 \text{close}_{\text{at}} d x (\text{var } (\text{Fr } y)) &\equiv \text{var } (\text{Fr } y) && \text{if } y \neq x \\
 \text{close}_{\text{at}} d x (\text{abs } t) &\equiv \text{abs } (\text{close}_{\text{at}} (d + 1) x t) \\
 \text{close}_{\text{at}} d x (\text{app } t_1 t_2) &\equiv \text{app } (\text{close}_{\text{at}} d x t_1) (\text{close}_{\text{at}} d x t_2)
 \end{aligned}$$

A distinctive aspect of locally nameless is that some terms do not represent ordinary lambda terms, owing to the possibility of $\text{Bd } n$ appearing under fewer than $n + 1$ abstractions. Such an occurrence is neither free (because it is not some $\text{Fr } x$) nor bound to an abstraction. *Local closure* is the predicate ruling out such ill-formed occurrences; only locally closed terms represent an α -equivalence classes of named terms. While it would be common to define this predicate as an **Inductive** datatype in Coq, in this work, local closure is an ordinary function defined recursively on the structure of terms, which unifies the presentation:

$$\mathbf{lc } t \equiv \mathbf{lc}_{\text{at}} 0 t \quad \text{where}$$

$$\begin{aligned}
\mathbf{lc}_{\text{at}} d (\text{var } (\text{Bd } n)) &\equiv n < d \\
\mathbf{lc}_{\text{at}} d (\text{var } (\text{Fr } n)) &\equiv \text{True} \\
\mathbf{lc}_{\text{at}} d (\text{abs } t) &\equiv \mathbf{lc}_{\text{at}} (d + 1) t \\
\mathbf{lc}_{\text{at}} d (\text{app } t u) &\equiv \mathbf{lc}_{\text{at}} d t_1 \wedge \mathbf{lc}_{\text{at}} d t_2
\end{aligned}$$

2.3.1 LNgén

The increased number of operations relative to a pure de Bruijn representation necessitates a greater number of basic lemmas. LNgén [9], a code generator used with Coq, complements the output of Ott [65] and supports the metatheory workflow advocated by Aydemir et al. [10]. Given a specification of a user's syntax (see Figure 2.2), LNgén produces Coq source files containing proof scripts for a multitude of basic infrastructural lemmas. Some of these are shown in Figure 2.1, which have been selected from Aydemir and Weirich [9] (their Figure 4). For instance, SUBST_FRESH asserts that substituting a free variable not present in a term has the same effect as applying the identity function. Each lemma in the figure is also cross-referenced to a corresponding lemma proved in Chapter 4 or 5 that establishes the same result for an arbitrary DTM using the DTM laws rather than induction on terms.

Unlike the properties shown early in the de Bruijn setting, note that several of these properties are not equations but set inclusions, and several only hold under ancillary side conditions, for instance stipulating that a term is closed or a variable is fresh. In this light, it may be somewhat surprising that the DTM laws consist of four unconditional equations not entirely dissimilar to, but much more general than, the equations used to instantiate Autosubst.

Theorem Name	Theorem	Statement
SUBST_SPEC	Thm. 4.5.4	$t[x \mapsto u] = (\backslash^x t)^u$
FV_SUBST_LOWER	Thm. 5.2.8	$\mathbf{fv} t \setminus \{x\} \subseteq \mathbf{fv} (t[x \mapsto u])$
FV_SUBST_UPPER	Thm. 5.2.10	$\mathbf{fv} (t[x \mapsto u]) \subseteq \mathbf{fv} t \setminus \{x\} \cup \mathbf{fv} u$
FV_OPEN_LOWER	Thm. 5.2.11	$\mathbf{fv} t \subseteq \mathbf{fv} (t^u)$
FV_OPEN_UPPER	Thm. 5.2.12	$\mathbf{fv} (t^u) \subseteq \mathbf{fv} t \cup \mathbf{fv} u$
SUBST_FRESH	Thm. 5.5.1	$t[x \mapsto u] = t$ where $x \notin \mathbf{fv} t$
SUBST_SUBST	Thm. 5.5.2	$t[x \mapsto u][x' \mapsto u'] = t[x' \mapsto u'][x \mapsto u[x' \mapsto u']]$ where $x \notin \mathbf{fv} u'$ and $x \neq x'$
SUBST_CLOSE	Thm. 5.5.4	$(\backslash^y t)[x \mapsto u] = \backslash^y (t[x \mapsto u])$ where $x \neq y$ and $y \notin \mathbf{fv} u$ and $\mathbf{lc} u$
OPEN_INJ	Thm. 5.5.7	$t^x = u^x \iff t = u$ where $x \notin (\mathbf{fv} t \cup \mathbf{fv} u)$

Figure 2.1: Some lemmas generated by LNgen, with references to proofs applicable to any DTM

```

exp, e, f, g :: '' ::=
| x          ::   :: var
| e1 e2      ::   :: app
| \ x . e    ::   :: abs (+ bind x in e +)
| ( e )      :: S :: paren {{ coq ([[e]]) }}
| { e2 / x } e1 :: M :: subst {{ coq (open_exp_wrt_exp [[x e1]] [[e2]]) }}

```

Figure 2.2: Snippet of an Ott specification for lambda calculus for input to LNgen

TEALEAVES

This chapter examines the role of Tealeaves in providing syntactical infrastructure within a formal metatheory workflow. The framework centers on a hierarchy of category-theoretic typeclasses [67, 68], including monads, decorated functors, decorated monads, decorated traversable monads (DTMs), and related abstractions. All of these structures are specialized to the category **Type** of Coq types and functions, which, for our purposes, closely resembles the category **SET**. This allows us to sidestep many categorical complications, with the exception of the following simplifying axioms:

Functional Extensionality Two functions are equal if they return equal outputs for equal inputs:

$$f = g \iff \forall x, fx = gx$$

Propositional Extensionality Two propositions are equal if they form a bi-implication:

$$(P = Q) = (P \iff Q)$$

Neither axiom is provable in Coq’s type theory, though both are known to be consistent with it. In principle, one could avoid assuming them by formalizing everything over setoids and systematically replacing propositional equality with setoid equivalence.

The general structure of a formalization using Tealeaves is shown in Figure 3.1. Currently, there are two ways to initialize a Tealeaves development. The simpler path is for a user to define their syntax as a type constructor $T: \mathbf{Type} \rightarrow \mathbf{Type}$ and then prove that it forms a specific structure: a traversable monad decorated by the monoid $\langle \mathbb{N}, +, 0 \rangle$ of natural numbers under addition. We examine this instantiation process in more detail below.

Once a user has established the required structure, they can import and use *backend* modules. A *backend* is a Coq module implementing a concrete first-order representation of variable binding. The instantiation described above affords the use of two backends: a parallel de Bruijn substitution following Section 2.2, and a single-variable substitution for locally nameless following Section 2.3. These modules provide various syntax operations, lemmas, and custom simplification tactics. As shown in Figure 3.1, the user remains responsible for the high-level metatheory of their system, such as confluence, soundness, and preservation lemmas. Tealeaves eliminates the boilerplate associated with defining and reasoning about low-level syntactic operations, such as substitution and enumeration of free variables. The long-term goal is to enable the community to contribute additional representational strategies—for example, a version of locally nameless supporting parallel substitution—as pluggable Tealeaves backends.

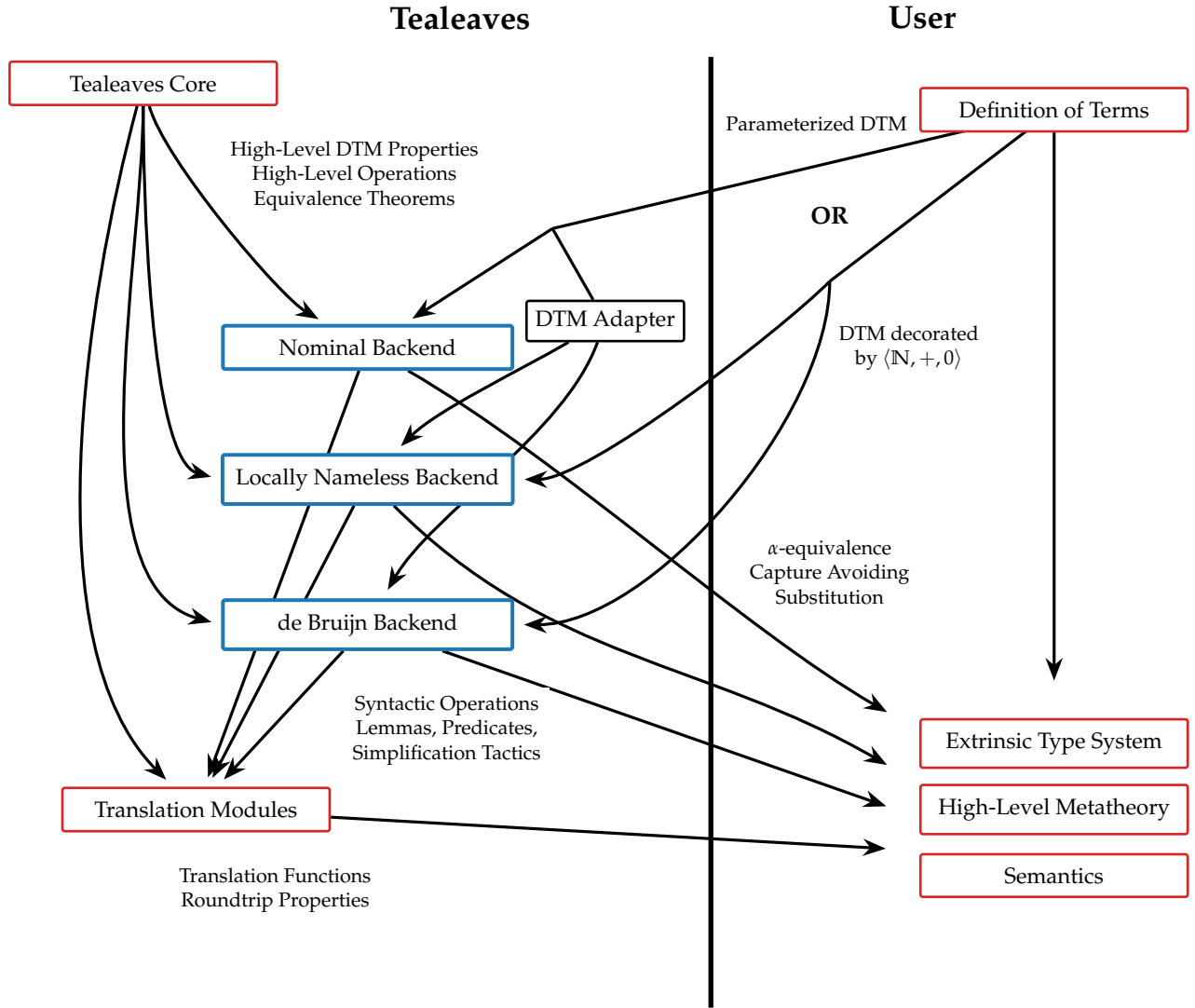


Figure 3.1: General structure of a Tealeaves development, with arrows representing data flow

The second, more general initialization path defines syntax as a binary type constructor of type $T: \mathbf{Type} \rightarrow \mathbf{Type} \rightarrow \mathbf{Type}$ and requires proving that it forms a *parameterized* decorated traversable monad, which is technically a *family* of DTMs. This richer structure will be examined in ?? and Section 5.7. Parameterized DTMs subsume DTMs. In particular, an adapter module is provided that proves the instance $T \mathbf{1}$ forms a DTM decorated by $\langle \mathbb{N}, +, 0 \rangle$, which makes this structure compatible with the locally nameless and de Bruijn backends. However, a parameterized DTM is required to use the fully nominal backend, which provides α -equivalence and capture-avoiding substitution as presented in Section 2.1. For now, let us ignore this extra parameterization and concentrate on ordinary DTMs.

The backend is responsible for choosing the type parameter V , representing variables, with which T is instantiated. For example, the de Bruijn backend defines functions over the type $T \mathbb{N}$ because de Bruijn indices are natural numbers. Additionally, while I often refer to a monad as simply being “decorated,” it is more correct to say that a DTM is decorated *by* a particular monoid W . Like V , the choice of W is fundamentally dictated by the backend, because the choice represents the *contextual* information needed to resolve variables. The de Bruijn indices and locally nameless backends happen to both use the monoid $\langle \mathbb{N}, +, 0 \rangle$, because in both cases, the contextual information represents the number of binders in scope at a particular occurrence. To use these backends, a user must use this same monoid when proving their syntax forms a DTM.

3.1 INSTANTIATING TEALEAVES

The following inductive datatype in Coq corresponds to the definition of lambda calculus terms from Section 2.2:

```
Inductive lam (V : Type) : Type :=
| var : V -> lam V
| abs : lam V -> lam V
| app : lam V -> lam V -> lam V.
```

Notably, this definition does not specify that `abs` binds a variable in its body. This information will be encoded exogenously as part of the definition of `binddt`, below. Compared to the definition of `term` used to instantiate Autosubst, the parameterization of `lam` by an underlying set of variables allows the same syntax to be reused with multiple backends.

Before the de Bruijn and locally nameless backends can be used, `lam` must be proven to form a traversable monad decorated by $\langle \mathbb{N}, +, 0 \rangle$. Tealeaves provides a total of three equivalent definitions of this abstraction, and proofs of their equivalence, but the simplest version to instantiate is based on two operations, `ret` and `binddt`, which the user defines. For `lam`, `ret` is simply `var`, the constructor coercing variables to terms. The other operation is defined as follows:

```

Fixpoint binddt_lam `{Applicative G} `(f: nat * V1 -> G (lam V2))
(t: lam V1): G (lam V2) := match t with
| var v      => f (0, v)
| abs t      => pure abs <*> binddt_lam (f (⊙ 1) t)
| app t1 t2 => pure app <*> binddt_lam f t1 <*> binddt_lam f t2
end.

```

This operation represents a generic structural recursion principle that supports defining operations on `lam` without incurring a dependency on the shape of `lam` itself. This particular instance is examined more thoroughly in Section 5.4. The parameter `{Applicative G}` indicates that `binddt` is parameterized by an applicative functor (Definition 5.3.5) named `G`. Operations `pure` and `<*>` are part of the structure of `G`. The parameter `f` is two-argument function. The first argument gives the number of binders in scope at an occurrence, and the second is the actual value of the variable. The `var` case is defined `f (0, v)` because there are no binders in scope in an atomic term. The operation `(f (⊙ 1))` (Def. 4.3.45) is the analogue of $(\uparrow)$ from Section 2.2 and reflects passing under a single binder. In general, the use of $(\odot)$ encodes the *binding policy*—a choice of which constructors bind variables in which of their arguments. We do not make the notion of “binding policy” precise except that it must satisfy the DTM laws; it is this flexibility that enables Tealeaves to be used with variadic binding, which we will examine in Chapter 6 after developing the necessary technical machinery.

The operations `var` and `binddt_lam` are registered as instances of operational typeclasses `Binddt` and `Return`.

```

Instance Return_lam: Return lam := var.
Instance Binddt_lam: Binddt nat lam lam := binddt_lam.

```

In the arguments to the `Binddt` class, the argument `nat` is the type of the underlying monoid, which we know must be $\langle \mathbb{N}, +, 0 \rangle$, while the two appearances of `lam` indicate that this operation substitutes lambda terms inside other lambda terms.

These operations must be shown to satisfy the appropriate equational axioms. These proofs are given by induction on terms, where individual cases mostly involve mechanical rewriting with a handful of basic equations. In fact, for such a simple syntax, users can invoke a tactic `derive_dtm` provided by Tealeaves to prove the axioms automatically, much like Autosubst’s `derive`.

```

Instance DTM_lam: DecoratedTraversableMonad nat lam.
Proof. derive_dtm. Qed.

```

The exact proofs synthesized by `derive_dtm`, and the basic equations they depend on, are detailed in Appendix A.

For more complicated kinds of syntax—particularly nominal syntax or syntax with variadic binding—the automation currently provided by Tealeaves is not (logically) complete and will fail to prove the DTM laws automatically. This should not be surprising: `binddt` is an arbitrary function defined by the user, so without constraining the form of this operation, it is difficult to ensure that a given tactic will always work. When the tactic fails, the culprit is typically the binding policy. The more complicated the binders, the less obvious it becomes that the DTM laws are satisfied. For variadic binders, the proof requires fairly deep theory, namely the representation theorem(s) for (decorated-)traversable functors, which will be examined in Sections 6.2 and 6.4. We will later prove a theorem stating that the DTM laws are always satisfied if the monad is freely generated from a decorated traversable functor, which suggests a fully automatic instantiation process is possible.

3.1.1 Typed Calculi

Tealeaves is also applicable to typed lambda calculi. In such cases, the typing judgment is defined extrinsically on the set of terms, rather than being incorporated into the definition of terms themselves. Tealeaves does not, and is not intended to, prove theorems about type systems—these are specific to a formal system, and typing rules are usually specific to the top-level constructor of a given term. Tealeaves supports reasoning about operations on syntactic datatypes that are, in a precise sense, *parametric* with respect to the shape of terms. Intuitively, such operations must specify their logic purely in terms of how individual variable occurrences are affected; they cannot make special decisions about individual constructors of syntax. Of course, users can define such operations by structural recursion on terms, as usual, and use these alongside the operations defined by Tealeaves.

Chapter 7 presents examples of using Tealeaves to prove type soundness theorems for simply-typed lambda calculus (STLC) and System F. Section 8.2.2 discusses how the approach taken by this work could be adapted to settings where terms are intrinsically well-scoped or well-typed.

3.2 DE BRUIJN INDICES BACKEND

The de Bruijn representation is implemented in the module `Backends.DB`. It provides the full suite of operations from Section 2.2. Its usage looks like this:

```
From Tealeaves Require Import Backends.DB.  
Notation "'term'" := (lam nat).  
Check DB.rename: (nat -> nat) -> term -> term.
```

```
Check DB.subst: (nat -> term) -> term -> term.
```

The operations exported by `Backends.DB` involve the type $T\mathbb{N}$ where T is a parameter assumed only to form a DTM decorated by $\langle\mathbb{N}, +, 0\rangle$. The type constructor T , along with the relevant typeclass instances, are *implicit parameters* of these operations. As a result, they can be applied to any datatype registered as an instance of the appropriate typeclass. In the example shown above, a notation is introduced to abbreviate $\text{lam } \mathbb{N}$, and the type annotations in the code implicitly specialize the generic operations `DB.rename` and `DB.subst` to the case $T = \text{lam}$.

A number of lemmas about these operations are also provided, including in particular the ones discussed in Section 2.2. Among these are the equations required to instantiate the `SubstLemmas` typeclass of `Autosubst`. In fact, the backend supplies a shim module that instantiates `Autosubst` for an arbitrary DTM, which makes available that library’s notations and tactics for use. This exists for demonstration purposes, but it is not practical to use: the `Autosubst`-defined tactics `asimpl` and `autosubst` do not work with Tealeaves out of the box. Their implementation relies on Coq’s built-in simplification tactics, such as `cbn` (“call by name”), which do not interact well with operations defined as special cases of `binddt` rather than with `Fixpoint`. Namely, these simplification procedures unroll `binddt` to expose its definition, leading to cryptic expressions users do not expect. Therefore, Tealeaves provides its own reimplement of `Autosubst`’s `asimpl` tactic that refolds `binddt` after simplification. We reimplement this tactic with minimal modifications except to replace its use of `cbn` with a custom simplification tactic tailored for use with `binddt`; the resulting functionality essentially subsumes `Autosubst`. As an example of compatibility, the Tealeaves repository includes a proof of confluence for the lambda calculus, adapted with from Schäfer et al. [64] with minimal modifications except to use Tealeaves’ version of the tactics.

3.2.1 Extensions of the de Bruijn Algebra

The DTM abstraction is more expressive than the σ -calculus upon which `Autosubst` is founded. For example, besides substitution and renaming, the backend defines a *predicate* `clat` (“closed at”) where `clat k t` holds if and only if no index in t resolves to a free variable at or above level k . For instance, a term with no free variables satisfies `clat 0 t`. Another operation, `level`, returns the minimum value k such that `clat k t` holds:

```
Check DB.cl_at: nat -> term -> Prop. (* all free vars are < k *)
Check DB.level: term -> nat.         (* smallest 'k' s.t. 'cl_at k' *)
```

One key lemma supplied by the de Bruin backend states that the effect of a substitution σ on a particular term t is determined only by the value of σ up to the level of t . Formally, this property

takes the following form.

Lemma 3.2.1 ( SUBST_POINTWISE). *Let T be a DTM decorated by $\langle \mathbb{N}, +, 0 \rangle$, let $t : T \mathbb{N}$ be an abstract de Bruijn term, and let k be a natural number such that $\mathbf{cl}_{\text{at}} k t$ holds. Let $\sigma_1, \sigma_2 : \mathbb{N} \rightarrow T \mathbb{N}$ be pointwise equal substitutions on variables up to but not including k . In other words, assume*

$$\forall (i \in \mathbb{N}), i < k \implies \sigma_1(i) = \sigma_2(i).$$

Then the substitutions have the same affect on t :

$$t[\sigma_1] = t[\sigma_2]$$

Remark 3.2.2. *Definitions and theorems featuring the Roqc logo () are hyperlinked to the corresponding formalization in Tealeaves.*

This sort of pointwise equational reasoning is a hallmark of Tealeaves and will be explored further in Section 5.5 and subsequent sections. Note that, unlike the operations in Section 2.2, $\mathbf{cl}_{\text{at}}$ maps terms to propositions, not terms; note also that Lemma 3.2.1 is a *conditional* property. In both cases, the additional expressiveness of Tealeaves over Autosubst arises from the assumption that T is a traversable functor, which is the subject of Chapter 5.

3.3 LOCALLY NAMELESS BACKEND

Our implementation of locally nameless is found in the module `Backends.LN`. The module begins by defining the type `LN` of locally nameless variables as the sum of `ℕ` and `atom`, where `atom` is an abstract type of which we assume only a decidable equality.

```
Inductive LN : Type :=
  | Fr : atom -> LN  (* free variables *)
  | Bd : nat  -> LN. (* bound variables *)
```

The backend exports all of the operations discussed in Section 2.3, shown in Figure 3.2. The backend also supplies proofs of all of the properties of locally nameless originally shown in Figure 2.1. Some of these lemmas are shown in Figure 3.3, using custom Coq notations supplied by the backend.

Simplification

As mentioned in Section 3.2, a common challenge with generic approaches like ours is that standard simplification tactics in Coq, like `simpl` or `cbn`, unfold definitions to reveal their underlying generic mechanism—here, the `binddt` operation. This can result in expressions that are cryptic

```

From Tealeaves Require Import Backends.LN.
Notation "'term'" := (lam LN).
Check FV: term -> atomset.
Check open: term -> term -> term.
Check close: atom -> term -> term.
Check subst: atom -> term -> term -> term.
Check lc_at: nat -> term -> Prop. (* all n < d + k in t *)
Check lc: term -> Prop.           (* = lc_at 0 *)
Check level: term -> nat.        (* the least 'k' s.t. 'lc_at k t' *)

```

Figure 3.2: Locally nameless operations defined by Tealeaves

```

Check subst_spec: forall (x: atom) (t u: term),
  t {x -> u} = (\[x] t) '(u)
Check subst_fresh: forall (x: atom) (t u: term),
  x ∉ FV t -> t {x -> u} = t.
Check open_inj: forall (x: atom) (t u: term),
  x ∉ (FV t ∪ FV u) ->
  t '(ret (Fr x)) = u '(ret (Fr x)) -> t = u.

```

Figure 3.3: Some locally nameless lemmas provided by Tealeaves

to users. More problematically, some equalities that would ordinarily hold by definition, if all operations were defined by `Fixpoint` and not with `binddt`, only hold up to propositional equality for the operations provided by Tealeaves. That is, what users ordinarily expect to be *definitions* can require *proofs*. For example, the following equation does not hold computationally for the **fv** operation defined by Tealeaves (Def. 5.2.5):

$$\mathbf{fv}(\text{app } t_1 \ t_2) = \mathbf{fv} \ t_1 \cup \mathbf{fv} \ t_2$$

Simplifying this goal with `cbn` will reveal an equality between two expressions involving `binddt` whose proof requires rewriting using the laws of applicative functors (shown in Def. 5.3.5).

To mitigate this issue, the locally nameless backend provides a custom simplification tactic, `simplify_LN`, that can perform controlled unfolding, rewriting, and refolding automatically. Using this tactic with Tealeaves provides much the same user experience as defining operations conventionally and simplifying with `cbn`. Let $t \rightsquigarrow u$ denote that t is rewritten to u by `simplify_LN`. Then the simplification of **fv** operation, for instance, matches its definition in Section 2.3:

$$\begin{array}{ll} \mathbf{fv}(\text{var}(\text{Bd } n)) & \rightsquigarrow \ \emptyset \\ \mathbf{fv}(\text{var}(\text{Fr } x)) & \rightsquigarrow \ \{x\} \\ \mathbf{fv}(\text{app } t_1 \ t_2) & \rightsquigarrow \ \mathbf{fv} \ t_1 \cup \mathbf{fv} \ t_2 \\ \mathbf{fv}(\text{abs } t) & \rightsquigarrow \ \mathbf{fv} \ t \end{array}$$

As an example of how this infrastructure is used in practice, consider the following typing rule for the simply-typed lambda calculus, which makes use of the cofinite quantification described by Aydemir et al. [10]:

$$\frac{\forall x, x \notin L \rightarrow \Gamma \cdot (x : \tau_1) \vdash t^x : \tau_2}{\Gamma \vdash \text{abs } t : \tau_1 \Rightarrow \tau_2} \text{ cof-abs}$$

This rule states that $\text{abs } t$ has type $\tau_1 \Rightarrow \tau_2$ if, for all variables x outside of a given set L , opening t by x (more precisely, by $\text{var } x$) has type τ_2 in an extended context where x has type τ_1 .

A typical infrastructural lemma used with a locally nameless representation is that all well-typed terms are locally closed. This is proved by induction on typing judgments (we emphasize that Tealeaves does not formalize properties about type systems). While proving that the typing rule above preserves local closure (source ) , the goal is $\mathbf{lc}(\text{abs } t)$ and the inductive hypothesis establishes $\mathbf{lc} \ t^x$ for some x . To solve this goal, we call `simplify_LN` to simplify the goal from $\mathbf{lc}(\text{abs } t)$ to $\mathbf{lc}_{\text{at}} \ 1 \ t$. This goal is dispatched by applying the following lemma exported by the `Backends.LN`:

$$\text{LC_OPEN_VAR}(\text{👉}): \quad \mathbf{lc} \ t^x \iff \mathbf{lc}_{\text{at}} \ 1 \ t$$

```

From Tealeaves.Backends.Adapters Require Import LNtoDB DBtoLN.
Check LNtokey: lam LN -> key.
Check toLN: key -> lam nat -> option (lam LN).
Check toDB: key -> lam LN -> option (lam nat).

```

Figure 3.4: Locally nameless and de Bruijn representation translation functions (source )

3.4 TRANSLATING REPRESENTATIONS

Different variable representation strategies have varying suitability for different tasks. Working in a representation-independent framework like Tealeaves enables us to select the most appropriate strategy for a given situation. For example, we use Autosubst-like automation to demonstrate confluence of the lambda calculus (source ). Then, after extending the calculus with a simple type system, we use the locally nameless and confinite quantification methodology to prove type soundness (source ). When multiple representations are considered in the same context—for example, if one writes a paper about a calculus with named variables, but formalizes their work in Coq using locally nameless—a gap is introduced between theory and practice unless the two representations can be formally related. A central contribution of this work is to provide a mechanism to relate different representations of variable binding.

Tealeaves provides a module that converts between de Bruijn and locally nameless terms for an arbitrary DTM, demonstrated in Figure 3.4. Note that these representation strategies are not per se equivalent: de Bruijn imposes a particular order among free variables, while locally nameless does not impose an order but imposes a choice of names. Translating between the two formats requires additional information to reconcile these differences. We encode this information into a datatype called `key`, implemented as a list of unique free variable names (i.e. no duplicates). A `key` must be provided by the user, and the conversion functions use this `key` to perform lookups during translation.

Note that the outputs of the translation functions are wrapped with the `option` (source ) monad (an `option A` is either `None` or `Some a`). This type signals the possibility that these functions can fail for arbitrary inputs. This is because, for an arbitrary term and an arbitrary `key`, there is no guarantee that the `key` lookups will succeed. For instance, attempting to translate a de Bruijn term `var 0` to locally nameless using an empty `key []` yields `None`. This is because the term contains a free variable, but the `key` does not contain a name for this variable:

$$\text{toLN } [] \text{ (var 0) = None}$$

On the other hand, if the `key` contains variable names $[x, y, z]$, indicating that x is the name of the

first free variable, then the translation yields a locally nameless term:

$$\text{toLN } [x, y, z] (\text{var } 0) = \text{Some } (\text{var } (\text{Fr } x))$$

For a more complex example, the same key maps the de Bruijn term

$$\text{app } (\text{abs } (\text{app } (\text{var } 1) (\text{var } 0))) (\text{app } (\text{var } 1) (\text{var } 0))$$

to the following locally nameless term, where $\text{bvar } n$ and $\text{fvar } x$ abbreviate $\text{var } (\text{Bd } n)$ and $\text{var } (\text{Fr } x)$, respectively:

$$\text{Some } (\text{app } (\text{abs } (\text{app } (\text{fvar } x) (\text{bvar } 0))) (\text{app } (\text{fvar } y) (\text{fvar } x)))$$

Or, in a more conventional notation, $(\lambda. 1\ 0) (1\ 0)$ is mapped to $\text{Some } ((\lambda. x\ 0) (y\ x))$. Using the same key with toDB maps the latter term to the former one.

Let $\text{dom } k$ denote the set of atoms represented in k , and recall that the level of a de Bruijn term is a (strict) upper bound of the maximum free variable represented in that term. The translation module supplies a proof that toLN and toDB form a partial bijection in the following sense:

Theorem 3.4.1. () *Any duplicate-free key k determines a partial bijection, via $\text{toDB } k$ and $\text{toLN } k$, between locally nameless terms $t: T\ \text{LN}$ that satisfy $\text{lc } t$ and $\text{fv } t \subseteq \text{dom } k$, and de Bruijn terms $t: T\ \text{IN}$ that satisfy $\text{cl}_{\text{at}} (\text{length } k) t$.*

$$\{t: T\ \text{LN} \mid \text{lc } t \text{ and } \text{fv } t \subseteq \text{dom } k\} \stackrel{\text{bij.}}{\simeq} \{t: T\ \text{IN} \mid \text{cl}_{\text{at}} (\text{length } k) t\}$$

The details of this translation process and its correctness are discussed in Section 5.6.

3.5 VARIADIC BINDERS

The approach taken by Tealeaves extends to syntax with variadic binders. To demonstrate this, we consider extend the lambda calculus with a `letin` construct that introduces a list of definition into the lexical scope of a body argument:

$$\begin{aligned} l &::= \text{nil} \mid \text{const } l \\ t &::= \text{var } v \mid \text{abs } t \mid \text{letin } l\ t \mid \text{app } t\ t \end{aligned}$$

The first argument to `letin` is a list of definitions to be bound in the body, the second argument. For simplicity, we continue to assume that are working with either de Bruijn or locally nameless, where binders are not explicitly named. Therefore, `letin` accepts a list of terms and not, for example, a list of pairs like (x, t) where x is the name being bound with definition t . For instance, syntax

that might on paper be written

$$\text{let } x = e, y = e' \text{ in } e''$$

can be concisely represented as $\text{letin } [e, e'] e''$. This expression opens two binders in e'' , referring to e and e' . In general, the number of binders opened in e'' equals the length of the list of definitions. The question of how many binders are opened in e and e' is a matter of the intended reading of letin constructor. We consider three distinct interpretations:

Simple: The constructor binds each of the definitions in the scope of the body, but does not bind any variables within the definitions themselves.

Telescoping: Each entry in the list of definitions binds one variable in all subsequent entries.

Mutually Recursive: All entries in the list are mutually visible, enabling self-reference.

With the “simple” binding policy, terms in l are invisible to other terms in l . The only effect of letin from a variable-binding perspective is to open length l binders in the body. This allows for programs such as the following one, which evaluates to 5 (for clarity we demonstrate with named variables, but the same pattern can be represented with de Bruijn or locally nameless):

$$\text{let } x = 3, y = 2 \text{ in } x + y$$

In the telescoping interpretation, later entries in the list of definitions can refer to definitions given earlier. Such a semantics supports writing the following program, which evaluates to 5:

$$\text{let } x = 3, y = x + 2 \text{ in } y$$

In a *mutually recursive* semantics for letin , all definitions in the list can reference other entries, including themselves. For instance, this sort of semantics can be used to write a non-terminating program, such as the following one:

$$\text{let } x = y, y = x \text{ in } x$$

To provide a concrete sense of how these semantics operate, Figure 3.5 shows the behavior of `LN.level` for a range of example terms under the different interpretations. For locally nameless terms, level t is the minimum k for which all de Bruijn indices that occur in t at some depth d satisfy $n < d + k$. Locally closed terms are precisely the ones that satisfy level $t = 0$, while $\text{abs } t$ is locally closed if and only if level $t \leq 1$. For readability, we do not explicitly show the `bvar` and `fvar` constructors for the terms in Figure 3.5.

Consider $\text{letin } [x] 0$ for instance. This expression binds the atomic term x as the first free variable in the body term, and the body is a reference to this variable. This term is locally

Term	Simple	Telescoping	Mutually Recursive
letin $[x]$ 0	0	0	0
letin $[0]$ 0	1	1	0
letin $[x, 0]$ 0	1	0	0
letin $[x, 1]$ 0	2	1	0
letin $[x, 0, 1]$ 2	2	0	0
letin $[1, 2, 3]$ 2	4	2	1

Figure 3.5: LN.level applied to locally nameless terms under different readings of letin (source )

closed under any interpretation. Now consider $\text{letin } [x, 1] \ 0$. Under the simple interpretation, the index 1 occurs where no bound variables are in scope, so the level of this term is 2. Under the telescoping interpretation, there is one variable in scope, referencing x ; the de Bruijn index exceeds its maximum allowable value (0) by 1, so the level is 1. Under the mutually recursive reading, there are two variables in scope at 1, and this term is locally closed.

Each of these readings of letin can be represented with Tealeaves—the distinction lies in how binddt uses $(\odot)$. This will be examined in greater detail in Section 6.2 after introducing the necessary theoretical machinery to prove the DTM laws for these terms. Unfortunately, the `derive_dtm` does not find such proofs automatically at this time, partly because they make non-trivial use of the theory of traversable functors developed in Chapter 6. To illustrate how the DTM instances are defined, the following code shows how the definition of `binddt_lam` from Section 3.1 is extended to implement the “simple” semantics:

```

Fixpoint binddt_lam `{Applicative G} `(f: nat * V1 -> G (lam V2))
  (t: lam V1): G (lam V2) := match t with
| letin defs body =>
  pure (@letin V2) <*> traverse (binddt_lam f) l
  <*> binddt (f  $\odot$  length l) body
| ...
end.

```

Note that list of definitions is traversed by substitution (`traverse` is defined in Chapter 5). The appearance of `f  $\odot$  length l` indicates that $(\text{length } l)$ -many binders are opened in the body term. This definition is no more complicated than the one given for lam without letin. The complexity lies in the *proof* that this definition satisfies the necessary equations.

3.6 NOMINAL BACKEND

Now we generalize to syntax with named variables. The syntax of the lambda calculus is defined as follows.

$$t ::= \text{var } v \mid \text{abs } b t \mid \text{app } t t$$

In Coq this is defined as follows.

```
Inductive nlam (B : Type) (V : Type) : Type :=
| var : V -> lam V
| abs : B -> lam V -> lam V
| app : lam V -> lam V -> lam V.
```

Formally, such syntax is represented as a two-argument type constructor, where $T B V$ indicates the set of lambda terms with B -valued binder labels and V -valued variables. As noted earlier, $\text{lam } V$ is isomorphic to $\text{nlam } \mathbf{1} V$.

For now, consider a *fixed choice* of parameter B . The definition of `binddt_lam` from Section 3.1 can be adapted to this choice with minimal changes, replacing the `abs` case as follows:

```
Fixpoint binddt_nlam {Applicative G} (f: list B * V1 -> G (nlam B V2))
(t: nlam B V1): G (nlam B V2) := match t with
| abs b t => pure (abs b) <*> binddt_lam (f ◊ [b]) t
| ...
end.
```

This definition is identical to the one earlier except that $\mathbb{N}$ has been replaced with $\text{list } B$, and the `abs` case uses `f ◊ [b]` to reflect passing under a binder with label b . With this operation, for fixed choice of B , $T B$ is a DTM decorated by the monoid $\text{list } B$ with `append` as the monoid operation. The observation that $\text{lam } V$ forms a DTM decorated by $\langle \mathbb{N}, +, 0 \rangle$ is a special case: note that $\text{list } \mathbf{1}$ is isomorphic to $\mathbb{N}$, so setting B to $\mathbf{1}$ recovers the DTM instance for `lam`.

Unfortunately, the extended operation `binddt_nlam` is not enough to generalize the theory to reason about nominal variable binding: capture-avoiding substitution renames binders, but `binddt_nlam` as shown above leaves the binder label on `abs` unchanged. In order to generalize the theory, we must recognize that T is also a functor in the B parameter, and this functorial structure must be incorporated into the picture. Incorporating this extra functoriality results in the notion of a *parameterized* DTM, which introduces several new theoretical considerations. We will examine the resulting structure in two stages: first in Section 4.6, where we introduce parameterized decorations, and then again in Section 5.7, where we consider how to generalize traversals to this structure.

The backend module for this representation is `Backends.Nominal`. This module remains a subject of ongoing work; currently, it provides fundamental operations such as computation of free variables, single-variable substitution, and alpha equivalence:

```

From Tealeaves Require Import Backends.Nominal.
Notation "'term'" := (nlam atom atom).
Check FV: term -> atomset.
Check subst: atom -> term -> term -> term.
Check alpha: term -> term -> Prop.

```

3.6.1 Translating Between Nominal and Locally Nameless

This module also proves that alpha equivalence is indeed an equivalence relation. Perhaps the most interesting infrastructure provided by Tealeaves is a translation between nominal terms and locally nameless terms:

```

From Tealeaves Require Import Backends.Adapters.LNandNominal. (* LN <-> Nominal *)
Check toLN: nlam atom atom -> nlam unit LN.
Check toNominal: nlam unit LN -> nlam atom atom.

```

These operations implicitly use of a derived typeclass instance proving that `nlam unit` is a traversable monad decorated by `IN`. The translation module proves two roundtrip properties that characterize the result composing the two operations in different orders. If we begin with a nominal term, translate to a locally nameless term, and translate back, we always arrive at something α -equivalent to the original term:

```

Check roundtrip_Nominal: forall (t: nlam atom atom), alpha t (toNominal (toLN t)).

```

In the other direction, translating a locally nameless term twice yields the original term precisely, assuming the term we start with is locally closed:

```

Check roundtrip_LN: forall (t: nlam unit LN), lc t -> t = toLN (toNominal t).

```

Capture-avoiding substitution for nominal terms is examined in Section 4.6. The translation discussed here is examined in Section 5.6. The definition of α -equivalence and the proof of the roundtrip properties, which depend on a coalgebraic characterization of DTMs, must wait until Section 6.5.

3.7 MULTISORTED TEALEAVES

A *multisorted* syntax has multiple grammatical categories (e.g., terms, formulas, types) and multiple kinds of variables (e.g. term variables, type variables). Theoretically, extending the DTM concept to a multisorted setting is straightforward and follows the same path as ordinary multisorted universal algebra (e.g., see Section 2 of [38]). Instead of working with a single set of expressions, one works with a family of sets parameterized by a set $\mathcal{S}$ of *sorts*. Categorically, this corresponds to moving from the category **SET** to the category $\mathcal{S}\text{-SET}$ of $\mathcal{S}$ -indexed families. In this setting, a multisorted algebraic signature gives rise to a free monad on $\mathcal{S}\text{-SET}$. The notions of traversable and decorated functors generalize straightforwardly to functors on this category.

Tealeaves includes a multisorted variant of DTMs parameterized by a set of sorts. When the sort set is a singleton, the abstraction reduces to the ordinary DTM concept. However, the implementation in Tealeaves does not faithfully follow the development of multisorted algebra, as doing so was found to be difficult in practice. To see why, suppose $\mathcal{S}$ is a family of sorts, represented as a datatype equipped with a decidable equality, and suppose $T: \mathcal{S} \rightarrow \mathbf{Type}$ represents an $\mathcal{S}$ -indexed type family. Now suppose $s_1, s_2: \mathcal{S}$ are two sorts and suppose we have a proof that $s_1 = s_2$ in Coq. Although this proof allows us to derive an equality $T s_1 = T s_2$, this equality does not hold *definitionally* in Coq. In particular, a term of type $T s_1$ is not automatically considered to have type $T s_2$. A basic implementation of multisorted DTMs using $\mathcal{S}$ -indexed type families was found to lead to frequent type errors that blocked convenient reasoning.

These kinds of technical complications concerning equalities between types are well-known, and several options exist for dealing with them. One possibility is to introduce a *heterogeneous* notion of equality (e.g., see [52]). However, the approach we take is to sidestep the issue by adopting the following pragmatic assumption:

In a multisorted setting, the underlying set of variables is the same for all sorts.

Suppose, for instance, that we wish to formalize the polymorphic lambda calculus (System F), which has both term and type variables, using de Bruijn indices. The assumption is that both term and type variables are represented with the set $\mathbb{N}$, without introducing a fundamental type distinction between indices that represent terms and those representing types. I believe this is a fair assumption in many cases, particularly considering that the intention of DTMs is to serve as a foundation for extrinsically scoped first-order syntax, where variables are fully concrete data that carry no information intrinsically except their raw value.

Under this assumption, the presentation of multisorted DTMs is only a minor extension of the single-sorted notion we will consider in most of this manuscript. Although we do not make a type distinction between different sorts of variables, note that multisorted DTMs do *not* allow ill-typed substitutions. For example, we cannot replace a variable that represents a term in the object language with an expression that represents a type.

The details of this technical compromise are presented in Appendix C. Note that multisorted DTMs are formalized separately from ordinary DTMs, and therefore the multisorted variant requires its own backends. Currently, Tealeaves provides a single multisorted backend, a port of the locally nameless backend. The development of this backend closely mirrors the single-sorted version, but it is more expressive. The additional expressiveness arises from the fact that we can reason about the interaction between substitutions that affect different sorts of variables, which leads to slightly different principles about when operations can be commuted, for instance.

DECORATED MONADS

Monoids, their homomorphisms, their actions, and the homomorphisms of their actions—all defined below—are basic and fundamental structures. A generalized theory of monoids can be defined in any *monoidal category* (Def. 4.1.4). By categorical duality, one also obtains a theory of comonoids, comonoid homomorphisms, co-actions, and so on. Decorated traversable monads are rooted in this generalized theory of monoids, considered in the category END_{Set} of endofunctors with composition as the monoidal product (Def. 4.1.46).

The *string diagram calculus* provides an elegant and intuitive notation for reasoning in monoidal categories, absorbing much of the otherwise distracting categorical bookkeeping into the notation. Section 4.1 is a self-contained introduction to this notation from an applied perspective. Section 4.2 is a diagrammatic overview of monads, algebras, and substitution. The contributions of this work begin in Section 4.3, which develops decorated functors and their monoidal categorical structure. Sections 4.4 and 4.5 define substitution for de Bruijn and locally nameless terms, respectively; 4.4 also derives the axioms of the de Bruijn algebra from those of decorated monads. Section 4.6 defines *parameterized families* of decorated monads, then defines ordinary capture-avoiding nominal substitution. However, deeper reasoning about locally nameless and nominal substitution must wait until developing additional machinery in Chapters 5 and 6.

4.1 INTRODUCTION TO DIAGRAMMATIC REASONING

This section introduces string diagrams in SET and outlines basic constructions that will be used in later sections. Assuming a background level of familiarity with category theory (see [8] for an introduction), my focus is on intuition. Every concept here will be applied or generalized in the study of decorated traversable monads, so readers may find it useful to revisit this section when encountering those generalizations.

Remark 4.1.1. *In this chapter and elsewhere, some standard lemmas are stated without proof; these are marked with Q.E.D symbol following the statement, as in this remark.* $\square$

4.1.1 The Category of Sets

Definition 4.1.2. *A category $\mathcal{C}$ is defined from the following data:*

- A class $\text{Obj}_{\mathcal{C}}$ of objects. We write $A \in \mathcal{C}$ to mean $A \in \text{Obj}_{\mathcal{C}}$.
- For any two objects $A, B \in \mathcal{C}$ a set $\text{Hom}_{\mathcal{C}}(A, B)$ of morphisms. We write $f: A \rightarrow B$ or $A \xrightarrow{f} B$ to mean $f \in \text{Hom}_{\mathcal{C}}(A, B)$.

- For any $A \in \mathcal{C}$ a designated morphism $A \xrightarrow{\text{id}_A} A$ called the identity at A .
- For any $A, B, C \in \mathcal{C}$ a composition function $(\circ_{A,B,C})$ of type

$$\text{Hom}_{\mathcal{C}}(B, C) \times \text{Hom}_{\mathcal{C}}(A, B) \rightarrow \text{Hom}_{\mathcal{C}}(A, C).$$

We write $(g \circ f)$ to mean $\circ(g, f)$.

Of course, composition must be associative, and id must act as an identity element for composition.

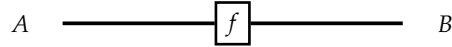
$$\text{id} \circ f = f = f \circ \text{id} \quad (h \circ g) \circ f = h \circ (g \circ f).$$

Example 4.1.3. The category of sets, written SET , is the category whose objects are sets and where $\text{Hom}(A, B)$ consists of all functions from A to B , with the usual notion of composition and identities.

There are two categories of foundational interest in this work. The first is SET . The second is the category END_{SET} of endofunctors on SET , introduced in Sec. 4.1.9 and treated as a natural extension of SET . It is convenient to reason in these categories by the notation of string diagrams. A set A is denoted by a simple wire:



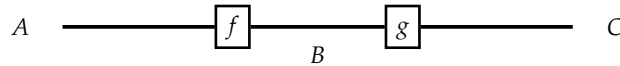
A function $f: A \rightarrow B$ is denoted by a box or (logic) gate whose input wire, shown on the left, is labeled with the type A , and whose output wire is labeled B :



Wires lack a meaningful length; diagrams are understood topologically, in terms of how they are connected.

The function $\text{id}_A: A \rightarrow A$ is treated specially by omitting it. In effect, this notation identifies an object with its identity function. If wires are thought of as transmitting information, then the identity function is a perfectly conductive and infinitely extendable wire. The identity law $\text{id} \circ f = f = f \circ \text{id}$ is implicitly encoded into the notation itself: all three expressions are represented by the same diagram.

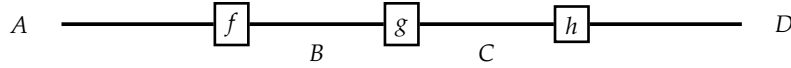
The composition of two functions $f: A \rightarrow B$ and $g: B \rightarrow C$ is depicted by arranging a sequence of boxes like beads along a string:



A slight notational incongruity arises here: function composition, written using $(\circ)$, follows the common mathematical convention that $(g \circ f)(a)$ means $g(fa)$. The diagrammatic convention of

this manuscript is that the leftmost function is applied first—thus, wires “propagate” information left-to-right.

The associativity law $(h \circ g) \circ f = h \circ (g \circ f)$ is encoded into the syntax by the fact that both sides are drawn identically as a chain of three boxes along a wire:



4.1.2 Monoidal String Diagrams

The default notion of composition baked into Definition 4.1.2 pipes the outputs of operations into the inputs of successive ones. This may be regarded as a notion of *vertical* composition (despite drawing them horizontally). The category SET and many others relevant to programming languages possess an extra structure: morphisms can also be composed in parallel, or *horizontally*, with independent operations acting on separate inputs. The corresponding notion is that of a monoidal category.

Definition 4.1.4. A monoidal category $\langle \mathcal{C}, \mathbf{I}, \otimes \rangle$ is a category $\mathcal{C}$ augmented with the following extra constructs:

- For any two $A, B \in \mathcal{C}$, an object $A \otimes B$ called their (monoidal) product.
- A designated object $\mathbf{I} \in \mathcal{C}$ called the (monoidal) unit.
- For two any morphisms $A \xrightarrow{f} C$ and $B \xrightarrow{g} D$, a monoidal product of morphisms:

$$A \otimes B \xrightarrow{f \otimes g} C \otimes D$$

These constructions are subject to natural isomorphisms of the following types:

$$\mathbf{1} \otimes A \simeq A \simeq A \otimes \mathbf{1} \quad (A \otimes B) \otimes C \simeq A \otimes (B \otimes C)$$

These must satisfy a few additional (unsurprising) equations known as the coherence conditions. These are not central to the material here and will be elided. A reader interested in the finer details may consult a reference such as [51].

Though it is common for the monoidal product to be denoted with $(\otimes)$, also commonly used to mean the tensor product of vector spaces, this product does not have to be the tensor product. In SET, we are interested in the monoidal structure given by the Cartesian product.

Example 4.1.5. $\langle \text{SET}, \mathbf{1}, \times \rangle$ is a monoidal category where $A \otimes B$ is the Cartesian product $A \times B$ and $f \otimes g \equiv f \times g$ is the pairwise mapping of functions f and g over their respective components. The unit object is the singleton set, denoted $\mathbf{1} = \{*\}$.

The Cartesian product is hardly the only monoidal categorical structure on SET. For example, the disjoint union $\sqcup$ with the empty set $\emptyset$ also satisfy the definition. This document will always refer to the Cartesian monoidal structure.

In the diagrammatic notation, the product $A \times B$ of two sets is drawn as a pair of adjacent wires.

$$A \times B \text{ ————— } A \times B \equiv \begin{array}{c} A \text{ ————— } A \\ B \text{ ————— } B \end{array}$$

The product of sets extends to an operation applying functions in parallel. Given $f: A \rightarrow C$ and $g: B \rightarrow D$, their product $A \times B \xrightarrow{f \times g} C \times D$ is defined

$$(a, b) \mapsto (fa, gb),$$

which is equivalent to either of the compositions

$$A \times B \xrightarrow{f \times B} C \times B \xrightarrow{C \times g} C \times D$$

and

$$A \times B \xrightarrow{A \times g} A \times D \xrightarrow{f \times D} C \times D.$$

(Note the convention of using an object's name to stand for its identity function.) Diagrammatic notation makes the equation obvious simply by sliding morphisms along their wires:

$$\begin{array}{c} A \text{ — } \boxed{f} \text{ — } C \\ B \text{ — } \boxed{g} \text{ — } D \end{array} = \begin{array}{c} A \text{ — } \boxed{f} \text{ — } C \\ B \text{ — } \boxed{g} \text{ — } D \end{array}$$

An (uncurried) function of two inputs, say $f: A \times B \rightarrow C$, is drawn with two input wires. A function with two outputs, for example the operation $\text{dup}: A \rightarrow A \times A$ that duplicates its input, is shown with two outputs. Naturally, a function with two inputs and two outputs, say $g: A \times B \rightarrow C \times D$, has two input and two output wires, and so on. (Note that $A \times B \rightarrow C \times D$ should be parsed as $(A \times B) \rightarrow (C \times D)$.) These examples are depicted below, respectively.

$$\begin{array}{c} A \text{ — } \boxed{f} \text{ — } C \\ B \text{ — } \end{array} \quad \begin{array}{c} A \text{ — } \boxed{\text{dup}} \text{ — } A \\ A \text{ — } \end{array} \quad \begin{array}{c} A \text{ — } \boxed{g} \text{ — } C \\ B \text{ — } \end{array}$$

The Monoidal Unit

The singleton set $\mathbf{1} = \{*\}$ consisting of a single canonical element has a special relationship with the Cartesian product. Namely, it is the unit of the operation in the sense of the following “equations”:

$$\mathbf{1} \times A \simeq A \simeq A \times \mathbf{1} \quad (4.1)$$

The notation $\simeq$ indicates that two sides are naturally isomorphic. Isomorphic here means for each A there are left and right *unitors* of the following types:

$$\mathbf{1} \times A \xrightarrow{\lambda_A^{-1}} A \quad A \xrightarrow{\lambda_A} \mathbf{1} \times A \quad A \times \mathbf{1} \xrightarrow{\rho_A^{-1}} A \quad A \xrightarrow{\rho_A} A \times \mathbf{1}$$

These must satisfy a requirement that composing λ and ρ with their inverses in either direction is the identity function. The naturality conditions yield the following commutative diagrams:

$$\begin{array}{ccccc} \mathbf{1} \times A & \xrightarrow{\lambda_A^{-1}} & A & & A & \xrightarrow{\lambda_A} & \mathbf{1} \times A & & A \times \mathbf{1} & \xrightarrow{\rho_A^{-1}} & A & & A & \xrightarrow{\rho_A} & A \times \mathbf{1} \\ \mathbf{1} \times f \downarrow & & \downarrow f & & f \downarrow & & \downarrow \mathbf{1} \times f & & f \times \mathbf{1} \downarrow & & \downarrow f & & f \downarrow & & \downarrow f \times \mathbf{1} \\ \mathbf{1} \times B & \xrightarrow{\lambda_B^{-1}} & B & & B & \xrightarrow{\lambda_B} & \mathbf{1} \times B & & B \times \mathbf{1} & \xrightarrow{\rho_B^{-1}} & B & & B & \xrightarrow{\rho_B} & B \times \mathbf{1} \end{array}$$

Like the identity function, the unit object is not drawn explicitly. Consequently, the notation makes no visual distinction between the objects A , $A \times \mathbf{1}$, or $\mathbf{1} \times \mathbf{1} \times A \times \mathbf{1}$. All of the unitor functions defined above are depicted identically to id_A :

$$A \xrightarrow{\quad\quad\quad} A$$

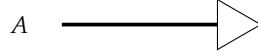
The $\mathbf{1}$ object by itself (or its identity function) is drawn as a blank diagram:

Although suppressing the explicit depiction of the monoidal unit and unitors can make the diagrammatic notation ambiguous, the axioms of monoidal categories ensure that equations between diagrams can be read up to any application of unitors that makes the equation well-typed.

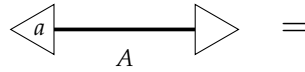
Morphisms from the unit object, i.e. functions of the form $\mathbf{1} \rightarrow A$, correspond precisely to the elements of A in an obvious way: $a \in A$ corresponds to the function $* \mapsto a$. These element-morphisms are depicted as a box originating with no input wires and an output of type A . We depict such boxes in the shape of a triangle:

$$\triangleleft_a \xrightarrow{\quad\quad\quad} A$$

Though it is not a general property of monoidal categories, it happens in $\mathbf{SET}$ that any set A has a unique function into $\mathbf{1}$. This be denoted as del_A and referred to as “deletion” since it maps all values to the unit element $*$, which contains no information. This function is drawn as a triangle-shaped “cap,” and the notation should suggest that an input of type A is discarded or somehow collapsed:

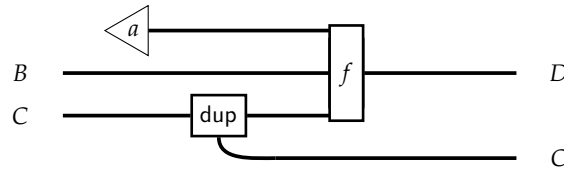


Consider an element $a \in A$ composed with the operation that deletes it. Formally this is represented by the function $\mathbf{1} \xrightarrow{a} A \xrightarrow{\text{del}} \mathbf{1}$, resulting in the unique function of type $\mathbf{1} \rightarrow \mathbf{1}$, which we have just seen is represented by an empty diagram. This yields the following equation stating that spontaneously creating an element and deleting it is the same as doing nothing:



Associator Morphisms

We present a more complicated operation to demonstrate the notation. Let $a \in A$ and let $A \times B \xrightarrow{f} D$. The following diagram depicts a function of type $B \times C \rightarrow D \times C$ that maps an input (b, c) to the output $(f(a, b, c), c)$, where dup is the operation that makes two copies of its inputs:



It subsumed by the notation that the Cartesian products $(A \times B) \times C$ and $A \times (B \times C)$ are interchangeable for all practical purposes. This is codified by a family of natural isomorphisms of the form $(A \times B) \times C \simeq A \times (B \times C)$, witnessed by *associator* morphisms denoted with the Greek letter α . A non-diagrammatic categorical account of this operation—one in which all functions are defined explicitly as compositions of more basic functions—would require the reassociator morphisms to be made explicit. By contrast, the above notation would be translated to the following composition:

$$\begin{aligned}
 B \times C &\xrightarrow{\rho_{B \times C}^{-1}} \mathbf{1} \times (B \times C) \xrightarrow{a \times (B \times C)} A \times (B \times C) \xrightarrow{\alpha_{A, B, C}^{-1}} (A \times B) \times C \\
 &\xrightarrow{A \times B \times \text{dup}} (A \times B) \times (C \times C) \xrightarrow{\alpha_{(A \times B), C, C}^{-1}} ((A \times B) \times C) \times C \xrightarrow{f \times C} D \times C
 \end{aligned}$$

The utility of the diagrammatic notation is to suppress trivial details, such as explicit depiction of the unitors and associators, that otherwise clutter morphisms and equations.

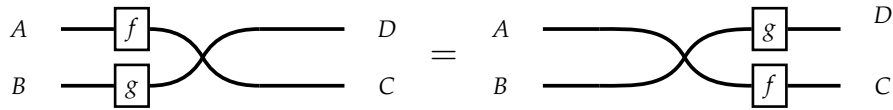
4.1.3 Crossing Wires

The Cartesian product has the property that $A \times B$ is naturally isomorphic to $B \times A$, realized by an operation $\text{swap}_{A,B}$ defined by $(a, b) \mapsto (b, a)$. Diagrammatically, this isomorphism is depicted by crossing wires, like so:

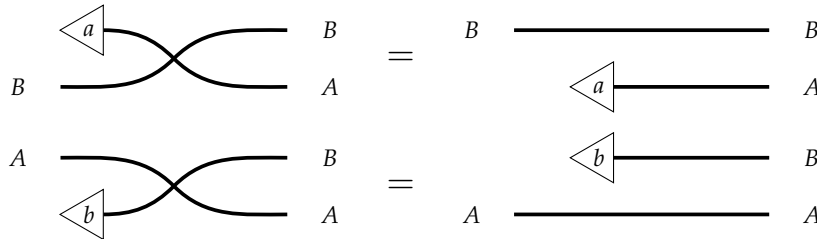


This operation makes SET into a *symmetric* monoidal category and the swap operation is called the *symmetry*. We now examine several properties of this operation. While these properties may seem visually self-evident, they require formal justification in general monoidal categories. For instance, wires cannot be arbitrarily crossed in END_{Set} , though some wires can be crossed with some others—Definition 5.3.21, for instance, defines traversable functors by stipulating that certain wires can be crossed over them.

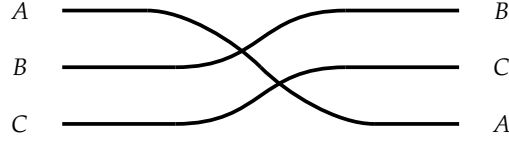
The fact that $\text{swap}_{A,B}$ is natural in A and B means that functions applied to the individual components can slide along the wires. Let $A \xrightarrow{f} C$ and $B \xrightarrow{g} D$. Then we can apply the function $f \times g$ to an input (a, b) , yielding (fa, gb) , and swap to obtain (gb, fa) . Or, we can swap the arguments immediately and then apply the function $g \times f$. This corresponds to the following equality:



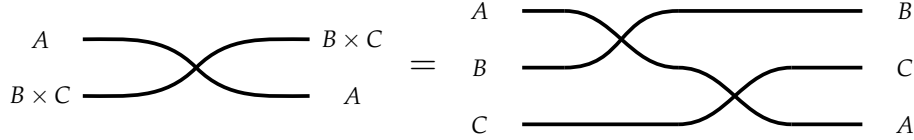
In particular, elements can be crossed over wires according to the following equations:



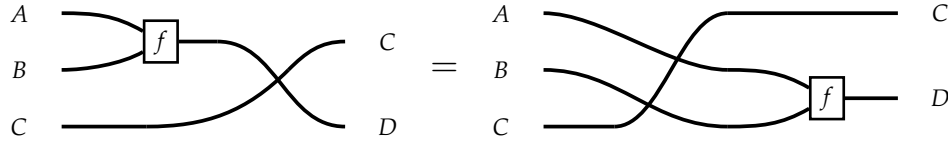
At face value, there is an ambiguity about whether the following diagram represents the operation $\text{swap}_{A,B \times C}$ that swaps the entire Cartesian product $B \times C$ with A , or the composition of multiple swapping operations:



The reader can verify that these are the same function, so there is no real ambiguity. That is, we have the following equality:



Operations with multiple inputs are subject to the following equality. Analogous properties hold for functions with multiple outputs or that cross from right to left.



4.1.4 Monoids, Homomorphisms, and Actions

Of all the algebraic structures, the notion of monoid has perhaps the best ratio between simplicity and utility. In certain contexts, we emphasize that these are monoids *in the category* SET to distinguish them from generalized monoids defined in an arbitrary monoidal category $\mathcal{C}$.

Definition 4.1.6. (🔗) A monoid is a triple $\langle M, e : M, m : M \times M \rightarrow M \rangle$ consisting of a set M , a unit e_M , and a binary operation m , subject to the following (implicitly universally quantified) equations.

$$m(e, x) = x \quad m(x, e) = x \quad m(x, m(y, z)) = m(m(x, y), z)$$

Or, expressed in an infix notation:

$$e \cdot x = x = x \cdot e \quad (x \cdot y) \cdot z = x \cdot (y \cdot z)$$

By abuse of terminology, we sometimes identify a monoid with its underlying set M . Because monoids are used so extensively in this text, we adopt a convention of drawing their operations as filled-in circles. Such a notation is unambiguous when the type of the wire is known and the monoid structure associated with the underlying set is clear from context.

$$\begin{array}{c} M \\ M \end{array} \curvearrowright M \equiv \begin{array}{c} M \\ M \end{array} \begin{array}{c} \diagup \\ \diagdown \end{array} \boxed{m} M \quad \bullet - M \equiv \triangleleft e - M$$

The monoid equations take the following form:

$$\begin{array}{c} \bullet \\ M \end{array} \curvearrowright M = M - M = \begin{array}{c} M \\ \bullet \end{array} \curvearrowright M$$

$$\begin{array}{c} M \\ M \\ M \end{array} \curvearrowright M = \begin{array}{c} M \\ M \\ M \end{array} \curvearrowright M$$

Concrete Monoids

Beyond the abstract properties of monoids, specific monoids play a crucial role in defining and reasoning about operations on terms. These monoids underpin fundamental operations such as the following:

- Enumerating variables in a term
- Verifying well-scopedness within a given context
- Determining whether a particular variable appears in a term

The most frequently used monoids are defined below.

Example 4.1.7. $\langle \text{list } A, [], ++ \rangle$ is the free monoid over A for any set A , where $[]$ is the empty list and $l_1 ++ l_2$ appends two lists.

The significance of list A as specifically a free monoid is examined in Section 5.1.2.

Example 4.1.8. $\langle \mathbb{N}, 0, + \rangle$ is the monoid of natural numbers under addition.

Note that $\mathbb{N}$ is equivalent to the free monoid over $\mathbf{1}$ by identifying 0 with $[]$ and successor with the operation $(l \mapsto \text{cons} * l)$.

Example 4.1.9. $\langle \mathbf{Prop}, \text{False}, \vee \rangle$ is a monoid, where $\mathbf{Prop}$ is the Coq type of propositions and False is the empty (i.e. unprovable) proposition. Note that the monoid equations,

$$\text{False} \vee P = P = P \vee \text{False} \quad (P \vee Q) \vee R = P \vee (Q \vee R),$$

rely on the axiom of propositional extensionality (see Ch. 3). We can likewise consider the monoid $\langle \mathbf{Prop}, \wedge, \text{True} \rangle$ of propositions under conjunction.

Next, we define the “subset” monoid over a type A . These subsets on A are actually formalized as *predicates* of type $A \rightarrow \mathbf{Prop}$. This monoid plays a fundamental role in defining set membership in Chapter 5.

Example 4.1.10. (🚫) *Given a set A , let $\text{subset } A$ denote $A \rightarrow \mathbf{Prop}$. The monoid of subsets under union has the constant-false operation (const False) as the unit. The monoid operation is defined*

$$(S \cdot S') a \equiv Sa \vee S' a.$$

Similarly, the subsets under intersection have (const True) as the unit where $S \cdot S'$ is defined

$$(S \cdot S') a \equiv Sa \wedge S' a.$$

We do not assume the ability to enumerate these set or to decide whether a subset contains a particular element $a : A$. We could also consider the type of *decidable* subsets of A , which have type $A \rightarrow \text{bool}$ and allow computing whether a particular $a : A$ is a member. Decidable subsets are not as convenient for use in Tealeaves because they do not form a functor on the category of types in Coq—see Example 5.3.15.

The definition of monoid generalizes to any monoidal category $\langle \mathcal{C}, \mathbf{I}, \otimes \rangle$ simply by interpreting diagrams of the same shapes shown above in $\mathcal{C}$.

Definition 4.1.11. *A generalized monoid consists of an object $M \in \mathcal{C}$ and morphisms of types $\mathbf{I} \rightarrow M$ and $M \otimes M \rightarrow M$, where these are subject to natural generalizations of the monoid equations.*

Monoid Homomorphisms

Definition 4.1.12. (🚫) *Where M and N are monoids, a monoid homomorphism $\phi : M \rightarrow N$ is a function between their underlying sets that preserves the monoidal structure:*

$$\begin{array}{c} \bullet \\ \text{---} \text{---} \text{---} \end{array} \begin{array}{c} \boxed{\phi} \\ \text{---} \end{array} \begin{array}{c} \text{---} \\ \text{---} \end{array} N = \begin{array}{c} \bullet \\ \text{---} \end{array} N \quad \phi e_M = e_N \quad (4.2)$$

$$\begin{array}{c} M \\ \text{---} \end{array} \begin{array}{c} \text{---} \\ \text{---} \end{array} \begin{array}{c} \bullet \\ \text{---} \end{array} \begin{array}{c} \boxed{\phi} \\ \text{---} \end{array} \begin{array}{c} \text{---} \\ \text{---} \end{array} N = \begin{array}{c} M \\ \text{---} \end{array} \begin{array}{c} \boxed{\phi} \\ \text{---} \end{array} \begin{array}{c} \text{---} \\ \text{---} \end{array} \begin{array}{c} \bullet \\ \text{---} \end{array} \begin{array}{c} \boxed{\phi} \\ \text{---} \end{array} \begin{array}{c} \text{---} \\ \text{---} \end{array} N \quad \phi(x \cdot y) = \phi x \cdot \phi y \quad (4.3)$$

The Trivial Monoid

The unit **1** can be equipped with a monoid structure in an obvious way ($* \cdot * = *$). According to the notation, the monoid operation, which has type $\mathbf{1} \times \mathbf{1} \rightarrow \mathbf{1}$, is drawn with a blank diagram. This makes it quite simple to verify the monoid equations:

=

Opposite Monoids

Crossing the input wires into the multiplication operation yields the *opposite* monoid.

Definition 4.1.13. (🚫) Let $\langle M, e, m \rangle$ be a monoid. The *opposite monoid* is the structure $\langle M, e, m^{\text{opp}} \rangle$, where m^{opp} is defined as follows:

$$\begin{array}{c} M \\ \diagdown \\ \boxed{m^{\text{opp}}} \\ \diagup \\ M \end{array} \quad M \quad \equiv \quad \begin{array}{c} M \\ \diagdown \\ \text{X} \\ \diagup \\ M \end{array} \quad M \quad m^{\text{opp}}(x, y) \equiv y \cdot x \quad (4.4)$$

The reader can check using the diagrammatic notation that the above operation respects the monoid laws. It is also easy to check that the opposite monoid of the opposite monoid yields the original monoid, which follows by a rule of the symmetry: crossing the same wires twice consecutively is the identity.

Commutativity and Idempotence

Definition 4.1.14. (🚫) A monoid is *commutative* when it validates the following equation.

$$\begin{array}{c} \diagdown \\ \bullet \\ \diagup \end{array} \quad M \quad = \quad \begin{array}{c} M \\ \diagdown \\ \text{X} \\ \diagup \\ M \end{array} \quad M \quad x \cdot y = y \cdot x \quad (4.5)$$

Definition 4.1.15. A monoid is *idempotent* when it validates the following equation (the first operation applied is the duplication function, discussed below):

$$M \quad \begin{array}{c} \text{O} \\ \bullet \end{array} \quad M \quad = \quad M \quad \text{---} \quad M \quad x \cdot x = x \quad (4.6)$$

Additionally, commutativity can be defined on a per-element basis. A particular element $a \in M$ is *commutative* when $a \cdot x = x \cdot a$ for all $x \in M$, and is *idempotent* when $a \cdot a = a$.

Definition 4.1.16. The *center* of a monoid $\langle M, e_M, \cdot \rangle$ consists of all elements that are commutative, forming a commutative submonoid of M .

Definition 4.1.17. The idempotent center of a monoid $\langle M, e_M, \cdot \rangle$ consists of all elements that are commutative and idempotent, forming a sub-semilattice of M .

Section 5.7, which defines parameterized DTMs, generalizes the above notion to that of the idempotent center of an applicative functor, a concept required to state the composition law.

Product Monoid

Let M and N be two monoids. Owing to the particular properties of **SET**, the class of monoids in this category is itself closed under Cartesian products. The reader may want to use the diagrammatic monoid laws and the properties of crossing wires to validate that the next definition constitutes a valid monoid.

Definition 4.1.18. (🎮) We can define a monoid structure on the Cartesian product $M \times N$ by combining operations pointwise:

$$e_{M \times N} \equiv (e_M, e_N) \quad (4.7)$$

$$(m_1, n_1) \cdot (m_2, n_2) \equiv (m_1 \cdot m_2, n_1 \cdot n_2) \quad (4.8)$$

The same approach can be used to define a monad formed by the composition of two monads under certain conditions (see Lemma 4.3.38).

4.1.5 Monoids Acting on a Set

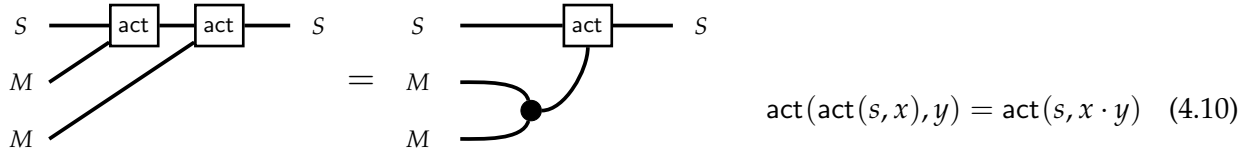
Monoids become particularly interesting when they *act* on a set. Generalized monoid actions provide a unifying framework for many concepts in this work, such as right modules (Def. 4.2.33), decorated functors (Def. 4.3.14), and monad algebras (Def. 4.2.24), all defined by some form of monoidal (co-)action on an object.

Definition 4.1.19. A right action of a monoid M on a set S of states is a function of the following type:

$$\text{act} : S \times M \rightarrow S \quad \text{This}$$

operation must satisfy the following equations:

$$\text{act}(s, e) = s \quad (4.9)$$



$$\text{act}(\text{act}(s, x), y) = \text{act}(s, x \cdot y) \quad (4.10)$$

The above definition coincides with that of a *semiautomaton*, a simple notion of state transition system. The partial application $\text{act}(-, x) : S \rightarrow S$ associates each element $x \in M$ with a function of type $S \rightarrow S$. Let us denote this function temporarily as $\llbracket x \rrbracket$. The axioms then state that the monoid unit acts as the identity function, and that the function $\llbracket x \cdot y \rrbracket$ behaves as the composition $\llbracket x \rrbracket; \llbracket y \rrbracket$, where the semicolon is left-to-right function composition ($f; g = g \circ f$). Thus, a right monoid action is a monoid homomorphism from M to the monoid $S \rightarrow S$ of functions under composition, provided composition is read with the semicolon.

Analogously, a *left* M -action is a function of type $M \times S \rightarrow S$ satisfying a slightly different associativity condition:

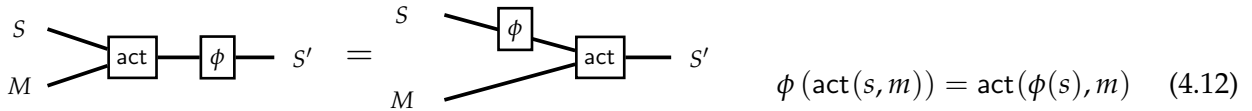
$$\text{act}(x, \text{act}(y, s)) = \text{act}(x \cdot y, s) \quad (4.11)$$

Here, the action of $x \cdot y$ on S is computed by first applying the action of y to S , followed by x . The previous construction provides a monoid homomorphism from M into $S \rightarrow S$, considered as a monoid under function composition ($\circ$).

Lemma 4.1.20. *A left M -action is equivalent to a right action of the opposite monoid. Likewise, right actions are equivalent to left actions of the opposite monoid.* $\square$

In cases where M is commutative, there is no distinction between a left or right action.

Definition 4.1.21. *A homomorphism of a monoid actions of M on the sets S and S' is a function of type $S \rightarrow S'$ that commutes with the actions in the following sense:*



$$\phi(\text{act}(s, m)) = \text{act}(\phi(s), m) \quad (4.12)$$

4.1.6 Monoids as Categories

In the categorical view, objects primarily exist to encode type information in the sense of governing which morphisms can be composed with which others. In the diagrammatic notation, objects serve only to label wires. This leads to thinking of a category as just a (possibly very large) monoid generalized to include a type system.

Lemma 4.1.22. *A monoid in SET is conceptually the same as thing as a category with a single object.*

Proof. The single object is immaterial can be denoted $*$. $\text{Hom}(*, *)$ is the set M itself. The morphisms of this category correspond to the elements of the monoid, the composition is monoid multiplication $M \times M \rightarrow M$, and the unit element $e \equiv * \xrightarrow{e} *$ serves as the identity arrow. $\square$

Thus, a monoid can be regarded as an untyped or uni-typed category: any morphisms may be composed. Likewise, a category can be regarded as a typed monoid: morphisms may be composed when the output type of one matches the input type of the other.

Any monoid M yields two actions on its underlying set: the left action associates x with $y \mapsto x \cdot y$, and the right action associates x with $y \mapsto y \cdot x$. The above construction yields an *embedding* (injective homomorphism) of M into $M \rightarrow M$, where functions are composed either left-to-right or right-to-left depending on which action is under consideration. If M is viewed as a category with one object, then a right M -action is the same thing as a *presheaf*, and a left action is a *co-presheaf*. This previous construction of an injective homomorphism is a small glimpse of a deeper result known as the Yoneda lemma (for a discussion of Yoneda from a relevant applied perspective see [19]).

4.1.7 Comonoids, Homomorphisms, and Co-Actions

A *comonoid* is the categorical dual of a monoid, obtained by reversing arrows. Thus, a comonoid, which I will call E for “environment,” is a set paired with a *comultiplication* of type $E \rightarrow (E \times E)$ and a *counit* of type $E \rightarrow \mathbf{1}$. To enhance the readability of increasingly complex diagrams, this manuscript depicts “comonadic” wires as red.⁵

Definition 4.1.23. (🚧) A comonoid consists of a set E paired with operations of the following shapes:

$$\begin{array}{ll}
 \begin{array}{c} E \text{ --- } \bullet \end{array} & \text{counit : } E \rightarrow \mathbf{1} \\
 \begin{array}{c} E \text{ --- } \bullet \begin{array}{l} \nearrow E \\ \searrow E \end{array} \end{array} & \text{comult : } E \rightarrow E \times E
 \end{array}$$

These must satisfy the following laws:

$$\begin{array}{c} E \text{ --- } \bullet \begin{array}{l} \nearrow \bullet \\ \searrow E \end{array} \end{array} = \begin{array}{c} E \text{ --- } \end{array} E \quad (\text{counit} \times \text{id}_E) \circ \text{comult} = \text{id} \quad (4.13)$$

$$\begin{array}{c} E \text{ --- } \bullet \begin{array}{l} \nearrow E \\ \searrow \bullet \end{array} \end{array} = \begin{array}{c} E \text{ --- } \end{array} E \quad (\text{id}_E \times \text{counit}) \circ \text{comult} = \text{id} \quad (4.14)$$

$$\begin{array}{c} E \text{ --- } \bullet \begin{array}{l} \nearrow \bullet \begin{array}{l} \nearrow E \\ \searrow E \end{array} \\ \searrow E \end{array} \end{array} = \begin{array}{c} E \text{ --- } \bullet \begin{array}{l} \nearrow E \\ \searrow \bullet \begin{array}{l} \nearrow E \\ \searrow E \end{array} \end{array} \quad (\text{comult} \times \text{id}_E) \circ \text{comult} = (\text{id}_E \times \text{comult}) \circ \text{comult} \quad (4.15)$$

⁵The color scheme has been chosen for the benefit of color-impaired readers, but the colors do not convey more information than the wire labels.

Equations (4.13)–(4.15) are not strictly well-typed. The left-side of (4.13), for instance, is of type $E \rightarrow \mathbf{1} \times E$. The equality must be read to implicitly insert a unitor $\mathbf{1} \times E \xrightarrow{\lambda^{-1}} E$. Similarly, (4.15) should be read to implicitly insert an associator $(E \times E) \times E \xrightarrow{\alpha} E \times (E \times E)$. In complex diagrams where multiple such insertions must be made for type correctness, the coherence laws required of unitors and associators ensure that arbitrary choices about *where* these are inserted does not affect the truth of equation.

Though monoids and comonoids are formally dual notions, their realization in SET differ significantly. Despite the shorthand “ M is a monoid,” the underlying set of a monoid carries no structural information about the monoid beyond its cardinality. In contrast, a comonoid in SET is uniquely determined by its underlying set.

Lemma 4.1.24. (🚫) *For each set E , there is exactly one comonoid on E . This comonoid, which will be called the duplication comonoid over E , is defined from the following operations.*

$$\begin{array}{ll} \text{del} & : E \rightarrow \mathbf{1} & \text{del } e & = \star \\ \text{dup} & : E \rightarrow E \times E & \text{dup } e & = (e, e) \end{array}$$

□

The stark contrast between the structure of monoids and comonoids in SET, despite their formally dual nature, arises from the inherent asymmetry in the definition of morphisms in SET. A function $A \rightarrow B$, as the arrow notation suggests, is a *directed* notion: $A \rightarrow B$ is hardly isomorphic to $A \leftarrow B$.

The ability to duplicate and discard information carried by “comonoidal” wires is a key aspect of the theory of decorated traversable monads.

Aside 4.1.25. *The operations suggest a relationship between comonoids and classical notions of information. Classically, the state of a composite system comprising A and B is determined by the state of the subsystems—thus, a vector in the Cartesian product $A \times B$ of their state spaces. The state of a composite quantum system, which might not be associated with a definite state of A or B , is represented by a vector in the tensor product $A \otimes B$. In the sorts of categories used to model quantum systems, where morphisms are linear maps that preserve structure (e.g. norms), comonoids are highly constrained and do not uniformly exist for all objects. The scarcity of comonoids in such a setting corresponds essentially to the no-cloning and no-deletion properties of quantum mechanics. For a deeper examination in this direction, see Abramsky [2].*

In SET, the theory of comonoids is largely unremarkable.

Definition 4.1.26. *A comonoid homomorphism, the dual of Definition 4.1.12, is a function that commutes with the comonoid operations.*

Lemma 4.1.27. *Every function of type $E \xrightarrow{f} J$ corresponds to a comonoid homomorphism between the unique comonoids over E and J .* □

Thus, the property of being a comonoid homomorphism is not very special. Co-commutativity, the dual of Definition 4.1.14, is also not very special.

Lemma 4.1.28. *Every comonoid in SET is co-commutative, satisfying*

$$\text{swap} \circ \text{comult} = \text{comult} \quad (4.16)$$

□

Product comonoids can be constructed like product monoids, but of course this just yields the unique comonoid for the product of their underlying sets.

Lemma 4.1.29. *The product of two comonoids E and J is a comonoid on $E \times J$ obtained by dualizing the diagrams shown in Definition 4.1.18. This yields the unique comonoid on $E \times J$.*

However, comonoid *coactions* and their homomorphisms, the duals of Definitions 4.1.19 and 4.1.21, are useful. Since comonoids are co-commutative, left and right co-actions are equivalent. The reader is invited to deduce the appropriate axioms by inverting the diagrams for monoid actions and expressing the result as ordinary equations. They will be led the following definition.

Definition 4.1.30. *A comonoid E acting on a set S of states is a function $\text{coact}: S \rightarrow E \times S$ such that $\text{coact}(s)$ returns (e, s) for some $e \in E$, called the annotation of s .*

In terms of the information it encodes, a comonoid co-acting on S is equivalent to a function from S to E . So, a coaction is a set paired with a function that tells us something about each of its elements.

Example 4.1.31. *Given any $e \in E$ and any set S , the function $s \mapsto (e, s)$ is a coaction on S , the constant coaction over e on S .*

Example 4.1.32. *Let $\text{list } A$ be the set of lists with elements of type A . The function $l \mapsto (\text{length } l, l)$ is a coaction on $\text{list } A$.*

Example 4.1.33. *Let TA be the set of tree data structures (for some class of trees, e.g. binary trees) storing data of type A . The function $t \mapsto (\text{depth } t, t)$, where depth computes the maximum depth of a tree, is a coaction on TA .*

Although decorated-traversable functors will not be defined formally until Chapter 5, an informal definition is accessible now: a concrete data structure, such as list or tree, whose members are containers with a finite number of elements, where each *element* can be annotated with information specific to its location in the structure. For instance, instead of annotating each $\text{list } A$ with its length, we can annotate each element in a particular list with its *index* (Ex. 4.3.16).

Coaction homomorphisms are obtained by dualizing Definition 4.1.21, yielding the following.

Definition 4.1.34. A homomorphism between two coactions of E on S and S' is a function that maps elements to ones with the same annotation.

Example 4.1.35. For the length-annotation coaction on lists, homomorphisms map lists to lists of the same length. For the depth-annotation coaction on trees, homomorphisms map trees to trees of the same depth.

4.1.8 Bimonoids

We must cover one more key topic before generalizing these notions to the category of endofunctors, that being the notion of a monoid and comonoid that interact in a compatible way.

Definition 4.1.36. A bimonoid is an object M equipped with the structures of both a monoid and a comonoid in such a way as to make the structures “compatible.” The compatibility condition can be phrased in one of two equivalent ways.

- The unit $\mathbf{1} \rightarrow M$ is a homomorphism between the trivial comonoid and the comonoid on M , and multiplication $M \times M \rightarrow M$ is a comonoid homomorphism from the product comonoid on $M \times M$ to the comonoid on M .
- The counit $M \rightarrow \mathbf{1}$ is a homomorphism from M to the trivial monoid, and comultiplication $M \rightarrow M \times M$ is a monoid homomorphism from M to the product monoid on $M \times M$.

Let us unpack these conditions using the latter characterization with reference to the diagrams in Definitions 4.1.12 and 4.1.18. Let $\langle M, e, m \rangle$ be a monoid. The comonoid structure on M is uniquely determined already. Recall that a monoid homomorphism must satisfy two equations, and so both comonoid operations being homomorphisms yields a total of four conditions.

Baton Law The unit $\mathbf{1} \rightarrow M$ must be mapped by $M \rightarrow \mathbf{1}$ to the unit of the trivial monoid on $\mathbf{1}$, resulting in a function (necessarily the identity) of type $\mathbf{1} \rightarrow \mathbf{1}$.

$$\bullet \text{---} \bullet = \text{del } e_M = * \quad (4.17)$$

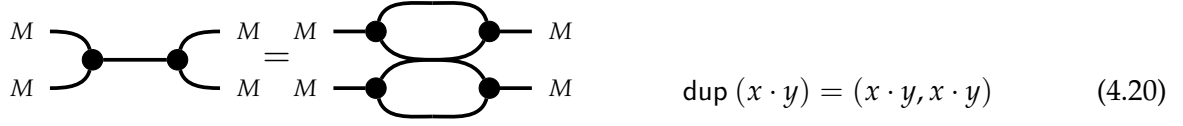
Cap Law Multiplication $M \times M \rightarrow M$, followed by $M \rightarrow \mathbf{1}$, is equivalent to applying $M \rightarrow \mathbf{1}$ to both sides of $\mathbf{1} \times \mathbf{1}$, followed by multiplication in the trivial monoid.

$$\begin{array}{c} M \\ \text{---} \end{array} \begin{array}{c} \text{---} \\ \text{---} \end{array} \bullet = \begin{array}{c} M \\ \text{---} \end{array} \bullet \quad \text{del } (x \cdot y) = \lambda_1^{-1}(*, *) = \rho_1^{-1}(*, *) \quad (4.18)$$

Cup Law Duplication $M \rightarrow M \times M$ maps the unit in M to the unit in $M \times M$.

$$\bullet \text{---} \bullet = \begin{array}{c} M \\ \text{---} \end{array} \bullet \quad \text{dup } e_M = (e_M, e_M) \quad (4.19)$$

Butterfly Law Multiplication $M \times M \rightarrow M$, followed by duplication $M \rightarrow M \times M$, is equivalent to duplicating both sides of $M \times M$, then applying the multiplication operation for the product monoid $M \times M$.



The four previous equations, being symmetrical, can also be read to state that the monoid operations are homomorphisms with respect to the comonoidal structure on their domain and codomains. This yields the dual characterization in Definition 4.1.36.

Lemma 4.1.37. *Any monoid structure on M already forms a bimonoid between M and the duplication comonoid on M . Thus, in $\mathbf{SET}$ there is no difference between monoids and bimonoids.* $\square$

The payoff of defining bimonoids is not immediately clear in $\mathbf{SET}$. In other categories, such as $\mathbf{END}_{\mathbf{SET}}$, the concept is more interesting. Compare Equations (4.19) and (4.20) with (4.89) and (4.90).

4.1.9 Diagrammatic Reasoning in $\mathbf{END}_{\mathbf{SET}}$

To see what these exercises in definitions give us, we switch settings and move from $\mathbf{SET}$ to the category $\mathbf{END}_{\mathbf{SET}}$ of endofunctors on $\mathbf{SET}$. It will be shown that $\mathbf{SET}$ embeds into $\mathbf{END}_{\mathbf{SET}}$ so that the latter can be thought of as an extension of $\mathbf{SET}$, rather than a completely new category. Essentially, we will introduce a class of wires into these diagrams, some of which will represent syntactic data structures.

Definition 4.1.38. (🚗) *An endofunctor on $\mathbf{SET}$ is a type constructor $F: \mathbf{SET} \rightarrow \mathbf{SET}$ equipped with a polymorphic operation*

$$\text{map}^F: \forall (A, B : \mathbf{SET}), (A \rightarrow B) \rightarrow FA \rightarrow FB$$

satisfying the following two equations:

$$\text{map}^F \text{id}_A = \text{id}_{FA} \quad (4.21)$$

$$\text{map}^F g \circ \text{map}^F f = \text{map}^F (g \circ f) \quad (4.22)$$

Remark 4.1.39. *We will sometimes refer to endofunctors on $\mathbf{SET}$ as simply functors. By abuse of terminology, we sometimes identify a functor with its underlying type constructor. The notation Ff is occasionally used as a shorthand for $\text{map}^F f$.*

For polymorphic operations like `map`, implemented in Tealeaves in the form of operational typeclasses [68], a superscript in this text indicates an argument to the corresponding `Class`. For

instance, a functor is defined on the type constructor $F: \text{Type} \rightarrow \text{Type}$ by providing an instance of $\text{Map } F$. More generally, superscripts generally reflect *ad-hoc polymorphism*. Subscripts in this text correspond to parametrically polymorphic arguments. For example, each functor provides its own (ad-hoc) definition of map , but any particular map^F behaves the same for all type arguments A, B . Functors in subscripts, e.g. F in id_F , indicate the operation that takes A to id_{FA} . Scripts will usually be suppressed when they can be inferred from the context.

The following concrete functors will be used throughout the text.

Example 4.1.40. (👉) *First-order data structures such as lists and trees, generally speaking, form functors whose map operations uniformly apply functions to the data items stored in the structure. For instance, list is a functor.*

$$\text{map}^{\text{list}} f ([]) \equiv [] \quad (4.23)$$

$$\text{map}^{\text{list}} f (\text{cons } x \, l) \equiv \text{cons } (f \, x) \left(\text{map}^{\text{list}} f \, l \right) \quad (4.24)$$

Example 4.1.41. (👉) *The subset functor maps a type A to the type $A \rightarrow \mathbf{Prop}$ of Coq-level predicates over A . Mapping f over a subset S forms the image of the subset:*

$$\begin{aligned} \text{map}^{\text{subset}} f &: (A \rightarrow \mathbf{Prop}) \rightarrow B \rightarrow \mathbf{Prop} \\ \text{map}^{\text{subset}} f \, S \, b &\equiv \exists a: A. S \, a \wedge f \, a = b \end{aligned}$$

That is, mapping f over a subset S yields a predicate that is true of $b: B$ if there exists an a such that S is true of a and $f \, a = b$.

Example 4.1.42. (👉) *Let E be any set. The set-forming operation $A \mapsto E \times A$ is a functor. For short, we abbreviate this functor as $E^\times$, and interpret $E^\times A$ as an A -value that exists in the context of extra information of type E . The map operation applies a function to A :*

$$\begin{aligned} \text{map}^{E^\times} f &: E \times A \rightarrow E \times B \\ \text{map}^{E^\times} f \, (e, a) &\equiv (e, f \, a) \end{aligned}$$

Example 4.1.43. *Let E be any set. A related functor to the prior one is given by the set-forming operation $A \mapsto (E \rightarrow A)$, abbreviated $\vec{E}$. A value of type $\vec{E} \, A$ is interpreted as a suspended computation of an A -value pending extra information of type E :*

$$\begin{aligned} \text{map}^{\vec{E}} f &: (E \rightarrow A) \rightarrow E \rightarrow B \\ \text{map}^{\vec{E}} f \, x &\equiv (e \mapsto f(xe)) \quad (\text{or } f \circ x) \end{aligned}$$

Definition 4.1.44. (👉) *A natural transformation $\phi: F \Rightarrow G$ between functors F and G is a polymorphic*

function

$$\phi: \forall (A: \text{SET}), FA \rightarrow GA$$

satisfying the follow equation for all $f: A \rightarrow B$:

$$\phi \circ \text{map}^F f = \text{map}^G f \circ \phi \quad (4.25)$$

Note the use of $\Rightarrow$ to record the type of natural transformations. We write $\phi_A: FA \rightarrow GA$ to indicate specializing ϕ to set A .

When functors represent concrete data structures, natural transformations are operations that manipulate the structure at a purely *structural* level, acting uniformly with respect to the data stored in them. Therefore they commute with mapping operations, which apply a function uniformly to each of the data stored in the structure while leaving the *shape* of the structure unchanged. The notion of shape is made precise in Chapter 6 (Def. 6.1.33)

Example 4.1.45. *Reversing a list without inspecting the contents is a natural transformation, as is duplication ($l \mapsto l ++ l$). Flattening a binary tree into a list in depth-first order is a natural transformation. A function that increments each value in a list or tree containing natural numbers is a map.*

The polymorphic identity function forms a natural transformation of type $F \Rightarrow F$ for any functor F . The composition of natural transformations $F \xrightarrow{\phi} G$ and $G \xrightarrow{\psi} H$ is a natural transformation $F \xrightarrow{\psi \circ \phi} H$. This forms a category.

Definition 4.1.46. () The category END_{SET} of endofunctors on SET is defined as follows:

- Objects are functors of type $\text{SET} \rightarrow \text{SET}$
- Morphisms are natural transformations between endofunctors
- The identity on F is the identity natural transformation

We may therefore use string diagrams to reason in END_{SET} , where the morphisms are *natural transformations* and wires have the type of *functors*. Functors, of course, can be composed in their own right.

Definition 4.1.47. () The composition $(G \circ F)$ maps has the following action on morphisms:

$$\text{map}^{G \circ F} f \equiv \text{map}^G (\text{map}^F f) \quad (4.26)$$

Furthermore, we can define a notion of composition of independent natural transformations in parallel.

Definition 4.1.48. If $\phi: F \Rightarrow F'$ and $\psi: G \Rightarrow G'$ are natural transformations, their horizontal composition is defined in either of two ways, which are equal by the naturality of ψ :

$$\psi \square \phi: G \circ F \Rightarrow G' \circ F'$$

$$(\psi \square \phi)_A = \text{one of} \begin{cases} G(FA) \xrightarrow{\text{map}^G \phi_A} G(F'A) \xrightarrow{\psi_{F'A}} G'(F'A) \\ G(FA) \xrightarrow{\psi_{FA}} G'(FA) \xrightarrow{\text{map}^{G'} \phi_A} G'(F'A) \end{cases}$$

This makes END_{Set} into a monoidal category.

Definition 4.1.49. END_{Set} forms monoidal category where the product of objects (functors) is defined $G \otimes F \equiv G \circ F$ and the horizontal composition of transformations $\psi \otimes \phi$ is given by $\psi \square \phi$. The monoidal unit is the identity functor $\mathbb{1}$, which maps each set and function to itself.

This furnishes the ability to reason about natural transformations by way of string diagrams. For example, the above equality has the following presentation:

$$\begin{array}{c} G \text{ --- } \boxed{\psi} \text{ --- } G \\ F \text{ --- } \boxed{\phi} \text{ --- } F \end{array} = \begin{array}{c} G \text{ --- } \boxed{\psi} \text{ --- } G \\ F \text{ --- } \boxed{\phi} \text{ --- } F \end{array}$$

4.1.10 Embedding SET into END_{Set}

We can mix diagrams involving SET and END_{Set} by interpreting sets as *constant* functors.

Definition 4.1.50. (🚗) The constant functor over A , denoted $\overline{A}$, maps all sets to A and morphisms to id_A .

$$\begin{aligned} \text{map}^{\text{const } A} : (B \rightarrow C) &\rightarrow A \rightarrow A \\ \text{map}^{\text{const } A} f a &= a \end{aligned} \tag{4.27}$$

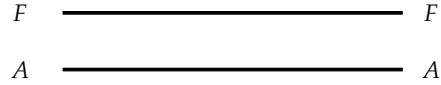
A morphism such as $f: A \rightarrow B$ can likewise be lifted to a natural transformation between constant functors by the following lemma.

Lemma 4.1.51. Given sets A and B , a natural transformation between constant functors $\overline{A} \Rightarrow \overline{B}$ is equivalent to a function from A to B .

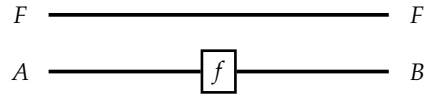
Mapping each set A to the constant functor $\overline{A}$, and each function $A \xrightarrow{f} B$ to the constant natural transformation $\overline{A} \xrightarrow{\overline{f}} \overline{B}$, forms a functor from the category SET to the category END_{Set} . Lemma 4.1.51 states that this functor is *full and faithful*. It is also injective on objects (distinct sets map to distinct constant functors). This forms an *embedding* and allows regarding SET as

essentially a subcategory of END_{Set} . This embedding underlies diagrams that mix functors, sets, functions, and natural transformations.

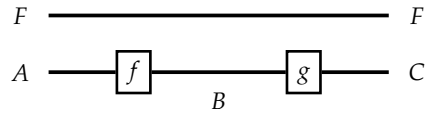
Given a functor F , the object $FA: \text{SET}$ is depicted with the following diagram, which formally represents the composition $F \circ \overline{A}$:



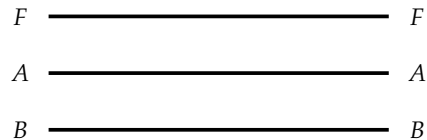
Given $A \xrightarrow{f} B$, the function $\text{map}^F: FA \rightarrow FB$ applies f to the A -wire.



The notation subsumes the functor laws in the sense that diagrams for $\text{map}^F \text{id}_A$ and id_{FA} are equal. Similarly, the diagram corresponding to the composition $\text{map}^F g \circ \text{map}^F f$ is identical to the diagram for $\text{map}^F (g \circ f)$, both consisting of morphisms f and g applied in sequence to the A wire.



However, the embedding does not justify representing a set like $F(A \times B)$ with a diagram like the following one:



Formally this diagram stands for the composition of functors $F \circ \overline{A} \circ \overline{B}$, which is equivalent to $F \circ \overline{A}$, so the diagram above actually represents the identity function on the set FA . Section 4.3 examines a different embedding to represent sets like $F(A \times B)$.

4.2 MONADS AND ALGEBRAS

We present background information on (co)monads and their (co)algebras from the perspective of string-diagrammatic reasoning. We state the necessarily properties of free monads that will later be used to prove that the decorated-traversable structure of a freely generated monad is inherited from that of its signature endofunctor.

4.2.1 Monads and Comonads

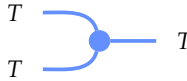
As a monoidal category, monoids and comonoids can be sensibly defined in END_{Set} , which yields monads and comonads.

Definition 4.2.1. (🚗) A monad is a monoid in END_{Set} .

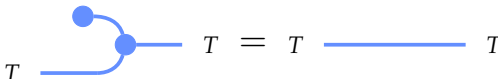
Definition 4.2.2. (🚗) A comonad is a comonoid in END_{Set} .

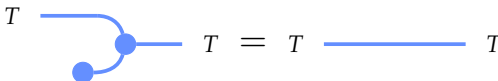
Explicitly, a monad T is a functor equipped with a natural transformation $\mathbb{1} \Rightarrow T$ from the identity functor to T , and another, called *join*, of type $T \circ T \Rightarrow T$. These have the following depictions. In anticipation of increasingly complex diagrams, we adopt a convention of drawing monadic wires in blue.

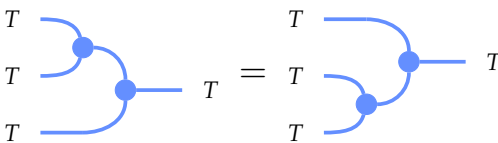
Definition 4.2.3. (🚗) A monad is a functor $T: \text{SET} \rightarrow \text{SET}$ that is equipped with two natural transformations of the following types:

	$\text{ret}: \forall (A: \text{SET}), A \rightarrow TA$
	$\text{join}: \forall (A: \text{SET}), T(TA) \rightarrow TA$

These are subject to the following equations:

	$\text{join}^T \circ \text{ret}^T = \text{id}_{TA} \quad (4.28)$
---	--

	$\text{join}^T \circ \text{map}^T(\text{ret}^T) = \text{id}_{TA} \quad (4.29)$
---	--

	$\text{join}^T \circ \text{join}^T = \text{join}^T \circ \text{map}^T(\text{join}^T) \quad (4.30)$
---	--

Remark 4.2.4. We use T to denote an arbitrary monad and F for an arbitrary functor, following a general convention in which variable names reflect the structural assumptions we are making about each object.

For functors that represent concrete data structures, a monad may be given the following interpretation: *ret* lifts lone elements of a set A into *singletons*, while *join* flattens a nested T -structure whose contents are *also* T -structures of type TA a single T -structure whose contents have type A .

Example 4.2.5. () *list* is a monad where ret^{list} maps any $a \in A$ into the singleton list $[a]$ and $\text{join}^{\text{list}}$ is the concatenation operation, flattening a list-of-lists such as $[[x, y], [z], [w, v]]$ into a list $[x, y, z, w, v]$.

$$\begin{aligned} \text{ret}^{\text{list}} &: A \rightarrow \text{list } A & \text{ret}^{\text{list}} a &\equiv [a] \\ \text{join}^{\text{list}} &: \text{list } (\text{list } A) \rightarrow \text{list } A & \text{join}^{\text{list}} l &\equiv \text{concat } l \end{aligned}$$

Example 4.2.6. () *subset* is a monad where ret^{set} maps any $a \in A$ into the predicate that tests for equality with a . join^{set} , which represents the union of a set of sets, accepts a higher-order predicate P on predicates of type A and returns a predicate on type A . This predicate is true of $a \in A$ if there exists a predicate Q on type A such that $Q a$ is true and $P Q$ is true.

$$\begin{aligned} \text{ret}^{\text{set}} &: A \rightarrow A \rightarrow \mathbf{Prop} & (\text{ret}^{\text{set}} a) a' &\equiv a = a' \\ \text{join}^{\text{set}} &: ((A \rightarrow \mathbf{Prop}) \rightarrow \mathbf{Prop}) \rightarrow A \rightarrow \mathbf{Prop} & (\text{join}^{\text{set}} P) a &\equiv \exists (Q: A \rightarrow \mathbf{Prop}). P Q \wedge Q a \end{aligned}$$

Example 4.2.7. () *option* A , isomorphic to $\mathbf{1} + A$, has the following constructors:

$$\begin{aligned} \text{None} &: \text{option } A \\ \text{Some} &: A \rightarrow \text{option } A \end{aligned}$$

This forms a monad where ret^{opt} (note the shorthand *opt*) is *Some* and join^{opt} flattens an “optional optional A ” into an optional A :

$$\begin{aligned} \text{join}^{\text{opt}} \text{None} &\equiv \text{None} \\ \text{join}^{\text{opt}} (\text{Some None}) &\equiv \text{None} \\ \text{join}^{\text{opt}} (\text{Some (Some } a)) &\equiv \text{Some } a \end{aligned}$$

Example 4.2.8. The environment functor $\vec{E}$ (Ex. 4.1.43) forms a monad where $\text{ret}^{\vec{E}}$ maps $a \in A$ to the constant function on a and $\text{join}^{\vec{E}}$ passes two copies of the environment:

$$\begin{aligned} \text{ret}^{\vec{E}} &: A \rightarrow E \rightarrow A & \text{ret } a &\equiv e \mapsto a \\ \text{join}^{\vec{E}} &: (E \rightarrow E \rightarrow A) \rightarrow E \rightarrow A & \text{join } f &\equiv e \mapsto f e e \end{aligned}$$

A monad homomorphism is a generalization of Definition 4.1.12.

Definition 4.2.9. () A monad homomorphism $\phi: T \Rightarrow U$ is a natural transformation between monads satisfying analogues of Equations (4.2) and (4.3):

$$\phi \circ \text{ret}^T = \text{ret}^U \tag{4.31}$$

$$\phi \circ \text{join}^T = \text{join}^U \circ \phi \circ \text{map}^T \phi \tag{4.32}$$

Example 4.2.10. (🔗) The function $\text{element_of}^{\text{list}}$, defined below, is both a monad homomorphism and monoid homomorphism from list to subset .

$$\begin{aligned}\text{element_of}^{\text{list}} &: \forall (A : \text{SET}), \text{list } A \rightarrow \text{subset } A \\ \text{element_of}^{\text{list}} [] &\equiv \text{const False} \\ \text{element_of}^{\text{list}} (\text{cons } a l) &\equiv (\text{ret}^{\text{set}} a) \vee \text{element_of}^{\text{list}} a l\end{aligned}$$

4.2.2 Kleisli Monads

The embedding of SET into END_{Set} allows for the following equivalent presentation of monads that is of more direct relevance to functional programming applications.

Definition 4.2.11. (🔗) A monad presented as a Kleisli triple is set-forming operation $T : \text{SET} \rightarrow \text{SET}$ equipped with two polymorphic operations

$$\begin{aligned}\text{ret} &: \forall (A : \text{SET}), A \rightarrow TA \\ \text{bind} &: \forall (A, B : \text{SET}), (A \rightarrow TB) \rightarrow TA \rightarrow TA\end{aligned}$$

subject to the following three laws:

$$\text{bind}^T \text{ret}^T = \text{id}_T \quad (4.33)$$

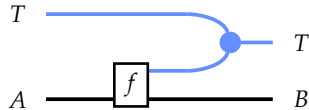
$$\text{bind}^T f \circ \text{ret}^T = f \quad (4.34)$$

$$\text{bind}^T g \circ \text{bind}^T f = \text{bind}^T (\text{bind}^T g \circ f) \quad (4.35)$$

Remark 4.2.12. A monad presented as a Kleisli triple is also known as an extension system. Abstractions presented in later sections in the “Kleisli” style will be referred to as generalized extension systems.

Definitions 4.2.1 and 4.2.11 are equivalent in a strict sense. One direction of this proof derives the Kleisli structure from the more traditionally category-theoretic structure, and is nicely illustrated diagrammatically.

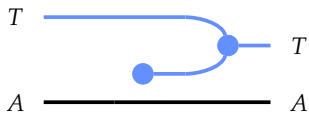
Lemma 4.2.13. (🔗) Every monad gives rise to a Kleisli-presented monad. The operation ret is shared by both presentations, and bind is defined as follows:



$$\text{bind}^T f \equiv \text{join}^T \circ \text{map}^T f \quad (4.36)$$

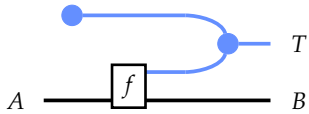
Proof.

Proof of (4.33): $\text{bind}^\top \text{ret}^\top = \text{id}_{\text{TA}}$



$$= \begin{array}{c} T \text{ --- } T \\ A \text{ --- } A \end{array} \quad Eqn. (4.29)$$

Proof of (4.34): $\text{bind}^\top f \circ \text{ret}^\top = f$

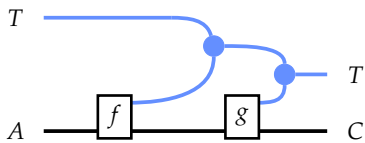


$=$

 Naturality of ret

$$= \begin{array}{c} \text{---} T \\ \boxed{f} \\ \text{---} A \quad \text{---} B \end{array} \quad \text{Eqn. (4.28)}$$

Proof of (4.35): $\text{bind}^\top g \circ \text{bind}^\top f = \text{bind}^\top (\text{bind}^\top g \circ f)$



Naturality of join

Diagrammatic equation (4.30) showing a blue line with two dots connected to a black line with two boxes labeled f and g .

The other direction of this equivalence derives join from bind and ret.

Lemma 4.2.14. () A Kleisli-presented monad $\langle T, \text{ret}^\top, \text{bind}^\top \rangle$ forms a monad where the map and join

operations are defined as follows:

$$\text{map}^T f \equiv \text{bind}^T (\text{ret}^T \circ f) \quad (4.37)$$

$$\text{join}^T f \equiv \text{bind}^T (\text{id}_{TA}) \quad (4.38)$$

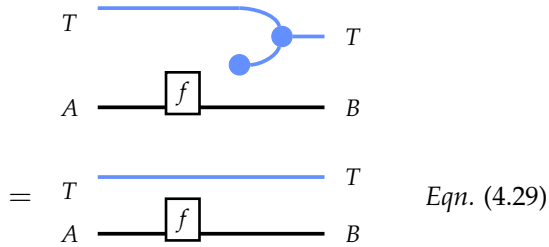
This direction of the equivalence must forgo the elegance of the diagrammatic reasoning and resort to ordinary equational reasoning. The diagrammatic notation is only sound if T is a functor, but in Definition 4.2.11, T is just a set-forming operation and map^T is a derived operation. The notation also assumes the basic operations are natural transformations, which bind is not. We must explicitly rewrite expressions using Equations (4.33)–(4.35).

Lemmas 4.2.13 and 4.2.14 establish that an instance of either of Definitions 4.2.1 and 4.2.11 is associated with an instance of the other. However, this does not establish equivalence. We must also show that one presentation uniquely determines the other.

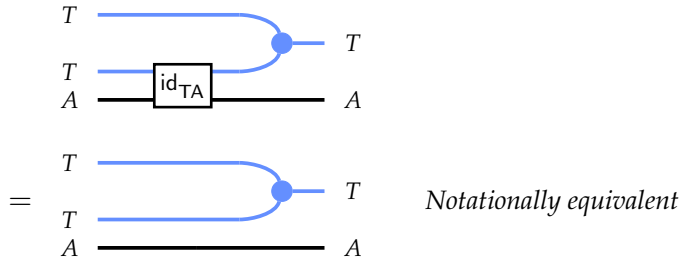
Lemma 4.2.15. (🐼) *Definitions 4.2.1 and 4.2.11 are equivalent.* □

Proof. First, we consider a monad presented in terms of map^T , and join^T (and ret^T , which is shared by both presentations). Then we define bind using (4.36), and then reconstruct map and join using (4.37) and (4.38). We must prove these reconstructed operations are the same as the original ones, which amounts to proving (4.37) and (4.38) (originally stated as definitions, now goals):

Proof of (4.37): $\text{map}^T f = \text{bind}^T (\text{ret} \circ f)$



Proof of (4.38): $\text{join}^T f = \text{bind}^T \text{id}_{TA}$



The other direction of the equivalence asserts that if bind is defined from the join and map , where join and map are defined from some instance of bind , then the two bind operations are

equal. For the reasons explained above, this direction of the proof proceeds by ordinary equational reasoning. $\square$

4.2.3 Free Monads

We now examine the relationship between free monads and syntax trees, previously presented in Section 1.2. The metatheory exported by Tealeaves does not require a decorated-traversable monad to be freely generated. However, decomposing syntax in terms of its free monad reveals an important structural property: the decorated-traversable nature of syntax monads arises naturally from the fact that their signature functors are themselves decorated-traversable. This is proved in parts across Theorems 4.3.44, 5.3.41 and 5.4.18. To support those proofs, we state a handful of standard lemmas regarding free monads without proof.

Recall that if Σ is the endofunctor encoding the signature of a formal language, a monad T can be defined by the following least fixed-point expression:

$$TV \equiv \mu X. (V + \Sigma X) \quad (4.39)$$

That is, the set TV consists of variables in V and all expressions well-formed by applying any of the constructors to expressions recursively drawn from TV . This forms a fixed point in the sense of the following natural isomorphism:

$$TV \simeq V + \Sigma (TV) \quad (4.40)$$

This characterization of T is associated with two functions into TV : one of type $V \rightarrow TV$, the other of type $\Sigma (TV) \rightarrow TV$. Both of these operations are in fact natural in the V argument:

$$\eta: \mathbb{1} \Rightarrow T \quad \mu^*: \Sigma \circ T \Rightarrow T$$

The former operation is precisely the unit of the monad, ret^T . The other operation encodes the constructors of the syntax, mapping something of type $\Sigma (TV)$, representing a function symbol applied to an appropriate number of term arguments, to something of type TV , representing a term.

Example 4.2.16. *The signature of the lambda calculus without binder labels is as follows:*

$$\Sigma^\lambda X := X + X \times X$$

The set of lambda terms is given the free monad $\text{lam } V \equiv \mu X. (V + \Sigma^\lambda X)$

Example 4.2.17. The signature of the lambda calculus with binder labels drawn from a set B is as follows:

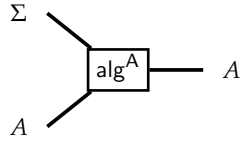
$$\Sigma^{\text{nlam}} B X := B \times X + X \times X$$

The set of lambda terms with binder labels is given by the free monad $\text{nlam } B V \equiv \mu X. (V + \Sigma^{\text{nlam}} B X)$

Remark 4.2.18. The precise claim in Example 4.2.17 is that $\text{nlam } B$ forms a free monad for a fixed choice of B . For short, we say nlam forms a monad in its second argument.

Functor Algebras

Definition 4.2.19. Given any functor $\Sigma: \text{SET} \rightarrow \text{SET}$, a Σ -algebra is set A called the carrier paired with a structure map, a function of type $\Sigma A \rightarrow A$.



$$\text{alg}^A: \Sigma A \rightarrow A$$

Remark 4.2.20. When no specific name is given, the structure map on algebra with carrier A will be denoted alg^A . By convention, we may refer to either of A or alg^A as the algebra, depending on the context.

Definition 4.2.21. A homomorphism of Σ -algebras alg^A and alg^B is a function between their carriers that commutes with the algebras:

$$\phi \circ \text{alg}^A = \text{alg}^B \circ \text{map}^\Sigma \phi \quad (4.41)$$

Label: (4.41)

The right-to-left direction of (4.40) equips TV with the structure of a $(V + \Sigma(-))$ -algebra. The fact that T is the *least* fixed point of this functor makes TV the *initial* algebra of this functor, which corresponds to the following property, whose (standard) proof we omit.

Lemma 4.2.22. Any $(V + \Sigma(-))$ -algebra induces a unique homomorphism into itself from TV . $\square$

Lemma 4.2.22 yields a higher-order function polymorphic function on TV :

$$\text{fold}^T: \forall (A: \text{SET}), ((V + \Sigma A) \rightarrow A) \rightarrow TV \rightarrow A$$

In fact, this operation taken as a *specification* of TV up to isomorphism by stating its elimination principle. Namely, to give a function from TV to A , it suffices to specify the function in two cases:

the base case that the term is atomic (i.e. just a variable); and the recursive case that TV is a constructor applied to a set of immediate subterms. These cases correspond to two functions:

$$\begin{aligned} \text{base} &: V \rightarrow A \\ \text{rec} &: \Sigma A \rightarrow A \end{aligned}$$

These functions are applied to T by $\text{fold}^T [\text{base}, \text{rec}]$ where $[\text{base}, \text{rec}]$ combines the two operations into a single operation with domain $V + \Sigma A$.

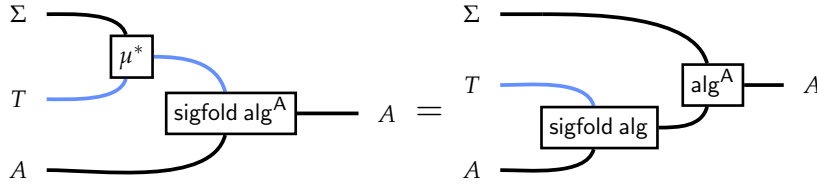
By specializing the base case to the identity function, we obtain a slightly less general but highly useful function:

$$\text{sigfold}^T : \forall (A : \text{SET}), (\Sigma A \rightarrow A) \rightarrow TA \rightarrow A$$

In this work, this operation is named `sigfold` because it collapses the structure of the signature functor Σ . The relationship between `fold` and `sigfold` is as follows:

$$\text{fold}^T ([f, g]) = \text{sigfold}^T g \circ \text{map}^T f \quad (4.42)$$

A crucial equality that will be used in several proofs is the following, which states that $\text{sigfold } \text{alg}^A$ for any Σ -algebra alg^A is a homomorphism from the Σ -algebra on TA given by μ_A^* to alg^A . This property is ultimately used to establish that a free generated monad T inherits the structure its decorated traversable structure from that of its signature endofunctor (Theorem 5.4.18).



$$\text{sigfold } \text{alg}^A \circ \mu_A^* = \text{alg}^A \circ \text{map}^\Sigma (\text{sigfold } \text{alg}^A) \quad (4.43)$$

The statement that T inherits the structure of a monad is given by the following standard lemma.

Lemma 4.2.23. *Where TV is defined by the fixpoint equation (4.39), T is a monad with the following*

operations:

$$\text{ret}^T \equiv \eta \quad (4.44)$$

$$\text{map}^T f \equiv \text{fold}^T [\text{ret}^T \circ f, \mu^*] \quad (4.45)$$

$$\text{join}^T \equiv \text{sigfold}^T \mu^* \quad (4.46)$$

$$\text{bind}^T f \equiv \text{fold}^T [f, \mu^*] \quad (4.47)$$

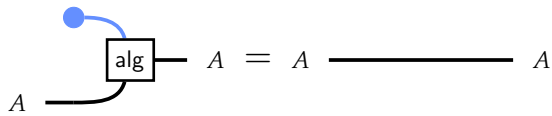
□

Each of the monad laws (4.28)–(4.30) can be derived from the properties of fold for the above definitions.

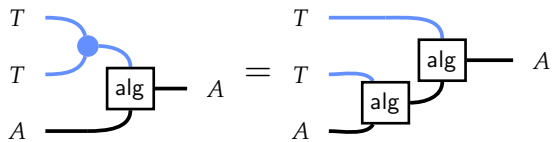
Monad Algebras

Many of the structures examined in this work are of algebras of *monad*—a notion that carries more structure than that of algebras for a functor. A monad algebra is required to interact with the monad operations of T so as to a left action of T on the object A . Compare the following to Def. 4.1.19.

Definition 4.2.24. *An algebra of a monad T , or a T -algebra, is an operation $\text{alg}^A: TA \rightarrow A$ subject to the following equations:*



$\text{alg} \circ \text{ret}^T = \text{id}_A \quad (4.48)$



$\text{alg} \circ \text{join}^T = \text{alg} \circ \text{map}^T \text{alg} \quad (4.49)$

Definition 4.2.25. *A homomorphism between two T -algebras is the same as homomorphism in the sense of Definition 4.2.21.*

The following lemma clarifies the relationship between functor algebras and monad algebras.

Lemma 4.2.26. *Let T be the monad generated by Σ . Then Σ -algebra induces a T -algebra, and a homomorphism between Σ algebras yields a homomorphism between the T -algebras.* □

The operation that lifts a Σ -algebra with carrier set A to a T -algebra is precisely sigfold^T . The

fact that sigfold alg^A forms a monad algebra yields the following equations:

$$\text{sigfold alg}^A \circ \text{ret}^T = \text{id}_A \quad (4.50)$$

$$\text{sigfold alg}^A \circ \text{join}^T = \text{sigfold alg}^A \circ \text{map}^T (\text{sigfold alg}^A) \quad (4.51)$$

The fact that a Σ -algebra homomorphism yields a T -algebra homomorphism means the following:

Lemma 4.2.27. *For all Σ -algebras $\langle A, \text{alg}^A \rangle$ and $\langle B, \text{alg}^B \rangle$ and all $f: A \rightarrow B$, the following holds:*

$$f \circ \text{alg}^A = \text{alg}^B \circ \text{map}^\Sigma f \implies f \circ \text{sigfold alg}^A = \text{sigfold alg}^B \circ \text{map}^T f \quad (4.52)$$

□

The monad T defined from a signature functor Σ is the *free monad* generated by Σ . This corresponds to the following property.

Definition 4.2.28. *The following inclusion morphism is a natural transformation $\iota: \Sigma \Rightarrow T$ embedding Σ into T :*

$$\iota_A \equiv \mu^* \circ \text{map}^\Sigma \text{ret}^T \quad (4.53)$$

The inclusion morphism can loosely be seen as a sense in which T is “greater than” or “contains” Σ . The next lemma provides a sense in which T is the “minimal” monad having this property.

Lemma 4.2.29. *Let U be any monad. Given any natural transformation $\phi: \Sigma \Rightarrow U$, there is a unique monad homomorphism $\psi: T \Rightarrow U$ satisfying the following property:*

$$\psi \circ \iota = \phi$$

This homomorphism is defined by defining a Σ -algebra structure on U :

$$(\text{join}^U \circ \phi_{UA}) : \Sigma(UA) \rightarrow UA$$

This algebra is then lifted over T after mapping A , the base case, to UA with ret^U

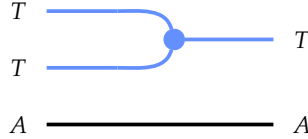
$$\text{sigfold}^T (\text{join}^U \circ \phi_{UA}) \circ \text{map}^T \text{ret}^U : TA \rightarrow UA$$

□

4.2.4 Free Algebras

A special class of T -algebras, the free algebras, arise from the fact that, just as a monoid acts on itself (c.f. Section 4.1.6), a monad has a left and right action on itself. This makes any set TA a T -algebra, with (4.48) and (4.49) reducing to two of the monad equations.

Definition 4.2.30. Let T be any monad and A any set. Then $\langle TA, \text{join}_A^T \rangle$ is a T -algebra, namely the free T -algebra generated by A . The algebra is defined by the following diagram:



$$\text{alg}^{TA} \equiv \text{join}_A^T \quad (4.54)$$

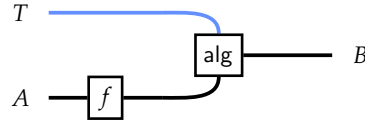
The following construction unpacks the sense in which these algebras are free.

Lemma 4.2.31. Let $\langle B, \alpha: TB \rightarrow B \rangle$ be any T -algebra and A be any set. Then for any function $A \xrightarrow{f} B$, there is a unique T -algebra homomorphism $\phi: TA \rightarrow B$ such that the following equality holds for all $a: A$.

$$\phi(\text{ret}^T a) = f a \quad (4.55)$$

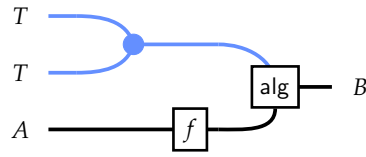
Moreover, all T -algebra homomorphisms with domain TA correspond to functions $A \rightarrow B$ in this manner.

Proof. Given f , ϕ is defined as follows:



$$\phi \equiv \text{alg}^B \circ \text{map}^T f$$

That this operation satisfies (4.55) is immediate using (4.48) and the fact that ret^T is a natural transformation. That this operation is a monad algebra homomorphism is the statement of the following equality, which holds by (4.49) and the fact that join^T is natural:



$$\text{alg}^B \circ \text{map} f \circ \text{join}^T = \text{alg}^B \circ \text{map}^T (\text{alg}^B \circ \text{map}^T f)$$

To establish uniqueness, assume $\psi: TA \rightarrow B$ is any T -algebra homomorphism from the free

T -algebra over A to B such that $\psi(\text{ret}^\top a) = fa$ for all $a: A$. Then $\phi = \psi$:

$$\begin{array}{llll}
\begin{array}{c} T \\ \hline \text{---} \end{array} & \begin{array}{c} \text{---} \end{array} & \begin{array}{c} B \\ \text{---} \end{array} & \text{alg}^B \circ \text{map}^\top f & \text{Test} \\
\begin{array}{c} A \\ \text{---} \end{array} & \begin{array}{c} \boxed{f} \end{array} & \begin{array}{c} B \\ \text{---} \end{array} & & \\
= & & & & \\
\begin{array}{c} T \\ \hline \text{---} \end{array} & \begin{array}{c} \text{---} \end{array} & \begin{array}{c} B \\ \text{---} \end{array} & \text{alg}^B \circ \text{map}^\top (\psi \circ \text{ret}^\top) & \text{Assumption of } \psi \\
\begin{array}{c} A \\ \text{---} \end{array} & \begin{array}{c} \boxed{\psi} \end{array} & \begin{array}{c} B \\ \text{---} \end{array} & & \\
= & & & & \\
\begin{array}{c} T \\ \hline \text{---} \end{array} & \begin{array}{c} \text{---} \end{array} & \begin{array}{c} B \\ \text{---} \end{array} & \psi \circ \text{join}^\top \circ \text{map}^\top \text{ret}^\top & \text{Eqn. (4.41)} \\
\begin{array}{c} A \\ \text{---} \end{array} & \begin{array}{c} \boxed{\psi} \end{array} & \begin{array}{c} B \\ \text{---} \end{array} & & \\
= & & & & \\
\begin{array}{c} T \\ \hline \text{---} \end{array} & \begin{array}{c} \text{---} \end{array} & \begin{array}{c} B \\ \text{---} \end{array} & \psi & \text{Eqn. (4.29)} \\
\begin{array}{c} A \\ \text{---} \end{array} & \begin{array}{c} \boxed{\psi} \end{array} & \begin{array}{c} B \\ \text{---} \end{array} & &
\end{array}$$

To establish that all homomorphisms with domain TA arise from functions $f: A \rightarrow B$ such that $\phi(\text{ret}^\top a) = fa$. Suppose $\phi: TA \rightarrow B$ is a homomorphism where B is equipped with the structure of a T -algebra alg^B . Now $\phi \circ \text{ret}^\top$ is a function from A to B . The above construction yields a unique homomorphism $\psi: TA \rightarrow B$ such that $\psi \circ \text{ret}^\top = \phi \circ \text{ret}^\top$. But ϕ has this property, so by uniqueness ψ is exactly ϕ . $\square$

Equation (4.55) expresses a decomposition of an arbitrary function f whose codomain is a T -algebra into two components, ϕ and ret :

$$A \xrightarrow{f} B = A \xrightarrow{\text{ret}^\top} TA \xrightarrow{\phi} B$$

The intuition of this property is that TA is the *minimal* T -algebra that is in some sense *above* A : all roads from A to a T -algebra B must pass through TA .

As an application of the above construction, let $A, B \in \text{SET}$ and let $A \xrightarrow{f} TB$ be any function from A to the free T -algebra over B . The above construction yields a homomorphism $TA \rightarrow TB$. This morphism is precisely $\text{bind}^\top f$, and all T -algebra homomorphisms from TA to TB are of the form $\text{bind}^\top f$.

4.2.5 Naïve Substitution

We now we consider how the monadic structure corresponds to a notion of substitution in the syntax. Let V be a set of variables. We obtain the following operation by specializing both

parameters of bind to V :

$$\text{bind}_{V,V}^T: (V \rightarrow T V) \rightarrow T V \rightarrow T V$$

An operation of type $\sigma: V \rightarrow \text{lam } V$ maps variables to terms, while $\text{bind } \sigma: \text{lam } V \rightarrow \text{lam } V$ applies σ to each of the variables in its argument. Recall the definition of single-variable substitution from Section 1.2:

$$\begin{aligned} \text{subst } x \ u &\equiv \text{bind}^T (\text{subst}_{\text{loc}} x \ u) \\ \text{subst}_{\text{loc}} x \ u \ v &\equiv \begin{cases} u & \text{if } v = x \\ \text{ret } v & \text{else} \end{cases} \end{aligned}$$

The monad laws correspond to basic properties of this operation.

Lemma 4.2.32. *The following properties of naïve substitution hold for all monads T and variables $x, y \in V$.*

$$t[x \mapsto x] = t \quad (4.56)$$

$$x[x \mapsto u] = u \quad (4.57)$$

$$y[x \mapsto u] = y \quad (\text{where } x \neq y) \quad (4.58)$$

$$(t[x \mapsto u_1])[y \mapsto u_2] = t[x \mapsto u_1[y \mapsto u_2]; y \mapsto u_2] \quad (\text{where } x \neq y) \quad (4.59)$$

In the last equation, $(x \mapsto \text{subst } y \ u_2 \ u_1; y \mapsto u_1)$ is the substitution that maps x to $u_1[y \mapsto u_2]$, y to u_2 , and ignores all other variables.

Proof.

Proof of (4.56):

$$\begin{aligned} &\text{subst } x \ (\text{ret } x) \ t \\ = &\text{bind } (\text{subst}_{\text{loc}} x \ (\text{ret } x)) \ t \\ = &\text{bind } \text{ret } t \\ = &t \end{aligned} \quad \text{Eqn. (4.33)}$$

Proof of (4.57) and (4.58):

$$\begin{aligned} &\text{subst } x \ u \ (\text{ret } y) \\ = &\text{bind } (\text{subst}_{\text{loc}} x \ u) \ (\text{ret } y) \\ = &\text{subst}_{\text{loc}} x \ u \ y \end{aligned} \quad \text{Eqn. (4.34)}$$

Now (4.57) and (4.58) follow based on whether $y = x$.

Proof of (4.59): We simplify the LHS as follows:

$$\begin{aligned}
& \text{subst } y \ u_2 \ (\text{subst } x \ u_1 \ t) \\
= & \text{bind } (\text{subst}_{\text{loc}} y \ u_2) \ (\text{bind } (\text{subst}_{\text{loc}} x \ u_1) \ t) \\
= & \text{bind } (\text{subst}_{\text{loc}} y \ u_2 \star \text{subst}_{\text{loc}} x \ u_1) \ t
\end{aligned}
\tag{Eqn. (4.35)}$$

Now consider the behavior of $(\text{subst}_{\text{loc}} y \ u_2 \star \text{subst}_{\text{loc}} x \ u_1)$ when applied to an arbitrary variable $v \in V$. Note that

$$\begin{aligned}
& (\text{subst}_{\text{loc}} y \ u_2 \star \text{subst}_{\text{loc}} x \ u_1) \ v \\
= & \text{bind } (\text{subst}_{\text{loc}} y \ u_2) \ (\text{subst}_{\text{loc}} x \ u_1 \ v) \\
= & \text{subst } y \ u_2 \ (\text{subst}_{\text{loc}} x \ u_1 \ v)
\end{aligned}
\tag{Eqn. (4.34)}$$

The equation holds by checking three cases to observe that this operation has the desired behavior:

$$\begin{aligned}
\text{subst } y \ u_2 \ (\text{subst}_{\text{loc}} x \ u_1 \ v) &= \text{subst } y \ u_2 \ u_1 && (\text{if } v = x) \\
\text{subst } y \ u_2 \ (\text{subst}_{\text{loc}} x \ u_1 \ v) &= u_2 && (\text{if } v = y) \\
\text{subst } y \ u_2 \ (\text{subst}_{\text{loc}} x \ u_1 \ v) &= \text{ret } v && (\text{if } v \neq x, v \neq y)
\end{aligned}$$

□

This is not a very expressive abstraction by itself—the only operation in Chapter 2 that can be defined in terms of bind^T is the locally nameless subst . Even for the naïve substitution defined above, the abstraction is too limited for practical use. For instance, suppose we want to define $x \in t$ to mean that x occurs in t . Then we should like to show that if x *does not* occur freely in t , any substitution replacing x has no effect on t :

$$t[x \mapsto u] = t \quad \text{if } x \notin t$$

However, $x \in t$ cannot be expressed in terms of bind , as bind can only define functions from terms to terms, whereas the occurrence relation $x \in (-)$ is a predicate.

4.2.6 Right Actions of Monads

Before introducing decorated functors, we consider a slight generalization of the monadic approach to syntax. Instead of working directly with a the monad T , the theory—including the extension to DTMs—can be generalized to an arbitrary functor U that equipped with a *right action* of T , a generalization of Definition 4.1.19.

Definition 4.2.33. (🚗) A right T -module is a right action of T in END_{Set} . Explicitly, it is a functor U equipped with a natural transformation $U \circ T \Rightarrow U$ satisfying the following equations:

$$\begin{array}{ccc}
 \begin{array}{c} U \\ \text{---} \end{array} \begin{array}{c} \text{---} \\ \text{---} \end{array} \begin{array}{c} \bullet \\ \bullet \end{array} \begin{array}{c} U \\ \text{---} \end{array} & \text{alg}^U: \forall (A: \text{SET}), U(TA) \rightarrow UA & \\
 \begin{array}{c} U \\ \text{---} \end{array} \begin{array}{c} \text{---} \\ \text{---} \end{array} \begin{array}{c} \bullet \\ \bullet \end{array} \begin{array}{c} U \\ \text{---} \end{array} & \text{alg}^U \circ \text{map}^U \text{ret}^T = \text{id}_U & (4.60) \\
 \begin{array}{c} U \\ \text{---} \end{array} \begin{array}{c} \text{---} \\ \text{---} \end{array} \begin{array}{c} \bullet \\ \bullet \end{array} \begin{array}{c} U \\ \text{---} \end{array} & \text{alg}^U \circ \text{alg}^U = \text{alg}^U \circ \text{map}^U \text{join}^T & (4.61) \\
 \begin{array}{c} U \\ \text{---} \end{array} \begin{array}{c} \text{---} \\ \text{---} \end{array} \begin{array}{c} \bullet \\ \bullet \end{array} \begin{array}{c} U \\ \text{---} \end{array} & &
 \end{array}$$

These structures yield an analogue of Definition 4.2.11 by precomposing alg^U with map^U .

Definition 4.2.34. (🚗) A Kleisli-presented right monad action is a monad T paired with a set-forming operation $U: \text{SET} \rightarrow \text{SET}$ equipped with a polymorphic operation

$$\text{bind}^{U,T} : \forall (A B: \text{SET}), (A \rightarrow TB) \rightarrow UA \rightarrow UB$$

subject to the following two laws:

$$\text{bind}^U \text{ret}^T = \text{id}_U \quad (4.62)$$

$$\text{bind}^U g \circ \text{bind}^U f = \text{bind}^U (\text{bind}^T g \circ f) \quad (4.63)$$

Example 4.2.35. As an example, consider a simple first-order logic with the following syntax of terms and formulas:

$$\begin{aligned}
 t &::= \text{var } v \mid \text{func } f(t, \dots, t) \mid \text{const } c \\
 \varphi &::= P(t, \dots, t) \mid \text{not } \varphi \mid \text{and } \varphi \varphi \mid \text{forall } v. \varphi \mid \text{exists } v. \varphi
 \end{aligned}$$

Let $\text{term } V$ denote the set of first-order terms and $\text{formula } V$ denote formulas. Term variables occur in formulas, and substitution replaces these with terms, not formulas. This corresponds to a right module structure of the term monad on the formula functor:

$$\text{bind}^{\text{formula}, \text{term}}: (V \rightarrow \text{term } V) \rightarrow \text{formula } V \rightarrow \text{formula } V$$

4.3 THE CATEGORY OF DECORATED FUNCTORS

Decorated functors generalize comonoid co-actions (Def. 4.1.30). Rather than acting on a set, such as list A , the coaction acts on *functors*, such as list. When functors represent concrete data structures, the coaction annotates the elements with information particular to their position within the structure. To develop this perspective, we now consider a different embedding of $\mathbf{SET}$ into $\mathbf{END}_{\mathbf{Set}}$ than the constant functor embedding considered in Section 4.1.10.

4.3.1 Embedding $\mathbf{SET}$ into $\mathbf{END}_{\mathbf{Set}}$ via **prod**

Definition 4.3.1. The functor **prod**: $\mathbf{SET} \rightarrow \mathbf{END}_{\mathbf{Set}}$ is defined as follows:

- The set A is mapped to the Cartesian-product-with- A functor, denoted $A^\times$ (Ex. 4.1.42)
- The function $f: A \rightarrow B$ is mapped to the natural transformation $f^\times: A^\times \Rightarrow B^\times$ that maps f over the first component of the Cartesian product:

$$\mathbf{prod} f(a, c) \equiv (fa, c) \quad (4.64)$$

Lemma 4.3.2. All natural transformations of type $A^\times \Rightarrow B^\times$ are of the form **prod** f for some function $f: A \rightarrow B$.

Corollary 4.3.3. **prod**: $\mathbf{SET} \rightarrow \mathbf{END}_{\mathbf{Set}}$ is a full faithful embedding of $\mathbf{SET}$ into $\mathbf{END}_{\mathbf{Set}}$.

This embedding allows us to represent objects like $F(A \times B)$ with diagrams like the following:

$$\begin{array}{ccc} F & \text{-----} & F \\ A^\times & \text{-----} & A^\times \\ B & \text{-----} & B \end{array}$$

Tensorial Strength

Unlike $\mathbf{SET}$, where $A \times B$ is isomorphic to $B \times A$, wires in $\mathbf{END}_{\mathbf{Set}}$ cannot generally be crossed because there is not always a natural transformation from $F \circ G$ to $G \circ F$, much less are they isomorphic—a list of trees is not equivalent to a tree of lists, for instance. However, the wires that arise from the **prod** embedding can be crossed left-to-right over any wire.

Definition 4.3.4. (🚗) Let $F: \mathbf{SET} \rightarrow \mathbf{SET}$ be any functor. The (tensorial) strength is a natural transformation called of type $\mathbf{st}_{A,B}^F: A \times FB \rightarrow F(A \times B)$ defined as follows:

$$\begin{array}{ccc} A^\times & \text{-----} & F \\ F & \text{-----} & A^\times \end{array} \quad \mathbf{st}_{A,B}^F(a, t) \equiv \mathbf{map}^F(b \mapsto (a, b)) t \quad (4.65)$$

Example 4.3.5. Let $e : E$ be any value and consider the functor list . The tensorial strength maps (e, l) to a copy of l where each element has been paired with e . For instance,

$$\text{st}_A^F(e, [x; y; z]) = [(e, x); (e, y); (e, z)]$$

Note in the above example, the strength duplicates e so that each element of the list receives its own copy, implicitly relying on the fact that every set forms a comonoid. In general, the strength satisfies the same sorts of desirable properties examined in Section 4.1.3. For instance, natural transformations can slide along either wire. Now we turn our attention to how **prod** preserves monoids and comonoids, yielding a particular class of monads and comonads in END_{Set} .

Preserving Monoidal Structure

Two important classes of functors between monoidal categories are ones that preserve the structure of monoids or comonoids. These are respectively the *lax* and *oplax* monoidal functors. A crucial fact about **prod** is that it preserves both.

Definition 4.3.6. A lax monoidal functor between monoidal categories $\langle \mathcal{C}, \mathbf{I}_{\mathcal{C}}, \otimes_{\mathcal{C}} \rangle$ and $\langle \mathcal{D}, \mathbf{I}_{\mathcal{D}}, \otimes_{\mathcal{D}} \rangle$ is one equipped with the following extra structure:

- A morphism in $\mathcal{D}$ of type $\epsilon : \mathbf{I}_{\mathcal{D}} \rightarrow F \mathbf{I}_{\mathcal{C}}$
- A natural transformation μ where $\mu_{A,B}$ has type $FA \otimes_{\mathcal{D}} FB \rightarrow F(A \otimes_{\mathcal{C}} B)$.

These are subject to unsurprising conditions detailed in [51].

Lax monoidal functors have just the right structure to map monoids in $\mathcal{C}$ to monoids in $\mathcal{D}$.

Lemma 4.3.7. Let M be a monoid in $\mathcal{C}$, which consists of two morphisms:

$$e : \mathbf{1}_{\mathcal{C}} \rightarrow M \quad m : M \otimes_{\mathcal{C}} M \rightarrow M$$

Let F be a lax monoidal functor from $\mathcal{C}$ to $\mathcal{D}$. Then FM is naturally equipped with the structure of a monoid in $\mathcal{D}$ by the following morphisms:

$$\mathbf{1}_{\mathcal{D}} \xrightarrow{\epsilon} F \mathbf{I}_{\mathcal{C}} \xrightarrow{F e} FM \quad FM \otimes_{\mathcal{D}} FM \xrightarrow{\mu} F(M \otimes_{\mathcal{C}} M) \xrightarrow{F m} FM$$

The dual notion is that of an oplax monoidal functor.

Definition 4.3.8. A oplax monoidal functor between monoidal categories $\langle \mathcal{C}, \mathbf{I}_{\mathcal{C}}, \otimes_{\mathcal{C}} \rangle$ and $\langle \mathcal{D}, \mathbf{I}_{\mathcal{D}}, \otimes_{\mathcal{D}} \rangle$ is equipped with the following natural transformations:

- A morphism in $\mathcal{D}$ of type $F \mathbf{I}_{\mathcal{C}} \rightarrow \mathbf{I}_{\mathcal{D}}$

- A natural transformation Δ where $\Delta_{A,B}$ has type $F(A \otimes_{\mathcal{C}} B) \rightarrow FA \otimes_{\mathcal{D}} FB$.

These are subject to conditions dual to those of Definition 4.3.6.

Lemma 4.3.9. *Let E be a comonoid in $\mathcal{C}$. Let F be a oplax monoidal functor from $\mathcal{C}$ to $\mathcal{D}$. Then FE can be equipped with a comonoid structure given by reversing the arrows in Lemma 4.3.7.*

The functor **prod** is both lax monoidal and oplax monoidal. Indeed, it has a stronger property: it is *strongly monoidal*.

Definition 4.3.10. A strongly monoidal functor is equipped with the following natural isomorphisms:

$$F\mathbf{I}_{\mathcal{C}} \simeq \mathbf{I}_{\mathcal{D}} \quad (4.66)$$

$$F(A \otimes_{\mathcal{C}} B) \simeq FA \otimes_{\mathcal{D}} FB \quad (4.67)$$

Such functors are both lax monoidal and oplax monoidal and therefore preserve monoids and comonoids.

Lemma 4.3.11. **prod** is a strong monoidal functor.

Proof. Given two sets A and B , we must verify the following natural isomorphisms:

$$\mathbf{1}^{\times} \simeq \mathbf{1} \quad (4.68)$$

$$(A \times B)^{\times} \simeq A^{\times} \circ B^{\times} \quad (4.69)$$

(4.68) is a natural isomorphism between the identity functor and the functor that forms the Cartesian product of its argument and the singleton set $\mathbf{1}$. (4.69) relates two functors: one forming the Cartesian product with $A \times B$, the other defined by the composition of functors that form the Cartesian product with B and A , respectively. The lemma boils down to verifying the following isomorphisms for all A, B , and C :

$$(\mathbf{1} \times C) \simeq C \quad (4.70)$$

$$(A \times B) \times C \simeq A \times (B \times C) \quad (4.71)$$

□

As a corollary, comonoids in **SET** yield comonads in **END_{Set}**. Let E be any set. Then $E^{\times}$ is a comonad, the *Reader comonad* or *Environment comonad* over E .

Corollary 4.3.12. (🔗) Let E be any set. The following operations constitute a comonad.

$E^{\times}$ 

$\text{extr}^{E^{\times}} : \forall (A : \mathbf{SET}), E \times A \rightarrow A$



$$\text{dup}^{E^\times} : \forall (A : \text{SET}), E \times A \rightarrow E \times E \times A$$

These operations have the following definitions:

$$\text{extr}^{E^\times}(e, a) \equiv a \quad (4.72)$$

$$\text{dup}^{E^\times}(e, a) \equiv (e, e, a) \quad (4.73)$$

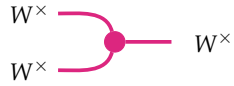
We think of these wires as carrying “contextual” information. They are drawn in red to assist in differentiating from wires carrying other kinds of information.

Just as **prod** maps comonoids to comonads, **prod** maps monoids to monads, the so-called *Writer monads* (often simply called “the” writer monad). In functional programming, the Writer monad is often used to model programs that write to a log [70].

Corollary 4.3.13. (🚗) *Let $\langle W, e, m \rangle$ be a monoid. Then **prod** W naturally forms a monad with the following operations:*



$$\text{ret}^{W^\times} : \forall (A : \text{SET}), A \rightarrow W \times A$$



$$\text{join}^{W^\times} : \forall (A : \text{SET}), W \times W \times A \rightarrow W \times A$$

These operations have the following definitions:

$$\text{ret}^{W^\times} a \equiv (e_W, a) \quad (4.74)$$

$$\text{join}^{W^\times}(w_1, w_2, a) \equiv (w_1 \cdot w_2, a) \quad (4.75)$$

Observe that an instance of the writer monad is also necessarily an instance of the reader comonad by the comonoid structure on W . In SET , this comonoid necessarily makes W into a bimonoid (Sec. 4.1.8). $\prod$ also has the right properties such that the bimonoid equations are also preserved by the embedding. To see this, we observe that crossing wires in SET using swap before applying **prod** is equivalent to crossing wires after applying **prod** using tensorial strength: that is, modulo the natural isomorphism (4.69),

$$(A \times B)^\times \xrightarrow{\text{prod}(\text{swap}_{A,B})} (B \times A)^\times$$

is equivalent to the tensorial strength commuting $A^\times$ with $B^\times$:

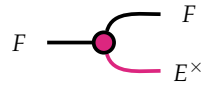
$$A^\times \circ B^\times \xrightarrow{\text{st}_A^{B^\times}} B^\times \circ A^\times.$$

Consequently, the writer monad satisfies the four bimonoid laws (4.17)–(4.20) as an object in END_{Set} , where the wires are crossed in END_{Set} using the strength. This makes $W^\times$ into a *bimonad*.

4.3.2 The Category of Decorated Functors

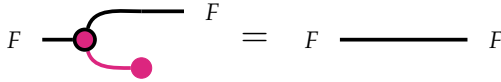
We first examine functors that are decorated by a *set* E , without assuming that this set forms a monoid. These functors form a category, DEC_E , but not a monoidal category. We will shortly incorporate an extra assumption that this set is actually a monoid W , which yields a monoidal category, DEC_W .

Definition 4.3.14. (🚗) *Let E be any set. A functor $F: \text{SET} \rightarrow \text{SET}$ is decorated by E if it is equipped with a right coaction of the environment comonad $E^\times$. Explicitly, a decorated functor is a pair $\langle F, \text{dec}^F \rangle$ where F is a functor and*

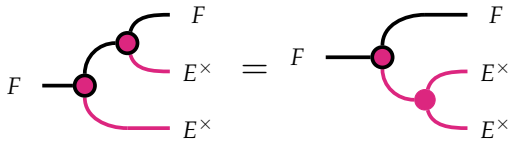


$$\text{dec}^F: \forall (A: \text{SET}), FA \rightarrow F(E \times A)$$

is a natural transformation subject to the following two laws:



$$\text{map}^F \text{extr}^{E^\times} \circ \text{dec}^F = \text{id}_F \quad (4.76)$$



$$\text{dec}^F \circ \text{dec}^F = \text{map}^F \text{dup}^{E^\times} \circ \text{dec}^F \quad (4.77)$$

Whenever we say “decorated functor,” the type of decorations—the set E —is implicit and assumed to be the same for all functors under consideration. As is our convention, we sometimes conflate a decorated functor with its underlying functor F , without specifying dec^F . In many contexts this is unambiguous, since each functor in mind is associated with a unique decoration. However, in some cases, such as Examples 4.3.16 and 4.3.17 below, the same functor has several decorations of interest and the notation dec^F ambiguous. In such cases, explicit disambiguation is required.

If F is a data structure, then dec^F annotates each of the elements of F with contextual information of type E . This annotation is unique to each element and often reflects information about its position in the structure (see Ex. 4.3.16). Equation (4.76) states that if we annotate every element and then immediately delete the annotations, we end up where we started. Thus, annotation

does not change F but only annotates its contents with extra data. Equation (4.77) states that if we annotate each element twice, leaving each item pair with two contexts values e and e' , then e and e' are identical. The fact that dec^F is natural intuitively means that it does not scrutinize the individual data items. Consider the operation dec^F specialized to the type parameter A :

$$\text{dec}_A^F: FA \rightarrow F(E \times A)$$

Because this operation does not examine A , it must be that the information in E is computed from the structure of F , and so the value of E at a particular occurrence of A is determined only by the location of the occurrence rather than its value. For contrast, let F be any functor and let A be equipped with a comonoidal coaction of type $A \rightarrow E \times A$. Mapping this over F *also* yields a function of type

$$FA \xrightarrow{F\alpha} F(E \times A)$$

that satisfies the two coaction laws. This operation annotates each A with information specific to its *value*, as computed by the coaction. The diagrammatic notation nicely illustrates the contrast by depicting the information flowing into the E wire as coming from either A or F .

We now consider some examples. Compare the next example with the constant coaction on a set (Ex. 4.1.31).

Example 4.3.15. *Let $e \in E$ be any element and consider any functor F . The function $\text{map}^F(a \mapsto (e, a))$ is a valid decoration on F , the constant decoration on F by e .*

Most decorations on data structures are more interesting. The following two examples are both coactions of the set of natural numbers on lists, having type

$$\text{dec}: \text{list} \Rightarrow \text{list} \circ \mathbb{N}^\times.$$

Example 4.3.16. *The index annotation on list pairs each element in a list with its index $i \in \mathbb{N}$, indexing left-to-right from 0. For example,*

$$\text{dec}^{\text{ix}}[x, y, z] = [(0, x), (1, y), (2, z)].$$

Example 4.3.17. *The length annotation on list pairs each value in a list of length $n \in \mathbb{N}$ with the value n . For example,*

$$\text{dec}^{\text{len}}[x, y, z] = [(3, x), (3, y), (3, z)].$$

Definition 4.3.18. *Decorated functors are closed under products and coproducts (of functors). Given decorated functors F and G , the coproduct $(F + G)$ mapping A to $FA + GA$ is decorated as follows:*

$$\text{dec}_A^{F+G} \equiv \text{dec}_A^F + \text{dec}_A^G \tag{4.78}$$

The product $(F \times G)$ mapping A to $FA \times GA$ is decorated analogously:

$$\text{dec}_A^{F \times G} \equiv \text{dec}_A^F \times \text{dec}_A^G \quad (4.79)$$

Recall that *binding policy* was the loose term used to refer to information about which constructors bind which variables in which of their arguments. In Tealeaves, such policies are codified by defining a decoration on the signature endofunctor of the language. In the lambda calculus with anonymous binders, the *abs* constructor binds a single variable in its argument, while *app* does not bind variables in its arguments. This corresponds to the following example.

Example 4.3.19. Σ^λ is decorated by $\langle \mathbb{N}, \cdot, 0 \rangle$ as follows (where by mild abuse of notation we give constructors of Σ^λ the same name as corresponding constructors of *lam*):

$$\begin{aligned} \text{dec} : \forall (A : \text{SET}), \Sigma^\lambda A &\rightarrow \Sigma^\lambda (\mathbb{N} \times A) \\ \text{dec} (\text{app } a_1 a_2) &\equiv \text{app } (0, a_1) (0, a_2) \\ \text{dec} (\text{abs } a) &\equiv \text{abs } (1, a) \end{aligned}$$

The preceding example generalizes to the nominal syntax *nlam* as follows.

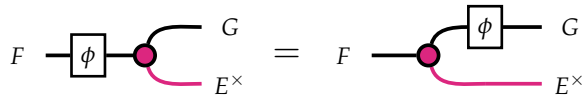
Example 4.3.20. Let B be a type of binder values. $\langle \Sigma^{\text{nlam}} B, \text{dec}^{\text{nlam}} \rangle$ is decorated by $\langle \text{list } B, [] ++ \rangle$ where

$$\begin{aligned} \text{dec}^{\text{nlam}} : \forall (A : \text{SET}), \Sigma^{\text{nlam}} B A &\rightarrow \Sigma^{\text{nlam}} B (\text{list } B \times A) \\ \text{dec} (\text{app } a_1 a_2) &\equiv \text{app } ([], a_1) ([], a_2) \\ \text{dec} (\text{abs } b a) &\equiv \text{abs } b ([b], a) \end{aligned}$$

Section 6.2 revisits the extension of lambda calculus with a *letin* constructor, previously introduced in Section 3.5 along with three distinct semantics for how this constructor opens binders within the list of definitions. In that section, the three semantics correspond to a different ways of decorating the list functor by $\mathbb{N}$: a constant co-action by $0 \in \mathbb{N}$ (Ex. 4.3.15), the index annotation (Ex. 4.3.16), or the length annotation (Ex. 4.3.17). This also means that any discussion of *list* as a decorated functor must specify which decoration of *list* is under consideration.

A homomorphism between decorated functors is a straightforward generalization of a comonoid coaction homomorphism.

Definition 4.3.21. () A homomorphism of decorated functors $\langle F, \text{dec}^F \rangle$ and $\langle G, \text{dec}^G \rangle$ is a natural transformation $\phi : F \Rightarrow G$ that commutes with the decorations of F and G .



$\text{dec}^G \circ \phi = \phi \circ \text{dec}^F \quad (4.80)$

Example 4.3.22. The operation `dropTail` is defined as follows:

$$\begin{aligned}\text{dropTail } [] &= [] \\ \text{dropTail } (\text{cons } x \, l) &= [x]\end{aligned}$$

This operation is a homomorphism of list with respect to the index annotation (Ex. 4.3.16), but it is not a homomorphism of the length annotation (Ex. 4.3.17).

Example 4.3.23. List reversal is a homomorphism with respect to the length annotation but not the index annotation.

Example 4.3.24. List duplication is not a homomorphism with respect to the length or index annotations.

Clearly the identity natural transformation is a homomorphism between any decorated functor and itself. Also, the composition of homomorphisms $\langle F, \text{dec}^F \rangle \Rightarrow \langle G, \text{dec}^G \rangle$ and $\langle G, \text{dec}^G \rangle \Rightarrow \langle H, \text{dec}^H \rangle$ gives a homomorphism $\langle F, \text{dec}^F \rangle \Rightarrow \langle H, \text{dec}^H \rangle$. Thus, decorated functors form a category.

Definition 4.3.25. () Let E be any set. The category DEC_E of functors decorated by E is given by the following data:

- Objects are functors equipped with a decoration by E (Def. 4.3.14).
- Morphisms are homomorphisms between decorated functors (Def. 4.3.21).

4.3.3 Decorated Functors as Extension Systems

Mirroring the two presentations of monads in Section 4.2, we now consider an alternate presentation of decorated functors. Let E be any set.

Definition 4.3.26. () A decorated functor presented as an extension system is a type constructor $F: \text{SET} \rightarrow \text{SET}$ paired with an operation

$$\text{mapd}^F: \forall (A, B: \text{SET}), (E \times A \rightarrow B) \rightarrow FA \rightarrow FB$$

subject to the following equations:

$$\text{mapd } \text{extract}^{E^\times} = \text{id}_{FA} \tag{4.81}$$

$$\text{mapd } g \circ \text{mapd } f = \text{mapd } (g \circ \text{cobind}^{E^\times} f) \tag{4.82}$$

The `mapd` (“map with decoration”) operation lifts a function $E \times A \rightarrow B$, viewed as a context-sensitive function, over the functor F . This allows maps over this data structure to examine not only the value of each item but also its associated context within F .

Lemma 4.3.27. (🚗) Any decorated functor has a presentation as an extension system by the following definition:

$$\begin{array}{c}
 F \\
 \text{---} \bullet \text{---} F \\
 \text{---} \text{---} \\
 A \text{---} \boxed{f} \text{---} B
 \end{array}
 \quad \text{mapd } f \equiv \text{map } f \circ \text{dec} \quad (4.83)$$

Proof.

Proof of (4.81): $\text{mapd } \text{extract}^{E^\times} = \text{id}_F$

$$\begin{array}{c}
 F \\
 \text{---} \bullet \text{---} F \\
 \text{---} \text{---} \\
 A \text{---} \text{---} A
 \end{array}
 = \begin{array}{c}
 F \\
 \text{---} \text{---} F \\
 \text{---} \text{---} A
 \end{array} \quad \text{Eqn. (4.76)}$$

Proof of (4.82): $\text{mapd } g \circ \text{mapd } f = \text{mapd } (g \circ \text{cobind}^{E^\times} f)$

$$\begin{array}{c}
 T \\
 \text{---} \bullet \text{---} T \\
 \text{---} \text{---} \\
 A \text{---} \boxed{f} \text{---} \boxed{g} \text{---} C
 \end{array}
 = \begin{array}{c}
 F \\
 \text{---} \bullet \text{---} F \\
 \text{---} \text{---} \\
 F \text{---} \bullet \text{---} F \\
 \text{---} \text{---} \\
 A \text{---} \boxed{f} \text{---} \boxed{g} \text{---} C
 \end{array} \quad \text{Naturality of dec}$$

$$= \begin{array}{c}
 F \\
 \text{---} \bullet \text{---} F \\
 \text{---} \text{---} \\
 F \text{---} \bullet \text{---} F \\
 \text{---} \text{---} \\
 A \text{---} \boxed{f} \text{---} \boxed{g} \text{---} C
 \end{array} \quad \text{Eqn. (4.77)}$$

□

As with monads and Kleisli triples, the two presentations of decorated functors are equivalent.

Lemma 4.3.28. (🚗) An instance of Definition 4.3.26 yields an instance of Definition 4.3.14 with the following operations:

$$\text{dec} \equiv \text{mapd } \text{id}_{E^\times} \quad (4.84)$$

$$\text{map } f \equiv \text{mapd } (f \circ \text{extract}^{E^\times}) \quad (4.85)$$

Lemma 4.3.29. (🚗) Lemmas 4.3.27 and 4.3.28 form an equivalence between Definitions 4.3.14 and 4.3.26.

An advantage of Definition 4.3.26 is it only requires defining mapd and proving two equations. By comparison, Definition 4.3.14 involves two operations constrained by five equations.

4.3.4 The Monoidal Category of Decorated Functors

In Examples 4.3.19 and 4.3.20, the underlying set of the decoration carries a monoidal structure. While the category DEC_E does not require E to be a monoid, when this happens, the category of decorated functors inherits the structure of a monoidal category (Def. 4.1.4). That is, there is a “natural” way to decorate both the identity functor and the composition of two decorated functors, and these constructions satisfy appropriate equations. In this section, let $\langle W, e, m \rangle$ be some fixed monoid.

Definition 4.3.30. (🎮) *The null decoration on any functor F is the constant decoration on F by the monoid unit:*

$$\begin{array}{c} F \\ \text{---} \boxed{\text{decnull}^F} \text{---} W^\times \end{array} \equiv \begin{array}{c} F \text{---} F \\ \bullet \text{---} W^\times \end{array} \quad \text{decnull}^F \equiv \text{map}^F (a \mapsto (e_W, a))$$

In particular, the null decoration is a natural way of decorating the identity functor.

Definition 4.3.31. (🎮) *The identity decorated functor is the identity functor paired with the null decoration $\langle 1, \text{decnull}^1 \rangle$.*

$$\begin{array}{c} 1 \text{---} \bullet \text{---} 1 \\ \bullet \text{---} W^\times \end{array} \equiv \begin{array}{c} 1 \text{---} 1 \\ \bullet \text{---} W^\times \end{array} \quad \text{dec}^1 \equiv \text{decnull}^1 \equiv \text{ret}^{W^\times} \quad (4.86)$$

Lemma B.1.1 proves that Definition 4.3.31 is a valid decorated functor.

The multiplication on W provides a way to combine two decorations to form a decoration on the composition of the underlying functors. For this, we use an operation called shifting.

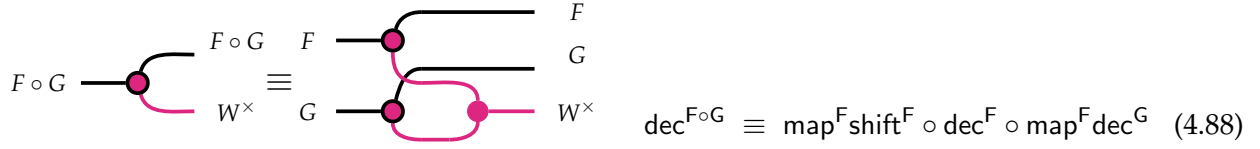
Definition 4.3.32. (🎮) *For any functor F and monoid W , the shift operation is a natural transformation defined as follows:*

$$\text{shift}: W^\times \circ (F \circ W^\times) \rightarrow F \circ W^\times$$

$$\begin{array}{c} W^\times \\ F \text{---} F \\ W^\times \end{array} \equiv \begin{array}{c} F \text{---} F \\ \bullet \text{---} W^\times \end{array} \quad \text{shift}^F \equiv \text{map}^F \text{join}^{W^\times} \circ \text{st}_W^F \quad (4.87)$$

This operation pushes an outer context into some functor F , adding it to the context at the leaves of G . This operation passes information down from the root of a tree towards its leaves.

Definition 4.3.33. (🚫) Let $\langle F, \text{dec}^F \rangle$ and $\langle G, \text{dec}^G \rangle$ be two decorated functors. Their composition is given by combining their decorations monoidally. This yields a decorated functor $\langle F \circ G, \text{dec}^{F \circ G} \rangle$ where $\text{dec}^{F \circ G}$ is defined as follows:



$$\text{dec}^{F \circ G} \equiv \text{map}^F \text{shift}^F \circ \text{dec}^F \circ \text{map}^F \text{dec}^G \quad (4.88)$$

Lemma B.1.2 proves that Definition 4.3.33 is a valid decorated functor.

The monoid laws ensure that these constructions give rise to a monoidal category.

Definition 4.3.34. (🚫) DEC_W is a monoidal category by the following data:

- The monoidal unit $\mathbf{I}$ is the identity functor paired with the null decoration.

$$\mathbf{I} \equiv \langle \mathbb{1}, \text{decnull}^{\mathbb{1}} \rangle$$

- The composition of two decorated functors $\langle F, \text{dec}^F \rangle$ and $\langle G, \text{dec}^G \rangle$ is given by Definition 4.3.33.

$$\langle F, \text{dec}^F \rangle \otimes \langle G, \text{dec}^G \rangle \equiv \langle F \circ G, \text{dec}^{F \circ G} \rangle$$

Lemma B.1.3 proves that this forms a monoidal category: specifically, that composition of decorated functors is associative and $\mathbf{I}$ acts as an identity.

Remark 4.3.35. Note that the monoid on W is part of the structure of DEC_W . We distinguish between functors that are decorated by a set or by a monoid—the difference is whether DEC inherits a monoidal structure.

While the monoidal category DEC_W appears to be novel in the context of programming languages, its construction closely resembles a well-known correspondence in abstract algebra, examined for example in [12, 18].

Lemma 4.3.36. Let H be a k -algebra. Then a bialgebra structure on H induces a monoidal structure on the category of right H -comodules.

In this context, the “bialgebra” corresponds to the bimonoid structure on any monoid W , which is lifted to form a bimonoid in END_{Set} , using the tensorial strength as a distributive law. “Right H -comodules” correspond to decorated functors, which are the right coactions of $W^\times$. What this connection implies for decorated monads, if anything, remains unclear, is left to future work.

4.3.5 Decorated Monads

In this section, let $\langle W, e, \cdot \rangle$ be a particular monoid. Because DEC_W forms a monoidal category, it is natural to examine the structure of monoids in this category.

Definition 4.3.37. (🚫) *A monad decorated by W , or just a decorated monad, is a monoid in DEC_W .*

By applying the forgetful functor $U: \text{DEC}_W \rightarrow \text{END}_{\text{Set}}$ that discards the decoration, we see that any decorated monad is, in particular, an ordinary monad (Def. 4.2.3). At the same time, as an object in DEC_W , it is equipped with a decoration $T \Rightarrow T \circ W^\times$ making T a decorated functor (Def. 4.3.14). What Definition 4.3.37 adds is a requirement that the monad operations ret^T and join^T commute with decorations dec^T and $\text{dec}^{T \circ T}$, which leads to the following equations:

$$\begin{array}{c} \text{blue dot} \text{---} \text{red dot} \\ \text{blue line} \text{---} \text{red line} \end{array} \begin{array}{c} T \\ W^\times \end{array} = \begin{array}{c} \text{blue dot} \text{---} \text{red dot} \\ \text{blue line} \text{---} \text{red line} \end{array} \begin{array}{c} T \\ W^\times \end{array} \quad \text{dec}^T \circ \text{ret}^T = \text{ret}^T \circ \text{ret}^{W^\times} \quad (4.89)$$

$$\begin{array}{c} T \\ T \end{array} \begin{array}{c} \text{blue dot} \text{---} \text{red dot} \\ \text{blue line} \text{---} \text{red line} \end{array} \begin{array}{c} T \\ W^\times \end{array} = \begin{array}{c} T \\ T \end{array} \begin{array}{c} \text{blue dot} \text{---} \text{red dot} \\ \text{blue line} \text{---} \text{red line} \end{array} \begin{array}{c} T \\ W^\times \end{array} \quad \text{dec}^T \circ \text{join}^T = \text{join}^T \circ \text{dec}^{T \circ T} \quad (4.90)$$

The equations above are written in a point-free style. By rewriting (4.89) in a pointed style, and recalling that $\text{ret}^{W^\times}$ pairs each element with e_W , we obtain the following equation.

$$\text{dec}^T \left(\text{ret}^T a \right) = \text{ret}^T (e, a) \quad (4.91)$$

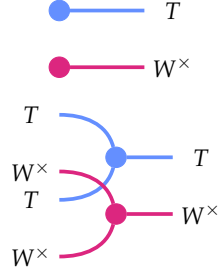
In the case where a decorated monad represents first-order abstract syntax (examined below), (4.91) can be interpreted as saying that the binding context of an atomic term is trivial—there are no binders in an expression like $\text{var } x$. (4.90) codifies the fact that the binding context is *cumulative*—a variable underneath multiple constructors inherits contextual information from all of them added together using the monoidal structure of contexts.

Alternate Characterization

We now examine an equivalent characterization of decorated monads. We first observe that the codomain of dec^T inherits the structure of a monad; this mirrors the construction of the product monoid (Def. 4.1.18) in SET , using the tensor strength in lieu of the Cartesian symmetry (i.e. the swap operation).

Lemma 4.3.38. *The composition $T \circ W^\times$ forms a monad with the following operations.*

$$\begin{aligned} \text{ret}^{T \circ W^\times} &: \forall (A: \text{SET}), A \rightarrow T(W \times A) \\ \text{join}^{T \circ W^\times} &: \forall (A: \text{SET}), T(W \times T(W \times A)) \rightarrow T(W \times A) \end{aligned}$$



$$\text{ret}^{T \circ W^\times} \equiv \text{ret}^T \circ \text{ret}^{W^\times} \quad (4.92)$$

$$\text{join}^{T \circ W^\times} \equiv \text{join}^T \circ \text{map}^T \text{shift}^T \quad (4.93)$$

□

Compare the following characterization of decorated monads to the dual characterizations of bimonoids in Definition 4.1.36.

Lemma 4.3.39. (🔗) *Let T form both a monad and a decorated functor. Then T forms a decorated monad if and only if $\text{dec}^T : T \Rightarrow T \circ W^\times$ is a monad homomorphism from T to the monad defined in Lemma 4.3.38.*

Proof. This is simply another way of reading Equations (4.89) and (4.90). □

4.3.6 Decorated Free Monads

We now illustrate how the decoration on a signature endofunctor Σ endows its free monad T with the structure of a decorated monad. We assume Σ is decorated by a monoid W , where the decoration codifies the binding context contributed by each constructor to each of its arguments. By Lemma 4.2.29, to define an operation of type $TA \rightarrow T(W \times A)$, it suffices to define two operations of the following types, respectively codifying the base case and recursive case of structural recursion on TA :

$$\begin{aligned} \text{base} &: A \rightarrow T(W \times A) \\ \text{rec} &: \Sigma(T(W \times A)) \rightarrow T(W \times A) \end{aligned}$$

The first case, which does not depend on Σ , is simple: the A -value is paired with empty context and lifted to an atomic term. The second operation factors in one “layer” of constructors. For this, we first apply the decoration on Σ to reflect what each constructor contributes to the context, before adding this to the inner W value and merging Σ into T with μ^* . This leads to the following definition of the decoration constructed on T .

Definition 4.3.40. *Given dec^Σ , the following operation defines a Σ -algebra on $T(W \times A)$ for any A :*

$$\alpha^{\text{dec}} \equiv (\mu^* \circ \text{map}^\Sigma \text{shift} \circ \text{dec}^\Sigma) : \Sigma \circ T \circ W^\times \Rightarrow T \circ W^\times$$

Now the induced decoration on the free monad T is defined as follows:

$$\text{dec}^T \equiv \text{sigfold}^T \alpha^{\text{dec}} \circ \text{map}^T (\text{ret}^T \circ \text{ret}^{W^\times}) \quad (4.94)$$

For instance, Definition 4.3.40 lifts the decoration on Σ^λ to the following operation on lam.

Example 4.3.41. The operation dec^{lam} induced by $\text{dec}^{\Sigma^\lambda}$ (Ex. 4.3.19) is as follows:

$$\begin{aligned}\text{dec}^{\text{lam}}(\text{var } v) &\equiv \text{var } (0, v) \\ \text{dec}^{\text{lam}}(\text{abs } t) &\equiv \text{abs } \left(\text{shift } \left(1, \text{dec}^{\text{lam}} t \right) \right) \\ \text{dec}^{\text{lam}}(\text{app } t_1 t_2) &\equiv \text{app } \left(\text{dec}^{\text{lam}} t_1 \right) \left(\text{dec}^{\text{lam}} t_2 \right)\end{aligned}$$

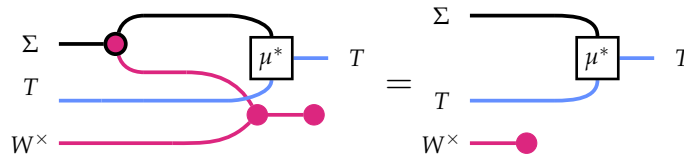
We must verify that Definition 4.3.40 endows T with the structure of a decorated monad. This requires verifying that dec^T is a natural transformation satisfying (4.76), (4.77), (4.89), and (4.90).

Theorem 4.3.42. dec^T endows T with the structure of a decorated functor.

Proof. First we verify that dec^T is a natural transformation. It is trivial to verify that $\text{map}^{T \circ W^\times} f$ forms a Σ -algebra homomorphism between α_A^{dec} and α_B^{dec} because α^{dec} is itself natural. This is used below to commute α^{dec} with sigfold^T by Lemma 4.2.27:

$$\begin{aligned}& \text{map}^{T \circ W^\times} f \circ \text{dec}^T \\ &= \text{map}^{T \circ W^\times} f \circ \text{sigfold}^T \alpha^{\text{dec}} \circ \text{map}^T \left(\text{ret}^T \circ \text{ret}^{W^\times} \right) && \text{Eqn. (4.94)} \\ &= \text{sigfold}^T \alpha^{\text{dec}} \circ \text{map}^{T \circ T \circ W^\times} f \circ \text{map}^T \left(\text{ret}^T \circ \text{ret}^{W^\times} \right) && \text{Lem. 4.2.27} \\ &= \text{sigfold}^T \alpha^{\text{dec}} \circ \text{map}^T \left(\text{ret}^T \circ \text{ret}^{W^\times} \right) \circ \text{map}^T f && \text{Eqn. (4.22) and naturality} \\ &= \text{dec}^T \circ \text{map}^T f && \text{Eqn. (4.94)}\end{aligned}$$

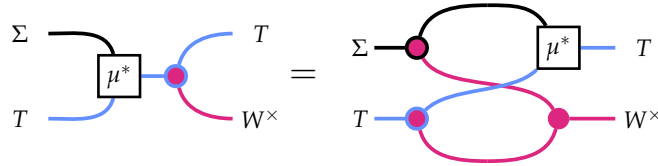
Next we verify the counit law. For this, we observe that for all A , $\text{map}^T \text{extract}$ is a homomorphism between the Σ -algebra on $T(W \times A)$ given by α^{dec} and the algebra on TA given by μ^* . This observation corresponds to the following diagrammatic equality:



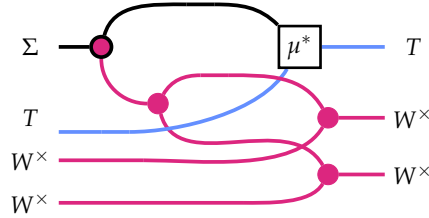
We also recall that $\text{join}^\top$ is precisely $\text{sigfold}^\top \mu^*$. We have the following:

$$\begin{aligned}
& \text{map}^\top \text{extract}^{W^\times} \circ \text{dec}^\top \\
= & \text{map}^\top \text{extract}^{W^\times} \circ \text{sigfold}^\top \alpha^{\text{dec}} \circ \text{map}^\top (\text{ret}^\top \circ \text{ret}^{W^\times}) && \text{Eqn. (4.94)} \\
= & \text{sigfold}^\top \mu^* \circ \text{map}^{\top \circ \top} \text{extract}^{W^\times} \circ \text{map}^\top (\text{ret}^\top \circ \text{ret}^{W^\times}) && \text{Lem. 4.2.27} \\
= & \text{sigfold}^\top \mu^* \circ \text{map}^\top (\text{ret}^\top \circ \text{extract}^{W^\times} \circ \text{ret}^{W^\times}) && \text{Eqn. (4.22) and naturality} \\
= & \text{sigfold}^\top \mu^* \circ \text{map}^\top \text{ret}^\top && \text{Eqn. (4.17)} \\
= & \text{id} && \text{Eqn. (4.29)}
\end{aligned}$$

Finally, we verify the duplication law. In this proof, we exploit the fact that we have already shown $\text{dec}^\top$ is a natural transformation satisfying (4.89). The keystone property is the following one, which states that $\text{dec}^\top$ is a Σ -algebra homomorphism between μ^* and α^{dec} . This can be inferred from the initiality of μ^* :



In the following proof, the algebra β^{dec} refers to the following Σ -algebra:



The reader can use the above equation, the duplication law for dec^Σ , the properties of $W^\times$ inherited from those of bimonoids, and string-diagrammatic reasoning to show that both $\text{dec}^\top$ and $\text{map}^\top \text{dup}^{W^\times}$ form homomorphisms from α^{dec} to β^{dec} . We now have the following proof of the

duplication law for dec^T :

$$\begin{aligned}
& \text{dec}^T \circ \text{dec}^T \\
&= \text{dec}^T \circ \text{sigfold}^T \alpha^{\text{dec}} \circ \text{map}^T (\text{ret}^T \circ \text{ret}^{W^\times}) && \text{Eqn. (4.94)} \\
&= \text{sigfold}^T \beta^{\text{dec}} \circ \text{map}^{T \circ T} \text{dec}^T \circ \text{map}^T (\text{ret}^T \circ \text{ret}^{W^\times}) && \text{Lem. 4.2.27} \\
&= \text{sigfold}^T \beta^{\text{dec}} \circ \text{map}^T (\text{ret}^T \circ \text{ret}^{W^\times} \circ \text{ret}^{W^\times}) && \text{Naturality and (4.89)} \\
&= \text{sigfold}^T \beta^{\text{dec}} \circ \text{map}^{T \circ T} \text{dup}^{W^\times} \circ \text{map}^T (\text{ret}^T \circ \text{ret}^{W^\times}) && \text{Naturality and def. of } \text{dup}^{W^\times} \\
&= \text{map}^T \text{dup}^{W^\times} \circ \text{sigfold}^T \alpha^{\text{dec}} \circ \text{map}^T (\text{ret}^T \circ \text{ret}^{W^\times}) && \text{Lem. 4.2.27} \\
&= \text{map}^T \text{dup}^{W^\times} \circ \text{dec}^T && \text{Eqn. (4.94)}
\end{aligned}$$

□

Now we must verify (4.89) and (4.90). We can take a conceptual shortcut by the following lemma, which is simply an equivalent definition of α^{dec} .

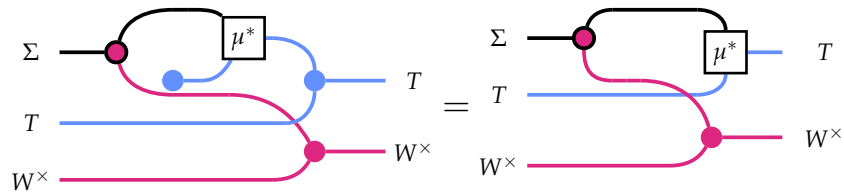
Lemma 4.3.43. *The monad decoration in Definition 4.3.40 is equivalent to the following one:*

$$\text{dec}^T = \text{sigfold}^T (\text{join}^{T \circ W^\times} \circ \iota \circ \text{dec}^\Sigma) \circ \text{map}^T \text{ret}^{T \circ W^\times} \quad (4.95)$$

Proof. This immediately reduces to the following equation:

$$\text{join}^{T \circ W^\times} \circ \iota \circ \text{dec}^\Sigma = \mu^* \circ \text{map}^\Sigma \text{shift} \circ \text{dec}^\Sigma \quad (4.96)$$

Expanding definitions using (4.53) and (4.93) reveals that (4.96) asserts the following diagrammatic equivalence, whose unremarkable proof we omit:



$$\text{join}^T \circ \text{map}^T \text{shift}^T \circ \mu^* \circ \text{map}^\Sigma \text{ret}^T \circ \text{dec}^\Sigma = \mu^* \circ \text{map}^\Sigma \text{shift} \circ \text{dec}^\Sigma$$

We now have an equation between the two definitions of dec^T given by (4.94) and (4.95):

$$\begin{aligned}
& \text{sigfold}^T (\text{join}^{T \circ W^\times} \circ \iota \circ \text{dec}^\Sigma) \circ \text{map}^T (\text{ret}^{T \circ W^\times}) \\
&= \text{sigfold}^T (\mu^* \circ \text{map}^\Sigma \text{shift} \circ \text{dec}^\Sigma) \circ \text{map}^T (\text{ret}^T \circ \text{ret}^{W^\times}) && \text{Eqns. (4.96) (4.92)}
\end{aligned}$$

□

Theorem 4.3.44. For a fixed monoid $\langle W, e, \cdot \rangle$, let Σ be decorated by W and let T be the free monad generated by Σ , assuming it exists. The operation dec^T in Definition 4.3.40 equips T with the structure of a decorated monad.

Proof. Theorem 4.3.42 demonstrated that dec^T makes T a decorated functor. Lemma 4.2.29 states that monad homomorphisms $T \Rightarrow U$ correspond precisely to natural transformations $\Sigma \Rightarrow U$. Lemma 4.3.39 demonstrated that a monad that is also a decorated functor is a decorated monad if and only if the decoration is a monad homomorphism. Equation (4.95) presented dec^T as the extension of the following natural transformation:

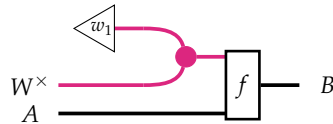
$$(\iota \circ \text{dec}^\Sigma) : \Sigma \Rightarrow T \circ W^\times$$

Therefore dec^T forms a monad homomorphism, and we are done. $\square$

4.3.7 Decorated Monads as an Extension System

In this section we fix some monoid $\langle W, e_W, \cdot \rangle$ and consider monads decorated by W . Following the style of Definitions 4.2.11 and 4.3.26, we reformulate decorated monads as extension systems. The following operation plays a role analogous to shift^T in the presentation of decorated monads as monoids in DEC_W .

Definition 4.3.45. (🚗) Let $f : W \times A \rightarrow B$ and $w_1 : W$. The pre-increment of f by w_1 , written $f \odot w_1$, is the operation that adds w_1 to any context fed to f :



$$(f \odot w_1) \equiv \lambda (w_2, a). f(w_1 \cdot w_2, a) \quad (4.97)$$

Definition 4.3.46. (🚗) A decorated monad presented as an extension system is a set-forming operation $T : \text{SET} \rightarrow \text{SET}$ paired with operations

$$\begin{aligned} \text{ret} &: \forall (A : \text{SET}), A \rightarrow TA \\ \text{bindd} &: \forall (A, B : \text{SET}), (W \times A \rightarrow TB) \rightarrow TA \rightarrow TB \end{aligned}$$

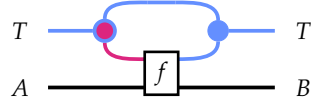
subject to the following equations:

$$\text{bindd } f \circ \text{ret}^T = f \circ \text{ret}^{W^\times} \quad (4.98)$$

$$\text{bindd } (\text{ret}^T \circ \text{extract}) = \text{id}_T \quad (4.99)$$

$$\text{bindd } g \circ \text{bindd } f = \text{bindd } ((w, a) \mapsto \text{bindd } (g \odot w) (f(w, a))) \quad (4.100)$$

Lemma 4.3.47. (☞) Any decorated monad gives rise to an extension system where bindd is defined



$$\text{bindd } f \equiv \text{join} \circ \text{map } f \circ \text{dec} \quad (4.101)$$

Proof.

Proof of (4.99) (☞): $\text{bindd } (\text{ret} \circ \text{extract}) = \text{id}_{TA}$

$$= \begin{array}{c} T \\ A \end{array} \quad \text{Eqns. (4.76) (4.29)}$$

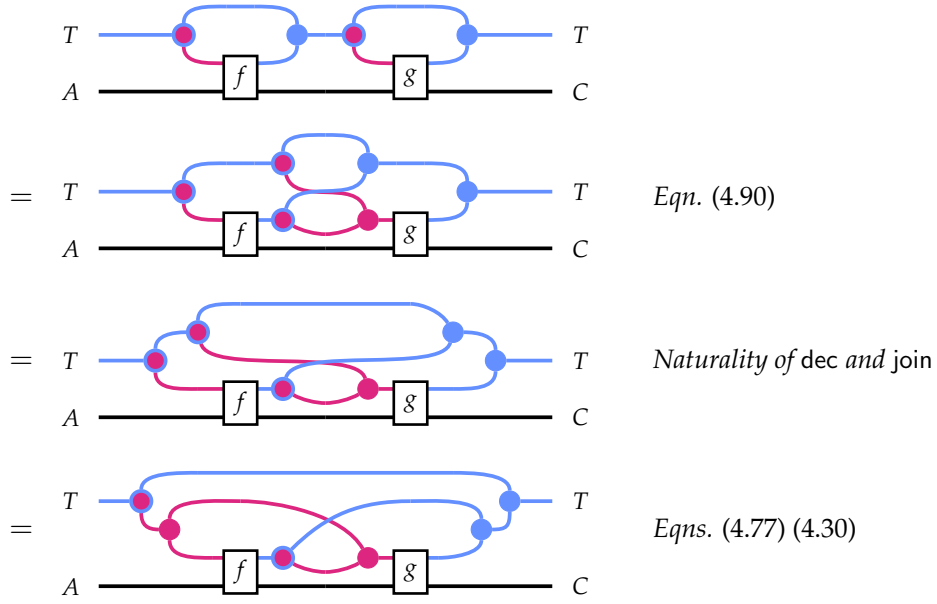
Proof of (4.98) (☞): $\text{bindd } f \circ \text{ret}^T = f \circ \text{ret}^{W \times}$

$$= \begin{array}{c} T \\ A \end{array} \quad \text{Eqn. (4.90)}$$

$$= \begin{array}{c} T \\ A \end{array} \quad \text{Naturality of ret}$$

$$= \begin{array}{c} T \\ A \end{array} \quad \text{Eqn. (4.28)}$$

Proof of (4.100) (☹): $\text{bindd } g \circ \text{bindd } f = \text{bindd } ((w, a) \mapsto \text{bindd } (g \odot w) (f(w, a)))$

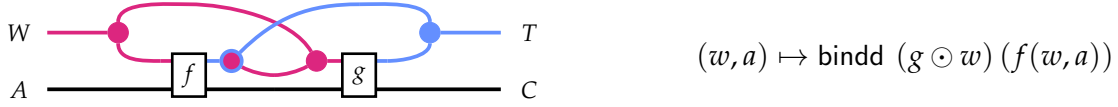


□

Example 4.3.48. The operation $\text{bindd}^{\text{lam}}$ corresponding to Example 4.3.41 is defined as follows:

$$\begin{aligned}\text{bindd}^{\text{lam}} f (\text{var } v) &\equiv f(0, v) \\ \text{bindd}^{\text{lam}} f (\text{abs } t) &\equiv \text{abs } (\text{bindd}^{\text{lam}} (f \odot 1) t) \\ \text{bindd}^{\text{lam}} f (\text{app } t_1 t_2) &\equiv \text{app } (\text{bindd}^{\text{lam}} f t_1) (\text{bindd}^{\text{lam}} f t_2)\end{aligned}$$

The following operation represents the (generalized analogue of) the Kleisli composition of g and f for a decorated monad:



In the context of syntax, the Kleisli composition codifies the combined effect of two context-sensitive substitutions performed at a particular variable, say v , underneath w binders in a term t . The first substitution replaces v with $f(w, v)$. It would be incorrect however to say that the effect of g on the new subterm $f(w, a)$ is to apply the substitution $\text{bindd } g$, as this would ignore the fact that this subterm is located in a context in which w binders are already in scope—information required by g . Thus, a copy of the original context w is made and added to g using $(\odot)$.

Lemma 4.3.49. (🐼) A decorated monad presented as an extension system forms a decorated monad. The operations map , dec , and join are defined from bindd as follows.

$$\text{dec}^T \equiv \text{bindd } \text{ret}^T \quad (4.102)$$

$$\text{join}^T \equiv \text{bindd } \text{extract}^{W^\times} \quad (4.103)$$

$$\text{map}^T f \equiv \text{bindd } (\text{ret}^T \circ f \circ \text{extract}^{W^\times}) \quad (4.104)$$

□

Lemma 4.3.50. (🐼) Definitions 4.3.37 and 4.3.46 are equivalent in the sense that an instance of one uniquely determines an instance of the other. □

Decorated Functor Instance

It follows from the above that a decorated monad is, in particular, a decorated functor, where mapd is related to bindd by the following equation:

$$\text{mapd}^T f = \text{bindd}^T (\text{ret}^T \circ f) \quad (4.105)$$

Lemma 4.3.51. *The following equations hold:*

$$\text{mapd}^\top f \circ \text{ret}^\top = \text{ret}^\top \circ f \circ \text{ret}^{W^\times} \quad (4.106)$$

$$\text{bindd}^\top g \circ \text{mapd}^\top f = \text{bindd}^\top ((w, a) \mapsto g(w, f(w, a))) \quad (4.107)$$

$$\text{mapd}^\top g \circ \text{bindd}^\top f = \text{bindd}^\top ((w, a) \mapsto (\text{mapd}^\top (g \odot w) (f(w, a)))) \quad (4.108)$$

4.4 DE BRUIJN SUBSTITUTION

We now illustrate how the theory of decorated monads, assumed to be decorated by $\langle \mathbb{N}, +, 0 \rangle$ and specialized to the type parameter $V = \mathbb{N}$, yields the de Bruijn algebra presented in Section 2.2.

We now reformulate the definitions of de Bruijn renaming and instantiation (Defs. 2.2.1 and 2.2.2) in a way that makes their structure more transparent. While these definitions remain equivalent to those presented earlier, they emphasize a different perspective. Rather than directly modifying the renaming or substitution during recursion, they maintain an accumulator variable, d for “depth”, recording the number of binders recursed under. Lifting the operation using $\uparrow_{\text{ren}}$ or $\uparrow$ is deferred until recursion bottoms out on a var case, at which point it is iterated d times (where $f^0 = \text{id}$ and $f^{n+1} = f \circ f^n$). This is a “localized” perspective, in the sense that all of the computational action takes place at the leaves, rather than interleaved throughout the recursion.

Let $\rho: \mathbb{N} \rightarrow \mathbb{N}$ be a de Bruijn renaming operation. Recall that $\uparrow_{\text{ren}} \rho = 0 \cdot (S \circ \rho)$ is the function that maps 0 to 0 and all variables $n + 1$ to $\rho(n) + 1$.

Definition 4.4.1. *The application of ρ to a term t is defined $\text{rename } \rho t = \text{rename}_0 \rho t$, where*

$$\begin{aligned} \text{rename}_d \rho (\text{var } n) &\equiv \text{var } \left(\left(\uparrow_{\text{ren}}^d \rho \right) n \right) \\ \text{rename}_d \rho (\text{abs } t) &\equiv \text{abs } (\text{rename}_{d+1} \rho t) \\ \text{rename}_d \rho (\text{app } t_1 t_2) &\equiv \text{app } (\text{rename}_d \rho t_1) (\text{rename}_d \rho t_2) \end{aligned}$$

Let $\sigma: \mathbb{N} \rightarrow \text{lam } \mathbb{N}$ be a substitution. Recall that $\uparrow \sigma = \text{var } 0 \bullet (\text{rename } (+1) \circ \sigma)$ is the function that maps 0 to var 0 and all variables $n + 1$ to $\text{rename } (+1) \sigma(n)$.

Definition 4.4.2. *The instantiation of σ in t is defined $\text{inst } \sigma t = \text{inst}_0 \sigma t$, where*

$$\begin{aligned} \text{inst}_d \sigma (\text{var } n) &\equiv \left(\uparrow^d \sigma \right) n \\ \text{inst}_d \sigma (\text{abs } t) &\equiv \text{abs } (\text{inst}_{d+1} \sigma t) \\ \text{inst}_d \sigma (\text{app } t_1 t_2) &\equiv \text{app } (\text{inst}_d \sigma t_1) (\text{inst}_d \sigma t_2) \end{aligned}$$

Now, we uncurry the $(\uparrow_{\text{ren}}^d)$ and $(\uparrow^d)$ operations, expressing them as functions on pairs whose first component is d . These operations express the logic of operations on syntax in terms of how they affect variables in a particular binding context, without mentioning the grammatical structure of the syntax.

$$\text{ren}_{\text{loc}} \rho: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N} \quad (\text{ren}_{\text{loc}} \rho)(d, n) = \left(\uparrow_{\text{ren}}^d \rho \right) n \quad (4.109)$$

$$\text{sub}_{\text{loc}} \sigma: \mathbb{N} \times \mathbb{N} \rightarrow T \mathbb{N} \quad (\text{sub}_{\text{loc}} \sigma)(d, n) = \left(\uparrow^d \sigma \right) n \quad (4.110)$$

Now we can express renaming and substitution using operations derived from the structure of lam as a decorated monad. Namely, we observe that $\text{rename } \rho \ t$ is equal to $\text{mapd}^{\text{lam}}(\text{ren}_{\text{loc}} \rho) \ t$ and $\text{inst } \sigma \ t$ is equal to $\text{bindd}^{\text{lam}}(\text{sub}_{\text{loc}} \sigma) \ t$. By extension, we can define renaming and instantiation for any monad decorated by $\langle \mathbb{N}, +, 0 \rangle$.

Definition 4.4.3. (🎮) *De Bruijn renaming is defined as follows:*

$$\begin{aligned} \text{rename}^T: (\mathbb{N} \rightarrow \mathbb{N}) &\rightarrow T \mathbb{N} \rightarrow T \mathbb{N} \\ \text{rename}^T \rho &\equiv \text{mapd}^T(\text{ren}_{\text{loc}} \rho) \end{aligned} \quad (4.111)$$

Definition 4.4.4. (🎮) *De Bruijn instantiation is defined as follows:*

$$\begin{aligned} \text{inst}^T: (\mathbb{N} \rightarrow T \mathbb{N}) &\rightarrow T \mathbb{N} \rightarrow T \mathbb{N} \\ \text{inst}^T \sigma &\equiv \text{bindd}^T(\text{sub}_{\text{loc}} \sigma) \end{aligned} \quad (4.112)$$

Equations (2.12)–(2.14) can now be derived as corollaries of the laws of decorated monads. For facilitate the proof, we introduce an alternative specification of ren_{loc} and sub_{loc} that avoids iterating $(\uparrow_{\text{ren}})$ and $(\uparrow)$. Instead, the following lemmas directly capture the logic of lifting and shifting operations when the index n occurs an arbitrary number d of binders.

Lemma 4.4.5. (🎮) *Iterating $(\uparrow_{\text{ren}})$ is characterized by the following equation:*

$$\left(\uparrow_{\text{ren}}^d \rho \right) n = \begin{cases} n & \text{if } n < d \\ \rho(n - d) + d & \text{if } n \geq d \end{cases}$$

Proof. The result can be observed by unrolling a handful of iterations:

$$\begin{aligned} (\uparrow_{\text{ren}}^0 \rho) &= \rho \\ (\uparrow_{\text{ren}}^1 \rho) &= 0 \cdot ((+1) \circ \rho) \\ (\uparrow_{\text{ren}}^2 \rho) &= 0 \cdot ((+1) \circ (0 \cdot ((+1) \circ \rho))) \\ &= 0 \cdot 1 \cdot ((+2) \circ \rho) \end{aligned}$$

In the last step, we simplify the expression using the following specification relating the cons operation to composition.

$$g \circ (x \cdot f) = g \, x \cdot (g \circ f) \quad (4.113)$$

By induction on d , we have the following pattern:

$$\left(\uparrow_{\text{ren}}^d \rho \right) = 0 \cdot 1 \dots \cdot (d-1) \cdot ((+d) \circ \rho)$$

The result holds by noting the following equivalence:

$$\left(\underbrace{x_0 \cdot x_1 \dots \cdot x_{d-1}}_{d \text{ values}} \cdot f \right) n = \begin{cases} x_n & \text{if } n < d \\ f(n-d) & \text{if } n \geq d \end{cases} \quad (4.114)$$

□

The same idea can be extended to $(\uparrow)$ after establishing a couple equations.

Lemma 4.4.6. () *Renaming satisfies the following equations:*

$$\text{rename } (+x) (\text{ret } y) = \text{ret } (x + y) \quad (4.115)$$

$$\text{rename } (+y) \circ \text{rename } (+x) = \text{rename } (+ (x + y)) \quad (4.116)$$

Proof.

Proof of (4.115):

$$\begin{aligned} & \text{rename } (+x) (\text{ret } y) \\ = & \text{mapd}^T (\text{ren}_{\text{loc}} (+x)) (\text{ret } y) && \text{Eqn. (4.111)} \\ = & \text{ret } ((\text{ren}_{\text{loc}} (+x)) (0, y)) && \text{Eqn. (4.106)} \\ = & \text{ret } (x + y) && \text{Eqn. (4.109)} \end{aligned}$$

Proof of (4.116): A simple calculation using (4.114) shows the following, used below:

$$\uparrow_{\text{ren}}^d (+y) \circ \uparrow_{\text{ren}}^d (+x) = \uparrow_{\text{ren}}^d (+ (x + y)) \quad (4.117)$$

$$\begin{aligned}
& \text{rename } (+y) \circ \text{rename } (+x) \\
= & \text{mapd}^T (\text{ren}_{\text{loc}} (+y)) \circ \text{mapd}^T (\text{ren}_{\text{loc}} (+x)) && \text{Eqn. (4.111)} \\
= & \text{mapd}^T (\lambda(d, n). \text{ren}_{\text{loc}} (+y) (d, \text{ren}_{\text{loc}} (+n) (d, n))) && \text{Eqn. (4.82)} \\
= & \text{mapd}^T \left(\lambda(d, n). \left(\uparrow_{\text{ren}}^d (+y) \right) \left(\uparrow_{\text{ren}}^d (+x) \right) n \right) && \text{Eqn. (4.109)} \\
= & \text{mapd}^T \left(\lambda(d, n). \left(\uparrow_{\text{ren}}^d (+ (x + y)) \right) n \right) && \text{Eqn. (4.117)} \\
= & \text{rename } (+ (x + y)) && \text{Eqn. (4.111)}
\end{aligned}$$

□

Now we can give a specification for the iterating the $(\uparrow)$ operation.

Lemma 4.4.7. (🐞) *Iterating $(\uparrow)$ is characterized by the following equation.*

$$\left(\uparrow^d \sigma \right) n = \begin{cases} \text{ret } n & \text{if } n < d \\ \text{rename } (n \mapsto n + d) (\sigma(n - d)) & \text{if } n \geq d \end{cases}$$

Proof. We begin by unrolling cases $d = 0, 1, 2$.

$$\begin{aligned}
& \left(\uparrow^0 \sigma \right) = \sigma \\
& \left(\uparrow^1 \sigma \right) = \text{ret } 0 \cdot (\text{rename } (+1) \circ \sigma) \\
& \left(\uparrow^2 \sigma \right) = \text{ret } 0 \cdot (\text{rename } (+1) \circ (\text{ret } 0 \cdot (\text{rename } (+1) \circ \sigma))) \\
& \quad = \text{ret } 0 \cdot (\text{rename } (+1) (\text{ret } 0)) \cdot (\text{rename } (+1) \circ \text{rename } (+1) \circ \sigma) && \text{Eqn. (4.113)} \\
& \quad = \text{ret } 0 \cdot \text{ret } 1 \cdot (\text{rename } (+2) \circ \sigma) && \text{Eqns. (4.115) (4.116)} \\
& \quad \vdots \\
& \left(\uparrow^d \sigma \right) = \text{ret } 0 \cdot \text{ret } 1 \cdot \dots \cdot \text{ret } (d - 1) \cdot (\text{rename } (+d) \circ \sigma)
\end{aligned}$$

□

The specification in Lemma 4.4.7 can be understood as follows. If $n < d$, then n is a bound variable and remains unchanged. If $n \geq d$, n refers to the $(n - d)^{\text{th}}$ free variable, which is replaced with $\sigma(n - d)$. However, each occurrence in $\sigma(n - d)$ now occurs under d additional bound variables, so we renaming the n^{th} free variable in this term to $n + d$. Using these more convenient descriptions, we can derive the equations necessary to instantiate Autosubst as follows.

Lemma 4.4.8. (🐞) *Proof of Equation 2.12: $\text{inst ret } t = t$*

Proof. Unfolding the notation reveals the following goal:

$$\text{bindd } (\text{sub}_{\text{loc}} \text{ret}) t = t$$

By (4.99), it suffices to demonstrate that $\text{sub}_{\text{loc}} \text{ret}$ maps each de Bruijn index n to $\text{ret } n$, regardless of the binding depth of the occurrence. That is, we must show

$$(\text{sub}_{\text{loc}} \text{ret}) = \text{ret} \circ \text{extract},$$

which is more simply expressed as follows:

$$\forall (d : \mathbb{N}), \left(\uparrow^d \text{ret} \right) n = \text{ret } n.$$

It suffices to proceed by case analysis on whether $n < d$ using the specification in Lemma 4.4.7. If $n < d$, the equation is immediate. Otherwise, the left-hand side simplifies to

$$\text{rename } (n \mapsto n + d) (\text{ret } (n - d)).$$

Now $\text{rename } \rho (\text{ret } x) = \text{ret } (\rho x)$ by (4.106), so the expression simplifies to $\text{ret } ((n - d) + d)$. Since $n \geq d$, $n - d + d = n$ and we are done. □

Lemma 4.4.9. (🔗) *Proof of Equation 2.13:* $\text{inst } \sigma (\text{ret } n) = \sigma(n)$

Proof. Unfolding notation reveals the following goal:

$$\text{bindd } (\text{sub}_{\text{loc}} \sigma) (\text{ret } n) = \sigma(n)$$

By (4.98), the latter is equivalent to $(\text{sub}_{\text{loc}} \sigma) (0, n)$, or $(\uparrow^0 \sigma) n$, which is $\sigma(n)$. □

We require the following property to prove Lemma 4.4.11.

Lemma 4.4.10. *The following holds:*

$$\text{mapd } (\text{ren}_{\text{loc}} (+d) \odot d') \circ \text{rename } (+d') = \text{rename } (+d + d') \quad (4.118)$$

Proof. The above is nearly identical to (4.116) except for the presence of a term of the form $(\odot d')$. Using Lemma 4.4.5, the operation $(\text{ren}_{\text{loc}} (+d) \odot d')$ has the following specification.

$$(\text{ren}_{\text{loc}} (+d) \odot d') (d'', n) = \begin{cases} n & \text{if } n < d' + d'' \\ (n - d' - d'') + d + d' + d'' & \text{if } n \geq d' + d'' \end{cases}$$

We begin by simplifying the LHS of (4.118) as follows:

$$\begin{aligned}
& \text{mapd } (\text{ren}_{\text{loc}} (+d) \odot d') \circ \text{rename } (+d') \\
= & \text{mapd } (\text{ren}_{\text{loc}} (+d) \odot d') \circ \text{mapd } (\text{ren}_{\text{loc}} (+d')) && \text{Definition} \\
= & \text{mapd } (\lambda(d'', n). (\text{ren}_{\text{loc}} (+d) \odot d') (d'', \text{ren}_{\text{loc}} (+d')(d'', n))) && \text{Eqn. (4.82)} \\
= & \text{mapd } (\lambda(d'', n). \text{ren}_{\text{loc}} (+d) (d' + d'', \text{ren}_{\text{loc}} (+d')(d'', n))) && \text{Simplify } (\odot)
\end{aligned}$$

This reduces the goal to two expressions of the form $\text{mapd } \rho$ for different ρ , so it suffices to show these inner expressions are equal, which reduces the goal to showing the following:

$$\text{ren}_{\text{loc}} (+d) (d' + d'', \text{ren}_{\text{loc}} (+d')(d'', n)) = \text{ren}_{\text{loc}} (+ (d + d')) (d'', n)$$

We proceed by splitting on whether (d'', n) represents a free or bound occurrence.

Bound Case ($n < d''$): In this case, the RHS reduces to n immediately by Lemma 4.4.5, while the LHS reduces as follows:

$$\begin{aligned}
& \text{ren}_{\text{loc}} (+d) (d' + d'', (\text{ren}_{\text{loc}} (+d')(d'', n))) \\
= & \text{ren}_{\text{loc}} (+d) (d' + d'', n) \\
= & n
\end{aligned}$$

Free Case ($n \geq d''$): In this case, the RHS reduces to $n + (d + d')$ immediately by Lemma 4.4.5, while the LHS reduces as follows:

$$\begin{aligned}
& \text{ren}_{\text{loc}} (+d) (d' + d'', \text{ren}_{\text{loc}} (+d')(d'', n)) \\
= & \text{ren}_{\text{loc}} (+d) (d' + d'', n - d'' + d' + d'') \\
= & \text{ren}_{\text{loc}} (+d) (d' + d'', n + d')
\end{aligned}$$

By $n \geq d''$, we have $n + d' \geq d' + d''$, so by Lemma 4.4.5 this reduces to

$$(n + d') - (d' + d'') + d + (d' + d'') = n + d + d'$$

and we are done. □

Lemma 4.4.11. (🚫) *Proof of Equation 2.14:* $t[\sigma][\tau] = t[\sigma * \tau]$.

Proof. Unfolding notation reveals the following goal:

$$\text{bindd } (\text{sub}_{\text{loc}} \tau) \circ \text{bindd } (\text{sub}_{\text{loc}} \sigma) = \text{bindd } (\text{sub}_{\text{loc}} (\text{bindd } (\text{sub}_{\text{loc}} \tau) \circ \sigma))$$

By (4.100), the left-hand side rewrites to the following:

$$\text{bindd}((d, n) \mapsto \text{bindd}(\text{sub}_{\text{loc}} \tau \odot d)(\text{sub}_{\text{loc}} \sigma(d, n)))$$

After this step, both sides are of the form $\text{bindd } \sigma'$ for different σ' , so it suffices to show these two localized substitutions are equal. This yields the following goal where the pair (d, n) is universally quantified:

$$\text{bindd}(\text{sub}_{\text{loc}} \tau \odot d)(\text{sub}_{\text{loc}} \sigma(d, n)) = \text{sub}_{\text{loc}}(\text{bindd}(\text{sub}_{\text{loc}} \tau) \circ \sigma)(d, n) \quad (4.119)$$

Now we use Lem. 4.4.7 to proceed by case analysis on whether n is free or bound.

Bound Case ($n < d$): In this case, $\text{sub}_{\text{loc}} \sigma'(n, d) = \text{ret } n$ for all σ' . Therefore (4.119) simplifies to the following goal:

$$\text{bindd}(\text{sub}_{\text{loc}} \tau \odot d)(\text{ret } n) = \text{ret } n$$

By (4.98) the left-hand side can be rewritten to $\text{sub}_{\text{loc}} \tau(d, n)$. Since $n < d$, this returns $\text{ret } n$ and we are done.

Free Case ($n \geq d$): In this case, (4.119) simplifies to the following equality.

$$\text{bindd}(\text{sub}_{\text{loc}} \tau \odot d)(\text{rename}(+d)(\sigma(n - d))) = \text{rename}(+d)(\text{bindd}(\text{sub}_{\text{loc}} \tau)(\sigma(n - d)))$$

Using the bindd/mapd composition law (4.107), the LHS simplifies as follows:

$$\begin{aligned} & \text{bindd}(\text{sub}_{\text{loc}} \tau \odot d)(\text{rename}(+d)(\sigma(n - d))) \\ = & \text{bindd}(\text{sub}_{\text{loc}} \tau \odot d)(\text{mapd}(\text{ren}_{\text{loc}}(+d))(\sigma(n - d))) \\ = & \text{bindd}(\lambda(d', n'). (\text{sub}_{\text{loc}} \tau \odot d)(d', \text{ren}_{\text{loc}}(+d)(d', n')))(\sigma(n - d)) \\ = & \text{bindd}(\lambda(d', n'). (\text{sub}_{\text{loc}} \tau (d + d', \text{ren}_{\text{loc}}(+d)(d', n')))(\sigma(n - d))) \end{aligned}$$

Using the mapd/bindd (4.108), the RHS simplifies as follows:

$$\begin{aligned} & \text{rename}(+d)(\text{bindd}(\text{sub}_{\text{loc}} \tau)(\sigma(n - d))) \\ = & \text{mapd}(\text{ren}_{\text{loc}}(+d))(\text{bindd}(\text{sub}_{\text{loc}} \tau)(\sigma(n - d))) \\ = & \text{bindd}(\lambda(d', n'). \text{mapd}(\text{ren}_{\text{loc}}(+d) \odot d')(\text{sub}_{\text{loc}} \tau(d', n')))(\sigma(n - d)) \end{aligned}$$

Once again we arrive at an equation between two expressions of the form $\text{bindd } \sigma' t$ for different σ' . Again we proceed by showing the inner operations are equal, yielding the following goal:

$$\text{sub}_{\text{loc}} \tau(d + d', \text{ren}_{\text{loc}}(+d)(d', n')) = \text{mapd}(\text{ren}_{\text{loc}}(+d) \odot d')(\text{sub}_{\text{loc}} \tau(d', n')) \quad (4.120)$$

Again we split on whether n' is free or bound. If $n' < d'$, the left-hand side of (4.120) reduces as follows:

$$\begin{aligned}
& \text{sub}_{\text{loc}} \tau (d + d', \text{ren}_{\text{loc}} (+d) (d', n')) \\
= & \text{sub}_{\text{loc}} \tau (d + d', n') && \text{Lem. 4.4.5} \\
= & \text{ret } n' && \text{Lem. 4.4.7}
\end{aligned}$$

The right-hand side reduces as follows:

$$\begin{aligned}
& \text{mapd} (\text{ren}_{\text{loc}} (+d) \odot d') (\text{sub}_{\text{loc}} \tau (d', n')) \\
= & \text{mapd} (\text{ren}_{\text{loc}} (+d) \odot d') (\text{ret } n') && \text{Lem. 4.4.7} \\
= & \text{ret} ((\text{ren}_{\text{loc}} (+d) \odot d') (0, n')) && \text{Eqn. (4.106)} \\
= & \text{ret} (\text{ren}_{\text{loc}} (+d) (d', n')) && \text{Simplify } (\odot) \\
= & \text{ret } n' && \text{Lem. 4.4.5}
\end{aligned}$$

On the other hand, if $n' \geq d'$, the left-hand side of (4.120) reduces as follows:

$$\begin{aligned}
& \text{sub}_{\text{loc}} \tau (d + d', \text{ren}_{\text{loc}} (+d) (d', n')) \\
= & \text{sub}_{\text{loc}} \tau (d + d', ((n' - d') + d + d')) && \text{Lem. 4.4.7} \\
= & \text{sub}_{\text{loc}} \tau (d + d', n' + d) && \text{Arithmetic}
\end{aligned}$$

Now $n' + d \geq d + d'$, by which this reduces to $\text{rename} (d + d') (\tau(n' - d'))$. Using Lemma 4.4.10, the right-hand side of (4.120) can be reduced as follows:

$$\begin{aligned}
& \text{mapd} (\text{ren}_{\text{loc}} (+d) \odot d') (\text{sub}_{\text{loc}} \tau (d', n')) \\
= & \text{mapd} (\text{ren}_{\text{loc}} (+d) \odot d') (\text{rename} (+d') (\tau(n' - d'))) && \text{Lem. 4.4.7} \\
= & (\text{rename} (+d + d') (\tau(n' - d'))) && \text{Lem. 4.4.10}
\end{aligned}$$

This establishes (4.120), which completes the proof. $\square$

4.5 LOCALLY NAMELESS SUBSTITUTION

In this section we provide generic implementations of the substitution-related operations of locally nameless, first defined in Section 2.3. We let continue to T be a monad decorated by $\langle \mathbb{N}, +, 0 \rangle$, but specialize the set of variables to LN (Sec. 3.3), the disjoint union of bound variables represented by de Bruijn indices ($\text{Bd } n$) or free variables represented by atoms ($\text{Fr } x$). The only assumption made of atom is that it comes equipped with a decidable equality, so we can test if two names are equal.

The first operation we consider, the substitution of free variables, is virtually identical to the naïve substitution operation defined for arbitrary monads (Sec. 4.2.5).

Definition 4.5.1. (🎮) *Locally nameless substitution is defined as follows:*

$$\begin{aligned} \text{subst}^T x u t &\equiv \text{bind}^T (\text{subst}_{\text{loc}} x u) t \\ \text{subst}_{\text{loc}} x u v &\equiv \begin{cases} u & \text{if } v = \text{Fr } x \\ \text{ret } v & \text{else} \end{cases} \end{aligned} \quad (4.121)$$

Definition 4.5.2. (🎮) *The locally nameless operation open is defined as follows:*

$$\begin{aligned} \text{open } u t &\equiv \text{bindd}^T (\text{open}_{\text{loc}} u) t \\ \text{open}_{\text{loc}} u (d, v) &\equiv \begin{cases} u & \text{if } v = \text{Bd } d \\ \text{var } (\text{Bd } (n - 1)) & \text{if } v = \text{Bd } n, n > d \\ \text{ret } v & \text{else} \end{cases} \end{aligned} \quad (4.122)$$

Note that the definition of $\text{open}_{\text{loc}} u (d, v)$ is directly lifted from the var cases of the definition given for $\{d \rightarrow u\} t$ in Section 2.3. Likewise, we can read the var cases for $\backslash^x t$ to define close^T as follows.

Definition 4.5.3. (🎮) *The locally nameless operation close is defined as follows:*

$$\begin{aligned} \text{close } x t &\equiv \text{mapd}^T (\text{close}_{\text{loc}} x) t \\ \text{close}_{\text{loc}} x (d, v) &\equiv \begin{cases} \text{Bd } d & \text{if } v = \text{Fr } x \\ \text{Bd } (n + 1) & \text{if } v = \text{Bd } n, n \geq d \\ v & \text{else} \end{cases} \end{aligned} \quad (4.123)$$

The decorated monad laws allow us to prove a basic property, the first one listed in Figure 2.1.

Theorem 4.5.4 (🎮 SUBST_SPEC). *The following equation decomposes substitution in terms of opening and closing.*

$$t[x \mapsto u] = \left(\backslash^x t \right)^u$$

Proof. The notation desugars to the following goal:

$$\text{bind}^T (\text{subst}_{\text{loc}} x u) t = \text{bindd}^T (\text{open}_{\text{loc}} u) \left(\text{mapd}^T (\text{close}_{\text{loc}} x) t \right)$$

First we consider the proof intuitively by considering the effect of the left and right operations on an individual occurrence of a variable v in t , aiming to show that both sides have the same effect on all occurrences.

- If v is $\text{Fr } x$, $\text{subst}_{\text{loc}} x u$ replaces it with u . On the right, $\text{close}_{\text{loc}} x$ replaces v with $\text{Bd } d$ —where d is the binding context of v —representing the first unbound de Bruijn index. The subsequent $\text{open}_{\text{loc}} u$, finding $\text{Bd } d$ at depth d , replaces this value with u .
- If v is not $\text{Fr } x$, $\text{subst}_{\text{loc}} x u$ leaves the occurrence alone. On the right, since $v \neq \text{Fr } x$, there are two possible cases.
 - If v is $\text{Bd } n$ and $n \geq d$, $\text{close}_{\text{loc}} x$ replaces v with $\text{Bd } (n + 1)$. Since $n + 1 > d$, the subsequent $\text{open}_{\text{loc}} u$ replaces it with $\text{var } (\text{Bd } (n + 1 - 1))$, leaving it unchanged.
 - Otherwise, $\text{close}_{\text{loc}} x$ leaves v alone. Since we know $v \neq \text{Bd } d$ and $v \neq \text{Bd } n$ where $n > d$, the subsequent $\text{open}_{\text{loc}} u$ replaces v with $\text{var } v$, leaving it unchanged.

Now we proceed with the formal argument. By (4.107), the right-hand side is equal to

$$\text{bindd}^T ((d, v) \mapsto \text{open}_{\text{loc}} u (d, \text{close}_{\text{loc}} x(d, v)))$$

Therefore, it suffices to show the following equality for all contexts d and variables v .

$$\text{subst}_{\text{loc}} x u (d, v) = \text{open}_{\text{loc}} u (d, \text{close}_{\text{loc}} x(d, v))$$

This proof proceeds by case analysis on d and v —it is precisely the argument given above. $\square$

Unfortunately, this is about as much mileage as we can get out of the decorated monad abstraction by itself. We will resume developing locally nameless metatheory in Section 5.2 after developing additional theoretical machinery.

4.6 NOMINAL SUBSTITUTION

We now turn our attention to capture-avoiding substitution for the nominal lambda calculus. To incorporate binder labels, the syntax is now a bifunctor, a functor of two arguments:

$$\text{nlam} : \text{SET} \rightarrow \text{SET} \rightarrow \text{SET}$$

Here, $\text{nlam } B V$ is the set of all abstract syntax trees for the lambda calculus where binder labels are drawn from B and variables are drawn from V . For the nominal backend, both B and V are instantiated to the set atom of primitive variable names. Depending on the context, it can be useful to characterize nlam as either a functor of two arguments, or as two related instances of a single-argument functor.

Definition 4.6.1. () A bifunctor $F: \text{SET} \rightarrow \text{SET} \rightarrow \text{SET}$ is a type constructor of two arguments paired with an operation map_2^F of the following type:

$$\text{map}_2^F: \forall (B B' V V': \text{SET}), (B \rightarrow B') \rightarrow (V \rightarrow V') \rightarrow F B V \rightarrow F B' V'$$

This operation is subject to obvious generalizations of (4.21) and (4.22).

Example 4.6.2. () niam is a two-argument functor where the functor instance in the B argument applies a function to each binder label:

$$\begin{aligned} \text{map}_2^{\text{niam}} g f (\text{var } v) &\equiv \text{var } (f v) \\ \text{map}_2^{\text{niam}} g f (\text{abs } b t) &\equiv \text{abs } (g b) (\text{map}_2^{\text{niam}} g f t) \\ \text{map}_2^{\text{niam}} g f (\text{app } t_1 t_2) &\equiv \text{app } (\text{map}_2^{\text{niam}} g f t_1) (\text{map}_2^{\text{niam}} g f t_2) \end{aligned}$$

Lemma 4.6.3. () A bifunctor can equivalently be described in terms of two operations

$$\begin{aligned} \text{bmap}^F: \forall (V B B': \text{SET}), (B \rightarrow B') \rightarrow F B V \rightarrow F B' V \\ \text{vmap}^F: \forall (B V V': \text{SET}), (V \rightarrow V') \rightarrow F B V \rightarrow F B V' \end{aligned}$$

that both satisfy the functor laws and, moreover, commute with each other.

By virtue of Lemma 4.6.3, we will sometimes examine niam in terms of map_2 and sometimes in terms of bmap and vmap , depending on which is more suitable. In some respects it is easier to specify niam as a two-argument functor because this involves fewer operations. However, viewing niam as a family of monads indexed by a family of homomorphisms has a more transparent relationship with the theory developed so far because it simply adds a new operation (bmap) to the existing theory. For niam , Lemma 4.6.3 yields two mapping operations respectively targeting the binder labels or the variables:

$$\text{bmap}^{\text{niam}} g \equiv \text{map}_2^{\text{niam}} g \text{id} \quad (4.124)$$

$$\text{vmap}^{\text{niam}} f \equiv \text{map}_2^{\text{niam}} \text{id } f \quad (4.125)$$

Here, $\text{vmap}^{\text{niam}}$ is the same map operation we considered in the previous sections, given a new name to emphasize that it maps over variables. The new operation is $\text{bmap}^{\text{niam}}$. These operations commute:

$$\text{bmap } g \circ \text{vmap } f = \text{vmap } f \circ \text{bmap } g \quad (4.126)$$

Equation (4.126) can be read as stating both that $\text{bmap } g: \text{niam } B \Rightarrow \text{niam } B'$ is a natural transformation between the functors $\text{niam } B$ and $\text{niam } B'$ and that $\text{vmap } f: \text{niam } B V \Rightarrow \text{niam } B V'$ is a natural transformation between $\text{niam } (-) V$ and $\text{niam } (-) V'$. By the decoration in Example 4.3.20

and Theorem 4.3.44, for any $B \in \text{SET}$, $\text{nlam } B$ forms a monad decorated with respect to the free monoid $\langle \text{list } B, [], ++ \rangle$.

Lemma 4.6.4. *For any $g: B \rightarrow B'$, $\text{bmap}^{\text{nlam}} g$ is a monad homomorphism between $\text{nlam } B$ and $\text{nlam } B'$.*

Proof. A simple proof by induction on nlam . □

Thus, the bifunctorial structure of nlam reveals a family of decorated monads for different values of B , and bmap defines a class of homomorphisms between the members of this family. This notion of “bifunctorial monad” is codified by the following unsurprising definition.

Definition 4.6.5. (🐼) *A parameterized family of monads is a bifunctor equipped with operations of the following types:*

$$\text{ret}^T : \forall (B, V : \text{SET}), V \rightarrow TBV$$

$$\text{join}^T : \forall (B, V : \text{SET}), TB(TBV) \rightarrow TBV$$

These operations are subject to the following conditions. First, for all B , ret^T and join^T must form a monad where vmap provides the functor structure on TB . These operations must be natural transformations in this sense:

$$\text{map}_2 g f \circ \text{ret}^T = \text{ret}^T \circ f \tag{4.127}$$

$$\text{map}_2 g f \circ \text{join}^T = \text{join}^T \circ \text{map}_2 g (\text{map}_2 g f) \tag{4.128}$$

Equivalently, for any $g: B \rightarrow B'$, $\text{bmap } g$ must form a monad homomorphism.

4.6.1 Nominal Decoration

We turn our attention to the question of how $\text{bmap } g$ interacts with the decoration on nlam . The first law is unsurprising: applying a function $g: B \rightarrow B'$ to each binder, followed by applying $\text{dec}^{\text{nlam } B'}$, is equivalent to applying $\text{dec}^{\text{nlam } B}$ first, followed by applying g to *every* binder value, including the values in the leaves resulting from the previous decoration:

Lemma 4.6.6. *For any $g: B \rightarrow B'$, $\text{bmap}^{\text{nlam}} g$ and dec^{nlam} have the following relation:*

$\text{nlam } B$

$\text{bmap } g$

$\text{nlam } B'$

=

$\text{nlam } B$

$\text{bmap } g$

$(\text{list } g)^\times$

$\text{nlam } B'$

$\text{list } B'$

$$\text{dec}^{\text{nlam } B'} \circ \text{bmap } g = \text{bmap } g \circ \text{vmap} \left(\text{map}^{\text{list}} g \times \text{id} \right) \circ \text{dec}^{\text{nlam } B} \tag{4.129}$$

The above equation is an instance of a slightly more general equation that characterizes the sense in which dec^{nlam} is a natural transformation:

$$\text{dec}^{\text{nlam } B'} \circ \text{map}_2 g f = \text{map}_2 g \left(\text{map}^{\text{list}} g \times f \right) \circ \text{dec}^{\text{nlam } B} \quad (4.130)$$

Equation (4.129) nearly states that $\text{bmap } g$ is a homomorphism in the sense of Definition 4.3.21. Making this precise requires a slightly more general notion of homomorphism between functors decorated by two different sets ($\text{list } B$ and $\text{list } B'$), where an extra operation is supplied ($\text{map}^{\text{list}} g$) that mediates between the decorations.

Definition 4.6.7. (🚗) *Let E_1 and E_2 be two sets and suppose $\langle F, \text{dec}^F \rangle$ is a functor decorated by E_1 and $\langle G, \text{dec}^G \rangle$ is decorated by E_2 . A heterogeneous homomorphism mediated by $f: E_1 \rightarrow E_2$ is a natural transformation $\phi: F \Rightarrow G$ that commutes with the decorations of F and G in this sense:*

$$\text{dec}^G \circ \phi = \phi \circ \text{map}^F f^\times \circ \text{dec}^F \quad (4.131)$$

Thus, Lemma 4.6.6 states that for each $g: B \rightarrow B'$, $\text{bmap}^{\text{nlam}} g$ is a heterogeneous homomorphism between the decorated functors $\text{nlam } B$ and $\text{nlam } B'$ mediated by $\text{map}^{\text{list}} g$. If this were the only additional machinery needed to reason about nominal syntax, then the theory would not change very much: $\text{bmap } g$ can be used to rename binders when needed, and this operation has a straightforward relationship with the decorated monadic structure on nlam . Namely, Definition 4.6.5 characterizes the relationship between bmap and the monadic structure, while Lemma 4.6.6 characterizes its relationship with the decorated structure.

Unfortunately, one part of the picture is missing: nlam can also be seen as a decorated functor *in the B parameter*, and this structure must be made explicit in order to rename binders correctly because binder-renaming is itself a context-sensitive operation. Consider the capture-avoiding nominal substitution operation subst in Section 2.1, particularly in terms of the substAvoid operation that renames each binder so as to avoid an expanding set of conflicting names. When a binder is renamed to a fresh value, the freshen operation requires contextual information about the binder itself, namely the current state of the avoid set, in order to ensure it is not renamed in a way that conflicts with earlier renaming higher in the syntax tree. So, we must know which binders are in scope at which binders. We now turn our attention to this additional structure.

Two-Argument Decorations

Definition 4.6.8. (🚗) *The following operation is the bifunctorial generalization of the shift function (Def. 4.3.32), where W is any monoid:*

$$\begin{aligned} \text{shift}_2^F &: W \times F(W \times B)(W \times V) \rightarrow F(W \times B)(W \times V) \\ \text{shift}_2^F(w, t) &\equiv \text{map}_2^F(\lambda(w', b). (w \cdot w', b)) (\lambda(w', v). (w \cdot w', v)) \end{aligned}$$

The following is a two-argument generalization of dec^{nlam} (Ex. 4.3.20), modified to annotate not just variables but binder occurrences with information about binders higher in the syntax tree.

Example 4.6.9. (🚗) *The following operation decorates nlam in both parameters simultaneously:*

$$\begin{aligned} \text{dec}^{\text{nlam}} : \text{nlam } B \ V &\rightarrow \text{nlam } (\text{list } B \times B) (\text{list } B \times V) \\ \text{dec}_2^{\text{nlam}} (\text{var } v) &\equiv \text{var } ([], v) \\ \text{dec}_2^{\text{nlam}} (\text{abs } b \ t) &\equiv \text{abs } ([], b) \left(\text{shift}_2 \left([b], \text{dec}_2^{\text{nlam}} t \right) \right) \\ \text{dec}_2^{\text{nlam}} (\text{app } t_1 \ t_2) &\equiv \text{app } \left(\text{dec}_2^{\text{nlam}} t_1 \right) \left(\text{dec}_2^{\text{nlam}} t_2 \right) \end{aligned}$$

Much like the two-argument map_2 operation, $\text{dec}_2^{\text{nlam}}$ can be decomposed into two functions, one that decorates binders and one that decorates variables:

$$\begin{aligned} \text{bdec}^{\text{nlam}} : \text{nlam } B \ V &\rightarrow \text{nlam } (\text{list } B \times B) \ V \\ \text{vdec}^{\text{nlam}} : \text{nlam } B \ V &\rightarrow \text{nlam } B (\text{list } B \times V) \end{aligned}$$

The vdec operation is precisely the decoration considered earlier witnessing $\text{nlam } B$ as a decorated functor, with a new name to emphasize that it decorates variables. The new operation is bmap . The two operations have the following relation to $\text{dec}_2^{\text{nlam}}$:

$$\text{dec}_2^{\text{nlam}} = \text{bdec}^{\text{nlam}} \circ \text{vdec}^{\text{nlam}}$$

As with $\text{map}_2/\text{bmap}/\text{vmap}$, we can analyze nlam either in terms of the two-argument decoration, or by examining the laws satisfied by the new operation bdec . We shall take the latter approach and focus on how this operation interacts with the others, namely vdec , vmap , bmap , ret and join .

One might conjecture that $\text{bdec}^{\text{nlam}}$ equips nlam with the structure of a decorated functor in the B parameter, where the decorating monoid is the free monoid $\text{list } B$. Would that it were so. But this is not the case: the operation does not fit Definition 4.3.14, because the monoid with respect to which the functor is decorated is not constant but *determined by B itself*. Consider that if nlam were decorated by $\text{list } B$, applying bdec twice would yield something of the following type:

$$\text{nlam } (\text{list } B \times \text{list } B \times B) \ V$$

However, the first application of bdec changes the type parameter from B to $\text{list } B \times B$. Therefore, applying bdec twice actually results in this type:

$$\text{nlam } (\text{list } (\text{list } B \times B) \times \text{list } B \times B) \ V$$

This shows that the theory must be generalized in a fundamental way. To develop this more general theory, we introduce a new fundamental operation: the *prefix decoration* on lists.

Definition 4.6.10. (🐼) *The list prefix decoration is an operation*

$$\text{dec}^{\text{pre}} : \text{list } B \rightarrow \text{list } (\text{list } B \times B)$$

that pairs each element of a list with the prefix of that element. This is formally defined as follows, where f is a helper function:

$$\begin{aligned} \text{dec}^{\text{pre}} l &\equiv f [] l \\ f p [] &\equiv [] \\ f p (\text{cons } x l) &\equiv \text{cons } (p, x) (f (p ++ [x]) l) \end{aligned}$$

This operation is much like the list index decoration in Example 4.3.16, but with a fundamental difference: the index decoration annotates each element with a constant value, namely its index in the list, which is independent of the functor itself, i.e. unchanged by map^{list} . The prefix decoration, by contrast, is entangled with map^{list} . In terms made more precise in Chapter 6, the index decoration is determined by the *shape* of the list, while the prefix decoration is determined by its *contents*. Thus, dec^{pre} does not form a decorated functor according to Definition 4.3.14 for the same reason that $\text{bdec}^{\text{nlam}}$ does not: the type of the contexts depends on the type parameter to list.

Fortunately, one does not have to throw the baby out with the bath water: there is still a comonad here. To define the comonad, we first note that dec^{pre} satisfies the following two laws, which can be discerned as analogues of Eqns. (4.76) and (4.77).

$$\text{map}^{\text{list}} (\text{extract}^{\text{list } B \times B}) \circ \text{dec}_B^{\text{pre}} = \text{id}_{\text{list } B} \quad (4.132)$$

$$\text{dec}_{\text{list } B \times B}^{\text{pre}} \circ \text{dec}_B^{\text{pre}} = \text{map}^{\text{list}} ((l, b) \mapsto (\text{dec}_B^{\text{pre}} l, l, b)) \circ \text{dec}_B^{\text{pre}} \quad (4.133)$$

The first equation is obvious: pairing each element with its prefix, and then discarding that prefix, leaves the list unchanged. The second law has the following interpretation. On the left, the prefix decoration is applied twice. The first application pairs each element with its prefix, and the second pairs each (prefix, element) pair with *its* prefix in the list of (prefix, element) pairs. On the right, after the first application of dec^{pre} , each (prefix, element) pair is paired with a value that is computed by pairing each element in the prefix with its prefix in that list. The takeaway from (4.133) is that the prefix of an element provides enough information to compute the prefix of each element in the prefix itself, too. To put this idea on formal ground, we observe that $\text{list } B \times B$ forms a comonad.

Definition 4.6.11. (👉) *The functor $Z: \text{SET} \rightarrow \text{SET}$ has the following action on sets:*

$$ZB \equiv \text{list } B \times B$$

The action on morphisms $\text{map}^Z f \equiv \text{map}^{\text{list}} f \times f$ applies a function to each element.

Remark 4.6.12. *The name Z was chosen simply because it does not conflict with other names used in this document, and a one-character name is convenient in string-diagrammatic reasoning. L would be an obvious choice too, but that name is used slightly differently below.*

Lemma 4.6.13. (👉) *$Z: \text{SET} \rightarrow \text{SET}$ forms a comonad, where the two operations*

$$\text{extract}^Z: \forall (B: \text{SET}), \text{list } B \times B \rightarrow B$$

$$\text{cojoin}^Z: \forall (B: \text{SET}), \text{list } B \times B \rightarrow \text{list } (\text{list } B \times B) \times \text{list } B \times B$$

are defined as follows:

$$\text{extract}^Z(l, b) \equiv b$$

$$\text{cojoin}^Z(l, b) \equiv (\text{dec}^{\text{pre}} l, l, b)$$

The comonad laws are straightforward to verify using Equations (4.132) and (4.133). With the comonad operations defined, those same equations can be expressed in the following form:

$$\begin{aligned} \text{map}^{\text{list}} (\text{extract}^Z) \circ \text{dec}_B^{\text{pre}} &= \text{id}_{\text{list } B} \\ \text{dec}_{\text{list } B \times B}^{\text{pre}} \circ \text{dec}_B^{\text{pre}} &= \text{map}^{\text{list}} \text{cojoin}^Z \circ \text{dec}_B^{\text{pre}} \end{aligned}$$

These equations, and a straightforward equation stating that dec^{pre} is a natural transformation of type $\text{list} \Rightarrow \text{list} \circ Z$, yield the following specification of the prefix decoration.

Lemma 4.6.14. (👉) *The prefix decoration on lists (Def. 4.6.10) constitutes a right co-action of Z on list .*

Note there is a slight conceptual circularity here where the comonadic structure of Z is defined in terms its coaction on what turns out to be a right comodule of that same comonad.

The fact that list is decorated by a comonad Z makes list a decorated functor in a more general sense. At this point, “decoration” is mildly overloaded as the object with respect to which a functor is decorated may be a set (Def. 4.3.14), a monoid (Def. 4.3.37), or a comonad. The first two meanings are also right coactions of a comonad, but they make specific assumptions about particular form of the comonad. As with the ordinary mapd operation on a decorated functor (Def. 4.3.26), we can use the coaction of Z on list to lift prefix-sensitive maps over lists.

Definition 4.6.15. (🐼) Composing dec^{pre} with map^{list} leads to the following operation and laws:

$$\begin{aligned} \text{mapd}^{\text{pre}} &: (\text{list } A \times A \rightarrow B) \rightarrow \text{list } A \rightarrow \text{list } B \\ \text{mapd}^{\text{pre}} \text{extract}^Z l &= l \end{aligned} \quad (4.134)$$

$$\text{mapd}^{\text{pre}} g \circ \text{mapd}^{\text{pre}} f = \text{mapd}^{\text{pre}} (g \circ \text{cobind}^Z f) \quad (4.135)$$

We are now in a position to spell out the structure of niam as a decorated functor in its first argument.

Lemma 4.6.16. For any V , bdec forms a coaction of the Z comonad on the functor $\text{niam}(-) V$.

It remains to investigate how bdec interacts with vmap , vdec , ret , and join . We will state these interactions without explicit proofs, which are straightforward. Regarding maps over variables, bdec and vmap commute, as they modify niam in different arguments independently of each other.

Lemma 4.6.17. $\text{bdec}^{\text{niam}}$ and $\text{vmap}^{\text{niam}} f$ commute for all $f: V \rightarrow V'$:

$$\text{bdec} \circ \text{vmap } f = \text{vmap } f \circ \text{bdec} \quad (4.136)$$

□

Regarding decoration, we consider the effect of decorating in V after decorating in B . The following property states that we can decorate in variables first, and then apply the prefix decoration to the contexts at the variables, and then decorate in the binders.

Lemma 4.6.18. bdec forms a heterogeneous homomorphism (Def. 4.6.7) between $\text{niam}B$ and $\text{niam}(ZB)$ mediated by dec^{pre} :

$$\text{vdec} \circ \text{bdec} = \text{bdec} \circ \text{vmap} (\text{dec}^{\text{pre}} \times \text{id}_V) \circ \text{vdec} \quad (4.137)$$

□

Lemma 4.6.19. Because there are no binders in atomic expressions, bdec has no effect on ret except to change the binder parameter to niam :

$$\text{bdec} \circ \text{ret}^{\text{niam}B} = \text{ret}^{\text{niam}ZB} \quad (4.138)$$

□

The relationship between bdec and join follows the same structure as Equation (4.90). Given a term of type $TB(TBV)$, we decorate in the binder arguments in both layers of T , as well as the outer layer of variables, to obtain something of type

$$T(ZB)(\text{list}B \times T(ZB)V).$$

Then the list B is added to the inner layer using shift , yielding $T(ZB)(T(ZB)V)$, which is then joined. Or, we can join the original term and then decorate.

Lemma 4.6.20. $\text{bdec} \circ \text{join}$ are related by the following equation:

$$\text{bdec} \circ \text{join} = \text{join} \circ \text{bdec} \circ \text{mapd}^{\text{n} \text{lam} B} \left((l, t) \mapsto \text{shift}^{\text{n} \text{lam} (-)^V} (l, \text{bdec } t) \right) \quad (4.139)$$

Now we consider the combined operation dec_2 that decorates in both binders and variables simultaneously. For this, we need the following structure, which is early equal to the comonad Z , but rather than considering pairs $\text{list } B \times B$, we allow the right component to have a type other than B (this other parameter will be the type of variables, which may or may not equal B in practice, but this extra parameterization is necessary to define the appropriate axioms of the underlying structure).

Definition 4.6.21. (🚗) $L: \text{SET} \rightarrow \text{SET} \rightarrow \text{SET}$ is a bifunctor defined

$$L B V \equiv \text{list } B \times V$$

The operation cojoin^L is defined as follows:

$$\begin{aligned} \text{cojoin}^L &: L B V \rightarrow L(ZB)(LBV) \\ \text{cojoin}^L(l, v) &= (\text{dec}^{\text{pre}} l, l, v) \end{aligned}$$

Note that cojoin^L is a mild abuse of notation: L is a bifunctor, so it does have the right form to be a comonad, though morally it is basically the same as Z .

Definition 4.6.22. (🚗) A parametrically decorated functor is a bifunctor $F: \text{SET} \rightarrow \text{SET} \rightarrow \text{SET}$ equipped with an operation dec_2 satisfying the following laws:

$$\begin{aligned} \text{dec}_2 &: F B V \rightarrow F(\text{list } B \times B)(\text{list } B \times V) \\ \text{dec}_2 \circ \text{map}_2 g f &= \text{map}_2 \left(\text{map}^{\text{list}} g \times g \right) \left(\text{map}^{\text{list}} g \times f \right) \circ \text{dec}_2 \end{aligned} \quad (4.140)$$

$$\text{dec}_2 \circ \text{dec}_2 = \text{map}_2 \text{cojoin}^Z \text{cojoin}^L \circ \text{dec}_2 \quad (4.141)$$

$$\text{map}_2 \left(\text{extract}^Z \right) \left(\text{extract}^{\text{list } B \times} \right) \circ \text{dec}_2 = \text{id} \quad (4.142)$$

Definition 4.6.23. (🚗) A parametrically decorated monad has the structure of both a parameterized family of monads (Def. 4.6.5) and a parametrically decorated functor (Def. 4.6.22) satisfying the following additional laws, which generalize Eqns. (4.89) and (4.90):

$$\text{dec}_2 \circ \text{ret}^{\text{n} \text{lam} B} = \text{ret}^{\text{n} \text{lam} (ZB)} \circ \text{ret}^{(\text{list } B)^\times} \quad (4.143)$$

$$\text{dec}_2 \circ \text{join}^{\text{n} \text{lam} B} = \text{join}^{\text{n} \text{lam} (ZB)} \circ \text{vmap} \left(\text{shift}_2 \circ \text{map}^{(\text{list } B)^\times} \text{dec}_2 \right) \circ \text{dec}_2 \quad (4.144)$$

Example 4.6.24. () nlam is a parametrically decorated monad.

Definition 4.6.25. () A parameterized decorated monad presented as an extension system is equipped with the following operations (implicitly polymorphic in all B and V).

$$\begin{aligned} \text{ret}^T &: V \rightarrow T B V \\ \text{bindd}_2 T &: (\text{list } B_1 \times B_1 \rightarrow B_2) \\ &\rightarrow (\text{list } B_1 \times V_1 \rightarrow T B_2 V_2) \\ &\rightarrow T B_1 V_1 \rightarrow T B_2 V_2 \end{aligned}$$

These operations are subject to the following three laws:

$$\text{bindd}_2 \rho \sigma \circ \text{ret}^{TB} = \sigma \circ \text{ret}^{(\text{list } B)} \quad (4.145)$$

$$\text{bindd}_2 (\text{extract}) \left(\text{ret}^{TB} \circ \text{extract} \right) = \text{id} \quad (4.146)$$

$$\text{bindd}_2 \rho_2 \sigma_2 \circ \text{bindd}_2 \rho_1 \sigma_1 = \text{bindd}_2 \left(\rho_2 \circ \text{cobind}^Z \rho_1 \right) (\star (\rho_2, \sigma_2, \rho_1, \sigma_1)) \quad (4.147)$$

In (4.147), the generalized Kleisli composition operation has the following definition:

$$\begin{aligned} \star (\rho_2, \sigma_2, \rho_1 \sigma_1) &: \text{list } B_1 \times V_1 \rightarrow T B_3 V_3 \\ \star (\rho_2, \sigma_2, \rho_1 \sigma_1) (l, v) &\equiv \text{bindd}_2 (\rho_2 \odot \text{mapd}^{\text{pre}} \rho_1 l) (\sigma_2 \odot \text{mapd}^{\text{pre}} \rho_1 l) (\sigma_1 (l, v)) \end{aligned} \quad (4.148)$$

Lemma 4.6.26. () Any parameterized decorated monad has a presentation as an extension system by the following definition:

$$\text{bindd}_2 g f \equiv \text{join} \circ \text{map}_2 g f \circ \text{dec}_2 \quad (4.149)$$

The operation bindd_2 simultaneously renames binders and substitutes variables, and is able to read the binding context of both types of parameters. Note that bindd_2 can be decomposed into two functions that rename binders and substitute variables separately:

$$\begin{aligned} \text{bmapd} &: (\text{list } B_1 \rightarrow B_2) \rightarrow T B_1 V_2 \rightarrow T B_2 V_2 \\ \text{vbindd} &: (\text{list } B_1 \rightarrow T B_2 V_2) \rightarrow T B_1 V_1 \rightarrow T B_1 V_1 \end{aligned}$$

The function bmapd is the composition of bmap and bdec (V_2 is an arbitrary type argument), while vbindd is nothing but the ordinary bindd^{TB_1} operation where B_1 is simply any parameter of T as a bifunctor. The bindd_2 operation is recovered as the composition

$$\text{bindd}_2 \rho \sigma = \text{bmapd } \rho \circ \text{vbindd } \sigma$$

This decomposition is inefficient—it performs in two recursions what can be performed in one—

though it clarifies the nature of the abstraction: T is a bifunctor, $T B$ is a DTM decorated by list B where B is an arbitrary parameter, and we have effectively added a new operation, bmapd , that performs context-sensitive renaming on the first type argument to T , representing binders. bmapd does not technically form a decorated functor in the sense we have defined, but it “decorated” in the sense forms a right comodule of the Z comonad, rather than something of the form $W^\times$.

bindd_2 is the operation that we will use to define capture-avoiding substitution, after solving one more basic problem: when renaming bound variables, it is not just the binding *context* we need—information about what binders were named in the original *input*, which corresponds to the list B_1 terms in the type of bindd —but also the *history* of decisions we made about what their names will be in the *output*, which has type list B_2 .

4.6.2 Hereditary Freshening

We have developed almost enough machinery to define capture-avoiding substitution. However, one more step is needed. Consider the operation substAvoid from Section 2.1. In this function, the value tracked during recursion is not the binding *context*, i.e. the set of binder labels passed under so far, as would be computed by the bdec and vdec operations. Rather, the value tracked is an accumulating set containing, in essence, the *history* of fresh names given to those binders using the *freshen* operation. That is, as binders are renamed, lower binders must know which names to avoid during freshening. These are the precisely the names of binders higher in the syntax tree in the final *output* of capture-avoiding substitution. The decorations annotate binders and variables with information about the input, which does not by itself tell us which names to avoid.

This is not a problem—at each binder, we simply use the context to compute the current state of the avoid set by “replaying” the process of freshening each name. We now consider some of the infrastructure we need to make this work.

Definition 4.6.27. (🐼) *The map with history operation on lists, hmap , is defined as follows:*

$$\text{hmap}: (\text{list } B \times A \rightarrow B) \rightarrow \text{list } A \rightarrow \text{list } B$$

$$\text{hmap } f [] = [] \tag{4.150}$$

$$\text{hmap } f (\text{cons } a l) = \text{cons } (f ([], a)) (\text{hmap } (f \odot (f ([], a))) l) \tag{4.151}$$

This operation is similar to map^{list} , applying a function f to each element in a list. However, it introduces an additional layer of generality: the mapped function can also access the prefix of the output computed so far. This operation is similar to the mapd^{pre} operation given in Definition 4.6.15, except that the latter operation provides the prefix of the input list, while hmap provides f with the prefix of the output.

We will use the hmap operation to iterate over a list of variables produce a new list of the

same length, with the specification that the final output has no duplicated entries. There are many ways to meet this specification—we could always make an up entirely new list—but ideally the final output will have some relation to the original input. Our implementation uses the following operation.

Definition 4.6.28. (🔒) *The freshening operation has the following type:*

$$\text{freshen} : \text{list atom} \rightarrow \text{atom} \rightarrow \text{atom}$$

The definition of this operation is not as important as its specification:

$$\forall l v, (\text{freshen } l v) \notin l \quad (4.152)$$

Under the hood, `freshen` is implemented using the fact that atoms are implemented as natural numbers. The operation returns v if $v \notin l$. Otherwise, it returns the minimum natural number that is not in l .

Now we consider how `hmap` and `freshen` can be used to rename bound variables. In `substAvoid`, we start with a top-level set of names we wish to avoid during binder renaming. This value is initially set to the union of three sets: $\{x\}$, where x is the name of free variable replaced by substitution; $\mathbf{fv } t$, where t is the term in which we perform substitution; and $\mathbf{fv } u$, where u is the term substituted for free occurrences of x . Let us call this set Γ . We shall consider an operation iterates over a list of binder names and replaces them all with fresh names. This operation has the following type:

$$\text{renbList} : \text{list atom} \rightarrow \text{list atom} \rightarrow \text{list atom}$$

The first argument will be Γ , while the second input is the list to be freshened. The specification we intend to meet is that for any l , `renbList` Γ l will be a list l' of the same size as l , where each name in l' is unique (no duplicates), and none of the elements in l' occur in Γ .

Definition 4.6.29. (🔒) *The local binder freshening operation is defined as follows:*

$$\begin{aligned} \text{renb} : \text{list atom} \rightarrow \text{list atom} \times \text{atom} \rightarrow \text{atom} \\ \text{renb } \Gamma (h, a) = \text{freshen } (\Gamma ++ h) a \end{aligned} \quad (4.153)$$

Now `renbList` can be defined using `hmap` to map `renb` over its input.

Definition 4.6.30. (🔒) *The binder freshening operation is defined as follows:*

$$\begin{aligned} \text{renbList} : \text{list atom} \rightarrow \text{list atom} \rightarrow \text{list atom} \\ \text{renbList } \Gamma = \text{hmap } (\text{renb } \Gamma) \end{aligned} \quad (4.154)$$

Example 4.6.31. (👉) For example, if the input list is $[b_1, b_2, b_3]$, then $\text{renbList } \Gamma [b_1, b_2, b_3]$ returns

$$\begin{aligned} & [\text{freshen } \Gamma b_1, \\ & \text{freshen } (\Gamma ++ [\text{freshen } \Gamma b_1]) b_2, \\ & \text{freshen } (\Gamma ++ [\text{freshen } \Gamma b_1] ++ [\text{freshen } (\Gamma ++ [\text{freshen } \Gamma b_1]) b_2]) b_3] \end{aligned}$$

It is clear that renbList meets its intended specification—the presence of Γ in (4.153) ensures no value in the output is an element of Γ , while the use of the history ensures no name in the output is an element of its prefix in the output, i.e. all names are unique. The following lemma codifies the effect renbList from the perspective of an individual occurrence in the input list.

Lemma 4.6.32. (👉) Let $l = l_1 ++ [x] ++ l_2$. Then the following holds:

$$\begin{aligned} \text{renbList } \Gamma l &= \text{renbList } \Gamma l_1 \\ & ++ [\text{freshen } (\Gamma ++ \text{renbList } \Gamma l_1) x] \\ & ++ \text{renbList } (\Gamma ++ \text{renbList } \Gamma l_1 ++ [\text{freshen } (\Gamma ++ \text{renbList } \Gamma l_1) x]) l_2 \end{aligned} \tag{4.155}$$

We also remind the reader that renbList does not change the size of these lists. □

This specification in Lemma 4.6.32 has the following implications. First, x is renamed to something that does not conflict with Γ or with any name in its prefix in the final output list (or rather, the prefix of whatever new name x is given). Second, everything in l_2 is renamed to something that does not conflict with Γ , with the prefix of x in the output list, or with the new name given to x . We say that the input list has been *hereditarily freshened*.

The strategy to define nominal substitution is to hereditarily freshen binder values to ensure no variable capture is possible, while ensuring that all bound variables are given the same new name as whichever abstraction they were bound to originally. Free variables will be put into the top-level avoid set Γ to ensure this process does not introduce conflicts.

To hereditarily freshen the binders in a syntax tree, we would like to define an operation like renbList to rename binders for an arbitrary (parameterized) decorated monad. To a first approximation, this would be the operation $\text{bmapd}(\text{renb}\Gamma)$, where Γ is an initial set of variables names to avoid, representing a set of free variable names that we must not capture. This operation is well-typed but incorrect: bmapd is analogous to mapd^{pre} for lists, and we need an operation analogous to hmap . A simple *history adapter* function does the trick.

Definition 4.6.33. (👉) The following function converts a function that expects an output history in its first argument to an operation that expects the input context:

$$\begin{aligned} \text{hadapt} &: (\text{list } B \times A \rightarrow B) \rightarrow (\text{list } A \times A \rightarrow B) \\ \text{hadapt } f(l, a) &\equiv f(\text{hmap } f l, a) \end{aligned} \tag{4.156}$$

Lemma 4.6.34. (👉) *A simple induction on lists shows that hmap and mapd^{pre} have the following relation:*

$$\text{hmap } f = \text{mapd}^{\text{pre}} (\text{hadapt } f) \quad (4.157)$$

So, we simply apply the same trick to define the operation that hereditarily freshens *every* binder in a syntax tree, given an initial avoid set Γ . We use hadapt to convert renb , an operation expecting an output history and a binder label, into a renb_ctx , an operation expecting an input context and a binder label.

Definition 4.6.35. (👉) *The localized binder renaming operation, parameterized by a top-level avoid set Γ , is defined as follows:*

$$\begin{aligned} \text{renb_ctx} &: \text{list atom} \rightarrow \text{list atom} \times \text{atom} \rightarrow \text{atom} \\ \text{renb_ctx } \Gamma &= \text{hadapt } (\text{renb } \Gamma) \end{aligned} \quad (4.158)$$

We can use the bmapd operation to lift this operation over binder labels on a syntax tree, or rather a parameterized DTM T , whose type parameter is instantiated to atom :

$$\begin{aligned} \text{freshen_binders} &: \text{list atom} \rightarrow T \text{ atom } V \rightarrow T \text{ atom } V \\ \text{freshen_binders } \Gamma &= \text{bmapd } (\text{renb } \Gamma) \end{aligned}$$

This operation only modifies binders—it does not effect and is not effected by the second parameter to T . It is one-half of capture-avoiding substitution, and we will complete the definition below after explaining how to handle the second type parameter. Also, we will use bindd_2 instead of bmapd , as soon as we have defined the second argument to bindd_2 (the first will be $\text{renb } \Gamma$).

To be sure, this operation is almost comically inefficient. At each binder in a term t , the binding context of the occurrence is iterated over with hmap to hereditarily freshen the entire list; then renb is called to pick a fresh name for this individual occurrence. It would obviously be more efficient to maintain the avoid set as a stateful value rather than to recompute the current state all over again at each site. It may be fairly said that Tealeaves provides an *executable specification* of capture-avoiding substitution, rather than an operation optimized for practical use. As our intended users are formal metatheorists prototyping a programming language, the inefficiency is not likely to be a real barrier. Rewriting techniques, such as were considered by Bellegarde and Hook in their monadic approach to syntax [14] (which, as will be shown in Chapter 8, is a special case of ours) can be used in future work to extract efficient operations from those provided by Tealeaves.

4.6.3 Capture-Avoiding Substitution Definition

We have seen how to rename binders to fresh values. Now we consider the final component of capture-avoiding substitution, namely the operation that modifies variable occurrences. We need a small amount of infrastructure first.

Lemma 4.6.36. *Given any list l : list atom and v : atom, if $v \in l$, then l uniquely decomposes into the form*

$$l = l_{\text{pre}} ++ [a] ++ l_{\text{post}}$$

for a uniquely determined prefix l_{pre} and postfix l_{post} where $a \notin l_{\text{post}}$. □

Let a term t be a nominal lambda term—an element of nlam atom atom , or more generally $T \text{ atom atom}$ where T is parameterized decorated monad. Consider an occurrence $(l, v) \in_d t$, where l : list atom and v : atom. Then l is precisely the set of binders that occur above v in the syntax tree, with the head element being the abstraction highest in the tree, i.e. closest to the root node. The occurrence of v is bound if and only if $v \in l$, in which case, it is bound *to* the occurrence in v that is closest to v , i.e. the last occurrence of v in l (earlier occurrence of v , if there are any, are said to be *shadowed* by the later occurrence). This leads to a unique way of decomposing l , which we formally define now.

Definition 4.6.37. *Let $l, l_{\text{pre}}, l_{\text{post}}$: list atom and v : atom. The predicate *bound* is defined as follows:*

$$\text{bound } l \ l_{\text{pre}} \ v \ l_{\text{post}} \equiv (l = l_{\text{pre}} ++ [v] ++ l_{\text{post}} \wedge v \notin l_{\text{post}})$$

That is, $\text{bound } l \ l_{\text{pre}} \ v \ l_{\text{post}}$ holds if, in the context l , v is bound and l_{pre} is the set of binders higher in the syntax tree than the one v to which v is bound, while l_{post} is the set of binders lower than it. l_{pre} is said to be the binding prefix, while l_{post} is the binding postfix (or suffix).

By Lemma 4.6.36, for any variable v and context l , either $v \notin l$ or there are unique l_{pre} and l_{post} such that $\text{bound } l \ l_{\text{pre}} \ v \ l_{\text{post}}$. The strategy to define capture-avoiding substitution is simple in the free variable case: the interest is in the bound variable case, since bound variables (might) have to be renamed. If an occurrence of v is bound, then the abstraction to which it is bound is freshen $h \ v$, where h : list atom is the result of hereditarily freshening the set of binders higher in the syntax tree than the abstraction to which v is bound: that is, it is $\text{renb_ctx } l_{\text{pre}} \ v$. This dynamic is captured in the following definition.

Definition 4.6.38. (🐼) *Let Γ : list atom be an initial set of names to avoid during binder renaming. The*

localized capture-avoiding substitution operation is defined as follows:

$$\text{subst}_{\text{loc}} \Gamma \ x \ u \ (l, v) = \begin{cases} u & v \notin l \text{ and } v = x \\ \text{ret}^\Gamma v & v \notin l \text{ and } v \neq x \\ \text{ret}^\Gamma (\text{renb_ctx} \Gamma) (l_{\text{pre}}, v) & v \in l, \text{ bound } l \ l_{\text{pre}} \ v \ l_{\text{post}} \end{cases} \quad (4.159)$$

Let us consider the bound variable case. If $v \in l$, it has a uniquely determined binding prefix l_{pre} . Unfolding definitions, we have the following equation(s) to explain how bound variables are renamed by $\text{subst}_{\text{loc}}$:

$$\begin{aligned} & \text{renb_ctx} \Gamma (l_{\text{pre}}, v) \\ = & \text{hadapt} (\text{renb} \Gamma) (l_{\text{pre}}, v) && \text{Eqn. (??)} \\ = & \text{renb} \Gamma (\text{hmap} (\text{renb} \Gamma) l_{\text{pre}}, v) && \text{Eqn. (4.156)} \\ = & \text{freshen} (\Gamma ++ \text{hmap} (\text{renb} \Gamma) l_{\text{pre}}) v && \text{Eqn. (4.153)} \\ = & \text{freshen} (\Gamma ++ \text{renbList} \Gamma l_{\text{pre}}) v && \text{Eqn. (4.154)} \end{aligned}$$

This is how we rename the occurrence of a bound occurrence of v when it occurs as a bound variable—but it is *also* how we rename v when it occurs as a binder label with context l_{pre} . Thus, this operation will preserve the binding structure of the tree. Now let us put all of this together to define capture-avoiding substitution. We have not defined how to compute the set $\mathbf{fv} \ t$ of free variables in t . This operation is defined in Chapter 5. For now, suppose this operation is defined and has the expected properties.

Definition 4.6.39. () Let T be a parameterized decorated monad and let $x : \text{atom}$ and $t, u : T \text{ atom atom}$. The capture-avoiding substitution of u for free occurrences of x in t is defined as follows. First, define the top-level avoid set as follows:

$$\Gamma \equiv \mathbf{fv} \ t \cup \mathbf{fv} \ u$$

Substitution is then defined as follows:

$$\text{subst} \ x \ u \ t \equiv \text{bindd}_2 (\text{renb_ctx} \Gamma) (\text{subst}_{\text{loc}} \Gamma \ x \ u) \ t \quad (4.160)$$

We will now turn our attention to analyzing the behavior of this definition, which we will compare and contrast with substAvoid from Section 2.1.

4.6.4 Capture-Avoiding Substitution Correctness

Let us consider an intuitive argument for the correctness of this operation. While this particular argument is not formalized in Coq, it is emblematic of the sort of formalized syntactic reasoning

exhibited in Sections 5.2 and 5.5, as well as the proof that α -equivalence classes are in bijection with locally nameless terms (defined in Section 5.6, proved in Section 6.5). The distinguishing feature of this style of reasoning is that it is entirely *local*: we show the operation is correct by showing that the operation is correct at each variable occurrence.

Lemma 4.6.40. *The subst operation above is correct and does not lead to variable capture.*

Proof. Let $t : T$ atom atom be an abstract nominal term. Let v be a variable occurring in t in a context where $l : \text{list atom}$ is the list of binders above t in the syntax tree, where the head of the list is the outermost binder (highest in the tree) and the tail element is the innermost. We shall consider the effect of substitution on this particular occurrence and show that substitution does not cause any variable capture—i.e., free variables remain free, and bound variables remain bound to the same location. We proceed by case analysis on whether v occurs free or bound; that is, whether $v \in l$:

v is free, $v \neq x$: In this case, $v \in \mathbf{fv} t$, and $\text{subst}_{\text{loc}}$ replaces v with $\text{ret}^\top v$, meaning v is left alone.

Correctness in this case means show that this occurrence of v is not bound in the final term. After substitution, the new binding context of v is $\text{renbList } \Gamma l$. Since $v \in \mathbf{fv} t$ and $\mathbf{fv} t \subseteq \Gamma$ (treating lists as sets), this reduces to the straightforward verification that $\text{renbList } \Gamma l$ does not contain any elements of Γ . This is true because the effect of $\text{renbList } \Gamma$ on an element of $x \in l$ is to rename that element to something of the form $\text{freshen } (\Gamma ++ h) x$, which is never an element of $\Gamma ++ h$.

v is free, $v = x$: In this case, $\text{subst}_{\text{loc}}$ replaces v with u . Correctness means showing the free variables in u do not inadvertently become bound in the final term. (Note that the bound occurrences in u necessarily remain bound to their respective abstractions, because the semantics of binding is that variables are bound to the innermost abstraction sharing the same name.) As in the previous case, this holds because $\mathbf{fv} u$ is a subset of Γ , and $\text{renbList } \Gamma l$ does not include any of the elements in Γ .

v is bound: In this case, the context l uniquely decomposes into something of the form

$$l = l_{\text{pre}} ++ [v] ++ l_{\text{post}}$$

where $x \notin l_{\text{post}}$. Now, $\text{subst}_{\text{loc}}$ replaces v with $\text{hadapt } (\text{renb } \Gamma) (l_{\text{pre}}, v)$. Unfolding definitions reveals this value is

$$\text{freshen } (\Gamma ++ \text{renbList } \Gamma l_{\text{pre}})$$

Lemma 4.6.32 gives a specification of $\text{renbList } \Gamma (l_{\text{pre}} ++ [v] ++ l_{\text{post}})$ as

$$\text{renbList } \Gamma (l_{\text{pre}} ++ [v] ++ l_{\text{post}}) = l'_{\text{pre}} ++ [\text{freshen } (\Gamma ++ l'_{\text{pre}}) v] ++ l'_{\text{post}}$$

where l'_{pre} is equal to $\text{renbList } \Gamma l_{\text{pre}}$ and where l'_{post} does not contain $\text{freshen } (\Gamma ++ l'_{\text{pre}}) v$, which is the new name of binder v after hereditarily freshening all binders. We have just shown that $\text{subst}_{\text{loc}}$ also replaces v with this same value, so this occurrence will remain bound to the same location in its new binding context.

By demonstrating this correctness at an arbitrary occurrence in t , it follows that the entire operation is correct. $\square$

The reader may wonder why the top-level free variable set does not need to include $\{x\}$, the name of the variable being substituted (which may not be in $\mathbf{fv } t \cup \mathbf{fv } u$) as it did in the definition of substAvoid in Section 2.1. First, note that the operation just defined is not quite equal to substAvoid . One major difference is that substAvoid does not make any more recursive calls after encountering an abstraction with the name x . We do not attempt to mimic effect of this design decision in the definition given. As a result, the operation defined above will perform more renaming of bound variables than substAvoid does.

Consider the evaluation of the expression

$$\text{substAvoid } (\{x\} \cup \mathbf{fv } t \cup \mathbf{fv } u) x u (\lambda b.t)$$

for some choice of x , u , b , and t . Assume $x \notin \mathbf{fv } (\lambda b.t) \cup \mathbf{fv } u$. Of course, in this case, it would be correct to halt immediately because there are no free occurrences to substitute, but that would be a different definition of substAvoid than the one we gave, and this extra step would be a source of inefficiency considering how often substAvoid calls itself recursively. Now substAvoid must include x as a variable to avoid for the following reason: when we name a binder label from b to z (perhaps allowing $z = b$, depending on freshening is defined), substAvoid spawns recursive subcalls to substAvoid to rename all free occurrences of b in t from b to z as well. Importantly, this renaming must take place *before* we can proceed with the operation substituting x . The concern is that if we rename b to x , then this initial renaming may introduce free occurrences of x in t . These free occurrences would then be replaced by the top-level operation that substitutes free occurrences of x with u .

We'll consider a real example. Let t be $(\lambda x.\lambda x.(zx))$, let u be $\lambda v.v$, and let us substitute y in t . That is, let us consider

$$\text{substAvoid } y (\lambda v.v) (\lambda x.\lambda x.(zx)),$$

which represents $(\lambda x.\lambda x.(zx)) [(\lambda v.v) / y]$. Note y does not occur anywhere in these terms. Figure 4.2 shows how this operation evaluates, with reference to an actual formalization of substAvoid in Coq. This demonstration defines subst in terms of substAvoid following Section 2.1, but deliberately omits the step of adding the substituted variable to the top-level avoid set. Using

our implementation of `freshen`, the substitution has the following computational behavior:

$$(\lambda x. \lambda x. (zx)) [(\lambda v. v) / y] = \lambda x. \lambda y. (z (\lambda v. v))$$

Note that the free occurrence of x has been incorrectly replaced with $\lambda v. v$. When y is added to the avoid set, the operation evaluates correctly:

$$(\lambda x. \lambda x. (zx)) [(\lambda v. v) / y] = \lambda x. \lambda v. (zv)$$

The variable v happens to have been chosen as the new name for the second abstraction binding x —it was not necessary to rename this abstraction like this, but it is still correct.

In Definition 4.6.39, there is no need to add the substituted variable to the avoid set, nor is there anything like “spawning recursive subcalls.” Our operation is conceptually very clean: the logic of renaming bound variables is handled entirely locally, and each variable and binder label is only affected once by a single operation. It is therefore perfectly safe to rename a bound variable to x if x is not in $\mathbf{fv} \, u \cup \mathbf{fv} \, t$, as there is no “subsequent” operation to be confused by this. Figure 4.1 demonstrates exactly this phenomenon, selected from a working Coq module, shown with minor edits for simplicity. This figure shows the computational behavior of how `Tealeaves` evaluates the substitution, which is as follows:

$$(\lambda x. \lambda x. (zx)) [(\lambda v. v) / y] = \lambda x. \lambda y. (zy)$$

Our implementation of `freshening` choose y as the new name for the second abstraction named x —just as before, it was not strictly necessary to rename x , but still not incorrect to do so. Our implementation of `freshen` happened to choose the name y , also the name of the variable being substituted. However, this did not cause any incorrect behavior, and our implementation returns something α -equivalent to the output given by the correct version of `substAvoid`.

I claim that Definition 4.6.39 is “clean” and easier to reason about than `substAvoid` because of the fact that the operation is defined without a notion of state. The definition is not given recursively, at least not directly, but is specified by its stateless behavior at individual variable occurrences. To formalize `substAvoid` as shown in Figure 4.2, I found it necessary to disable termination checking—a drastic measure potentially introducing inconsistency in the logic—to obtain a version of `substAvoid` that can be evaluated efficiently. An early attempt to define `substAvoid` using well-founded recursion was both very complicated and led to a function that could not be evaluated in a reasonable time, presumably due to its (completely unnecessary) simplification of the proof of termination. (Experts in Coq may have found a better way to do this, but the fact such expertise would be required speaks to the unfortunate nature of `substAvoid` as a non-structurally-recursive operation.) On the other hand, the definition of `bindd2` is simple and structurally recursive, although admittedly Definition 4.6.39 will not have good performance for the reasons

```

From Tealeaves Require Import
  Examples.LambdaNominal.Categorical
  Backends.Nominal.Barendregt.

Goal subst y (λ v, `v) (λ x, λ x, `z . `x) = (λ x, λ y, `z . `y)
  reflexivity.
Qed.

```

Figure 4.1: Attempting to substitute y in a term where it doesn't occur, using the Tealeaves definition, introduces bound occurrences of y in that term, but the output is nonetheless correct (source )

discussed above, namely the frequent recomputation of the current state of the avoid set, unless memoization or other techniques are used. Still, this definition is relatively to easy to reason about, which is a considerable advantage over `substAvoid`.

```

#[local] Unset Guard Checking. (* DISABLES TERMINATION CHECKING! *)

From Tealeaves Require Export
  Examples.LambdaNominal.Syntax.

Fixpoint substAvoid
  (l: list name) (* l is the avoid set *)
  (x : name) (u : term name name)
  (t : term name name) {struct t}
  : term name name :=
  match t with
  | tvar y => if y == x then u else tvar y
  | tap t1 t2 =>
    tap (substAvoid l x u t1) (substAvoid l x u t2)
  | lam b t =>
    if b == x then lam b t
    else let z := freshen l b in
      lam z (substAvoid (l ++ [z]) x u (substAvoid (l ++ [z]) b (tvar z) t))
  end.

(* INCORRECT: x should be added to the avoid set! *)
Definition subst_INCORRECT (x : name) (u : term name name) (t : term name name) :=
  substAvoid (FV t ++ FV u) x u t.

Definition subst_CORRECT (x : name) (u : term name name) (t : term name name) :=
  substAvoid ([x] ++ FV t ++ FV u) x u t.

Goal subst_INCORRECT y (λ v, `v) (λ x, λ x, `z · `x) = (λ x, λ y, `z · `y).
  Fail reflexivity.
Abort.

Goal subst_INCORRECT y (λ v, `v) (λ x, λ x, `z · `x) = (λ x, λ y, `z · (λ v, `v)).
  reflexivity.
Qed.

Goal subst_CORRECT y (λ v, `v) (λ x, λ x, `z · `x) = (λ x, λ v, `z · `v).
  reflexivity.
Qed.

```

Figure 4.2: Attempting to substitute y in a term where it doesn't occur using `substAvoid`, without adding y to the avoid list, leads to the incorrect substitution of a bound y (source .

DECORATED TRAVERSABLE MONADS

Chapter 4 extended monads on SET with a suitable coaction, enabling substitutions that depend on variable binding contexts. While this abstraction admits the equational laws of de Bruijn algebra, it lacks the expressiveness needed to reason effectively about locally nameless or nominal variable binding. It also cannot define operations like $\mathbf{fv}$ or $\mathbf{lc}$, which return not terms but other kinds of objects, such as sets or propositions. This chapter incorporates an orthogonal structure, *traversability*, captured by a distributive law over applicative functors [36, 44], to address this limitation.

Traversable functors capture the remaining essential aspect of first-order abstract syntax: the idea that datatypes like *list* and *tree* can be seen as *containers* of variable occurrences. When the term monad is traversable, these occurrences can be manipulated and reasoned about an aggregate. Beyond aggregation, we will show that traversability supports operations such as zipping syntax trees of the same shape (Sec. 6.1.5) or lifting relations, not just functions, over terms (Sec. 6.1.5). Traversability also underlies powerful reasoning principles used to establish properties whose truth is intuitively clear but whose proofs are otherwise elusive, e.g. Lemma 5.3.30. Thus, the traversal structure does not just support new operations, but new ways of reasoning about the operations defined in Chapter 4.

Because the formal definition of traversability is difficult to appreciate upon first presentation, we begin by presenting the weaker notion of *monoidally foldable* functors as a conceptual stepping stone in Section 5.1. This is already very useful—Section 5.2 uses this abstraction to define the remaining operations of locally nameless, as well as several of the lemmas from Figure 2.1. Section 5.3 highlights the shortcomings of this abstraction to motivate the additional power of traversals. Section 5.4 reincorporates the decorated structure of the monad. Section 5.5 uses this combined abstraction to the remaining lemmas of locally nameless. Section 5.6 defines operations that translate between de Bruijn, locally nameless, and nominal syntax, although the correctness of these operations rely on deep principles that will not be presented until Chapter 6. Finally, Section 5.7 incorporates traversability into the notion of a *parameterized* decorated monad previously discussed in Section 4.6.

5.1 MONOIDAL FOLDS

A *monoidal fold* collapses a data structure containing values drawn from a monoid M into a single value using the multiplication of M . The class of functors that support such an operation—called *linearizable* in this work—can be seen as a crude approximation of data structures that contain a finite number of elements in a definite order.

5.1.1 Algebras of the list Monad

A particular class of monad algebras of interest in this work are those of the list monad—also known as monoids.

Definition 5.1.1. Let M be any monoid. We define a list (monad-)algebra $\text{mconcat}_M^{\text{list}}$ on M as follows:

$$\text{mconcat}_M^{\text{list}} [] \equiv e_M \quad (5.1)$$

$$\text{mconcat}_M^{\text{list}} (\text{cons } m \, l) \equiv m \cdot \text{mconcat}_M^{\text{list}} l \quad (5.2)$$

Here, mconcat means *monoidal concatenation*, not to be confused with list concatenation, concat (Ex. 4.2.5). Actually, the latter operation, which is the join operation of the list monad, is indeed a special case of mconcat when M is a free monoid. The fact that $\text{mconcat}_M^{\text{list}}$ constitutes a monad algebra will be proved after the following lemma.

Lemma 5.1.2. For any monoid M , $\text{mconcat}_M^{\text{list}} : \text{list } M \rightarrow M$ is a monoid homomorphism.

Proof. The unit equality $\phi(e_{\text{list } M}) = e_M$ holds by the definition (5.1). The other condition that must be shown,

$$\text{mconcat}_M^{\text{list}} (l_1 ++ l_2) = \text{mconcat}_M^{\text{list}} l_1 \cdot \text{mconcat}_M^{\text{list}} l_2, \quad (5.3)$$

is proven by a straightforward induction on l_1 using with left unit and associativity laws of M . $\square$

Remark 5.1.3. The superscript on $\text{mconcat}^{\text{list}}$ is required because this operation will soon be generalized to a whole class of functors. The subscript, which implicitly includes the entire monoid structure of M , will usually be left implicit.

Consider any list algebra $\langle A, \text{mconcat}^{\text{list}} : \text{list } A \rightarrow A \rangle$. The monad algebras laws specialize to the following equations:

$$\text{mconcat}^{\text{list}} [a] = a \quad (5.4)$$

$$\text{mconcat}^{\text{list}} (\text{concat } l) = \text{mconcat}^{\text{list}} (\text{map}^{\text{list}} \text{mconcat}^{\text{list}} l) \quad (5.5)$$

We can now prove that Definition 5.1.1 forms a legitimate algebra.

Lemma 5.1.4. For any monoid M , $\text{mconcat}_M^{\text{list}}$ is a list-algebra.

Proof. Equation (5.4) is straightforward using the right unit law of M .

$$\text{mconcat}_M^{\text{list}} [m] = \text{mconcat}_M^{\text{list}} (\text{cons } m \, []) = m \cdot \text{mconcat}_M^{\text{list}} [] = m \cdot e_M = m$$

Equation (5.5) is proven by induction on l . In the case case that l is empty, we observe $\text{concat } []$ and $\text{map mconcat}^{\text{list}} []$ are both equal to $[],$ which gives the following:

$$\text{mconcat}_M^{\text{list}} (\text{concat } []) = \text{mconcat}^{\text{list}} \left(\text{map mconcat}^{\text{list}} [] \right) = \text{mconcat}_M^{\text{list}} [] = e_M$$

The inductive case is a corollary of (5.3). Here, $l: \text{list } A$ and $l': \text{list } (\text{list } A):$

$$\begin{aligned} & \text{mconcat}_M^{\text{list}} (\text{concat } (\text{cons } l \ l')) \\ = & \text{mconcat}_M^{\text{list}} (l \ ++ \ \text{concat } l') && \text{Simplify concat} \\ = & \text{mconcat}_M^{\text{list}} l \cdot \text{mconcat}_M^{\text{list}} (\text{concat } l') && \text{Eqn. (5.3)} \\ = & \text{mconcat}_M^{\text{list}} l \cdot \text{mconcat}_M^{\text{list}} \left(\text{map mconcat}_M^{\text{list}} l' \right) && \text{I.H.} \\ = & \text{mconcat}_M^{\text{list}} \left(\text{cons } \left(\text{mconcat}_M^{\text{list}} l \right) \left(\text{map mconcat}_M^{\text{list}} l' \right) \right) && \text{Eqn. (5.2)} \\ = & \text{mconcat}_M^{\text{list}} \left(\text{map mconcat}_M^{\text{list}} (\text{cons } l \ l') \right) && \text{Eqn. (4.24)} \end{aligned}$$

□

Lemma 5.1.5. *Every list-algebra $\langle A, \text{mconcat}_A^{\text{list}} : \text{list } A \rightarrow A \rangle$ endows A with the structure of a monoid A by the following definitions:*

$$\begin{aligned} e_A & \equiv \text{mconcat}_A^{\text{list}} [] \\ a_1 \cdot a_2 & \equiv \text{mconcat}_A^{\text{list}} [a_1, a_2] \end{aligned}$$

Proof. This is a worthwhile exercise. The crucial steps are captured by the following equations:

$$\begin{aligned} \text{concat } [[], [a]] &= [a] \\ \text{concat } [[a], []] &= [a] \\ \text{concat } [[a_1, a_2], [a_3]] &= \text{concat } [[a_1], [a_2, a_3]] \end{aligned}$$

□

Lemma 5.1.6. *A list-algebra is equivalent to a monoid.*

Proof. Lemmas 5.1.5 and 5.1.4 establish an association between the two notions. Furthermore, it is straightforward to check that this correspondence is unique in both directions. □

The next lemma is left as an exercise.

Lemma 5.1.7. *Given any monoid M , a list-algebra homomorphism is equivalent to a monoid homomorphism (Def. 4.1.12). In particular, the following equality holds for all monoid homomorphisms $\phi: M \rightarrow M'$*

and lists $l: \text{list } M$.

$$\phi \left(\text{mconcat}_M^{\text{list}} l \right) = \text{mconcat}_{M'}^{\text{list}} \left(\text{map}^{\text{list}} \phi l \right) \quad (5.6)$$

The mapReduce Operation

The free algebras of list are exactly the monoids of the form $\langle \text{list } A, [], ++ \rangle$. Lemma 4.2.31, which characterizes the free algebras of a monad, specializes to list as follows: given any function $f: A \rightarrow M$ where M is a monoid, there is an operation

$$\text{mapReduce}_M^{\text{list}} f: \text{list } A \rightarrow M$$

defined $\text{mapReduce}_M^{\text{list}} f l = \text{mconcat}_M^{\text{list}} (\text{map}^{\text{list}} f l)$. This operation satisfies the following equations, mirroring (5.51) and (5.52).

$$\text{mapReduce}_M^{\text{list}} f [a] = f a \quad (5.7)$$

$$\text{mapReduce}_M^{\text{list}} f (\text{concat } l) = \text{mapReduce}_M^{\text{list}} f \left(\text{mapReduce}_M^{\text{list}} f l \right) \quad (5.8)$$

For instance, given a nested list such as $[[u, v, t], [], [x, y, z], [w, q]]$, the LHS of (5.8) first flattens the list, then applies $\text{mapReduce}_M^{\text{list}} f$ to obtain the following:

$$(f u) \cdot (f v) \cdot (f t) \cdot (f x) \cdot (f y) \cdot (f z) \cdot (f w) \cdot (f q).$$

On the other hand, the RHS applies $\text{mapReduce}_M^{\text{list}} f$ to each of the inner lists, and then applies $\text{mapReduce}_M^{\text{list}} f$ to the result, to obtain an equal expression per the monoid laws:

$$((f u) \cdot (f v) \cdot (f t)) \cdot e_M \cdot ((f x) \cdot (f y) \cdot (f z)) \cdot ((f w) \cdot (f q)).$$

Remark 5.1.8. In Haskell terminology, $\text{mapReduce}_M^{\text{list}}$ is called *mapfold*. However, this usage somewhat overloads the term *fold*.

Lemma 5.1.7 and the functor law (4.22) imply the following equality:

$$\phi \left(\text{mapReduce}_M^{\text{list}} f l \right) = \text{mapReduce}_M^{\text{list}} (\phi \circ f) l \quad (5.9)$$

$\text{mapReduce}_M^{\text{list}}$ is a versatile mechanism for defining operations that manipulate list as a container. For instance, we can compute the length of a list, define a predicate testing whether an arbitrary value x is an element, and quantify predicates universally or existentially over its elements.

Definition 5.1.9 (Container Operations). *The container operations in Tealeaves are operations such as*

the following, where $P: A \rightarrow \mathbf{Prop}$ and $a: A$:

$$\text{length}^{\text{list}}: \text{list } A \rightarrow \mathbb{N} \quad \equiv \quad \text{mapReduce}_+^{\text{list}} (\text{const } 1) \quad (5.10)$$

$$\text{element_of}^{\text{list}}: \text{list } A \rightarrow A \rightarrow \mathbf{Prop} \quad \equiv \quad \text{mapReduce}_\cup^{\text{list}} \left(\text{ret}^{\text{subset}} \right) \quad (5.11)$$

$$(a \in -)^{\text{list}}: \text{list } A \rightarrow \mathbf{Prop} \quad \equiv \quad \text{mapReduce}_\vee^{\text{list}} \left(\text{ret}^{\text{subset}} a \right) \quad (5.12)$$

$$\text{ForAll}^{\text{list}} P: \text{list } A \rightarrow \mathbf{Prop} \quad \equiv \quad \text{mapReduce}_\wedge^{\text{list}} P \quad (5.13)$$

$$\text{ExistsAny}^{\text{list}} P: \text{list } A \rightarrow \mathbf{Prop} \quad \equiv \quad \text{mapReduce}_\vee^{\text{list}} P \quad (5.14)$$

The subscript $+$ indicates specializing mapReduce to the monoid $\langle \mathbb{N}, 0, + \rangle$; $\wedge$ and $\vee$ refer to conjunction and disjunction of propositions (Ex. 4.1.9); $\cup$ refers to the subsets under union (Ex. 4.1.10).

Recall that $\text{ret}^{\text{subset}}$ (Ex. 4.2.6) maps any a to the predicate testing for equality with a . The difference between $\text{element_of}^{\text{list}} l a$ and $a \in l$ is that the former maps each element in l to a singleton, takes their union, and tests if a is in this set; the latter maps each element x to the proposition $x = a$ and takes their disjunction. The equivalence of these definitions can be derived from (5.9). Note that $x \in l$ is identical to $\text{Forany} (\text{ret}^{\text{subset}} a) l$. The following lemma justifies reading the combinators Forall and Forany as logical quantification.

Lemma 5.1.10. *$\text{ForAll}^{\text{list}} P l$ is true if and only if all of the elements of l satisfy P . Likewise $\text{Forany } P l$ holds if P holds for any of the elements in l . That is, the following equivalences hold:*

$$\text{ForAll}^{\text{list}} P l \quad \iff \quad \forall (a: A), a \in l \implies P a \quad (5.15)$$

$$\text{ExistsAny}^{\text{list}} P l \quad \iff \quad \exists (a: A), a \in l \wedge P a \quad (5.16)$$

Proof. This is a straightforward induction on l . We take ForAll as an example. In the case $l = \text{nil}$, the LHS evaluates to True as follows.

$$\text{ForAll}^{\text{list}} P \text{ nil} = \text{mapReduce}_\wedge^{\text{list}} P [] = e_\wedge = \text{True}.$$

Now, $a \in []$ evaluates to False by

$$(a \in []) = \text{mapReduce}_\vee^{\text{list}} \left(\text{ret}^{\text{subset}} a \right) [] = e_\vee = \text{False}.$$

Therefore, the RHS is also vacuously True . In the inductive case, we simplify the LHS as follows:

$$\text{ForAll}^{\text{list}} P (\text{cons } a l) = P a \wedge \left(\text{mapReduce}_\wedge^{\text{list}} P l \right)$$

Now $a' \in (\text{cons } a \, l) \iff (a' = a \vee a' \in l)$. By the I.H we have the following.

$$(\text{mapReduce}^{\text{list}} P \, l) \iff \forall (a' : A), a' \in l \implies P \, a'$$

Therefore, it only remains to prove the following obvious equivalence:

$$(P \, a \wedge \forall (a' : A), a' \in l \implies P \, a') \iff \forall (a' : A), (a = a' \vee a' \in l) \implies P \, a'$$

□

This property is useful for a few reasons. The operations defined on the left are defined from `mapReduce`, and are therefore subject to reasoning principles like (5.7) and (5.8). The statements on the right are arguably more intuitive and useful during the course of proofs about operations on syntax (see Lemma 5.5.1 for an example).

We can also prove the following intuitive properties about the element-of relation ($x \in -$).

Lemma 5.1.11. *Let $f : A \rightarrow B$ be any function. Then for all $l : \text{list } A$, for all $a : A$ and $b : B$, the following properties hold:*

$$b \in \text{map}^{\text{list}} f \, l \iff \exists (a : A), a \in l \wedge f a = b \quad (5.17)$$

$$a \in l \implies f a \in \text{map}^F f \, l \quad (5.18)$$

Proof. The notation $b \in \text{map}^{\text{list}} f \, l$ unfolds to $\text{mapReduce}_{\vee}^{\text{list}} (\text{ret}^{\text{subset}} b) (\text{map}^{\text{list}} f \, l)$. The functor law (4.22) implies this is equal to $\text{mapReduce}_{\vee}^{\text{list}} (\text{ret}^{\text{subset}} b \circ f)$. Note that the inner expression is equal to the predicate $\lambda a. (b = f a)$. Then (5.17) follows from (5.16). Equation (5.18) follows logically from (5.17) with $f a$ in place of b and a itself acting as the element a' satisfying $a' \in l$ and $f a' = f a$. □

5.1.2 Monoidal Folds over Functors

We now generalize from list to functors with “list-like” structure. These are functors from which we can extract a list of elements, which provides a simple mechanism to manipulate them as containers. In fact, there are a total of three equivalent definitions:

1. Functors equipped with a natural transformation to list (Definition 5.1.13)
2. Functors equipped with a `mapReduce` operation satisfying certain laws (see Lemma 5.1.20)
3. Functors equipped with an `mconcat` operation satisfying certain laws (see Lemma 5.1.21)

This trinity is worth emphasizing because it generalizes to traversable functors, which is fully exploited in Chapter 6 to develop some of the deepest parts of the theory.

Remark 5.1.12. Lacking a standard term for this class of functors, I call them “linearizable” to recognize that we can extract an ordered list of elements from them; there are no connotations of linear algebra or linear logic. In Haskell, such functors are called “foldable”—an unfortunate overloading of “fold.” Fortunately, the issue becomes moot when we move from linearizable functors to traversable functors, a more expressive notion.

Definition 5.1.13. A linearizable functor F is one paired with a natural transformation to list:

$$\text{tolist}^F : \forall (A : \text{SET}), F A \rightarrow \text{list } A$$

Example 5.1.14. A constant functor is linearizable where $\text{tolist}^{\text{const } A}$ returns $[]$.

Example 5.1.15. The identity functor is linearizable where $\text{tolist}^{\mathbb{1}}$ is ret^{list} .

Example 5.1.16. Given two linearizable functors F and G , their product $F \times G$ is linearizable where tolist is defined

$$\text{tolist}^{F \times G} (x, y) = \text{tolist}^F x ++ \text{tolist}^G y.$$

Similarly, their coproduct $F + G$ is linearizable where tolist is defined as follows:

$$\text{tolist}^{F+G} (\text{inl } x) = \text{tolist}^F x$$

$$\text{tolist}^{F+G} (\text{inr } y) = \text{tolist}^G y$$

Example 5.1.17. Σ^A is a linearizable functor where tolist is defined as follows:

$$\text{tolist} (\text{app } a_1 a_2) = [a_1, a_2]$$

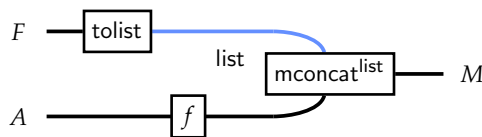
$$\text{tolist} (\text{abs } a) = [a]$$

Fundamentally, this is just tolist for the functor $(\mathbb{1} + (\mathbb{1} \times \mathbb{1}))$.

Given an operation enumerating the contents of F as a list, we obtain a generalization of $\text{mapReduce}^{\text{list}}$ by precomposition with tolist^F .

Definition 5.1.18. Let F be linearizable and let M be a monoid. The following operation is given:

$$\text{mapReduce}^F : \forall (\text{Monoid } M), (A \rightarrow M) \rightarrow F A \rightarrow M$$



$$\text{mapReduce}_M^F f \equiv \text{mapReduce}_M^{\text{list}} f \circ \text{tolist}^F \quad (5.19)$$

Remark 5.1.19. The diagram above implicitly uses the equality $\text{mapReduce}^{\text{list}} f = \text{mconcat}^{\text{list}} \circ \text{map}^{\text{list}} f$ to emphasize $\text{mconcat}^{\text{list}}$ as the most fundamental operation. The list wire is drawn in blue to emphasize that list is a monad.

A corollary of Equation (5.6) is that this more general operation commutes with any monoid homomorphism $\phi: M \rightarrow M'$:

$$\phi \circ \text{mapReduce}^F f = \text{mapReduce}^F (\phi \circ f) \quad (5.20)$$

In fact, this provides an alternate characterization of linearizable functors as follows.

Lemma 5.1.20. *A linearizable functor F is equivalently characterized as a functor equipped with an operation*

$$\text{mapReduce}^F: \forall (\text{Monoid } M), (A \rightarrow M) \rightarrow F A \rightarrow M$$

such that (5.20) is satisfied for all monoid homomorphisms ϕ and where the following holds.

$$\text{mapReduce}^F (g \circ f) = \text{mapReduce}^F g \circ \text{map}^F f \quad (5.21)$$

Proof. Definition 5.1.18 derived mapReduce^F from tolist^F and proved (5.20). Equation (5.21) is a corollary of the functor law (4.22). Now suppose mapReduce^F is given as a primitive operation and define tolist^F as follows:

$$\text{tolist}_A^F \equiv \text{mapReduce}_{\text{list } A}^F (x \mapsto [x]) \quad (5.22)$$

We must show this a natural transformation. Let $f: A \rightarrow B$ be any function. Now

$$\text{map}^{\text{list}} f: \text{list } A \rightarrow \text{list } B$$

is a monoid homomorphism—a corollary of Lemma 4.2.31 and the fact that map is a special case of bind . The naturality property is verified as follows:

$$\begin{aligned} & \text{map}^{\text{list}} f \circ \text{tolist}^F \\ = & \text{map}^{\text{list}} f \circ \text{mapReduce}^F (x \mapsto [x]) && \text{Eqn. (5.22)} \\ = & \text{mapReduce}^F (x \mapsto \text{map}^{\text{list}} f [x]) && \text{Eqn. (5.20)} \\ = & \text{mapReduce}^F ((x \mapsto [x]) \circ f) && \text{Naturality of } \text{ret}^{\text{list}} \\ = & \text{mapReduce}^F (x \mapsto [x]) \circ \text{map}^F f && \text{Eqn. (5.21)} \\ = & \text{tolist}^F \circ \text{map}^F f && \text{Eqn. (5.22)} \end{aligned}$$

Now we show that an instance of one of these operations uniquely determines an instance of the other. Suppose tolist^F is given as a primitive operation. We want to show that this operation is equal to the one defined by (5.22) where mapReduce is defined from tolist^F as given by (5.19). That

is, we must show

$$\text{tolist}_A^F = \text{mapReduce}_{\text{list } A}^{\text{list}} \left(\text{ret}_A^{\text{list}} \right) \circ \text{tolist}_A^F$$

Recall from Definition 4.2.30 the monoid on list A is such that $\text{mconcat}_{\text{list } A}^{\text{list}}$ is equivalent to $\text{join}_A^{\text{list}}$. Therefore, the monad law (4.29) implies that $\text{mapReduce}_{\text{list } A}^{\text{list}} \text{ret}^{\text{list}}$ is equal to the identity function, so the above equation holds.

In the other direction, suppose mapReduce^F is given as a primitive operation. We want to show that this is equivalent to the operation defined by (5.19) using the tolist operation defined by (5.22). That is, we must show for all $f: A \rightarrow M$ the following:

$$\text{mapReduce}_M^F f = \text{mapReduce}_M^{\text{list}} f \circ \left(\text{mapReduce}_{\text{list } A}^F \text{ret}_A^{\text{list}} \right)$$

For this, we recall that

$$\text{mapReduce}_M^{\text{list}} f: \text{list } A \rightarrow M$$

is a monoid homomorphism by Lemma 4.2.31. We have the following:

$$\begin{aligned} & \text{mapReduce}_M^F f \circ \left(\text{mapReduce}_M^{\text{list}} \text{ret}^{\text{list}} \right) \\ = & \text{mapReduce}_M^{\text{list}} \left(\text{mapReduce}_M^{\text{list}} f \circ \text{ret}^{\text{list}} \right) \end{aligned} \quad \text{Eqn. (5.20)}$$

$$= \text{mapReduce}_M^{\text{list}} f \quad \text{Eqn. (5.4)}$$

Thus, an instance of either tolist or mapReduce is *uniquely* associated with an instance of the other. $\square$

Instead of an operation like mapReduce^F , linearizable functors can also be characterized by an operation generalizing $\text{mconcat}^{\text{list}}$.

Lemma 5.1.21. *A linearizable functor can be equivalently specified as one equipped with a polymorphic operation*

$$\text{mconcat}^F: \forall (\text{Monoid } M), F M \rightarrow M$$

that commutes with monoid homomorphisms:

$$\phi \left(\text{mconcat}_M^F t \right) = \text{mconcat}_{M'}^F \left(\text{map}^F \phi t \right) \quad (5.23)$$

Proof. We prove equivalence with the mapReduce -based specification of linearizability. Given mapReduce^F , define mconcat^F as $\text{mapReduce}^F \text{id}_M$. Then (5.23) is a consequence of (5.20) and (5.21). Given mconcat^F , $\text{mapReduce}^F f$ is defined $\text{mconcat}^F \circ \text{map}^F f$, where (5.20) follows from (5.23) and (5.21) follows from (4.22). That either of these operations uniquely determine an instance of the other is equally straightforward. $\square$

Remark 5.1.22. *In Haskell terminology, this polymorphic version of $\text{mconcat}^{\text{list}}$ is known simply as “fold.” As mentioned, I avoid this overloaded terminology.*

Because of Lemmas 5.1.20 and 5.1.21, it does not matter whether we regard tolist^F , mapReduce^F , or mconcat^F as the primitive operation and the others as being defined from it. However, the reader should be cautioned that there are typically many non-equal choices of linearizable structure for a given functor—for instance, given any definition of tolist^F , we can define a new one by composition with any permutation. Thus, linearizability is not per se a *property* of functors but an explicit choice of *structure*. In Section 5.3.1 we consider badly-behaved implementations of tolist before showing how to rule out such behavior axiomatically.

The following corollary is essentially the uniqueness statement of Lemma 5.1.20 in the direction that starts with mapReduce as a primitive operation and re-derives mapReduce by way of tolist .

Corollary 5.1.23. *Let F be a linearizable functor. Then $\text{mapReduce}_M^F f$ decomposes as follows.*

$$\text{mapReduce}_M^F f = \text{mapReduce}_M^{\text{list}} f \circ \text{tolist}_M^F \quad (5.24)$$

Remark 5.1.24. *The significance of this corollary is that it is true regardless of whether tolist , mapReduce , or mconcat is taken as the most primitive operation, provided they are defined with respect to the same linearizability structure. In this sense, the equation is an invariant of linearizable functors, and not just a definition.*

Equation (5.24) is of substantial importance in Tealeaves. It is used below to prove several general properties of linearizable functors. In particular, it is used to reason about the following generalizations of the container operations in Definition 5.1.9.

Definition 5.1.25. *Let F be a linearizable functor. The following operations are given.*

$$\text{length}^F: F A \rightarrow \mathbb{N} \quad \equiv \quad \text{mapReduce}_+^F(\text{const } 1) \quad (5.25)$$

$$\text{element_of}^F: F A \rightarrow A \rightarrow \mathbf{Prop} \quad \equiv \quad \text{mapReduce}_{\text{subset } A}^F \left(\text{ret}^{\text{subset}} \right) \quad (5.26)$$

$$(x \in _): F A \rightarrow \mathbf{Prop} \quad \equiv \quad \text{mapReduce}_\vee^F \left(\text{ret}^{\text{subset}} x \right) \quad (5.27)$$

$$\text{ForAll}^F P: F A \rightarrow \mathbf{Prop} \quad \equiv \quad \text{mapReduce}_\wedge^F P \quad (5.28)$$

$$\text{ExistsAny}^F P: F A \rightarrow \mathbf{Prop} \quad \equiv \quad \text{mapReduce}_\vee^F P \quad (5.29)$$

Corollary 5.1.23 provides an avenue for reasoning about generalized container operations by relating them to operations on list. This has several corollaries of its own, stated below.

Corollary 5.1.26. *Let F be a linearizable functor and consider some term $t: F A$. Then the following*

equivalences hold.

$$\begin{aligned}
a \in t & \iff a \in (\text{tolist}^F t) \\
\text{ForAll}^F P t & \iff \text{ForAll}^{\text{list}} P (\text{tolist}^F t) \\
\text{ExistsAny}^F P t & \iff \text{ExistsAny}^{\text{list}} P (\text{tolist}^F t)
\end{aligned}$$

Proof. We take $a \in t$ as an example. The notation $a \in t$ unfolds to $\text{mapReduce}^F (\text{ret}^{\text{subset}} a)$. Using (5.24), we have the following chain of equalities.

$$a \in t = \text{mapReduce}^F (\text{ret}^{\text{subset}} a) t = \text{mapReduce}^{\text{list}} (\text{ret}^{\text{subset}} a) (\text{tolist}^F t) \equiv a \in \text{tolist } t$$

The other equations are similar. □

Corollary 5.1.26 captures the principle the elements occurring in a term t are precisely the ones occurring in $\text{tolist } t$. The significance of this fact is that given a property like $a \in t$ for an arbitrary linearizable functor, we cannot perform induction on t because we do not know anything about F except that it is linearizable. However, we can perform induction on $\text{tolist}^F t$. This approach is used in the subsequent two corollaries, which generalize Lemmas 5.1.10 and 5.1.11 to arbitrary linearizable functors.

Corollary 5.1.27. *Let F be a linearizable functor and consider some term $t : T A$. Then the following hold.*

$$\text{ForAll}^F P t \iff \forall (a : A), a \in t \implies Pa \tag{5.30}$$

$$\text{ExistsAny}^F P t \iff \exists (a : A), a \in t \wedge Pa \tag{5.31}$$

Proof. Applying Corollary 5.1.26 reduces the goal to the following one, which holds by Lemma 5.1.10.

$$\text{ForAll}^{\text{list}} P (\text{tolist } t) \iff \forall (a : A), a \in (\text{tolist } t) \implies Pa$$

$$\text{ExistsAny}^{\text{list}} P (\text{tolist } t) \iff \exists (a : A), a \in (\text{tolist } t) \wedge Pa$$

□

Corollary 5.1.28. *Let F be a linearizable functor and consider some term $t : T A$ and function $f : A \rightarrow B$. Then the following hold for all a and b .*

$$b \in \text{map}^F f t \iff \exists (a : A), a \in t \wedge fa = b \tag{5.32}$$

$$a \in t \implies fa \in \text{map}^F f t \tag{5.33}$$

Proof. Equation (5.32) is proved by using the above decomposition to express the goal as follows.

$$b \in \left(\text{tolist}^F \left(\text{map}^F f t \right) \right) \iff \exists (a : A), a \in \left(\text{tolist}^F t \right) \wedge fa = b$$

Using the naturality of tolist to replace $\text{tolist}^F (\text{map}^F f t)$ with $\text{map}^{\text{list}} f (\text{tolist}^F t)$ reduces the goal to an instance of (5.17). Equation (5.33) follows similarly. $\square$

Corollaries 5.1.27 and 5.1.28 demonstrate an important principle: the factorization property that follows from Lemma 5.1.20 allows proving a statement about F , an *arbitrary* linearizable functor, by reducing the goal to a proposition about lists. This can be seen as recovering a limited form of induction. In general, there is no way to perform induction on an opaque type constructor, since it is not a known, concrete datatype. However, when faced with goals such as Equation (5.32), we can rewrite all generalized container operations to ones involving $\text{mapReduce}^{\text{list}}$ and $\text{tolist}^F t$, and proceed by induction on $\text{tolist}^F t$, performing the sorts of concrete reasoning on lists exhibited in Lemmas 5.1.10 and 5.1.11.

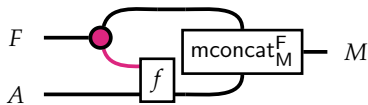
In Chapter 6, we take the above principle much further. Lemma 5.1.20 is generalized to the representation theorem for traversable functors, Theorem 6.1.28, and then to the representation theorem for decorated traversable monads, Theorem 6.3.18. There, too, we show that operations definable for arbitrary decorated traversable monads factorize through a concrete datatype, recovering a limited but powerful form of induction in the context of abstract first-order syntactical datatypes.

Monoidal Folds over Decorated Functors

Before moving on, we consider how functors equipped with both a mapReduce operation and a decoration. We do not provide impose any additional assumptions regarding how dec and mapReduce interact.

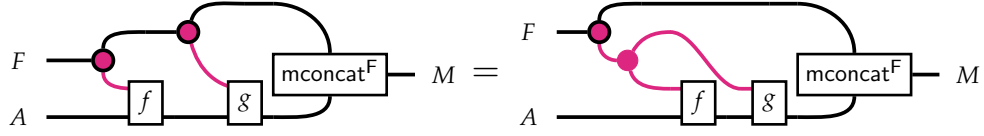
Definition 5.1.29. Let $\langle F, \text{mapReduce}^F, \text{dec}^F \rangle$ form both a linearizable functor and a functor decorated by E . We can define an context-sensitive version of mapReduce as follows:

$$\text{mapdReduce}^F : \forall (\text{Monoid } M), (E \times A \rightarrow M) \rightarrow F A \rightarrow M$$



$$\text{mapdReduce}_M^F f \equiv \text{mconcat}_M^F \circ \text{map}^F f \circ \text{dec}^F \quad (5.34)$$

Lemma 5.1.30. This operation satisfies the following composition law with mapd^F :



$$\text{mapdReduce}^F g \circ \text{mapd}^F f = \text{mapdReduce}^F (\lambda(e, a). g(e, f(e, a))) \quad (5.35)$$

Proof. Visually, this can be seen as simply a matter of bending wires and applying the decoration co-associativity law (4.77). We can also give the explicit equational proof:

$$\begin{aligned}
& \text{mapdReduce}^F g \circ \text{mapd}^F f \\
= & \text{mconcat}^F \circ \text{map}^F g \circ \text{dec}^F \circ \text{map}^F f \circ \text{dec} && \text{Unfold definitions} \\
= & \text{mconcat}^F \circ \text{map}^F g \circ \text{map}^F (\text{map}^{E^\times} f) \circ \text{dec}^F \circ \text{dec} && \text{Naturality of dec} \\
= & \text{mconcat}^F \circ \text{map}^F (g \circ \text{map}^{E^\times} f) \circ \text{dec}^F \circ \text{dec} && \text{Eqn. (4.22)} \\
= & \text{mconcat}^F \circ \text{map}^F (g \circ \text{map}^{E^\times} f) \circ \text{map}^F \text{dup}^{E^\times} \circ \text{dec} && \text{Eqn. (4.77)} \\
= & \text{mconcat}^F \circ \text{map}^F (g \circ \text{map}^{E^\times} f \circ \text{dup}^{E^\times}) \circ \text{dec} && \text{Eqn. (4.22)} \\
= & \text{mapdReduce}^F (\lambda(e, a). g(e, f(e, a))) && \text{Fold definitions}
\end{aligned}$$

□

The context-sensitive version of mapReduce^F provides context-sensitive versions of the container operations given in Definition 5.1.25.

Definition 5.1.31 (Decorated Container Operations). *Let F be a decorated and linearizable functor. The following operations are given where $P: E \times A \rightarrow \mathbf{Prop}$, $e: A$, $a: A$:*

$$\text{element_of_dec}: F A \rightarrow E \times A \rightarrow \mathbf{Prop} \quad \equiv \quad \text{mapReduce}_{\text{subset } A}^F (\text{ret}^{\text{subset}}) \quad (5.36)$$

$$((e, a) \in_d _): F A \rightarrow \mathbf{Prop} \quad \equiv \quad \text{mapdReduce}_V^F (\text{ret}^{\text{subset}}(e, a)) \quad (5.37)$$

$$\text{ForAllDec } P: F A \rightarrow \mathbf{Prop} \quad \equiv \quad \text{mapdReduce}_\wedge^F P \quad (5.38)$$

$$\text{ExistsAnyDec } P: F A \rightarrow \mathbf{Prop} \quad \equiv \quad \text{mapdReduce}_V^F P \quad (5.39)$$

The $(\in_d)$ predicate above (note the d subscript to indicate *decoration*) provides a notion of *contextual occurrence*. By definition, $(e, a) \in_d t$ is equal $(e, a) \in \text{dec}^F t$. That is, a leaf a appears in t with a context e if and only iff the pair (e, a) occurs in $\text{dec } t$. The following lemma relates contextual occurrence with context-agnostic occurrence.

Lemma 5.1.32. *Let $t : F A$ where F is a linearizable functor decorated by E . The following hold for all $a : A$.*

$$a \in t \iff \exists (e : E), (e, a) \in_d t \quad (5.40)$$

$$(e, a) \in_d t \implies a \in t \quad (5.41)$$

Proof. $(e, a) \in_d t$ is equivalent to $(e, a) \in \text{dec } t$. We have $t = \text{map}^F \text{extract}^{E^\times} (\text{dec } t)$ by (4.76). Thus, by (5.32), we have the following, which establishes (5.40):

$$a \in t \iff \exists e a', (e, a') \in \text{dec } t \wedge \text{extract}^{E^\times}(e, a') = a$$

Now (5.41) follows logically from (5.40), or from the application of (5.33) to $(e, a) \in \text{dec } t$ where f is $\text{extract}^{E^\times}$. $\square$

The following lemma is context-sensitive analogue of Corollary 5.1.26.

Lemma 5.1.33. *Let F be a linearizable functor decorated by E . Let $t : F A$ be a term. The following hold for all context-sensitive predicates P .*

$$\text{ForAllDec}^F P t \iff \forall (e : E) (a : A), (e, a) \in_d t \implies P(e, a) \quad (5.42)$$

$$\text{ExistsAnyDec}^F P t \iff \exists (e : E) (a : A), (e, a) \in_d t \wedge P(e, a) \quad (5.43)$$

Proof. For instance, (5.42) is proven using (5.24) to rewrite the goal into the form

$$\text{ForAll}^{\text{list}} P (\text{tolist } (\text{dec } t)) \iff \forall (e : E) (a : A), (e, a) \in \text{tolist } (\text{dec } t) \implies P(e, a)$$

after which the goal holds by Lemma 5.1.10. (5.43) follows similarly. $\square$

The following properties characterize how this notion of occurrence interacts with mapd and map . In particular, these operations do not change the context of an element—given an occurrence of a in some term t with context e , mapping a function over t replaces a by another occurrence whose context is still e .

Corollary 5.1.34. *Let F be a linearizable functor decorated by E . Let $t : F A$ be a term. The following hold for all functions and all context values $e : E$.*

$$(e, b) \in_d \text{mapd}^F f t \iff \exists (a : A), (e, a) \in_d t \wedge f(e, a) = b \quad (5.44)$$

$$(e, b) \in_d \text{map}^F f t \iff \exists (a : A), (e, a) \in_d t \wedge f a = b \quad (5.45)$$

Proof. The notation $(e, b) \in_d \text{mapd}^F f t$ unfolds to

$$\text{mapdReduce}_V^F \left(\text{ret}^{\text{subset}}(e, b) \right) \left(\text{mapd}^F f t \right)$$

By (5.35), this is equal to $\text{mapReduce}_V^F (\lambda(e', a).(e, b) = (e', (f(e', a)))) t$, expressed more concisely as

$$\text{ExistsAnyDec}^F (\lambda(e', a).(e, b) = (e', (f(e', a)))) t.$$

By Lemma 5.1.33, this is equivalent to

$$\exists (e' : E) (a : A), (e', a) \in_d t \wedge (e, b) = (e', f(e', a))$$

The above condition is logically equivalent to the right-hand side of (5.44). The other condition is similar. $\square$

5.1.3 Monoidal Folds over Monads

We now consider the appropriate notion of “linearizable” when the underlying functor also forms a monad. For such structures, we impose additional conditions regarding how `toList` and `mapReduce` interact with `ret` and `join`. The following is the analogue of Definition 5.1.13 for monads.

Definition 5.1.35. A linearizable monad T is one equipped with a monad homomorphism to the list monad. That is, besides satisfying Definition 5.1.13, the following equalities must hold:

$$\text{toList}^T (\text{ret}^T a) = [a] \tag{5.46}$$

$$\text{toList}^T (\text{join}^T t) = \text{concat} \left(\text{toList}^T \left(\text{map}^T \text{toList} t \right) \right) \tag{5.47}$$

The next two lemmas are analogues of Lemmas 5.1.20 and 5.1.21.

Lemma 5.1.36. A linearizable monad can be equivalently specified in terms of an operation mapReduce^T satisfying Equations (5.20) and (5.21) as well as the following additional equalities:

$$\text{mapReduce}^T f (\text{ret}^T a) = f a \tag{5.48}$$

$$\text{mapReduce}^T f (\text{join}^T t) = \text{mapReduce}^T \left(\text{mapReduce}^T f \right) t \tag{5.49}$$

Proof. The unique correspondence between toList^T and mapReduce^T follows from Lemma 5.1.20, so it only remains to check that the respective sets of equations imply each other. Suppose T is equipped with a monad homomorphism toList^T and suppose mapReduce is defined from toList^T by

(5.19). First we verify (5.48):

$$\begin{aligned}
& \text{mapReduce}^T f \left(\text{ret}^T a \right) \\
= & \text{mapReduce}^{\text{list}} f \left(\text{tolist}^T \left(\text{ret}^T a \right) \right) && \text{Eqn. (5.24)} \\
= & \text{mapReduce}^{\text{list}} f [a] && \text{Eqn. (4.31)} \\
= & f a && \text{Eqn. (5.7)}
\end{aligned}$$

Next we verify (5.49):

$$\begin{aligned}
& \text{mapReduce}^T f \left(\text{join}^T t \right) \\
= & \text{mapReduce}^{\text{list}} f \left(\text{tolist}^T \left(\text{join}^T t \right) \right) && \text{Eqn. (5.24)} \\
= & \text{mapReduce}^{\text{list}} f \left(\text{concat} \left(\text{tolist}^T \left(\text{map}^T \text{tolist}^T t \right) \right) \right) && \text{Eqn. (4.32)} \\
= & \text{mapReduce}^{\text{list}} f \left(\text{mapReduce}^{\text{list}} f \right) \left(\text{tolist}^T \left(\text{map}^T \text{tolist}^T t \right) \right) && \text{Eqn. (5.8)} \\
= & \text{mapReduce}^{\text{list}} f \left(\text{mapReduce}^{\text{list}} f \right) \left(\text{map}^T \text{tolist}^T \left(\text{tolist}^T t \right) \right) && \text{Naturality of tolist} \\
= & \text{mapReduce}^{\text{list}} f \left(\text{mapReduce}^{\text{list}} f \circ \text{tolist}^T \right) \left(\text{tolist}^T t \right) && \text{Eqn. (5.21)} \\
= & \text{mapReduce}^T f \left(\text{mapReduce}^T f \right) t && \text{Eqn. (5.24)} (\times 2)
\end{aligned}$$

Now suppose mapReduce^T is a primitive operation satisfying Definition 5.1.35. We verify that tolist , defined according to (5.22), is a monad homomorphism. First we check the unit law:

$$\begin{aligned}
& \text{tolist}^T \left(\text{ret}^T a \right) && \text{Unfold definitions} \\
= & \text{mapReduce}^T \text{ret}^{\text{list}} \left(\text{ret}^T a \right) && \text{Eqn. (5.48)} \\
= & \text{ret}^{\text{list}} a
\end{aligned}$$

For the join law, we observe that $\text{mapReduce}_{\text{listA}}^{\text{list}}$ is equal to $\text{bind}_A^{\text{list}} f$, or $\text{join}_A^{\text{list}} \circ \text{map}^{\text{list}} f$.

$$\begin{aligned}
& \text{tolist}^T (\text{join}^T t) \\
= & \text{mapReduce}^T \text{ret}^{\text{list}} (\text{join}^T t) && \text{Eqn. (5.22)} \\
= & \text{mapReduce}^T (\text{mapReduce}^T \text{ret}^{\text{list}}) t && \text{Eqn. (5.49)} \\
= & \text{mapReduce}^T \text{tolist}^T t && \text{Eqn. (5.22)} \\
= & \text{mapReduce}_{\text{listA}}^{\text{list}} \text{tolist}^T (\text{tolist}^T t) && \text{Eqn. (5.24)} \\
= & \text{join}^{\text{list}} (\text{map}^{\text{list}} \text{tolist}^T (\text{tolist}^T t)) && \text{mapReduce}_{\text{listA}}^{\text{list}} f = \text{bind}_A^{\text{list}} f \\
= & \text{join}^{\text{list}} (\text{tolist}^T (\text{map}^T \text{tolist}^T t)) && \text{Naturality of tolist}
\end{aligned}$$

□

It also follows that a linearizable monad satisfies the following condition:

$$\text{mapReduce}^T g (\text{bind}^T f t) = \text{mapReduce}^T (\text{mapReduce}^T g \circ f) t \quad (5.50)$$

Lemma 5.1.37. *A linearizable monad is equivalently one such that every monoid is a monad algebra of T , such that monoid homomorphisms commute with the algebras. That is, there is an operation*

$$\text{mconcat}^T : \forall (\text{Monoid } M), T M \rightarrow M$$

that commutes with monoid homomorphisms and satisfies Equations (4.48) and (4.49), which specialize to mconcat^T as follows.

$$\text{mconcat}^T (\text{ret}^T m) = m \quad (5.51)$$

$$\text{mconcat}^T (\text{join}^T t) = \text{mconcat}^T (\text{map}^T \text{mconcat}^T t) \quad (5.52)$$

Now we consider several corollaries of these equivalent characterizations.

Lemma 5.1.38. *Let T be a linearizable monad. Then ret^T is injective.*

Proof. Suppose $\text{ret}^T a = \text{ret}^T a'$. Then $\text{tolist} (\text{ret}^T a) = \text{tolist} (\text{ret}^T a')$. Since tolist is a monad homomorphism, this is equivalent to $[a] = [a']$. Since ret^{list} is injective, we have $a = a'$. □

The next property extends Corollary 5.1.28 to give specifications for the monad operations.

Lemma 5.1.39. *Let T be a linearizable monad. The following hold.*

$$a \in \text{ret}^T a' \iff a = a' \quad (5.53)$$

$$a \in \text{join}^T t \iff \exists (u : T A), u \in t \wedge a \in u \quad (5.54)$$

$$b \in \text{bind}^T f t \iff \exists (a : A), a \in t \wedge a \in f b \quad (5.55)$$

Proof. We take $a \in \text{join}^T t$ as an example. The notation unfolds to $\text{mapReduce}^T (\text{ret}^{\text{subset}} a) (\text{join } t)$. Note that these each of the expressions below defines a proposition, and then by our assumption of propositional extensionality, equality between propositions coincides with bi-implication.

$$\begin{aligned} & \text{mapReduce}_V^T (\text{ret}^{\text{subset}} a) (\text{join } t) \\ = & \text{mapReduce}_V^T (\text{mapReduce}_V^T (\text{ret}^{\text{subset}} a)) t && \text{Eqn. (5.49)} \\ = & \exists (u : T A), u \in t \wedge \text{mapReduce}_V (\text{ret}^{\text{subset}} a) u && \text{Eqn. (5.31)} \\ = & \exists (u : T A), u \in t \wedge \exists (a' : A), a' \in u \wedge a = a' && \text{Eqn. (5.31)} \\ = & \exists (u : T A), u \in t \wedge a \in u \end{aligned}$$

The other conditions are proved similarly. □

Corollary 5.1.40. *Let T be a free monad generated by a signature functor Σ . Then T is a linearizable monad if and only if Σ is a linearizable functor.*

Proof. This follows immediately from Lemma 4.2.29, which gives an equivalence between natural transformations of type

$$\text{tolist}^\Sigma : \Sigma \rightarrow \text{list}$$

and monad homomorphisms of type

$$\text{tolist}^T : T \rightarrow \text{list}.$$

□

Monoidal Folds over Decorated Monads

Now we incorporate the decoration structure into linearizable monads. Let W be a monoid and assume T is decorated by W . We begin by characterizing how mapdReduce interacts with ret and join .

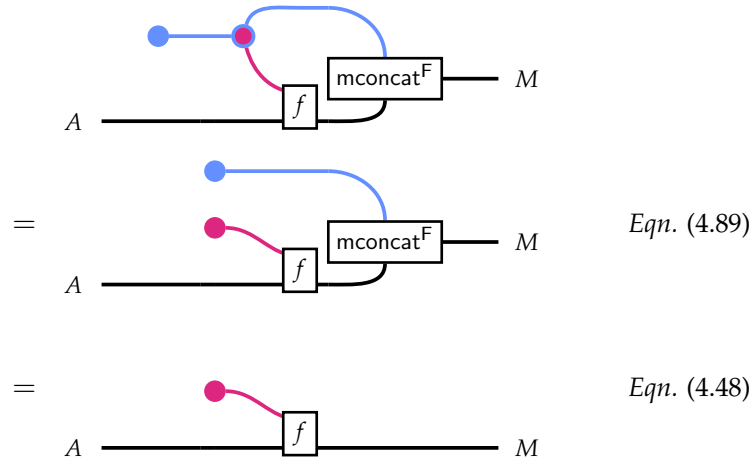
Lemma 5.1.41. *Let T be a linearizable monad decorated by W . The following hold for all $f : W \times A \rightarrow M$*

where M is a monoid.

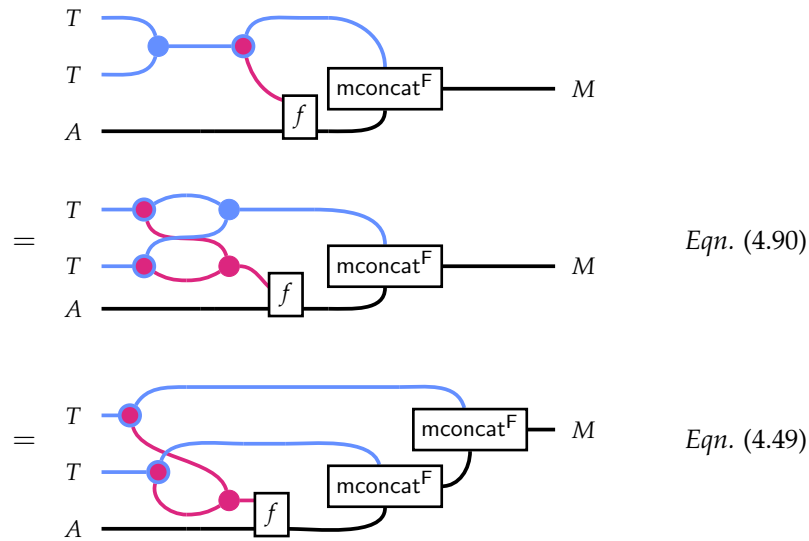
$$\text{mapdReduce}^T f (\text{ret } a) = f(e_M, a) \quad (5.56)$$

$$\text{mapdReduce}^T f (\text{join } t) = \text{mapdReduce}^T \left(\lambda(w, u). \text{mapdReduce}^T (f \odot w) \right) t \quad (5.57)$$

Proof. We give a string-diagrammatic proof of (5.56) as follows:



We give an abbreviated string-diagrammatic proof of (5.57), showing the key steps of the butterfly law for decorated monads and the monad algebra law for join^T :



□

Lemma 5.1.42. *Let T be a linearizable monad decorated by W . The following hold.*

$$(w, a) \in_d \text{ret}^\top a' \iff w = e_W \wedge a = a' \quad (5.58)$$

$$(w, a) \in_d \text{join}^\top t \iff \exists w_1 w_2 u, (w_1, u) \in_d t \wedge (w_2, a) \in u \wedge w = w_1 \cdot w_2 \quad (5.59)$$

Proof. Recall that $(w, a) \in_d t$ is equivalent to $(w, a) \in \text{dec } t$. Therefore, $(w, a) \in_d \text{ret } a'$ is equivalent to $(w, a) \in \text{ret } (e_M, a')$ by (4.89). Equation (5.53) implies $w = e_W$ and $a = a'$. The proof of (5.59) consists of unfolding definitions, applying (5.43), and first-order logic.

$$\begin{aligned} & (w, a) \in_d \text{join}^\top t \\ = & \text{ExistsAnyDec}^\top ((w, a) = -) (\text{join } t) \\ = & \text{ExistsAnyDec}^\top (\lambda (w_1, u). \text{ExistsAnyDec}^\top (((w, a) = -) \odot w_1) u) t \\ = & \exists w_1 u, (w_1, u) \in_d t \wedge (\text{ExistsAnyDec}^\top (((w, a) = -) \odot w_1) u) \\ = & \exists w_1 u, (w_1, u) \in_d t \wedge (\exists w_2 a', (w_2, a') \in_d u \wedge (((w, a) = -) \odot w_1) (w_2, a')) \\ = & \exists w_1 u, (w_1, u) \in_d t \wedge (\exists w_2 a', (w_2, a') \in_d u \wedge (w, a) = (w_1 \cdot w_2, a')) \\ = & \exists w_1 w_2 u, (w_1, u) \in_d t \wedge (w_2, a) \in u \wedge w = w_1 \cdot w_2 \end{aligned}$$

□

The following corollary is proved similarly to the preceding one.

Corollary 5.1.43. *For any $t: T A$ and $f: W \times A \rightarrow T B$ the following holds.*

$$(w, b) \in_d \text{bindd}^\top f t \iff \exists w_1 w_2 a, (w_1, a) \in_d t \wedge (w_2, b) \in_d f (w_1, a) \wedge w = w_1 \cdot w_2 \quad (5.60)$$

5.1.4 Relating Monoidal Folds by Different Functions

We now discuss several reasoning principles about operations defined as monoidal folds. These are used in Section 5.2 to reason about the operations of locally nameless. In each of these propositions, let F be a linearizable functor.

Lemma 5.1.44. *Let $f, g: A \rightarrow M$ where M is a monoid and $t: F A$. If $fa = ga$ for all elements in t , then the result of mapping f over t and reducing is equal to mapping g over t and reducing:*

$$(\forall (a: A), a \in t \implies fa = ga) \implies \text{mapReduce}^F f t = \text{mapReduce}^F g t$$

Proof. This follows by re-expressing the goal using (5.24):

$$(\forall (a: A), a \in \text{tolist } t \implies fa = ga) \implies \text{mapReduce}^{\text{list}} f (\text{tolist } t) = \text{mapReduce}^{\text{list}} g (\text{tolist } t).$$

This property is then a straightforward induction on $\text{tolist } t$. $\square$

Corollary 5.1.45. *Suppose F is decorated by E . Let $f, g: E \times A \rightarrow M$ where M is a monoid and $t: F A$. Then the following holds:*

$$(\forall (e: E) (a: A), (e, a) \in_d t \implies f(e, a) = g(e, a)) \implies \text{mapdReduce}^F f t = \text{mapdReduce}^F g t$$

Proof. This is equivalent to the statement of the previous property for $\text{dec}^F t$. $\square$

Definition 5.1.46. A pre-ordered set $\langle A, \leq \rangle$ is a set paired with a binary relation that is reflexive and transitive.

Definition 5.1.47. () A pre-ordered monoid is a monoid M that also forms a pre-ordered set $\langle M, \leq \rangle$ such that the monoid operation is order-preserving in both arguments. That is,

$$\begin{aligned} m_1 \leq m'_1 &\implies m_1 \cdot m_2 \leq m'_1 \cdot m_2 \\ m_2 \leq m'_2 &\implies m_1 \cdot m_2 \leq m_1 \cdot m'_2 \end{aligned}$$

Lemma 5.1.48. Let $f, g: A \rightarrow M$ where M is a pre-ordered monoid and $t: F A$. If f is less than g for all elements in l , then the result of reducing by f over l is less than that of reducing by g :

$$(\forall (a: A), a \in t \implies f a \leq g a) \rightarrow \text{mapReduce}^F f t \leq \text{mapReduce}^F g t$$

Proof. The goal is equivalent to the following one:

$$(\forall (a: A), a \in \text{tolist } t \implies f a \leq g a) \rightarrow \text{mapReduce}^F f (\text{tolist } t) \leq \text{mapReduce}^{\text{list}} g (\text{tolist } t)$$

We proceed by induction on the list $l \equiv \text{tolist } t$. Suppose l is $[\]$. Then the goal reduces to the following trivial one:

$$(\forall (a: A), \text{False} \implies f a \leq g a) \rightarrow e_M \leq e_M.$$

In the case $l = \text{cons } a' l'$, the goal reduces to the following:

$$(\forall (a: A), a = a' \vee a \in l' \implies f a \leq g a) \rightarrow (f a' \cdot \text{mapReduce}^F f l' \leq g a' \cdot \text{mapReduce}^{\text{list}} g l')$$

The induction hypothesis states the following:

$$(\forall (a: A), a \in l' \implies f a \leq g a) \rightarrow \text{mapReduce}^F f l' \leq \text{mapReduce}^{\text{list}} g l'.$$

The goal follows by applying monotonicity of the monoid operation. $\square$

Corollary 5.1.49. *Suppose F is decorated by E . Let $f, g: E \times A \rightarrow M$ where M is a preordered monoid*

and $t : F A$. Then the following holds.

$$(\forall (e : E) (a : A), (e, a) \in_d t \implies f(e, a) \leq g(e, a)) \rightarrow \text{mapdReduce}^F f t \leq \text{mapdReduce}^F g t$$

Proof. This is equivalent to the statement of the previous property for $\text{dec}^F t$. $\square$

Definition 5.1.50. A lax homomorphism between pre-ordered monoids M and N is a function such that satisfies the following ordered variants of the ordinary monoid laws.

$$e_N \leq \phi(e_M) \tag{5.61}$$

$$\phi(x_1) \cdot \phi(x_2) \leq \phi(x_1 \cdot x_2) \tag{5.62}$$

Lemma 5.1.51. Let $f : A \rightarrow M$ where M is a pre-ordered monoid and $\phi : M \rightarrow N$ be a lax homomorphism. Then the following inequality holds for all $t : F A$.

$$\text{mapReduce}(\phi \circ f) t \leq \phi(\text{mapReduce} f t)$$

Proof. Again we proceed by reducing to the case of list. In case l is the empty list, the left-hand side reduces to $\text{mapReduce}^{\text{list}} [],$ or just e_N . The right-hand side reduces to $\phi(e_M),$ so the goal follows by (5.61). In the inductive case, we must show something of the form $\phi(x) \cdot y \leq \phi(x \cdot z)$ where the inductive hypothesis states $y \leq \phi(z).$ Now we apply monotonicity in the second argument and (5.62) as follows.

$$\phi(x) \cdot y \leq \phi(x) \cdot \phi(z) \leq \phi(x \cdot z)$$

$\square$

It straightforward to extend Lemma 5.1.51 to $\text{mapdReduce}.$

5.2 LOCALLY NAMELESS MONOIDAL FOLDS

We previously defined the substitution operations of locally nameless in Section 4.5. Now, we define the remaining operations **fv** and **lc** as monoidal folds. Let $T : \text{END}_{\text{Set}}$ be a linearizable monad decorated by the monoid $\langle \mathbb{N}, 0, + \rangle.$ We will represent abstract locally nameless syntax trees as terms of type $T \text{ LN}.$

5.2.1 Elements of Terms

We begin by consider the properties of the context-sensitive occurrence relation $(\in_d)$ (Def. 5.1.31). The elements of a term are its occurrences, and the contextual occurrence $(d, v) \in t$ means that variable $v : \text{LN}$ occurs at a depth of $d : \mathbb{N}$ binders in $t.$ We recall that **subst**, **open**, and **close** are all defined in terms of **bindd** or its special cases like **mapd**. Corollaries 5.1.34 and 5.1.43 immediately

imply the following properties characterizing contextual occurrence after performing one of the locally nameless substitution operations. These lemmas will be used shortly in higher-level proofs.

Lemma 5.2.1. (🚫) *The following equivalences hold:*

$$(d, v) \in_d t[x \mapsto u] \iff \exists (d_1, v_1), (d_2, v_1) \in_d t \wedge (d_2, v) \in_d \text{subst}_{\text{loc}} x u v_1 \wedge d = d_1 + d_2 \quad (5.63)$$

$$(d, v) \in_d t^u \iff \exists (d_1, v_1), (d_2, v_1) \in_d t \wedge (d_2, v) \in_d \text{open}_{\text{loc}} u (d_1, v_1) \wedge d = d_1 + d_2 \quad (5.64)$$

$$(d, v) \in_d \backslash^x t \iff \exists v_1, (d, v_1) \in_d t \wedge \text{close}_{\text{loc}} x (d, v_1) = v \quad (5.65)$$

We now give a more specific characterization of context-agnostic occurrence in t after performing a substitution.

Lemma 5.2.2. (🚫) *For all $t, u: T \text{ LN}$, $x \in \text{atom}$, and $v \in \text{LN}$, the following holds:*

$$v \in t[x \mapsto u] \iff (v \in t \wedge v \neq \text{Fr } x) \vee (\text{Fr } x \in t \wedge v \in u) \quad (5.66)$$

Proof. By (5.55), $v \in t[x \mapsto u]$ if and only if there exists some variable y such that $y \in t$ and $v \in \text{subst}_{\text{loc}} x u y$. Let y be such a variable and consider two possibilities. If $y = \text{Fr } x$, then we have $\text{Fr } x \in t$, and $v \in \text{subst}_{\text{loc}} x u (\text{Fr } x) = u$, and we are done. If $y \neq \text{Fr } x$, then $\text{subst}_{\text{loc}} x u y = \text{ret } y$. By (5.53), $v \in \text{ret } y$ if and only if $v = y$, in which case $v \in t$ and $v \neq \text{Fr } x$. $\square$

5.2.2 Local Closure

We now turn our attention to the local closure predicate. $\mathbf{lc}$ is defined using `mapdReduce` using the monoid $\mathbf{Prop}_{\wedge} \equiv \langle \mathbf{Prop}, \wedge, \text{True} \rangle$ of Coq propositions under conjunction, which we gave the short name `ForAllDec`. An LN variable satisfies the local closure property if LN is a free variable or a de Bruijn term $\text{Bd } n$ where n is strictly less than its the value of its binding context. We generalize this definition slightly to allow some “slack,” where a de Bruijn index cannot exceed its depth by a given amount. This leads to the following *local* definition of local closure.

Definition 5.2.3. (🚫) *Local closure is defined $\mathbf{lc } t \equiv \mathbf{lc}_{\text{at}} 0 t$ where $\mathbf{lc}_{\text{at}}$ is defined as follows:*

$$\begin{aligned} \mathbf{lc}_{\text{at}}: \mathbb{N} \rightarrow T \text{ LN} \rightarrow \mathbf{Prop} \\ \mathbf{lc}_{\text{at}} k \equiv \text{ForAllDec } (\mathbf{lc}_{\text{loc}} k) \quad \text{where} \quad \mathbf{lc}_{\text{loc}} k v \equiv \begin{cases} n < d + k & \text{if } v = \text{Bd } n \\ \text{True} & \text{else} \end{cases} \end{aligned} \quad (5.67)$$

As a corollary of Lemma 5.1.33, we can derive the following equivalences for $\mathbf{lc}$ and $\mathbf{lc}_{\text{at}}$:

Corollary 5.2.4. (🔗) *The following specifications characterize local closure:*

$$\mathbf{lc}_{\text{at}} k t \iff \forall (d \in \mathbb{N})(n \in \mathbb{N}), (d, \text{Bd } n) \in_d t \implies n < d + k \quad (5.68)$$

$$\mathbf{lc} t \iff \forall (d \in \mathbb{N})(n \in \mathbb{N}), (d, \text{Bd } n) \in_d t \implies n < d \quad (5.69)$$

5.2.3 Free Variables

The **fv** operation is defined using `mapReduce` for the monoid `list atom`. The definition of **fv**_{loc} can be read from the `var` cases of **fv** in Section 2.3.

Definition 5.2.5. (🔗) *The operation computing sets of free variables is defined as follows:*

$$\begin{aligned} \mathbf{fv} : T \text{ LN} &\rightarrow \text{list atom} \\ \mathbf{fv}^T t &\equiv \text{mapReduce}_{\text{list atom}}^T \mathbf{fv}_{\text{loc}} t \quad \text{where} \quad \mathbf{fv}_{\text{loc}} v \equiv \begin{cases} [x] & \text{if } v = \text{Fr } x \\ [] & \text{else} \end{cases} \end{aligned} \quad (5.70)$$

Remark 5.2.6. *Tealeaves includes additional machinery to convert from `list atom` to `atomset` (🔗), implemented using the `MSets` module in the `Roqc` standard library. This affords the ability to reason about collections of atoms as sets, following an approach similar taken by `Metalib`, a component of `LNgen` [9]. However, `atomset` does not form a monoid in our framework, as the monoid laws do not hold up to propositional equality in `Coq`. Nonetheless, we occasionally treat **fv** as returning a set when convenient.*

Lemma 5.2.7. (🔗) *x is in **fv** t if and only if there is an occurrence of the form $\text{Fr } x$ in t :*

$$x \in \mathbf{fv} t \iff \text{Fr } x \in t \quad (5.71)$$

Proof. Unfolding notation on both sides leads to the following goal:

$$\left(\text{mapReduce}_{\text{list atom}}^T \mathbf{fv}_{\text{loc}} t \right) x \iff \text{mapReduce}_{\text{list atom}}^T (_ = \text{Fr } x) t$$

The left side can be expressed as `evalAt x (mapReducelist atomT fvloc t)` where `evalAt x f = fx`. Now

$$\text{evalAt } x : \text{subset } A \rightarrow \mathbf{Prop}$$

is a homomorphism from $\langle \text{subset } A, \emptyset, \cup \rangle$ to $\langle \mathbf{Prop}, \text{False}, \vee \rangle$, as the following equations show:

$$\begin{aligned} \text{evalAt } x \emptyset &= \text{False} \\ \text{evalAt } x (S_1 \cup S_2) &= S_1 x \vee S_2 x \end{aligned}$$

Using (5.20), the goal reduces to showing

$$\text{mapReduce}_{\vee}^{\top} (\text{evalAt } x \circ \mathbf{fv}_{\text{loc}}) t \iff \text{mapReduce}_{\vee}^{\top} (_ = \text{Fr } x) t.$$

Thus, it suffices to show for all v that $x \in (\mathbf{fv}_{\text{loc}} v)$ holds if and only if $v = \text{Fr } x$, which is immediate by case analysis. $\square$

We are now in a position to prove some of the core properties of locally nameless listed in Figure 2.1. For the first theorem, `FV_SUBST_LOWER`, we give two proofs of a somewhat different nature. First, we examine a proof that uses the characterization of the set-membership relation given by Lemma 5.2.1. Then, we give a different proof which is more directly rooted in first principles of `mapReduce`. The first proof is arguably higher-level and closer in practice to how an informal argument might proceed. The second exhibits the use of pointwise (in)equational reasoning, a general theme which extends to many other proofs about syntactic operations.

Theorem 5.2.8 ( `FV_SUBST_LOWER`).

$$\mathbf{fv } t \setminus \{x\} \subseteq \mathbf{fv } (t[x \mapsto u])$$

Proof. We can rewrite the goal as an implication

$$\forall (y : \text{atom}), y \in (\mathbf{fv } t \setminus \{x\}) \implies y \in \mathbf{fv } (t[x \mapsto u])$$

Using (5.71), the left-hand side is equivalent to $\text{Fr } y \in t \wedge y \neq x$. Also by (5.71), $y \in \mathbf{fv } (t[x \mapsto u])$ only if $\text{Fr } y \in t[x \mapsto u]$. Using (5.66), the latter is equivalent to the disjunction

$$(\text{Fr } y \in t \wedge \text{Fr } y \neq \text{Fr } x) \wedge (\text{Fr } x \in t \wedge \text{Fr } y \in u).$$

Thus, the conclusion holds immediately via the left disjunct. $\square$

We now give another proof that invokes the properties of `mapdReduce` over preordered monoids more directly. The pre-ordered monoid here is `atomset` under union, preordered by set inclusion. Note that the property in question is an inequality for this monoid.

Theorem 5.2.9 ( `FV_SUBST_LOWER, alt.`).

$$\mathbf{fv } t \setminus \{x\} \subseteq \mathbf{fv } (t[x \mapsto u])$$

Proof. We begin with the observation that

$$(_ \setminus \{x\}) : \text{subset atom} \rightarrow \text{subset atom}$$

is a homomorphism with respect to subsets under union. That is, the following hold.

$$\begin{aligned}\emptyset \setminus \{x\} &= \emptyset \\ (s_1 \setminus \{x\}) \cup (s_2 \setminus \{x\}) &= (s_1 \cup s_2) \setminus \{x\}\end{aligned}$$

Thus, the left side can be rewritten as follows:

$$\begin{aligned}(\mathbf{fv} \, t) \setminus \{x\} \\ &= \text{mapReduce} (\mathbf{fv}_{\text{loc}}) \setminus \{x\} && \text{Eqn. (5.70)} \\ &= \text{mapReduce} ((_ \setminus \{x\}) \circ \mathbf{fv}_{\text{loc}}) && \text{Eqn. (5.20)}\end{aligned}$$

while the right-hand side is

$$\begin{aligned}\mathbf{fv} (\text{subst } x \, u \, t) \\ &= \text{mapReduce} (\mathbf{fv}_{\text{loc}}) \circ \text{bind} (\text{subst}_{\text{loc}} \, x \, u) \, t && \text{Eqns. (5.70) and (4.121)} \\ &= \text{mapReduce} (\text{mapReduce } \mathbf{fv}_{\text{loc}} \circ \text{subst}_{\text{loc}} \, x \, u) && \text{Eqn. (5.50)} \\ &= \text{mapReduce} (\mathbf{fv} \circ \text{subst}_{\text{loc}} \, x \, u) && \text{Eqn. (5.70)}\end{aligned}$$

Now we invoke the properties of folding over preordered monoids. By Lemma 5.1.48, it suffices to show for each $v \in t$ the following inclusion:

$$(\mathbf{fv}_{\text{loc}} \, v) \setminus \{x\} \subseteq \mathbf{fv} (\text{subst}_{\text{loc}} \, x \, u \, v)$$

That is, we can prove the top-level inequality by demonstrating the inequality at each individual occurrence in t . We proceed by case analysis. If $v = \text{Fr } y$ and $y = x$, the goal reduces to $\emptyset \subseteq \mathbf{fv} \, u$, which is vacuously true. If $v = \text{Fr } y$ and $y \neq x$, the goal reduces to $\{y\} \subseteq \{y\}$. If v is $\text{Bd } n$, the goal reduces to $\emptyset \subseteq \emptyset$. This completes the proof. $\square$

We prove the next result similarly. This time, we must invoke the rules for lax monoidal homomorphisms.

Theorem 5.2.10 ( FV_SUBST_UPPER).

$$\mathbf{fv} (t[x \mapsto u]) \subseteq \mathbf{fv} \, t \setminus \{x\} \cup \mathbf{fv} \, u$$

Proof. We previously showed that the LHS expression can be rewritten to

$$\text{mapReduce} (\mathbf{fv} \circ \text{subst}_{\text{loc}} \, x \, u) \, t.$$

On the right, we recall that $(_ \setminus \{x\})$ is a monoid homomorphism. This observation allows push-

ing the $\setminus \{x\}$ under mapReduce, to leave the RHS in the following state:

$$\text{mapReduce } ((_ \setminus \{x\}) \circ \mathbf{fv}_{\text{loc}}) \cup \mathbf{fv } u$$

To push the union under mapReduce, we note that $(_ \cup \mathbf{fv } u)$ is a *lax* homomorphism on subset atom. Specifically, we have the following (in)equalities:

$$\begin{aligned} \emptyset &\subseteq \emptyset \cup \mathbf{fv } u \\ (S_1 \cup S_2) &= (S_1 \cup \mathbf{fv } u) \cup (S_2 \cup \mathbf{fv } u) \end{aligned}$$

By Lemma 5.1.51, we find the following inequality:

$$\text{mapReduce } ((_ \cup \mathbf{fv } u) \circ (_ \setminus \{x\})) \circ \mathbf{fv}_{\text{loc}} \subseteq (\mathbf{fv } t \setminus \{x\} \cup \mathbf{fv } u)$$

It is of course valid to replace the RHS of an inequality with a lesser value. Doing so leaves the top-level goal in the following form:

$$\text{mapReduce } (\mathbf{fv} \circ \text{subst}_{\text{loc}} x u) t \subseteq \text{mapReduce } ((_ \cup \mathbf{fv } u) \circ (_ \setminus \{x\})) \circ \mathbf{fv}_{\text{loc}}$$

Using Lemma 5.1.48, it suffices to show the following inequality for all variables in t :

$$\mathbf{fv} (\text{subst}_{\text{loc}} x u v) \subseteq (\mathbf{fv}_{\text{loc}} v \setminus \{x\}) \cup \mathbf{fv } u$$

We proceed by case analysis. If $v = \text{Fr } x$, the goal reduces to the following true inclusion:

$$\mathbf{fv } u \subseteq (\{x\} \setminus \{x\}) \cup \mathbf{fv } u$$

In case $v = \text{Fr } y$ and $y \neq x$, goal reduces to the following inclusion:

$$\{y\} \subseteq (\{y\} \setminus \{x\}) \cup \mathbf{fv } u$$

In the event $v = \text{Bd } n$, the goal reduces to $\emptyset \subseteq \mathbf{fv } u$ and we are done. □

Theorem 5.2.11 ( FV_OPEN_LOWER).

$$\mathbf{fv } t \subseteq \mathbf{fv} (t^u) \tag{5.72}$$

Proof. Using (5.71), the goal is equivalent to the following:

$$\forall (x : \text{atom}), \text{Fr } x \in t \implies \text{Fr } x \in t^u$$

By an immediate corollary of (5.64), we have

$$\text{Fr } x \in t'' \iff \exists d v, (d, v) \in_d t \wedge \text{Fr } x \in \text{open}_{\text{loc}} u (d, v).$$

Assume $\text{Fr } x \in t$. By (5.40), there is some d such that $(d, \text{Fr } x) \in_d t$. It suffices to show that open_{loc} leaves this occurrence unchanged. That is, we simply confirm the following:

$$\text{Fr } x \in \text{open}_{\text{loc}} u (d, \text{Fr } x)$$

This follows by simplifying open_{loc} and using (5.53). □

Theorem 5.2.12 (🚫 FV_OPEN_UPPER).

$$\mathbf{fv} (t'') \subseteq \mathbf{fv} t \cup \mathbf{fv} u \tag{5.73}$$

Proof. Using (5.71), the goal is equivalent to the following:

$$\forall (x: \text{atom}), \text{Fr } x \in t'' \implies \text{Fr } x \in t \vee \text{Fr } x \in u$$

Assume $\text{Fr } x \in t''$. We have seen that this implies there are d and v such that $(d, v) \in_d t$ and $\text{Fr } x \in \text{open}_{\text{loc}} u (d, v)$. We proceed by case analysis on v ; there two cases consistent with $\text{Fr } x \in \text{open}_{\text{loc}} u (d, v)$. First, we could have $v = \text{Bd } d$ and $\text{Fr } x \in u$, in which case we are done. The other case is that v is some other value such that $\text{Fr } x \in \text{ret } v$. By (5.53), the latter is true iff $v = \text{Fr } x$. In this case, we have $(d, \text{Fr } x) \in t$. By (5.41), this implies $\text{Fr } x \in t$ and we are done. □

Unfortunately, these are the only lemmas from Figure 2.1 that can be proven for decorated linearizable monads. Indeed, the remaining lemmas in that figure *do not hold* in general and require a more sophisticated abstraction for reasons we examine in the next section.

5.3 THE CATEGORY OF TRAVERSABLE FUNCTORS

While linearizable monads provide a way to collapse a structure into a monoid using `mapReduce`, this operation alone is not enough to fully capture the sense in which first-order terms are containers of variable occurrences, or the sense in which we would like to define operations that *iterate* over the occurrences.

For present purposes, there are two related limitations. First, the `mapReduce` operation is not expressive enough to define the partial translation operations defined in Section 3.4, which return values wrapped in the option monad. These operations can be thought of as iterating over the occurrences in a term one at a time to translate them individually, returning an error if and only if they encounter an error at one of these occurrences. However, this kind of “iteration,” such as it

is, does not return a value in a monoid but an optional term, depending on whether the operation succeeds, so it is not an instance of `mapReduce`.

Second—and this is a severe limitation—the axioms of `mapReduce` fail to rule out badly behaved implementations that do not satisfy that properties one might expect of such an operation. We begin by turning attention to this topic before defining traversable functors as a way of ruling out such examples.

5.3.1 Pathological Monoidal Folds

The properties proved in Section 5.1.4 relate folds by different functions, which was used extensively in to reason about the generalized locally nameless **fv** operation in Section 5.2. Now consider the following candidate property which relates *maps* by different functions.

Definition 5.3.1. *Let F be a linearizable functor. Let $f, g: A \rightarrow B$. The linearizable structure on F is said to be full if for all $t: F A$, whenever f and g are equal when applied to elements in t , mapping f over t gives the same result as mapping g over t .*

$$(\forall (a: A), a \in t \implies f a = g a) \implies \text{map}^F f t = \text{map}^F g t \quad (5.74)$$

Note that the above definition is agnostic about whether linearizability is defined in terms of `tolist`, `mapReduce`, or `mconcat`. However defined, the intuition for Definition 5.3.1 is that the operation must include every element, without skipping any.

Example 5.3.2. *The `mapReducelist` operation defined in Section 5.1.1 is full.*

Example 5.3.3. *The `tolist` operation for Σ^λ defined in Example 5.1.17 is full.*

In general, a law-abiding implementation of `mapReduceF` need not be full—the axioms leave freedom to skip elements.

Example 5.3.4. *We will define a monad homomorphism from `lam` to `list` that does not satisfy (5.74). By Corollary 5.1.40, a monad homomorphism corresponds precisely to a natural transformation from Σ^λ to `list`. So, consider the following one:*

$$\begin{aligned} \text{tolist}^{\Sigma^\lambda} (\text{app } a_1 a_2) &= [a_1] \\ \text{tolist}^{\Sigma^\lambda} (\text{abs } a) &= [a] \end{aligned}$$

This operation corresponds to the following implementation of `mapReducelam`, which applies its argument to only the left-most variable occurring in a term.

$$\begin{aligned} \text{mapReduce}^{\text{lam}} f (\text{var } v) &= f v \\ \text{mapReduce}^{\text{lam}} f (\text{app } t_1 t_2) &= \text{mapReduce}^{\text{lam}} f t_1 \\ \text{mapReduce}^{\text{lam}} f (\text{abs } t) &= \text{mapReduce}^{\text{lam}} f t \end{aligned}$$

The reader can verify this operation satisfies both (5.48) and (5.49). However, it is not full: any two functions that are equal on the left-most variable in t will satisfy the premise of (5.74), but if they disagree on any other elements, the conclusion does not hold.

In Example 5.3.4, $x \in t$ holds if and only if x is the left-most variable in t . Note that the specifications of ForAll and ExistsAny given in Corollary 5.1.27 are still true for the preceding example, but the names of the combinators are misnomers, as they only quantify over the left-most element of terms. More precisely, a linearizable structure *defines* what it means to be an element, but this does not need to coincide with one's intuition that any occurrence of $\text{var } v$ in lam should be considered an element.

It is far from obvious how to constrain operations like toList and mapReduce axiomatically to rule out this behavior. The trick is to generalize from monoids to *monoidal functors*, thus moving from linearizable functors to *traversable* functors. This both radically increases the expressiveness of the abstraction and introduces a number of critical reasoning principles, such as the fullness of mapReduce .

5.3.2 Applicative Functors

Lax monoidal functors between arbitrary monoidal categories were introduced in Definition 4.3.6, with the motivating example being the embedding $(A \mapsto A^\times)$ of SET into END_{SET} . In this section, we are interested specifically in lax monoidal functors of type $\text{SET} \rightarrow \text{SET}$. Recall that such a functor $G: \text{SET} \rightarrow \text{SET}$ is equipped with operations

$$\begin{aligned} \epsilon &: \mathbf{1} \rightarrow G \mathbf{1} \\ (\otimes) &: \forall (A, B: \text{SET}), G A \times G B \rightarrow G (A \times B) \end{aligned}$$

subject to equations similar to monoid laws. The following definition is equivalent, but presented in a more useful form.

Definition 5.3.5. *An applicative functor is a type constructor $G: \text{SET} \rightarrow \text{SET}$ equipped with operations*

$$\begin{aligned} \text{pure}^G &: \forall (A: \text{SET}), A \rightarrow G A \\ (\otimes)^G &: \forall (A B: \text{SET}), G (A \rightarrow B) \rightarrow G A \rightarrow G B \end{aligned}$$

subject to the following equations (note that $(\otimes)$ is left-associative):

$$\text{pure id} \otimes a = a \quad (5.75) \quad g \otimes (f \otimes a) = \text{pure } (_ \circ _) \otimes g \otimes f \otimes a \quad (5.77)$$

$$\text{pure } f \otimes \text{pure } a = \text{pure } (fa) \quad (5.76) \quad f \otimes \text{pure } a = \text{pure } (\text{evalAt } a) \otimes f \quad (5.78)$$

We mention the following fact without proof (see [55], Exercise 2).

Lemma 5.3.6. *Any expression formed from pure and $\otimes$ (only) can be rewritten using (5.75)–(5.78) into a normal form of the following shape, where the $\{t_i\}$ do not mention pure :*

$$\text{pure } f \otimes t_1 \otimes t_2 \dots \otimes t_n$$

Remark 5.3.7. *The infix notation $(_ \circ _)$ in (5.77) denotes the function $(\lambda g. \lambda f. g \circ f)$. This convention extends to other infix binary operators, with the first argument understood to appear on the left. In (5.78), $\text{evalAt } a$ denotes $(f \mapsto fa)$. The operator $(\otimes)$ is known as (applicative or effectful) application.*

Definitions 5.3.5 and 4.3.6 are equivalent in settings like SET or the internal category of Coq, where all functors are equipped with a suitably well-behaved tensorial strength [59]. However, the applicative presentation is much more convenient for present purposes.

Lemma 5.3.8. *A lax monoidal functor is applicative where pure^G and $(\otimes)^G$ are defined as follows:*

$$A \xrightarrow{\rho_A} A \times \mathbf{1} \xrightarrow{A \times \epsilon} A \times G \mathbf{1} \xrightarrow{\text{st}} G(A \times \mathbf{1}) \xrightarrow{G\rho^{-1}} G A \quad (5.79)$$

$$G(A \rightarrow B) \times G B \xrightarrow{(\otimes)} G(A \rightarrow B \times B) \xrightarrow{G \text{eval}} G B \quad (5.80)$$

In the final operation, eval is the function that maps (f, a) to fa .

Lemma 5.3.9. *An applicative functor is, in particular, a lax monoidal functor with the following definitions:*

$$\text{map}^G f a = \text{pure}^G f \otimes a \quad (5.81)$$

$$\epsilon^G = \text{pure}^G * \quad (5.82)$$

$$a \otimes^G b = \text{pure}^G (\lambda ab. (a, b)) \otimes^G a \otimes^G b \quad (5.83)$$

Remark 5.3.10. *When providing specific instances of applicative functors, I will sometimes define the structure in terms of pure and $(\otimes)$, and sometimes in terms of map , pure , and $(\otimes)$, depending on which presentation is cleanest.*

Examples of Applicative Functors

We examine a few examples of interesting and useful applicative functors. Recall that a lax monoidal functor G has the property that a monoid on a set M yields a monoid on $G M$ (Lem. 4.3.7). It should not be surprising that the simplest examples of such functors are given by monoids themselves, realized as constant functors. We also indicate how some of these examples will be used in later sections.

Lemma 5.3.11. *Let M be a monoid. The constant functor $\overline{M}$ (Def. 4.1.50) forms an applicative functor, while a monoid homomorphism forms an applicative homomorphism.*

Proof. For this lemma, we use Definition 4.3.6. The morphism $\mathbf{1} \rightarrow \overline{M} \mathbf{1}$ corresponds exactly to the monoid unit e_M , while the natural transformation

$$\overline{M} A \times \overline{M} B \rightarrow \overline{M} (A \times B)$$

is exactly (the constant natural transformation corresponding to) multiplication in M . The monoid laws and applicative homomorphism laws reduce to the ones for monoids. $\square$

In fact, is straightforward to show that for *any* constant functor $\overline{A}$, operations satisfying Definition 5.3.5 correspond to a monoid structure on A .

Example 5.3.12. The option functor (Ex. 4.2.7) is an applicative functor where $\text{pure}^{\text{opt}} a$ is $\text{Some } a$ and $(\otimes^{\text{opt}})$ is defined as follows:

$$\begin{aligned} \text{Some } f \otimes \text{Some } a &\equiv \text{Some } (fa) \\ \text{None } \otimes _ &\equiv \text{None} \\ _ \otimes \text{None} &\equiv \text{None} \end{aligned}$$

Traversing by option provides a semantics for map-like operations that can fail, such as the representation translations presented in Section 5.6.

Example 5.3.13. Define an applicative structure on list where $\text{pure}^{\text{list}} a$ is the singleton $[a]$ and $(\otimes^{\text{list}})$ is defined as follows:

$$\begin{aligned} [] \otimes _ &\equiv [] \\ \text{cons } f \ l \otimes l' &\equiv (\text{map}^{\text{list}} f \ l') \ ++ \ (l \otimes l') \end{aligned}$$

Note that $(\otimes^{\text{list}})$ applies a list of functions to each value in a list of arguments, concatenating the entire set of results. The associated multiplication operation forms the Cartesian product of two lists. For instance,

$$[a, b, c] \otimes [1, 2] = [(a, 1), (a, 2), (b, 1), (b, 2), (c, 1), (c, 2)]$$

Example 5.3.14. Define an applicative structure on subset as follows:

$$\begin{aligned} \text{pure}^{\text{subset}} a &\equiv \text{ret}^{\text{subset}} a \\ (S \otimes S') b &\equiv \exists (f: A \rightarrow B) (a: A), S f \wedge S' a \wedge f a = b \end{aligned}$$

That is, $\text{pure } a$ forms a singleton. If S is a set of functions and S' is a set of values, then $S \otimes S'$ is the set of all values of the form fa where f is one of the functions and a is one of the values.

Alternatively, the multiplication $(S \otimes S') (a, b)$ is defined $S a \wedge S' b$. Traversing by subset provides a semantics for map-like operations that are non-deterministic. This also has a different application: it is used in Section 6.1.5 to define *relations*, rather than functions, on syntax trees.

Example 5.3.15. *The following type constructor does not form an applicative functor in Coq’s type theory:*

$$\text{decidable_subset } A \equiv A \rightarrow \text{bool}$$

While the type $\text{decidable_subset } A$ has the benefit that we can compute whether such a set contains a particular element, this does not form a functor if A is allowed to be an arbitrary type. Given a function from $f: A \rightarrow B$, there is not generally a procedure to determine whether a given $b: B$ is in the image of f .

This (non-)example affects our ability to compute certain predicates—e.g. the property of being α -equivalent—as functions that return boolean values. Instead, our approach formalizes such predicates as proposition-valued functions using the subset functor. This does not preclude formalizing decidable predicates entirely; rather, any such formalization would have to avoid relying on reasoning principles that require decidable subsets to form a functor.

Example 5.3.16. *Let W be a monoid. Define an applicative structure on $W^\times$ as follows:*

$$\begin{aligned} \text{pure}^{W^\times} a &\equiv \text{ret}^{W^\times} a \\ (w_1, f) \otimes (w_2, a) &\equiv (w_1 \cdot w_2, f a) \end{aligned}$$

If a value of type $W \times A$ is regarded as an A value in a monoidal context, then $\text{pure}^{W^\times}$ places elements into an empty context; $\otimes$ applies a contextual function to a contextual value and places the result into a context formed by the products of the two contexts. Traversing by $W^\times$ provides a semantics for map-like operations that generate a log value at every element and accumulate the individual log values into a combined log.

Example 5.3.17. *Any monad comes equipped with an applicative functor where pure^T is ret^T and $(\otimes)^T$ and $(\otimes^T)$ are defined as follows:*

$$\begin{aligned} f \otimes a &= \text{bind}^T \left(\lambda (f': A \rightarrow B). \text{map}^T f' a \right) f \\ a \otimes b &= \text{bind}^T \left(\lambda (a': A). \text{map}^T (\lambda (b': B). (a', b')) b \right) a \end{aligned}$$

Examples 5.3.12, 5.3.13, and 5.3.14 all correspond to monad instances defined in Examples 4.2.7, 4.2.5, and 4.2.6, respectively. Example 5.3.16 corresponds to the Writer monad defined in Corollary 4.3.13.

Example 5.3.18. *The identity functor is an applicative functor where $\text{pure}^{\mathbb{I}}$ is the identity function, $f \otimes a$ is just fa , and $a \otimes b$ is just (a, b) .*

Lemma 5.3.19. *The composition of two applicative functors is also applicative, where the operations are defined as follows:*

$$\text{pure}^{G_1 \circ G_2} a \equiv \text{pure}^{G_1} (\text{pure}^{G_2} a) \quad (5.84)$$

$$f \circledast^{G_1 \circ G_2} a \equiv \text{pure}^{G_1} (\circledast^{G_2}) \circledast^{G_1} f \circledast^{G_1} a \quad (5.85)$$

$$a \otimes^{G_1 \circ G_2} b \equiv \text{map}^{G_1} (\otimes^{G_2}) (a \otimes^{G_1} b) \quad (5.86)$$

□

Applicative Homomorphisms

Homomorphisms between applicative functors are defined in a way that closely mirrors homomorphisms between monoids.

Definition 5.3.20. *An applicative morphism $\phi: G_1 \Rightarrow G_2$ is a natural transformation that respects the monoidal structure of applicative functors:*

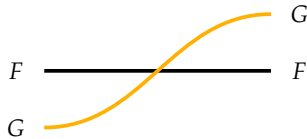
$$\phi (\text{pure}^{G_1} a) = \text{pure}^{G_2} a \quad (5.87) \quad \phi (f \circledast a) = \phi f \circledast \phi a \quad (5.88)$$

Of course, homomorphisms between constant applicative functors correspond exactly to homomorphisms between monoids.

5.3.3 Traversals as Applicative Distributive Laws

A traversable functor is the analogue of a linearizable functor where monoids are generalized to applicative functors. Rather than *collapsing* a functor into a monoid, we *distribute* a functor over a monoidal functor, which is to say we cross a functor wire over a monoidal wire. Extending the color scheme by which monadic wires are drawn in blue and comonoidal wires are drawn in red, applicative wires are drawn in yellow. The following definition mirrors the characterization of linearizable functors in terms of mconcat (Lem. 5.1.21).

Definition 5.3.21 (Traversable Functor). *A traversable functor $\langle F, \text{dist} \rangle$ is a functor F equipped with an operation*



$$\text{dist}^F : \forall (G : \text{Applicative}) (A : \text{SET}), F(G A) \rightarrow G(F A)$$

subject to the following axioms:

$$\text{dist}_1^F = \text{id}_{FA} \quad (5.89)$$

$$\text{dist}_{G_1 \circ G_2}^F = \text{map}^{G_1} \text{dist}_{G_2}^F \circ \text{dist}_{G_1}^F \quad (5.90)$$

$$\text{dist}_{G_2}^F \circ \text{map}^F(\phi_A) = \phi_{FA} \circ \text{dist}_{G_1}^F \quad (5.91)$$

In (5.91), ϕ is universally quantified over all homomorphisms between applicative functors G_1 and G_2 .

Remark 5.3.22. In the diagram for (5.89), the dotted line indicates the identity functor, which is ordinarily not drawn. In (5.90), the boxes highlight the difference between applying a single distribution of a composition of two functors, and a sequence of two individual distributions.

We give a few examples of traversable functors before examining the axioms in greater detail and demonstrating several *non-examples* to illustrate their practical significance.

Example 5.3.23. The list functor is traversable. Let G be any applicative functor and compare the following definition of $\text{dist}_G^{\text{list}}$ to the operation $\text{mconcat}^{\text{list}}$ given in Definition 5.1.1.

$$\text{dist}_G^{\text{list}} [] \equiv \text{pure}^G [] \quad (5.92)$$

$$\text{dist}_G^{\text{list}} (\text{cons } a \, l) \equiv \text{pure}^G \text{cons } \ast^G a \ast^G \text{dist}_G^{\text{list}} l \quad (5.93)$$

For example, consider a list of several option values, say $l = [\text{Some } a, \text{Some } b, \text{Some } c]$. Then

$$\text{dist}_{\text{opt}}^{\text{list}} l = \text{Some } [a, b, c]$$

On the other hand, applying $\text{dist}_{\text{opt}}^{\text{list}}$ to $[\text{Some } a, \text{Some } b, \text{None}]$ results in

$$\text{Some cons } \ast^{\text{opt}} \text{Some } a \ast^{\text{opt}} (\text{Some cons } \ast^{\text{opt}} \text{Some } b \ast^{\text{opt}} \text{None})$$

which simplifies to None . As another example, consider a list of values paired with a monoidal

value, say $l = [(w_1, a), (w_2, b), (w_3, c)]$. Then

$$\text{dist}_{W \times}^{\text{list}} l = (w_1 \cdot w_2 \cdot w_3, [a, b, c])$$

Example 5.3.24. Let $\Delta: \text{SET} \rightarrow \text{SET}$ be the diagonal (or duplication) functor mapping any set A to $A \times A$, where $\text{map}^\Delta f$ maps f over both components. This functor is traversable where dist^Δ is defined as follows:

$$\text{dist}_G^\Delta (\text{pair } a_1 a_2) = \text{pure}^G \text{pair} \otimes a_1 \otimes a_2$$

Example 5.3.25. The following two definitions **do not** constitute legitimate traversable structures on Δ because they do not satisfy Equation (5.89):

$$\begin{aligned} \text{dist}_G (\text{pair } a_1 a_2) &= \text{pure}^G \text{pair} \otimes a_1 \otimes a_1 \\ \text{dist}_G (\text{pair } a_1 a_2) &= \text{pure}^G \text{pair} \otimes a_2 \otimes a_1 \end{aligned}$$

Traversing Δ by the identity functor using the first operation maps (a_1, a_2) to (a_1, a_1) , while traversing using the second operation returns (a_2, a_1) .

The previous example demonstrates that traversals are not allowed to replace one element by another or to rearrange them. That is not to say law-abiding dist must visit the elements in any *particular* order, provided they are eventually inserted into their appropriate position. Consider the following example which traverses a list “backwards.”

Example 5.3.26. list is also traversable by the following definition, where $\text{dist}^{\text{list}}$ refers to the operation given in Example 5.3.23.

$$\text{dist}^{\text{rev}} l \equiv \text{map}^G \text{reverse} \left(\text{dist}_G^{\text{list}} (\text{reverse } l) \right)$$

This operation reverses the list before applying the distribution, but then maps over the applicative functor to reverse the list again afterwards, returning the final result to its original order. For instance, using this operation on the example $[(w_1, a), (w_2, b), (w_3, c)]$ results in

$$\text{dist}_{W \times}^{\text{list}} l = (w_3 \cdot w_2 \cdot w_1, [a, b, c]).$$

Notice that the list is given in its original order, while the logging effect represented by the applicative functor is in a reversed order. This operation satisfies both (5.89) and (5.90).

More generally, any permutation and its inverse can be used in place of reverse in the previous example. Thus, while every dist operation codifies some order on the elements of a data structure, a traversable functor does not inherently have a “preferred” order in which to visit its elements. The next example shows that elements cannot be visited twice.

Example 5.3.27. *The following operation does not constitute a legitimate traversable structure on Δ :*

$$\text{dist}_G (\text{pair } a_1 a_2) = \text{pure}^G (x, y, z \mapsto \text{pair } x z) \circledast a_1 \circledast a_1 \circledast a_2$$

The operation in Example 5.3.27 appears to duplicate the element a_1 , but the duplicated element is discarded by the function $(x, y, z \mapsto \text{pair } x z)$. What this operation duplicates is the *applicative effect* of a_1 —the value added to the log if G is the Writer monad, for instance. Unlike the operations shown in Example 5.3.25, this operation satisfies (5.89). However, it does not satisfy (5.90). As an example, consider the following term of type $\Delta (\text{list } (\text{list } \mathbb{N}))$.

$$l = ([[], [2]], [[3]])$$

Now the LHS of (5.90) simplifies as follows.

$$\begin{aligned} \text{dist}_{\text{list} \circ \text{list}} l &= [[(x, y, z \mapsto \text{pair } x z)]] \circledast [[[], [2]]] \circledast [[[], [2]]] \circledast [[3]] \\ &= [[[], [], [], [(2, 3)]]] \end{aligned}$$

While the RHS of (5.90) simplifies as follows.

$$\begin{aligned} \text{map dist}_{\text{list}} (\text{dist}_{\text{list}} l) &= \text{map dist}_{\text{list}} ([[(x, y, z \mapsto \text{pair } x z)]] \circledast [[[], [2]]] \circledast [[[], [2]]] \circledast [[3]]) \\ &= [[[], [], [(2, 3)], [(2, 3)]]] \end{aligned}$$

The broader point demonstrated in the previous example is that dist is not permitted to duplicate applicative effects. Combined with the no-rearrangement property exhibited in Example 5.3.25, one begins to sense that an applicative functor must visit every element exactly once, and otherwise leave the structure unmodified.

Example 5.3.28. Σ^λ is traversable where $\text{dist}^{\Sigma^\lambda}$ is defined as follows:

$$\begin{aligned} \text{dist}_G^{\Sigma^\lambda} (\text{app } a_1 a_2) &\equiv \text{pure}^G_{\text{app}} \circledast a_1 \circledast a_2 \\ \text{dist}_G^{\Sigma^\lambda} (\text{abs } a) &\equiv \text{pure}^G_{\text{abs}} \circledast a \end{aligned}$$

Compare the previous example to the poorly-behaved definition of tolist seen in Example 5.3.4, which discards the value a_2 in the app case. It is difficult to see how to similarly disregard a_2 in the above example without running afoul of (5.89).

Monoidal Folds over Traversable Functors

A traversable functor is indeed a linearizable functor by specializing the applicative functor to a constant, i.e. a monoid (Lem. 5.3.11). Of the three equivalent definitions of linearizable functors,

dist corresponds to the presentation in terms of mconcat.

Lemma 5.3.29. *A traversable functor can be equipped with an mconcat operation satisfying the conditions of Lemma 5.1.21.*

Proof. Let M be a monoid. The mconcat operation is defined by specializing dist to the constant applicative functor on M . This operation specializes to the following type:

$$\begin{array}{ccc} & & \overline{M} \\ & \nearrow & \\ F & \text{---} & F \\ & \searrow & \\ & & \overline{M} \end{array} \quad \text{dist}_{\overline{M}}^F: F M \rightarrow M$$

Lemma 5.3.11 observed that an applicative homomorphism from $\overline{M}$ to $\overline{N}$ is equivalent to a monoid homomorphism $M \rightarrow N$. Therefore this operation commutes with monoid homomorphisms by Equation (5.91), making F a linearizable functor. $\square$

One of the advantages of taking dist^F rather than mconcat^F as a primitive operation is that dist^F is subject to Equations (5.89) and (5.90), which have no analogue in Lemma 5.1.21. The examples shown earlier indicated that these equations tightly constrain the freedom of dist, hence of mconcat, to rearrange elements, skip them, or visit them twice. It should not be surprising to find that that derived linearizable structure on F is necessarily full. In fact, we can strengthen the property by raising the implication to an equivalence.

Lemma 5.3.30. *The following equivalence holds for all traversable functors F , $t: F A$ and $f, g: A \rightarrow B$.*

$$(\forall (a: A), a \in t \implies fa = ga) \iff (\text{map}^T f t = \text{map}^T g t) \quad (5.94)$$

Combined with the functor identity law (4.21), (5.94) also implies the following law.

$$(\forall (a: A), a \in t \implies fa = a) \iff (\text{map}^T f t = t) \quad (5.95)$$

The infrastructure necessary to prove Lemma 5.3.30 will not be presented until Chapter 6. We will return to this property at that time, but not before we use this property in Section 5.5 to complete the remaining proofs about the locally nameless representation.

5.3.4 The Category of Traversable Functors

We now examine the (monoidal) categorical structure of traversable functors.

Definition 5.3.31. *A traversable homomorphism $\psi: F_1 \Rightarrow F_2$ is a natural transformation between traversable functors that commutes with distribution.*

$$\begin{array}{c} F_1 \\ \hline \boxed{\psi} \\ \hline G \end{array} \begin{array}{c} G \\ \hline \hline \end{array} = \begin{array}{c} F_1 \\ \hline \hline \end{array} \begin{array}{c} G \\ \hline \boxed{\psi} \\ \hline G \end{array} \begin{array}{c} G \\ \hline \hline \end{array} \quad \text{dist}_G^{F_2} \circ \psi = \text{map}^G(\psi) \circ \text{dist}_G^{F_1} \quad (5.96)$$

Definition 5.3.32. *The category TRAV of traversable functors is given by the following data:*

- Objects are traversable functors $\langle F, \text{dist}^F \rangle$
- Morphisms are traversable homomorphisms

Much as with linearizable functors and linearizable monads, or with decorated functors and decorated monads, there is a predictable pattern for deriving a notion of traversable monad: first show that the category TRAV is monoidal—which amounts to showing it contains the identity function and is closed under composition—and then consider monoids in this category. We do so now.

Definition 5.3.33 (Identity Traversable Functor). *The identity functor is canonically traversable by the identity function:*

$$\begin{array}{c} \mathbb{1} \\ \hline \hline \end{array} \begin{array}{c} G \\ \hline \hline \end{array} \equiv \begin{array}{c} G \\ \hline \hline \end{array} \begin{array}{c} G \\ \hline \hline \end{array} \quad \text{dist}_G^{\mathbb{1}} = \text{id}_G \quad (5.97)$$

The fact this defines a traversable functor is clear, as all of the equations hold trivially. It is almost as easy to verify that traversable functors are closed under composition.

Definition 5.3.34 (Composition of Traversables). *Let $\langle F_1, \text{dist}^{F_1} \rangle$ and $\langle F_2, \text{dist}^{F_2} \rangle$. Then $F_1 \circ F_2$ is a traversable functor where $\text{dist}^{F_1 \circ F_2}$ is defined as follows:*

$$\begin{array}{c} F_1 \circ F_2 \\ \hline \hline \end{array} \begin{array}{c} G \\ \hline \hline \end{array} \equiv \begin{array}{c} F_1 \\ \hline \hline \end{array} \begin{array}{c} G \\ \hline \hline \end{array} \begin{array}{c} F_2 \\ \hline \hline \end{array} \begin{array}{c} G \\ \hline \hline \end{array} \quad \text{dist}_G^{F_1 \circ F_2} = \text{dist}_G^{F_1} \circ \text{map}^{F_1} \text{dist}_G^{F_2} \quad (5.98)$$

Definition 5.3.35 (Monoidal Structure on TRAV). *TRAV is a monoidal category by the following data:*

- The monoidal unit **I** is the identity functor paired with the identity traversal:

$$\mathbf{I} \equiv \langle \mathbb{1}, \text{dist}^{\mathbb{1}} \rangle$$

- The composition of two traversable functors $\langle F_1, \text{dist}^{F_1} \rangle$ and $\langle F_2, \text{dist}^{F_2} \rangle$ is given by Def. 5.3.34.

$$\langle F_1, \text{dist}^{F_1} \rangle \otimes \langle F_2, \text{dist}^{F_2} \rangle \equiv \langle F_1 \circ F_2, \text{dist}^{F_1 \circ F_2} \rangle$$

To show that this forms a monoidal category, we must check that $\text{dist}^{\mathbb{1} \circ F}$ and $\text{dist}^{F \circ \mathbb{1}}$ are equivalent to dist^F , as well as to check that $\text{dist}^{(F \circ G) \circ H}$ and $\text{dist}^{F \circ (G \circ H)}$. These proofs are trivial.

5.3.5 Traversable Functors as Extension Systems

As with decorated functors and monads, traversable functors can be characterized in terms of an operation that lifts functions of a particular form over the functor. The following definition is the traversable analogue of the specification for mapReduce^F given by Lemma 5.1.20.

Definition 5.3.36. *A traversable functor presented as an extension system is a set-forming operation $F: \text{SET} \rightarrow \text{SET}$ equipped with an operation*

$$\text{traverse}^F: \forall (G: \text{Applicative}) (A: \text{SET}), (A \rightarrow G B) \rightarrow F A \rightarrow G (F B) \quad (5.99)$$

subject to the following equations, where (5.102) is quantified over applicative homomorphisms:

$$\text{traverse}_{\mathbb{1}}^F \text{id} = \text{id}_F \quad (5.100)$$

$$\text{map}^{G_1} (\text{traverse}_{G_2}^F g) \circ \text{traverse}_{G_1}^F f = \text{traverse}_{G_1 \circ G_2}^F (\text{map}^{G_1} g \circ f) \quad (5.101)$$

$$\phi_F \circ \text{traverse}_{G_1}^F f = \text{traverse}_{G_2}^F (\phi \circ f) \quad \left(\text{all } \phi: G_1 \xrightarrow{\text{appl}} G_2 \right) \quad (5.102)$$

Lemma 5.3.37. *Definitions 5.3.21 and 5.3.36 are equivalent.*

Proof. Given map and dist , traverse can be defined as follows:

$$\text{traverse}_G f = \text{dist}_G \circ \text{map } f \quad (5.103)$$

Given traverse , map and dist can be defined as follows:

$$\text{map } f = \text{traverse}_{\mathbb{1}} f \quad (5.104)$$

$$\text{dist}_G = \text{traverse}_G \text{id} \quad (5.105)$$

The fact that the relevant axioms in each case are satisfied is easily shown, as is the fact that either presentation uniquely determines the other. $\square$

5.3.6 Traversable Monads

Definition 5.3.38 (Traversable Monad). *A traversable monad is a monoid in the category TRAV .*

A traversable monad is, of course, both an ordinary monad and a traversable functor. However, as an object in TRAV , it is subject to the additional conditions that both ret and join must

commute with the dist operations of their domain and codomain. Explicitly, this leads to the following axioms:

$$\begin{array}{c} \text{G} \\ \text{G} \end{array} \begin{array}{c} \text{G} \\ \text{T} \end{array} \begin{array}{c} \text{G} \\ \text{T} \end{array} = \begin{array}{c} \text{G} \\ \text{G} \end{array} \begin{array}{c} \text{G} \\ \text{T} \end{array} \begin{array}{c} \text{G} \\ \text{T} \end{array} \quad \text{dist}_G^T \circ \text{ret}^T = \text{map}^G(\text{ret}^T) \quad (5.106)$$

$$\begin{array}{c} \text{T} \\ \text{T} \\ \text{G} \end{array} \begin{array}{c} \text{G} \\ \text{T} \end{array} \begin{array}{c} \text{G} \\ \text{T} \end{array} = \begin{array}{c} \text{T} \\ \text{T} \\ \text{G} \end{array} \begin{array}{c} \text{G} \\ \text{T} \end{array} \begin{array}{c} \text{G} \\ \text{T} \end{array} \quad \text{dist}_G^T \circ \text{join}^T = \text{map}^G(\text{join}^T) \circ \text{dist}_G^T \circ \text{map}^T(\text{dist}_G^T) \quad (5.107)$$

Example 5.3.39. *lam is a traversable monad where dist^{lam} is defined as follows.*

$$\begin{aligned} \text{dist}_G^{\text{lam}}(\text{var } v) &\equiv \text{pure}^G \text{var} \otimes v \\ \text{dist}_G^{\text{lam}}(\text{app } t_1 t_2) &\equiv \text{pure}^G \text{app} \otimes \text{dist}_G^{\text{lam}} t_1 \otimes \text{dist}_G^{\text{lam}} t_2 \\ \text{dist}_G^{\text{lam}}(\text{abs } t) &\equiv \text{pure}^G \text{abs} \otimes \text{dist}_G^{\text{lam}} t \end{aligned}$$

Naturally, a traversable monad is necessarily a linearizable monad. Recalling that $\text{map}^{\overline{\text{M}}} f$ is the identity function, equations (5.51) and (5.52) are precisely (5.106) and (5.107) specialized to constant applicative functors.

5.3.7 Traversable Free Monads

We now show that a freely generated monad inherits traversability, including the traversable monads laws, from its generating functor.

Theorem 5.3.40. *If Σ is a traversable functor, then T is a traversable functor with the following traversal:*

$$\text{dist}_{G,A}^T \equiv \text{sigfold}^T \left(\text{map}^G \mu_A^* \circ \text{dist}_{G,A}^\Sigma \right) \circ \text{map}^T \left(\text{map}^G \text{ret}_A^T \right) \quad (5.108)$$

Proof. First, we want to show that this constitutes a natural transformation:

$$\text{map}^{G \circ T} f \circ \text{dist}_G^T = \text{dist}_G^T \circ \text{map}^{T \circ G} f$$

It is trivial to verify that $\text{map}^{G \circ T} f$ is a Σ -algebra homomorphism, as both map^G and dist_G^Σ are natural transformations:

$$\text{map}^G \left(\text{map}^T f \right) \circ \text{map}^G \mu_A^* \circ \text{dist}_{G,A}^\Sigma = \text{map}^G \mu_B^* \circ \text{dist}_{G,B}^\Sigma \circ \text{map}^G \left(\text{map}^T f \right)$$

Thus, we have the following:

$$\begin{aligned}
& \text{map}^{G \circ T} f \circ \text{dist}_G^T \\
= & \text{map}^G \left(\text{map}^T f \right) \circ \text{sigfold}^T \left(\text{map}^G \mu_A^* \circ \text{dist}_G^\Sigma \right) \circ \text{map}^G \text{ret}^T && \text{Eqns. (5.108) (4.26)} \\
= & \text{sigfold}^T \left(\text{map}^G \mu_B^* \circ \text{dist}_G^\Sigma \right) \circ \text{map}^T \left(\text{map}^G \left(\text{map}^T f \right) \right) \circ \text{map}^T \left(\text{map}^G \text{ret}^T \right) && \text{Lem. 4.2.27} \\
= & \text{sigfold}^T \left(\text{map}^G \mu_B^* \circ \text{dist}_G^\Sigma \right) \circ \text{map}^T \left(\text{map}^G \text{ret}^T \right) \circ \text{map}^T \left(\text{map}^G f \right) && (4.22) \text{ and naturality} \\
= & \text{dist}_G^T \circ \text{map}^{T \circ G} f && \text{Eqns. (5.108) (4.26)}
\end{aligned}$$

Next we verify that traversing by $\mathbb{1}$ is the identity function. Equation (5.100) specializes as follows, where we have unfolded all instances of $\text{map}^\mathbb{1}$, which is the identity function:

$$\text{sigfold}^T \left(\mu_A^* \circ \text{dist}_\mathbb{1}^\Sigma \right) \circ \text{map}^T \text{ret}^T = \text{id}_{TA}$$

By assumption, we know $\text{dist}_\mathbb{1}^\Sigma = \text{id}$. This reduces the goal the one of the monad laws (4.29), because $\text{sigfold}^T \mu^*$ is precisely the definition of join^T (Lem. 4.2.23). For space we do not give the proof of the traversal linearity law (5.90), though it is very much in the same line as the ones above and below.

□

Just as a freely generated monad is decorated if the generating functor is decorated (Theorem 4.3.44), a monad generated by a traversable functor is traversable.

Theorem 5.3.41. *If Σ is a traversable functor, then T is a traversable monad.*

Proof. First we verify the unit law (5.106) using Equation (4.50).

$$\begin{aligned}
& \text{sigfold}^T \left(\text{map}^G \mu_A^* \circ \text{dist}_G^\Sigma \right) \circ \text{map}^T \left(\text{map}^G \text{ret}^T \right) \circ \text{ret}^T \\
= & \text{sigfold}^T \left(\text{map}^G \mu_A^* \circ \text{dist}_G^\Sigma \right) \circ \text{ret}^T \circ \left(\text{map}^G \text{ret}^T \right) && \text{Naturality of ret} \\
= & \text{map}^G \text{ret}^T && \text{Eqn. (4.50)}
\end{aligned}$$

Next we verify the join law (5.107) using Equation (4.51)

$$\begin{aligned}
& \text{sigfold}^T \left(\text{map}^G \mu_A^* \circ \text{dist}_G^\Sigma \right) \circ \text{map}^T \left(\text{map}^G \text{ret}^T \right) \circ \text{join}^T \\
= & \text{sigfold}^T \left(\text{map}^G \mu_A^* \circ \text{dist}_G^\Sigma \right) \circ \text{join}^T \circ \left(\text{map}^{T \circ T \circ G} \text{ret}^T \right) && \text{Naturality of join} \\
= & \text{sigfold}^T \left(\text{map}^G \mu_A^* \circ \text{dist}_G^\Sigma \right) \circ \text{map}^T \left(\text{sigfold}^T \left(\text{map}^G \mu_A^* \circ \text{dist}_G^\Sigma \right) \right) \circ \left(\text{map}^{T \circ T \circ G} \text{ret}^T \right) && \text{Eqn. (4.51)} \\
= & \text{sigfold}^T \left(\text{map}^G \mu_A^* \circ \text{dist}_G^\Sigma \right) \circ \text{map}^T \left(\text{sigfold}^T \left(\text{map}^G \mu_A^* \circ \text{dist}_G^\Sigma \right) \circ \text{map}^{T \circ G} \text{ret}^T \right) && \text{Eqn. (4.22)} \\
= & \text{sigfold}^T \left(\text{map}^G \mu_A^* \circ \text{dist}_G^\Sigma \right) \circ \text{map}^T \text{dist}_G^T && \text{Eqn. (5.108)} \\
= & \text{sigfold}^T \left(\text{map}^G \mu_A^* \circ \text{dist}_G^\Sigma \right) \circ \text{map}^{T \circ G} \left(\text{join}^T \circ \text{ret}^T \right) \circ \text{map}^T \text{dist}_G^T && \text{Eqns. (4.21) (4.28)} \\
= & \text{sigfold}^T \left(\text{map}^G \mu_A^* \circ \text{dist}_G^\Sigma \right) \circ \text{map}^{T \circ G} \text{join}^T \circ \text{map}^{T \circ G} \text{ret}^T \circ \text{map}^T \text{dist}_G^T && \text{Eqn. (4.22)} \\
= & \text{map}^G \text{join}^T \circ \text{sigfold}^T \left(\text{map}^G \mu_A^* \circ \text{dist}_G^\Sigma \right) \circ \text{map}^T \left(\text{map}^G \text{ret}^T \right) \circ \text{map}^T \text{dist}_G^T && \text{Lem. 4.2.27} \\
= & \text{map}^G \text{join}^T \circ \text{dist}_G^T \circ \text{map}^T \left(\text{dist}_G^T \right) && (5.108)
\end{aligned}$$

□

Lemma 5.3.30 has the following analogue for bind.

Lemma 5.3.42. *The following implication holds for all $t: T A$ and $(f, g: A \rightarrow T B)$.*

$$(\forall (a: A), a \in t \implies fa = ga) \implies (\text{bind}^T f t = \text{bind}^T g t) \quad (5.109)$$

Combined with the bind identity law (4.33), (5.109) implies the following for all $(t: T A)$ and $(f: A \rightarrow T A)$.

$$(\forall a: A, a \in t \implies fa = \text{ret } a) \implies (\text{bind}^T f t = t) \quad (5.110)$$

Note that the implication in Lemma 5.3.42 is one-sided and not an equivalence as in Lemma 5.3.30. Such an equivalence does not hold for an arbitrary traversable monad. A counterexample to the stronger condition can be seen by considering a traversable monad that is not freely generated, such as list.

Example 5.3.43. *list is a traversable monad with the definitions from Ex. 4.2.5 and Ex. 5.3.23. Now consider the list $l = [1, 2]$ and the following two functions of type $\mathbb{N} \rightarrow \text{list } \mathbb{N}$:*

$$f(1) = [3, 4] \quad f(2) = [5] \quad g(1) = [3] \quad g(2) = [4, 5].$$

Now $\text{bind}^{\text{list}} f l = [3, 4, 5] = \text{bind}^{\text{list}} g l$, but f and g are not equal on all elements of l . Now consider

$$h(1) = [] \quad h(2) = [1, 2]$$

Now $\text{bind}^{\text{list}} h l = [1, 2] = l$, but $h a \neq \text{ret}^{\text{list}}$ for any $a \in l$. Therefore neither of (5.109) and (5.110) are equivalences for an arbitrary traversable monad.

Remark 5.3.44. The above counterexamples arise from the fact that $\text{join}^{\text{list}}$ discards structural information about the inner lists when flattening a list-of-lists. I conjecture that when restricted to freely generated traversable monads on SET , which are the syntax-like ones, the above properties can be strengthened to equivalences. How to nicely capture this from within Tealeaves is an open question.

Traversable monads can also be characterized as an extension system by combining bind and traverse into a single operation.

Definition 5.3.45. A traversable monad presented as an extension system is a type constructor $T : \text{SET} \rightarrow \text{SET}$ equipped with a bindt operation (for “bind with traverse”)

$$\text{bindt}^T : \forall (G : \text{Applicative}) (A : \text{SET}), (A \rightarrow G (T B)) \rightarrow T A \rightarrow G (T B) \quad (5.111)$$

subject to the following equations, where (5.114) is quantified over applicative homomorphisms.

$$\text{bindt}_1^T \text{ret}^T = \text{id}_T \quad (5.112)$$

$$\text{map}^{G_1} (\text{bindt}_{G_2}^T g) \circ \text{bindt}_{G_1}^T f = \text{bindt}_{G_1 \circ G_2}^T (\text{map}^{G_1} (\text{bindt}_{G_2}^T g) \circ f) \quad (5.113)$$

$$\phi_T \circ \text{bindt}_{G_1}^T f = \text{bindt}_{G_2}^T (\phi_T \circ f) \quad (5.114)$$

The following equivalence, an analogue of Lemma 5.1.37, is stated without proof. Rather than giving a proof here, we will prove an even stronger statement—Theorem 5.4.22—after incorporating the decoration structure in Section 5.4.

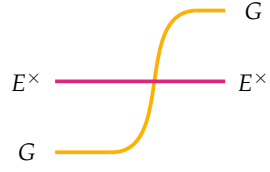
Lemma 5.3.46. Definitions 5.3.38 and 5.3.45 are equivalent.

5.4 THE CATEGORY OF DECORATED TRAVERSABLE FUNCTORS

We have shown that functors decorated by a monoid, or functors equipped with a traversal, both form monoidal categories, giving rise to decorated monads and traversable monads. Now we integrate the two concepts into a single abstraction: decorated-traversable functors. To define this abstraction, we must consider how the traversal and decoration interact on a functor equipped with both structures. We start by observing that the environment functor $E^\times$ is itself traversable.

Recall that the *tensorial strength* (Def. 4.3.4) is the operation $\text{st}_A^F : A \times F B \rightarrow F (A \times B)$ that maps an outer A -value over a functor to annotate every element in F (inasmuch as F is thought of as having “elements”) with a copy of this value. This operation can also be read as a distributive law $(A^\times) \circ F \Rightarrow F \circ (A^\times)$ that commutes any functor of the form $A^\times$ to the right of any other functor in SET .

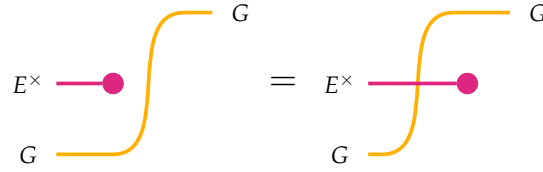
Lemma 5.4.1. $E^\times$ is canonically traversable where $\text{dist}_G^{E^\times}$ is simply the tensorial strength.



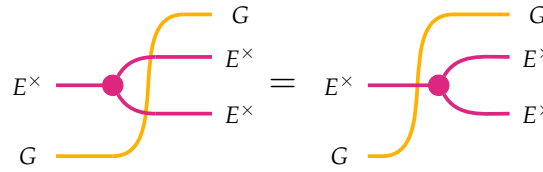
$$\text{dist}_G^{E^\times} \equiv \text{st}_E^G \quad (5.115)$$

Because $\text{dist}^{E^\times}$ is simply tensorial strength, the operation also enjoys the nice properties discussed in Section 4.1.3. In fact, the comonad operations on $E^\times$ both commute with the traversal, establishing the next lemma.

Lemma 5.4.2. For any set E , the Reader comonad on $E^\times$ is a comonad in TRAV . That is, the counit $\text{extract}^{E^\times} : E^\times \Rightarrow 1$ commutes with $\text{dist}^{E^\times}$ and dist^1 , while $\text{dup}^{E^\times}$ commutes with $\text{dist}^{E^\times}$ and $\text{dist}^{E^\times \circ E^\times}$.



$$\text{extract}^{E^\times} = \text{map}^G \text{extract}^{E^\times} \circ \text{dist}_G^{E^\times} \quad (5.116)$$

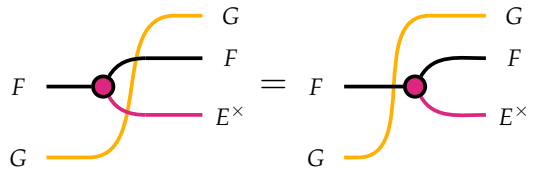


$$\text{dist}^{E^\times \circ E^\times} \circ \text{dup}^{E^\times} = \text{map}^G \text{dup}^{E^\times} \circ \text{dist}_G^{E^\times} \quad (5.117)$$

□

It is only natural to consider a similar condition for decorated functors.

Definition 5.4.3. A decorated traversable functor $\langle F, \text{dec}^F, \text{dist}^F \rangle$ is both a decorated functor $\langle F, \text{dec}^F \rangle$ (Def. 4.3.14) and a traversable functor $\langle F, \text{dist}^F \rangle$ (Def. 5.3.21) such that dec^F and dist^F are related by the following equation:



$$\text{dist}_G^{F \circ E^\times} \circ \text{dec}^F = \text{map}^G \text{dec}^F \circ \text{dist}_G^F \quad (5.118)$$

Example 5.4.4. Σ^λ is a decorated (by N) traversable functor with the decoration defined in Ex. 4.3.19 and distribution defined in Ex. 5.3.28.

We do not give a proof of (5.118) for the above example, as the proof relies on a general strategy of rewriting discussed in the Chapter A, which gives a fully explicit proof of the axioms instance for lam.

Definition 5.4.5. A homomorphism $\phi: F \Rightarrow G$ between two decorated traversable functors F and G is a natural transformation that commutes with both decorations and distributions. That is, ϕ must satisfies Def. 4.3.21 and Def. 5.3.31.

Because homomorphisms of both decorated and traversable functors contain the identity and are closed under composition, decorated traversable functors form a category.

Definition 5.4.6. Let $E \in \text{SET}$ be a set. The category DEC_E of decorated-traversable functors has decorated traversable functors as objects and their homomorphisms as morphisms.

5.4.1 Decorated Traversable Functors as Extension Systems

Much like the other class of functors considered so far, decorated traversable functors can be presented in terms of a single operation mapdt consolidating the effects of both mapd (Def. 4.3.26) and traverse (Def. 5.3.36). This operation lifts a function that is both context-sensitive and generates an applicative effect over a decorated traversable functor.

Definition 5.4.7. A decorated traversable functor is a set constructor $F: \text{SET} \rightarrow \text{SET}$ paired with an operation

$$\text{mapdt}^F: \forall (G: \text{Applicative}) (A B: \text{SET}), (E \times A \rightarrow G B) \rightarrow F A \rightarrow G (F B)$$

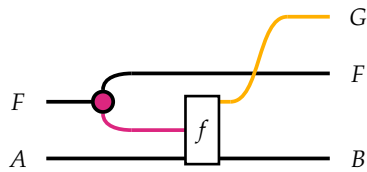
such that the following hold.

$$\text{mapdt}^F \text{extract}^{E^\times} = \text{id}_{FA} \quad (5.119)$$

$$\text{map}^{G_1} (\text{mapdt}^F g) \circ \text{mapdt}^F f = \text{mapdt}^F (\text{map}^{G_1} g \circ \text{mapdt}^{E^\times} f) \quad (5.120)$$

Remark 5.4.8. We draw attention to the appearance of $\text{mapdt}^{E^\times}$ in the composition law (5.120). This implicitly invokes the fact that $E^\times$ is itself a traversable functor decorated by E , as proven by (5.117).

Lemma 5.4.9. (🐼) A decorated traversable functor admits a presentation as an extension system. The operation mapdt is defined as follows:



$$\text{mapdt}^F f \equiv \text{dist}_G^F \circ \text{map}^F f \circ \text{dec}^T \quad (5.121)$$

□

Lemma 5.4.10. (🚗) *A decorated traversable functor presented as an extension system forms a decorated traversable functor. The operations map , dec , and dist are defined from mapdt as follows:*

$$\text{map}^F f \equiv \text{mapdt}_{\mathbb{I}} (f \circ \text{extract}^{E^\times}) \quad (5.122)$$

$$\text{dec}^F \equiv \text{mapdt}_{\mathbb{I}} \text{id}_{E \times A} \quad (5.123)$$

$$\text{dist}_G^F \equiv \text{mapdt}_G (\text{extract}^{E^\times}) \quad (5.124)$$

□

Lemma 5.4.11. (🚗) *Definitions 5.4.3 and 5.4.7 are equivalent.*

□

The following lemma is the analogue of Lemmas 5.3.30 and 5.3.42 for decorated traversable functors.

Lemma 5.4.12. *The following equivalences hold for all $t: F A$ and $(f: E \times A \rightarrow A)$.*

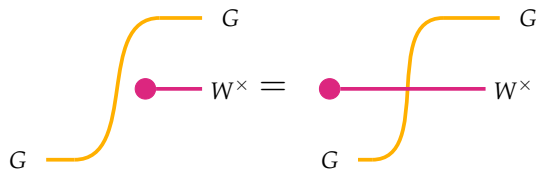
$$\forall (e: E) (a: A), (e, a) \in_d t \implies f(e, a) = g(e, a) \iff \text{mapd}^F f t = \text{mapd}^F g t \quad (5.125)$$

$$\forall (e: E) (a: A), (e, a) \in_d t \implies f(e, a) = a \iff \text{mapd}^F f t = t \quad (5.126)$$

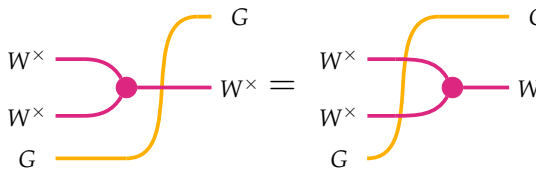
5.4.2 Monoidal Structure

Just as functors decorated by a monoid form a monoidal category DEC_W (Def. 4.3.34), and traversable functors form the monoidal category TRAV (Def. 5.3.35), decorated traversable functors form a monoidal category DECTRAV_W . We develop this category now.

Lemma 5.4.13. *The writer monad is a traversable monad.*



$\text{map}^G \text{ret}^{W^\times} = \text{dist}_G^{W^\times} \circ \text{ret}^{W^\times} \quad (5.127)$



$\text{dist}^{W^\times} \circ \text{join}^{W^\times} = \text{map}^G \text{join}^{W^\times} \circ \text{dist}_G^{W^\times \circ W^\times} \quad (5.128)$

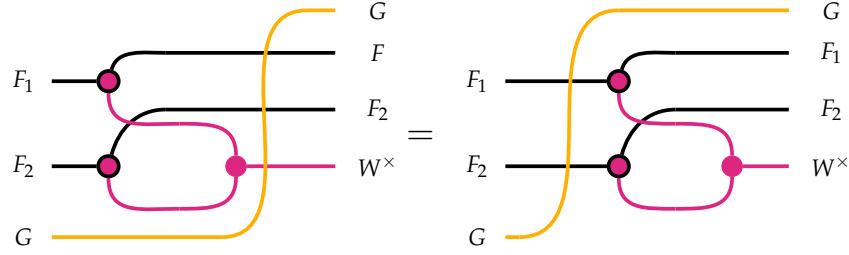
The traversability of $W^\times$ endows DECTRAV_W with a monoidal structure.

Lemma 5.4.14. *The identity functor is decorated traversable.*

Proof. By unfolding the decoration on the identity functor (Def. 4.3.31) and distribution over the identity functor (Def. 5.3.33), this statement reduces to (5.127). $\square$

Lemma 5.4.15. *The composition of decorated traversable functors is decorated traversable.*

Proof. This is the statement of the following equality, which is proven by applying (5.118) and (5.128):



$\square$

Definition 5.4.16. DECTRAV_W is a monoidal category with the following definitions.

- The monoidal unit $\mathbf{I}$ is the identity functor paired with the null decoration and identity traversal:

$$\mathbf{I} \equiv \langle \mathbb{1}, \text{dec}^{\mathbb{1}}, \text{dist}^{\mathbb{1}} \rangle$$

- The composition of two decorated traversable functors $\langle F, \text{dec}^{F_1}, \text{dist}^{F_1} \rangle$ and $\langle F_2, \text{dec}^{F_2}, \text{dist}^{F_2} \rangle$ is defined as follows:

$$\langle F_1, \text{dec}^{F_1}, \text{dist}^{F_1} \rangle \otimes \langle F_2, \text{dec}^{F_2}, \text{dist}^{F_2} \rangle \equiv \langle F_1 \circ F_2, \text{dec}^{F_1 \circ F_2}, \text{dist}^{F_1 \circ F_2} \rangle$$

Definition 5.4.17. A decorated-traversable monad is a monoid in DECTRAV_W . Explicitly, it is a tuple $\langle T, \text{ret}^T, \text{join}^T, \text{dec}^T, \text{dist}^T \rangle$ where T is a functor such that all of the following hold:

- $\langle T, \text{ret}^T, \text{join}^T, \text{dec}^T \rangle$ is a decorated monad (Def. 4.3.37)
- $\langle T, \text{ret}^T, \text{join}^T, \text{dist}^T \rangle$ is a traversable monad (Def. 5.3.38)
- $\langle T, \text{dec}^T, \text{dist}^T \rangle$ is a decorated traversable functor (Def. 5.4.3)

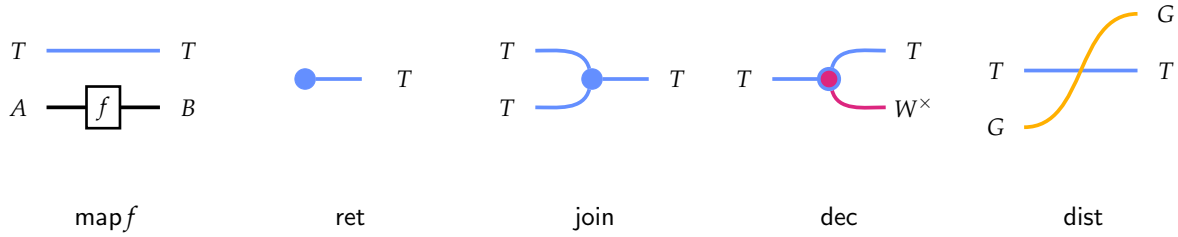
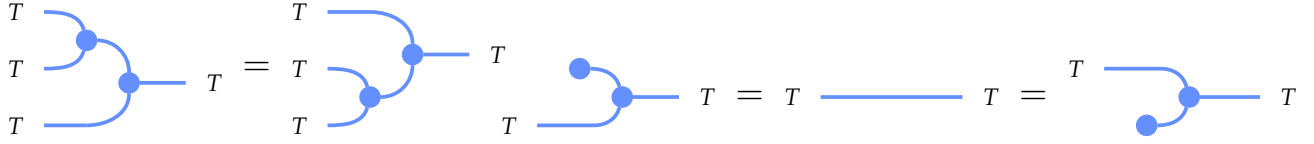


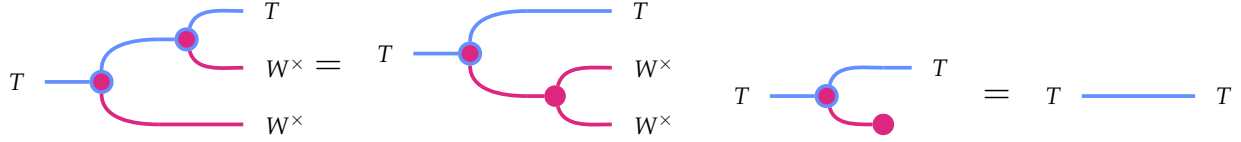
Figure 5.1: The five constituent operations of a DTM

These operations are summarized in Figure 5.1. The monad operations are required to commute with both decorations and traversals, and likewise the decoration and traversal commute with each other. This yields a total of 5 operations (including map) subject to 19 equations. The full set of equations are shown in Figure 5.2. Note that the equations stating that ret, join, dec, and dist are natural transformations, and the two function laws for map, are implicit in the string diagrammatic notation, leaving 13 string diagram identities.



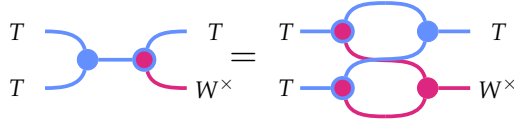
(a) Monad associativity

(b) Monad left and right unit laws

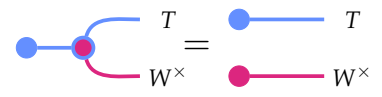


(c) Decorated functor coassociativity

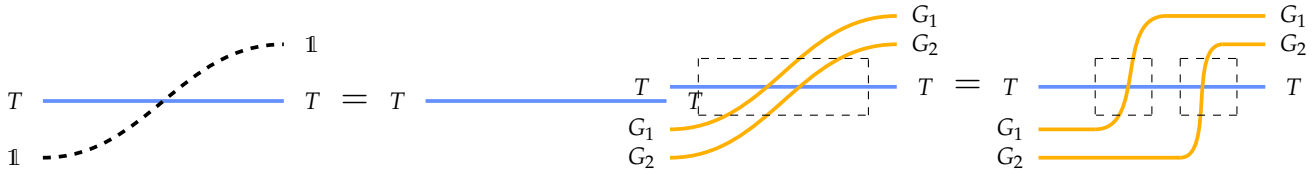
(d) Decorated functor identity



(e) Decorated monad butterfly

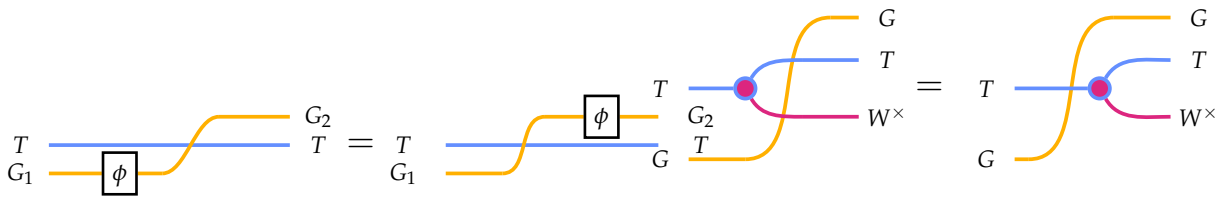


(f) Decorated monad cup law



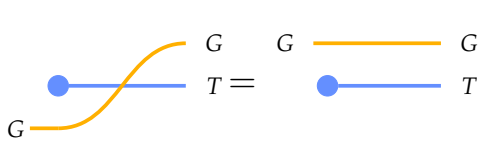
(g) Traversable functor identity law

(h) Compositionality diagrams

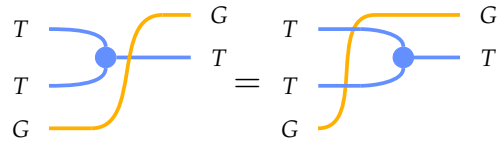


(i) Homomorphism law diagrams

(j) Decorated traversable functor law



(k) Traversable monad unit law



(l) Traversable monad join law

Figure 5.2: DTM axioms

Theorem 5.4.18. *If Σ is a decorated traversable functor, then T is a decorated traversable monad.*

Proof. We use the characterization of a DTM as a structure that is simultaneously a decorated monad, a traversable monad, and a decorated traversable functor. The former two conditions were proved in Theorem 4.3.44 and 5.3.41. The remaining condition we must verify is the following proof stating that decoration and traversal commute.

$$\begin{aligned}
& \text{dist}_G^{T \circ W^\times} \circ \text{dec}^T \\
= & \text{dist}_G^{T \circ W^\times} \circ \text{sigfold}^T \alpha^{\text{dec}} \circ \text{map}^T \left(\text{ret}^T \circ \text{ret}^{W^\times} \right) && \text{Eqn. (4.94)} \\
= & \text{sigfold}^T \gamma \circ \text{map}^T \left(\text{dist}_G^{T \circ W^\times} \right) \circ \text{map}^T \left(\text{ret}^T \circ \text{ret}^{W^\times} \right) && \text{Eqn. (4.94)} \\
= & \text{sigfold}^T \gamma \circ \text{map}^T \left(\text{dist}_G^{T \circ W^\times} \circ \text{ret}^T \circ \text{ret}^{W^\times} \right) && \text{Eqn. (4.94)} \\
= & \text{sigfold}^T \gamma \circ \text{map}^T \left(\text{map}^G \left(\text{ret}^T \circ \text{ret}^{W^\times} \right) \right) && \text{Eqn. (4.94)} \\
= & \text{map}^G \left(\text{sigfold}^T \alpha^{\text{dec}} \right) \circ \text{dist}_G^T \circ \text{map}^{T \circ G} \left(\text{ret}^T \circ \text{ret}^{W^\times} \right) \\
= & \text{map}^G \left(\text{sigfold}^T \alpha^{\text{dec}} \right) \circ \text{map}^{G \circ T} \left(\text{ret}^T \circ \text{ret}^{W^\times} \right) \circ \text{dist}_G^T && \text{Naturality of } \text{dist}_G^T \\
= & \text{map}^G \left(\text{sigfold}^T \alpha^{\text{dec}} \circ \text{map}^T \left(\text{ret}^T \circ \text{ret}^{W^\times} \right) \right) \circ \text{dist}_G^T && \text{Eqn. (4.22)} \\
= & \text{map}^G \text{dec}^T \circ \text{dist}_G^T && \text{Eqn. (4.94)}
\end{aligned}$$

□

5.4.3 Decorated Traversable Monads as Extension Systems

Now we consider the presentation of DTMs as extension systems, a perspective that offers a more concise and practical specification of the same structure. This presentation axiomatizes an operation binddt that generalizes bindd (Def. 4.3.46), bindt (Def. 5.3.45), and mapdt (Def. 5.4.7).

Definition 5.4.19. () *A decorated traversable monad presented as an extension system is a type constructor $T : \text{SET} \rightarrow \text{SET}$ equipped with two operations of the following types:*

$$\begin{aligned}
& \text{ret} \quad (A : \text{SET}) && : A \rightarrow T A \\
& \text{binddt} \quad (\text{Applicative } G) \quad (A B : \text{SET}) && : (W \times A \rightarrow G (T B)) \rightarrow (T A \rightarrow G (T B))
\end{aligned}$$

These subject to the following laws:

$$\text{binddt}_G f \circ \text{ret}^T = f \circ \text{ret}^{W \times} \quad (5.129)$$

$$\text{binddt}_{\mathbb{I}} \left(\text{ret}^T \circ \text{extract}^{W \times} \right) = \text{id}_{T A} \quad (5.130)$$

$$\text{map}^{G_1} \left(\text{binddt}_{G_2} g \right) \circ \left(\text{binddt}_{G_1} f \right) = \text{binddt}_{G_1 \circ G_2} (g \star f) \quad (5.131)$$

$$\phi \circ \text{binddt}_{G_1} f = \text{binddt}_{G_2} (\phi \circ f) \quad \left(\text{all } \phi: G_1 \xrightarrow{\text{appl}} G_2 \right) \quad (5.132)$$

In (5.131), the generalized Kleisli composition operation of functions

$$f: W \times A \rightarrow G_1 (T B)$$

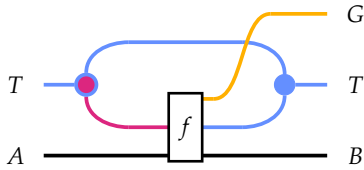
and

$$g: W \times B \rightarrow G_2 (T C)$$

is a function of type $W \times A \rightarrow G_1 (G_2 (T C))$, defined:

$$(g \star f) (w, a) \equiv \text{map}^{G_1} \left(\text{binddt}_{G_2} (g \odot w) \right) (f(w, a)) \quad (5.133)$$

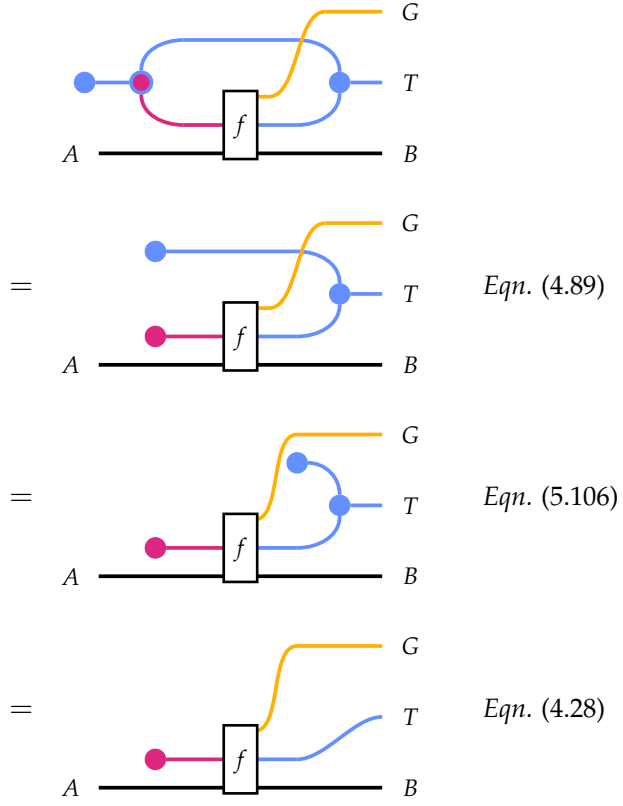
Lemma 5.4.20. (👉) Every DTM gives rise to an extension system for the following definition of binddt :



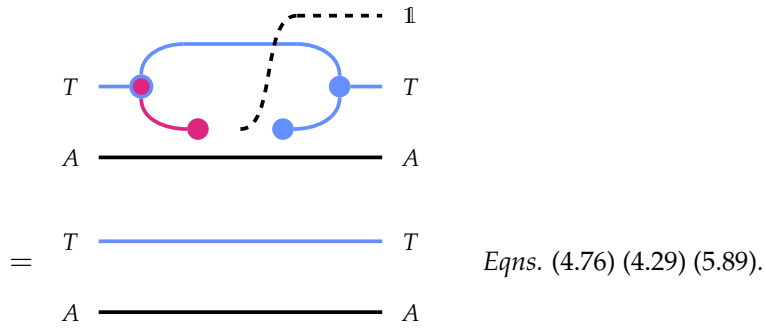
$$\text{binddt}_G^T f = \text{map}^G \text{join}^T \circ \text{dist}_G^T \circ \text{map}^T f \circ \text{dec}^T \quad (5.134)$$

Proof.

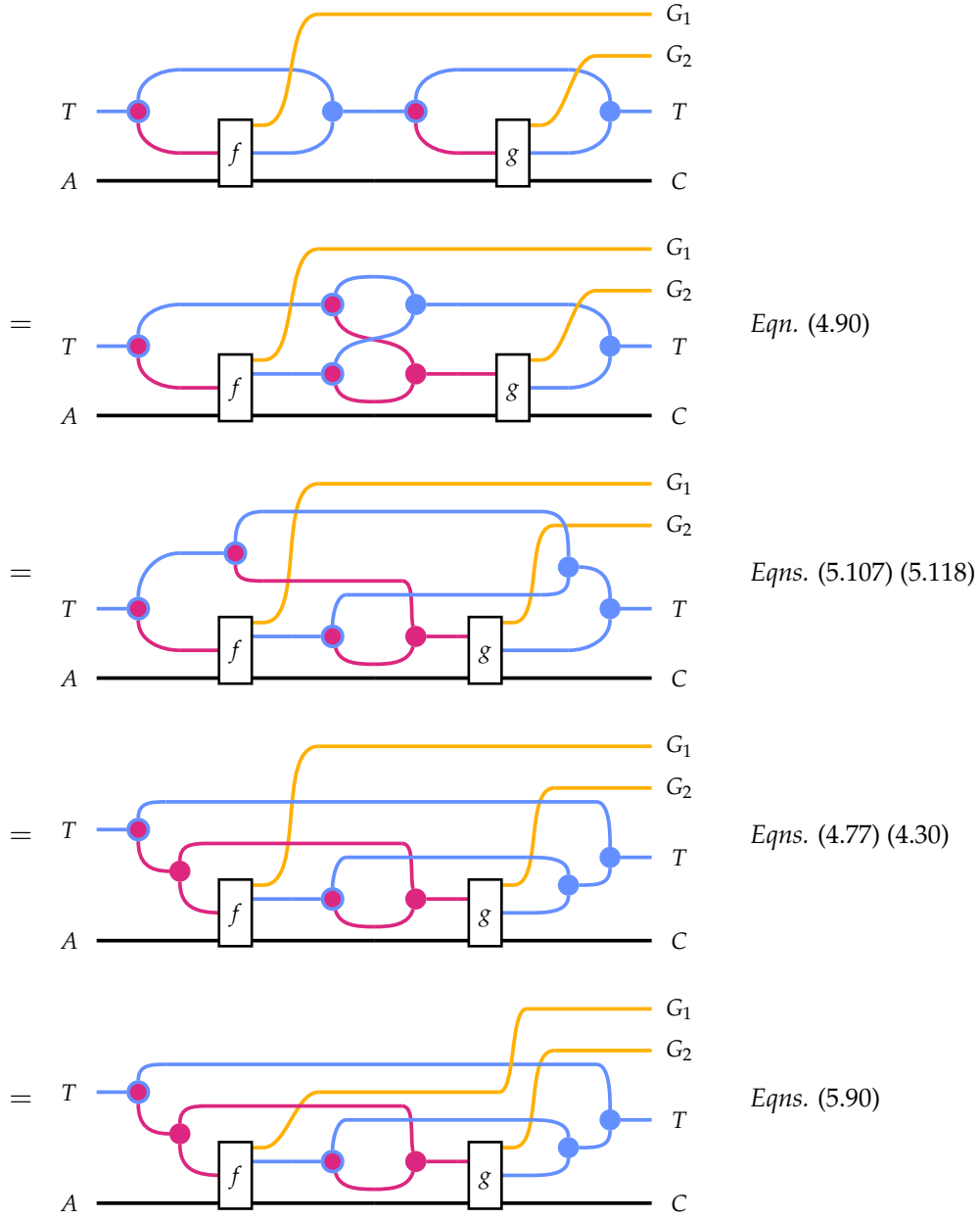
Equation (5.129): $\text{binddt}_G f \circ \text{ret}^T = f \circ \text{ret}^{W \times}$



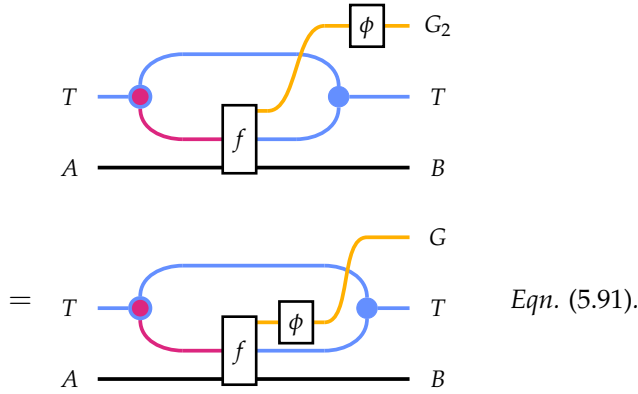
Equation (5.130): $\text{binddt}_{\mathbb{I}} (\text{ret}^T \circ \text{extract}^{W \times}) = \text{id}_{T A}$



Equation (5.131): $\text{map}^{G_1} \left(\text{binddt}_{G_2}^T g \right) \circ \left(\text{binddt}_{G_1}^T f \right) = \text{binddt}_{G_1 \circ G_2}^T (g \star f)$



Equation (5.132): $\phi \circ \text{binddt}_{G_1}^\top f = \text{binddt}_{G_2}^\top (\phi \circ f)$



□

An interesting aspect of the above proof is that each of the equations in Figure 5.2 is used exactly once.

Lemma 5.4.21. *A DTM presented as an extension system gives rise to a decorated traversable monad with the following definitions.*

$$\text{map}^\top f \equiv \text{binddt}_\mathbb{I} \left(\text{ret} \circ f \circ \text{extract}^{W^\times} \right) \quad (5.135)$$

$$\text{dec}^\top \equiv \text{binddt}_\mathbb{I} \text{ret}^\top \quad (5.136)$$

$$\text{join}^\top \equiv \text{binddt}_\mathbb{I} \text{extract}^{W^\times} \quad (5.137)$$

$$\text{dist}_G^\top \equiv \text{binddt}_G \left(\text{map}^G \text{ret}^\top \circ \text{extract}^{W^\times} \right) \quad (5.138)$$

Theorem 5.4.22. *Lemma 5.4.20 and 5.4.21 form an equivalence between Definitions 5.4.17 and 5.4.19.*

We also have the following laws, or definitions, depending on which operations are considered primitive.

Lemma 5.4.23. *The following definitions satisfy the axioms of their respective structures, assuming $\text{binddt}^\top$*

satisfies Equations (5.129)–(5.132).

$$\begin{aligned}
\text{map}^T f &\equiv \text{binddt}_{\mathbb{I}} \left(\text{ret} \circ f \circ \text{extract}^{W^\times} \right) \\
\text{mapd}^T f &\equiv \text{binddt}_{\mathbb{I}} (\text{ret} \circ f) \\
\text{bind}^T f &\equiv \text{binddt}_{\mathbb{I}} \left(f \circ \text{extract}^{W^\times} \right) \\
\text{traverse}^T f &\equiv \text{binddt}_G \left(\text{map}^G \text{ret} \circ f \circ \text{extract}^{W^\times} \right) \\
\text{mapdt}^T f &\equiv \text{binddt}_G \left(\text{map}^G \text{ret} \circ f \right) \\
\text{bindd}^T f &\equiv \text{binddt}_{\mathbb{I}} f \\
\text{bindt}^T f &\equiv \text{binddt}_G \left(f \circ \text{extract}^{W^\times} \right)
\end{aligned}$$

The following lemma is an analogue of Lemmas 5.3.30, 5.3.42 and 5.4.12.

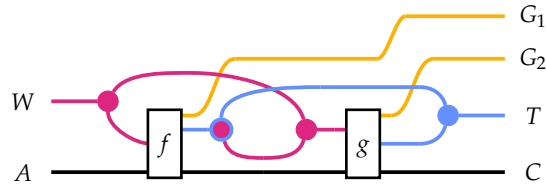
Lemma 5.4.24. *The following implications hold for all $t: T A$ and $(f, g: W \times A \rightarrow T B)$.*

$$\forall (w: W) (a: A), (w, a) \in_d t \implies f(w, a) = g(w, a) \implies \text{bindd}^T f t = \text{bindd}^T g t \quad (5.139)$$

$$\forall (w: W) (a: A), (w, a) \in_d t \implies f(w, a) = a \implies \text{bindd}^T f t = t \quad (5.140)$$

5.4.4 Kleisli Composition for DTMs

The Kleisli composition of two substitutions g and f , used in the composition law (5.129), is visualized as follows:



$$\lambda(w, a). \text{map}^{G_1} \left(\text{binddt}_{G_2} (g \odot w) \right) (f(w, a)) \quad (5.141)$$

A useful fact that does not depend on Equations (5.129)–(5.132) is that preincrement distributes linearly over Kleisli composition.

Lemma 5.4.25. *For all g, f , and w we have the following:*

$$(g \star f) \odot w = (g \odot w) \star (f \odot w) \quad (5.142)$$

Proof. This has a nice diagrammatic proof we give here.

$$\begin{array}{c}
\begin{array}{c} \text{Diagram 1: } W, A \text{ inputs, } f, g \text{ boxes, } G_1, G_2, T, C \text{ outputs. } \end{array} \\
= \begin{array}{c} \text{Diagram 2: } W, A \text{ inputs, } f, g \text{ boxes, } G_1, G_2, T, C \text{ outputs. } \end{array} \quad \text{Eqn. (4.20)} \\
= \begin{array}{c} \text{Diagram 3: } W, A \text{ inputs, } f, g \text{ boxes, } G_1, G_2, T, C \text{ outputs. } \end{array} \quad \text{Definition of } \text{dup}^{W \times} \\
= \begin{array}{c} \text{Diagram 4: } W, A \text{ inputs, } f, g \text{ boxes, } G_1, G_2, T, C \text{ outputs. } \end{array} \quad \text{Eqn. (5.128) and properties of tensor strength}
\end{array}$$

□

5.4.5 Proofs of DTM Instances

Definition 5.4.26 (DTM instance for Lam). *The operation $\text{binddt}^{\text{lam}}$ is defined as follows:*

$$\begin{aligned}
\text{binddt}_G f (\text{var } v) &\equiv f(0, v) \\
\text{binddt}_G f (\text{abs } t) &\equiv \text{pure}^G \text{abs} \circledast \text{binddt}_G (f \odot 1) t \\
\text{binddt}_G f (\text{app } t_1 t_2) &\equiv \text{pure}^G \text{app} \circledast \text{binddt}_G f t_1 \circledast \text{binddt}_G f t_2
\end{aligned}$$

In order to use Tealeaves, a user is obligated to prove the type constructor corresponding to their syntax is a DTM, particularly as defined in Definition 5.4.19. In the case of lambda calculus, our tactic `derive_dtm` can prove this automatically. For more general kinds of syntax, the automation is not complete and will fail to prove the instance, so a user must write at least

parts of the proofs of Eqns. (5.129)–(5.132) by hand. Therefore it is worth a close look at how these proofs proceed. Appendix A gives a fully detailed proof of the DTM laws for the lambda calculus. Sections 6.2 and 6.4 discuss the finer details of proving these laws when the syntax involves variadic or mutually recursive binders.

5.4.6 Heterogeneous Substitution

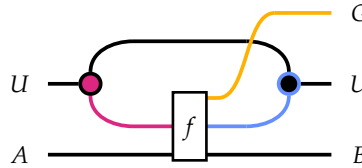
We can also consider substitution of one type of variable inside a different type of expression with a slightly more general definition. This abstraction arises by generalizing the monad T to an arbitrary right actions of T (Def. 4.2.33) everywhere in the theory of DTMs.

Definition 5.4.27. *Let T be a decorated traversable monad. A right module of T is a type constructor $U: \text{SET} \rightarrow \text{SET}$ equipped with an operation of the following type.*

$$\text{binddt}_{G,A,B}^{U,T} : (W \times A \rightarrow G(TB)) \rightarrow (U A \rightarrow G(U B))$$

subject to generalizations of Equations (5.130), (5.131), (5.132) obtained by replacing binddt^T everywhere with binddt^U , except for the occurrence of binddt^T in the definition of Kleisli composition. Note that there is no analogue of (5.129) as we do not assume there is an operation ret^U .

Diagrammatically, the binddt^U operation can be visualized as follows, assuming some action of type $U \circ T \rightarrow U$ satisfying Equations (4.60) and (4.61).



The fact that we do not assume the existence of ret^U in Definition 5.4.27 corresponds to a situation where U represents a syntax within which T -variables occur and can be substituted with T -valued subexpressions, but for which there are no U variables. This was shown in Example 4.2.35 for the syntax of first-order logic, where terms appear in formulas but there are no formula variables. If, for example, we extended the logic with a least fixed point operator [39], which adds formula variables to the language, then the more general abstraction first discussed in Appendix C is required. The details of the multisorted abstraction are mostly very similar to those of DTMs in general, though there are a few novel technical details which are examined in Appendix C

5.5 LOCALLY NAMELESS METATHEORY

With the properties established in Lemmas 5.3.30, 5.3.42, 5.4.12 and 5.4.24, which required the additional properties of traversable, rather than merely linearizable, functors, we can resume developing the metatheory of the locally nameless operations defined across Sections 4.5 and 5.2. Namely, we will prove the remaining exemplary lemmas from Figure 2.1 in Section 2.3. Throughout this section, we assume T is a traversable monad decorated by $\langle \mathbb{N}, +, 0 \rangle$. Terms of type $t : T \text{ LN}$ represent abstract locally nameless terms, where $\text{LN} = \mathbb{N} + \text{atom}$. Metavariables t and u are arbitrary terms, while x, y, z are arbitrary atoms and v is an arbitrary LN variable (i.e. an atom or index). We begin with a simple and particularly illustrative example.

Theorem 5.5.1 ( SUBST_FRESH). *Substitution of a variable that does not occur in a term has no effect:*

$$t[x \mapsto u] = t \quad \text{where } x \notin \mathbf{fv} \, t$$

Proof. The notation unfolds to $\text{bind}(\text{subst}_{\text{loc}} \, x \, u) \, t = t$. Using (5.71), the side condition is equivalent to $\text{Fr } x \notin t$. In particular $v \in t$ implies $v \neq \text{Fr } x$. The following corollary is immediate:

$$\forall (v : \text{LN}), v \in t \implies \text{subst}_{\text{loc}} \, x \, u \, v = \text{ret } v$$

By Lemma 5.3.42, this immediately implies $\text{bind}(\text{subst}_{\text{loc}} \, x \, u) \, t = t$. □

Theorem 5.5.2 ( SUBST_SUBST). *If $x \notin \mathbf{fv} \, u'$ and $x \neq y$, the following holds:*

$$t[x \mapsto u][y \mapsto u'] = t[y \mapsto u'][x \mapsto u[y \mapsto u']]$$

Proof. Using (4.35), the LHS is equal to the following:

$$\text{bind}(\text{bind}(\text{subst}_{\text{loc}} \, y \, s) \circ \text{subst}_{\text{loc}} \, x \, u) \, t$$

The RHS is equal to the following:

$$\text{bind}(\text{bind}(\text{subst}_{\text{loc}} \, x \, (\text{bind}(\text{subst}_{\text{loc}} \, y \, s) \, u)) \circ \text{subst}_{\text{loc}} \, y \, s) \, t.$$

It suffices to show the two inner expressions are extensionally equal. That is, we must prove the following for all variables $v : \text{LN}$.

$$\text{bind}(\text{subst}_{\text{loc}} \, y \, s) (\text{subst}_{\text{loc}} \, x \, u \, v) = \text{bind}(\text{subst}_{\text{loc}} \, x \, (\text{bind}(\text{subst}_{\text{loc}} \, y \, s) \, u)) (\text{subst}_{\text{loc}} \, y \, s \, v)$$

Note that by Equation (5.109), it would suffice to show the above equality under the assumption $v \in \text{LN}$, but we do not need this assumption. We proceed by case analysis on the form of v .

Bound: Suppose $v = \text{Bd } n$. Both inner expressions simplify to expressions of the form

$$\text{bind } (\text{subst}_{\text{loc}} y z) (\text{ret } (\text{Bd } n))$$

for different values of y and z . By (4.34), both sides are equal to $\text{subst}_{\text{loc}} y z (\text{ret } (\text{Bd } n))$, which simplifies to $\text{ret } (\text{Bd } n)$, and we are done.

Free, Equals x : Suppose $v = \text{Fr } x$. The LHS simplifies to $\text{bind } (\text{subst}_{\text{loc}} y s) u$. Now (4.34) and the assumption $x \neq y$ makes the RHS equal to $\text{bind } (\text{subst}_{\text{loc}} y s) u$, and we are done.

Free, Equals y : Suppose $v = \text{Fr } y$. Now $x \neq y$ and (4.34) reduce the goal to

$$s = \text{bind } (\text{subst}_{\text{loc}} x (\text{bind } (\text{subst}_{\text{loc}} y s) u)) s.$$

Now, $x \notin \mathbf{fv} s$ and Theorem 5.5.1 imply the RHS is equal to s .

Free, Not x or y : Suppose $v = \text{Fr } z$ where $x \neq y, z$. Now (4.34) reduces the goal to

$$\text{ret } v = \text{ret } v.$$

and we are done. □

To prove Theorem 5.5.4, we will need the following lemma, whose proof is left as an exercise.

Lemma 5.5.3. () For all $u : T \text{ LN}$ and $x : \text{atom}$, if $\mathbf{lc} u$ and $x \notin \mathbf{fv} u$, then for all $d : \mathbb{N}$,

$$\text{mapd } (\text{close}_{\text{loc}} x \odot d) t = t$$

□

Theorem 5.5.4 ( SUBST_CLOSE). For non-equal variables x and y , if y is not a free variable of u and u is locally closed, then substituting u for x in t commutes with closing t by y . That is, if $y \notin \mathbf{fv} u$ and $\mathbf{lc} u$ then

$$(\backslash^y t)[x \mapsto u] = \backslash^y (t[x \mapsto u])$$

Proof. The notation unfolds to the following:

$$\text{bind } (\text{subst}_{\text{loc}} x u) (\text{mapd } (\text{close}_{\text{loc}} y) t) = \text{mapd } (\text{close}_{\text{loc}} y) (\text{bind } (\text{subst}_{\text{loc}} x u) t).$$

Applying variants of (4.107) and (4.108) that govern the composition of bind and with mapd , the goal is equivalent to the following one:

$$\text{bindd } (\lambda (d, v). \text{subst}_{\text{loc}} x u (\text{close}_{\text{loc}} y (d, v))) t = \text{bindd } (\text{mapd } (\text{close}_{\text{loc}} y) \circ \text{subst}_{\text{loc}} x u) t.$$

By Lemma 5.4.24, it suffices to show the following for all $(d, v) \in_d t$:

$$\text{subst}_{\text{loc}} x u (\text{close}_{\text{loc}} y (d, v)) = \text{mapd} (\text{close}_{\text{loc}} y) (\text{subst}_{\text{loc}} x u v)$$

We proceed by case analysis on v and d :

$v = \text{Bd } n, n > d$: The LHS simplifies to $\text{Bd } (n + 1)$. By (4.106) (characterizing the composition of mapd and ret), the RHS is equal to $\text{close}_{\text{loc}} y (\text{Bd } n)$ which evaluates to $\text{Bd } (n + 1)$.

$v = \text{Bd } n, n < d$: Simplifying both sides and applying (4.106) makes both sides equal to $\text{Bd } n$.

$v = \text{Bd } d$: The LHS simplifies to $\text{subst}_{\text{loc}} x u y$, and since $y \neq x$ this is equal to $\text{ret } y$. Similarly, the RHS is equal to $\text{ret} (\text{close}_{\text{loc}} y (d, \text{Bd } d))$ which is $\text{ret } y$.

$v = \text{Fr } x$: The LHS is equal to u . The right-hand side is equal to $\text{mapd} (\text{close}_{\text{loc}} y \odot d) u$. By Lemma 5.5.3, the assumptions $y \notin \mathbf{fv } u$ and $\mathbf{lc } u$ imply this is equal to u .

$v = \text{Fr } y$: The LHS simplifies to $\text{Bd } d$ while the RHS simplifies to

$$\text{mapd} (\text{close}_{\text{loc}} y \odot d) (\text{ret} (\text{Fr } y))$$

(Note that the hypotheses of Lemma 5.5.3 are not satisfied.) By (4.106), this reduces to $\text{ret} (\text{close}_{\text{loc}} y (d, \text{Fr } y))$, which further simplifies to $\text{Bd } d$, and we are done.

$v = \text{Fr } z, z \neq x, y$: The LHS simplifies to $\text{Fr } z$. By (4.106), the RHS reduces to the following:

$$\text{mapd} (\text{close}_{\text{loc}} y \odot d) (\text{ret} (\text{Fr } z))$$

By Lemma 5.5.3, the RHS is equal to $\text{Fr } z$ and we are done.

□

The final locally nameless lemma, injectivity of opening a term by a fresh variable, has a different nature than earlier ones. While so far we have considered the effect of operations applied to a single term, the injectivity property relates the same operation applied to two different terms. Our first step is to express opening by an atomic term as a map (rather than a substitution).

Definition 5.5.5. (🐼) The operation $\text{varopen}_{\text{loc}}$ is defined as follows:

$$\text{varopen}_{\text{loc}} x (d, v) \equiv \begin{cases} x & \text{if } v = \text{Bd } d \\ (\text{Bd } (n - 1)) & \text{if } v = \text{Bd } n, n > d \\ v & \text{else} \end{cases}$$

Note that $\text{varopen}_{\text{loc}} x (d, v) = \text{open}_{\text{loc}} (\text{ret } x) (d, v)$.

Note that we have the following chain of equalities characterizing t^x :

$$\text{open}(\text{var } x) t = \text{bindd}^\top(\text{open}_{\text{loc}}(\text{var } x)) t = \text{mapd}^\top(\text{varopen}_{\text{loc}} x) t \quad (5.143)$$

Using the coalgebraic presentation of DTMs in Chapter 6, the same technique used to obtain results like Lemma 5.4.24 can also be used to prove the following property.

Lemma 5.5.6. () For all $t, u: T A$ and $f: W \times A \rightarrow B$, if f satisfies the follow injectivity condition

$$(\forall (w: W) (a, a': A), (w, a) \in_d t \implies (w, a') \in_d u \implies f(w, a) = f(w, a') \implies a = a')$$

then if $\text{mapd } f t = \text{mapd } g u$, we can infer $t = u$. □

We can now prove than opening by a fresh variable is injective.

Theorem 5.5.7 () (**OPEN_INJ**). For all terms t and u and atoms v : atom that are not in the free variables of either term, opening by $\text{ret}^\top(\text{Fr } x)$ is injective. That is,

$$t^x = u^x \iff t = u \quad \text{where } x \notin (\mathbf{fv } t \cup \mathbf{fv } u)$$

Proof. The right-to-left implication is trivial. For the other direction, using (5.143), the notation $t^x = u^x$ unfolds to the following statement:

$$\text{mapd}^\top(\text{varopen}_{\text{loc}} x) t = \text{mapd}^\top(\text{varopen}_{\text{loc}} x) u$$

Using (5.71), the right-hand side is logically equivalent to

$$\forall (v: \text{LN}), (v \in t \vee v \in u) \implies v \neq \text{Fr } x.$$

Using this fact, we can prove the injectivity property from Lemma 5.5.6 for $\text{varopen}_{\text{loc}}$. Let d, v, v' such that $(d, v) \in_d t$ and $(d, v') \in_d u$. Recall that $v \in t \iff \exists d, (d, v) \in_d t$, so the above property implies neither of v or v' is equal to $\text{Fr } x$. The following is easily shown:

$$\text{varopen}_{\text{loc}} x (d, v) = \text{varopen}_{\text{loc}} x (d, v') \implies v = v'$$

(Note that without the assumption that v is fresh for t and u , we might have a case like $v = \text{Bd } d$ and $v' = \text{Fr } x$.) Now the conclusion follows from Lemma 5.5.6. □

5.6 TRANSLATING VARIABLE BINDING REPRESENTATIONS

In this section, we consider operations that translate between different representations of variable binding. Although many such representations have been proposed—often accompanied by minor

variations—none is universally optimal. De Bruijn indices excel in automated rewriting [64], locally nameless is often favored for its intuitive appeal [61, 10], and nominal syntax aligns with programmer expectations, though it is hardly ideal for purposes of formal metatheory. A shared foundation that supports multiple representations and allows for verified translation between them is therefore highly desirable.

5.6.1 Locally Nameless and de Bruijn

First we consider a simple operation that converts locally nameless syntax to de Bruijn syntax. Let T be a monad decorated by natural numbers under addition. To a first approximation, we would like a bijection between $T\mathbb{N}$, representing de Bruijn terms, and $T\text{LN}$, representing locally nameless terms. The two representations are identical in their handling of bound variables, and so we must specify the logic of the translation on free variables—atoms, for locally nameless, and indices n greater than $d - 1$, where d is the binding depth of the occurrence, for de Bruijn.

Keys

Replacing free de Bruijn indices with names requires an arbitrary choice of name for each free variable represented in a term. This choice is facilitated by a datatype, `key`. A key is defined as `list atom`, subject to a constrain that the entries must be unique. A key k encodes a bijection between the atoms in the list and a prefix of $\mathbb{N}$, specifically $\{0, 1, \dots, (\text{length } k) - 1\}$. Keys support the following operations:

$$\begin{aligned} \text{getName} &: \text{key} \rightarrow \mathbb{N} \rightarrow \text{option atom} \\ \text{getIndex} &: \text{key} \rightarrow \text{atom} \rightarrow \text{option } \mathbb{N} \end{aligned}$$

$\text{getName } k \ n$ returns the name associated with the n^{th} free de Bruijn variable, which is the atom at index n in the key, if it exists. Of course, this index might not exist, so the output is wrapped in `option`. Conversely, $\text{getName } k \ x$ returns index in k at which x occurs, if x occurs in k . These operations form a partial bijection between $\mathbb{N}$ and `atom` in the following sense.

Definition 5.6.1 (Partial bijection). *A partial bijection between A and B comprises functions $f: A \rightarrow \text{option } B$ and $g: B \rightarrow \text{option } A$ satisfying the following property for all a, b :*

$$f \ a = \text{Some } b \iff g \ b = \text{Some } a.$$

Lemma 5.6.2. () *`getName` and `getIndex` form a partial bijection between $\mathbb{N}$ and `atom`.* □

When starting with a de Bruijn term, a key must be provided externally, because it encodes arbitrary information about which names to assign to free variables. When starting with a locally

nameless term, we can extract a key from the term by enumerating its free variables using **fv** and removing duplicates (🚫). This works because the traversal structure encodes the (essentially arbitrary) information about the order of occurrences in a term.

Translation Operations

Definition 5.6.3. (🚫) *The translation operation from de Bruijn to locally nameless is defined as follows:*

$$\begin{aligned} \text{toLN } k \, t &\equiv \text{mapdt}_{\text{opt}}^T (\text{toLN}_{\text{loc}} \, k) \, t & (5.144) \\ \text{toLN}_{\text{loc}} \, k \, (d, n) &\equiv \begin{cases} \text{map}^{\text{opt}} \text{Fr} (\text{getName } k \, (n - d)) & \text{if } n \geq d \\ \text{Some } (\text{Bd } n) & \text{else} \end{cases} \end{aligned}$$

Given a key k , toLN_{loc} maps a de Bruijn index n under d binders with the following logic. If n is a free variable, i.e. if $n > d - 1$, then we look up the name corresponding to the $(n - d)^{\text{th}}$ free variable. This returns an optional atom, which is coerced to an optional LN by mapping the Fr constructor over option. Otherwise, we leave the variable unmodified but wrap the value in Some . This function is a context-sensitive map wrapped in an applicative functor, so it is lifted over the term with mapdt . The operation returns None if and only if the localized operation fails at any occurrence. The translation in the other direction is defined similarly.

Definition 5.6.4. (🚫) *The translation operation from locally nameless to de Bruijn is defined as follows:*

$$\begin{aligned} \text{toDB } k \, t &\equiv \text{mapdt}_{\text{opt}}^T (\text{toDB}_{\text{loc}} \, k) \, t & (5.145) \\ \text{toDB}_{\text{loc}} \, k \, (d, v) &\equiv \begin{cases} \text{map}^{\text{opt}} (+d) (\text{getIndex } k \, x) & \text{if } v = \text{Fr } x \\ \text{Some } n & \text{if } v = \text{Bd } n \end{cases} \end{aligned}$$

To reason about these operations, we invoke a property extending Lemma 5.4.12.

Lemma 5.6.5. *The following holds for all traversable functors decorated by E , applicative functors G , and functions $f: E \times A \rightarrow GB$.*

$$\left((e: E) (a: A), (e, a) \in_d t \implies f(e, a) = \text{pure}^G a \right) \implies \text{mapdt}^F f \, t = \text{pure}^G t$$

Theorem 5.6.6. (🚫) *Let $t: T \, \text{LN}$ be a locally nameless term satisfying $\text{lc } t$. Let k be a key satisfying the following:*

$$\forall (x \in \mathbf{fv} \, t), x \in k$$

Then the following roundtrip property holds:

$$\text{map}^{\text{opt}} (\text{toLN } k) (\text{toDB } k \, t) = \text{Some } (\text{Some } t)$$

Proof. Using (5.120), the two operations are fused together into a single operation:

$$\text{mapdt}_{\text{opt} \circ \text{opt}}^{\top} \left(\text{map}^{\text{opt}} (\text{toLN } k) \circ \text{mapdt}_{\text{opt}}^{\mathbb{N} \times} (\text{toDB}_{\text{loc}} k) \right) t = \text{Some } (\text{Some } t)$$

This operation traverses t with respect to the applicative functor $(\text{option} \circ \text{option})$. By Lemma 5.6.5, it suffices to show that the inner operation maps (d, v) to $\text{Some } (\text{Some } v)$ for all $(d, v) \in_d t$. By local closure of t , invoking (5.42), we have two cases to consider: either v is $\text{Bd } n$ where n is less than d , or v is $\text{Fr } x$ where $x \in k$. In the $v = \text{Bd } n$ case, the goal reduces to showing the following:

$$\text{map}^{\text{opt}} (\text{toLN}_{\text{loc}} k) (\text{Some } (d, n)) = \text{Some } (\text{Some } (\text{Bd } n))$$

This follows immediately from the fact $n < d$. In the $v = \text{Fr } x$ case, the goal reduces to showing the following:

$$\text{map}^{\text{opt}} (\text{toLN}_{\text{loc}} k) (\text{Some } (d, \text{map}^{\text{opt}} (+d) (\text{getIndex } k x))) = \text{Some } (\text{Some } (\text{Fr } x))$$

This in return reduces the following:

$$\text{toLN}_{\text{loc}} k (d, \text{map}^{\text{opt}} (+d) (\text{getIndex } k x)) = \text{Some } (\text{Fr } x)$$

Because we know $x \in k$, by the partial bijection property, $\text{getIndex } k x$ returns a value of the form $\text{Some } i$ where $\text{getName } k i = \text{Some } x$. Thus, we are left with the follow goal:

$$\text{toLN}_{\text{loc}} k (d, \text{Some } (i + d)) = \text{Some } (\text{Fr } x)$$

This holds immediately by the definition of toLN_{loc} . □

The other direction follows similarly.

Theorem 5.6.7. () *Let $t : T N$ be a de Bruijn term satisfying $\text{cl}_{\text{at}} t j$ for some j , i.e. suppose t does not contain free de Bruijn variables higher than j . Let k be a key satisfying $j < \text{length}_k$. Then the following roundtrip property holds:*

$$\text{map}^{\text{opt}} (\text{toDB } k) (\text{toLN } k t) = \text{Some } (\text{Some } t)$$

□

Although we have not pursued this topic, it would be essentially straightforward to additionally verify that these operations commute with substitution in appropriate sense, since all operations involved are ultimately just special cases of bindd . (The main question here lies in deciding precisely *what* sense we should like translation to commute with substitution.) Such a verification would further increase our confidence that the translation is correct and potentially

provide an avenue to “transport” properties proved using one representation to properties stated for the other.

5.6.2 Locally Nameless and Nominal Syntax

We now consider an operation that translates between locally nameless terms and fully nominal terms. In this section, let

$$T: \text{SET} \rightarrow \text{SET} \rightarrow \text{SET}$$

form a bifunctor satisfying the following conditions:

- For any B , $(T B)$ forms a decorated traversable monad with respect to the free monoid list B .
- For any $g: B \rightarrow B'$ $\text{bmap}^T g: T B V \rightarrow T B' V$ forms a monad homomorphism
- The decoration bdec^T : satisfies the laws presented in Section 4.6.

A term of type $t: T \text{atom} \text{atom}$ represents an abstract nominal term. An abstract value $t: T \mathbf{1} \text{LN}$ represents an abstract locally nameless term—note that the choice $\mathbf{1}$ for the type of binder labels indicates that binders are not labeled, or rather, they are labeled with the non-informative value $*$: $\mathbf{1}$. Because the free monoid list $\mathbf{1}$ is isomorphic to $\mathbb{N}$, we shall treat $T \mathbf{1}$ as if the decorating monoid is $\langle \mathbb{N}, +, 0 \rangle$.

We begin by consider the operation translating a nominal term to a locally nameless. This operation is straightforward: binders labels are deleted (replaced with $*$), free variables v are replaced with $\text{Fr } v$, and bound variables are replaced with de Bruijn indices determined by the length of their binding postfix.

Definition 5.6.8. (🐼) *The translation operation from nominal to locally nameless is defined as follows:*

$$\begin{aligned} \text{nomToLN } t &\equiv \text{mapdp}^T (_ \mapsto *) \text{ toLN}_{\text{loc}} t \\ \text{toLN}_{\text{loc}} (l, v) &\equiv \begin{cases} \text{Fr } v & \text{if } v \notin l \\ \text{Bd } (\text{length } l_{\text{post}}) & \text{if bound } l \text{ pre } v \text{ post} \end{cases} \end{aligned} \tag{5.146}$$

For example, if the variable x occurs in the binding context $[x, y, x, z]$, then we have

$$\text{bound } ([x, y, x, z]) \ [x, y] \ x \ [z]$$

and the occurrence will be replaced with $\text{Bd } (\text{length } [z]) = \text{Bd } 1$. In the binding context $[a, b, c]$, assuming none of these are equal to x , the occurrence will be replaced with $\text{Fr } x$.

The translation in the other direction is slightly more obscure. Suppose we start with a locally nameless term $t: T \mathbf{1} \text{LN}$. The free variables in this term must be left alone, because free variable

names are significant. Each of the binder labels must be given a name, although not a completely arbitrary one—the name must not capture any of the free variables. Therefore, our first step is to compute $\mathbf{fv} \, t$ to use as an avoid set. Note that because $T \, \mathbf{1}$ is a DTM decorated by $\langle \mathbb{N}, +, 0 \rangle$, this $\mathbf{fv}$ operation is the same one given in Definition 5.2.5.

We now consider an operation that takes a locally nameless binding context (formally, a list of $\mathbf{1}$ values) and a locally nameless binder (always equal to $*$) and assigns it a name that avoids a given conflict set and guarantees uniqueness relative to the name assigned (by the same function) to any of the binders in its context. That is, we need the following.

Definition 5.6.9. *The following operation accepts an avoid set, a set of names higher in the syntax tree, and a locally nameless “binder” (i.e. a value of $\mathbf{1}$) and computes a fresh name for this binder.*

$$\text{historyToName} : \text{list atom} \rightarrow \text{list atom} \times \mathbf{1} \rightarrow \text{atom}$$

$$\text{historyToName } \Gamma (h, _) = \text{freshen } (\Gamma ++ h)$$

Now, we use the hadapt function (Def. 4.6.33) to convert an operation that expects a history to one that expects a context.

Definition 5.6.10. *The follow operation accepts an avoid set, a locally nameless binding context, and a locally nameless binder, and computes a fresh name for this binder.*

$$\text{bdToName} : \text{list atom} \rightarrow \text{list } \mathbf{1} \times \mathbf{1} \rightarrow \text{atom} \tag{5.147}$$

$$\text{bdToName } \Gamma = \text{hadapt } (\text{historyToName } \Gamma) \tag{5.148}$$

At this point, we can define the following operation that assigns unique names to each binder in a locally nameless term:

$$\text{nameBinders} : \forall (V : \text{SET}), T \, \mathbf{1} \, V \rightarrow T \, \text{atom} \, V$$

The remaining step is to incorporate an operation that assigns a name to locally nameless variables of the form $\text{Bd } n$. Here, we run into a technical complication: a priori, we have no guarantee that a value of this form is less than its binding depth, i.e. satisfies local closure. In this case, the index is ill-formed, and there is no reasonable default behavior to handle this case. One option would be to apply the same trick in Section 5.6.1 of wrapping the value in option ; this would work, but this applicative functor would be an unfortunate complication when proving correctness of the translation later. Instead, we will do something less sophisticated: since there is no *reasonable* default behavior, we will implement *unreasonable* behavior.

Definition 5.6.11. () *The following operation accepts an avoid set, a locally nameless binding context, and a de Bruijn index, and returns the same name assigned by the nameBinders operation to the abstraction*

to which the index is bound, if it exists. Otherwise, an arbitrary default value is returned.

$$\begin{aligned} \text{bdToName} &: \text{list atom} \rightarrow \text{list } \mathbf{1} \times \mathbb{N} \rightarrow \text{atom} \\ \text{bdToName } \Gamma (l, n) &= \begin{cases} \text{bdToName } \Gamma (\text{replicate } * (\text{length } l - (n + 1)), *) & \text{if } n < \text{length } l \\ 1337 & \text{else} \end{cases} \end{aligned}$$

In this above definition, the occurrence of 1337 happens to rely on the fact that `atom` is transparently implemented as the type of natural numbers, so the computation is well-typed—any default value can be inserted in its place. Let us carefully examine the logic of this operation before resuming the implementation of the translation.

The first argument to `bdToName` is the avoid set, which is necessary to avoid assigning names that would conflict with any `Fr x`. Consider the case when the index n is less than the length of the binding context. Then n represents a variable bound to the n^{th} element of the list *relative to the tail*. The value $(\text{length } l - (n + 1))$ measures the length of the binding prefix, the set of all abstractions higher in the context than the abstraction to which n is bound. For instance, if n is 0, n is bound to the final element and $\text{length } l - 1$ is the size of the binding prefix. The `replicate` function accepts a value, in this case $*$, and a length, and computes a list of that length by repeating the value this many times. Thus, the value computed is precisely the binding prefix. This context is paired with $*$, and so the entire expression computes exactly the name assigned to this binder by the translation function on binders.

In the case that $n \geq \text{length } l$, we simply pick a default value. This will not be a problem because we validate correctness of the translation later, we will assume that the input is locally closed, and so this case will not occur. If we had wanted to, we could have done something similar in Section 5.6.1; the choice is essentially an implementation detail. Having defined these operation, we can now define the operation translating LN occurrences to names.

Definition 5.6.12. *The following operation accepts an avoid set and a locally nameless variable in context and returns a name.*

$$\begin{aligned} \text{lnToName} &: \text{list atom} \rightarrow \text{list } \mathbf{1} \times \text{LN} \rightarrow \text{atom} & (5.149) \\ \text{lnToName } \Gamma (l, v) &= \begin{cases} x & \text{if } v = \text{Fr } x \\ \text{bdToName } \Gamma (l, n) & \text{if } v = \text{Bd } n \end{cases} \end{aligned}$$

The correctness of this translation will be shown in Section 6.5, the final technical section of this work.

5.7 TRAVERSING BINDERS

We return to the concept of a parameterized family of decorated monads considered in Section 4.6, turning our attention to the question of how to incorporate the traversable structure into this abstraction. We begin by noting that $\text{n\textit{lam}}$ is traversable in its binder parameter as well. However, none of the results in this work use traversability in the B argument: it is enough to assume that $T\ B$ is a traversable monad decorated by the free monoid $\text{list}\ B$ for any choice of B , where the bmap and bdec operations satisfy the properties presented in Section 4.6. Nonetheless, we present some preliminary results relating concerning traversability in the binder parameter.

Definition 5.7.1. *A traversable bifunctor is a bifunctor $F: \text{SET} \rightarrow \text{SET} \rightarrow \text{SET}$ equipped with an operation*

$$\text{dist}_2^F: \forall (G: \text{Applicative}) (B, V: \text{SET}), F (GB) (GV) \rightarrow G(FBV)$$

satisfying the obvious two-argument generalizations of the laws in Definition 5.3.21

It is straightforward to show that if F is a traversable bifunctor, then it is traversable in both argument individually by mapping pure^G over the other argument and using dist_2 . Therefore, following the same pattern in Section 4.6, we can split dist_2 into two operations bdist and vdist that distribute over an applicative functor in the binder and variable parameters of a bifunctor, respectively. vdist is the same dist operation we have already been considering, so we can concentrate on the unique properties of bdist .

The converse of the above observation is not quite true: bdist and vdist do not provide an obvious way to recover the dist_2 operation. Intuitively, dist_2 imposes a total order among the entire set of occurrences B or V values. However, while bdist and vdist individually provide a total order among binders and variables, respectively, they do not impose an order relating B values with V values or vice versa. The difference in expressiveness between dist_2 and $\text{bdist}/\text{vdist}$ in practical applications is likely marginal, although dist_2 is the more fundamental operation.

We continue to use $\text{n\textit{lam}}$ as a case study to motivate the appropriate axioms of the bdist operation. The major interesting fact about $\text{bdist}^{\text{n\textit{lam}}}$ is that it does *not* form a decorated traversable functor with the $\text{bdec}^{\text{n\textit{lam}}}$ operation.

5.7.1 Commutative-Idempotent Applicative and Traversable Functors

Just we can consider commutative and/or impotent monoids (Definition 4.1.14 and Def. 4.1.15, respectively), we can consider commutative and idempotent applicative functors.

Definition 5.7.2. *An applicative functor G is idempotent if the following holds for all A and $a: GA$:*

$$a \otimes a = \text{map}^G \text{dup } a \tag{5.150}$$

where $\text{dup } a = (a, a)$.

Definition 5.7.3. An applicative functor G is commutative if the following holds for all $a: GA, b: GB$:

$$a \otimes b = \text{map}^G \text{swap } (b \otimes a) \quad (5.151)$$

Example 5.7.4. The constant applicative functor over any commutative (resp. idempotent) monoid is itself commutative (resp. idempotent). Similarly, the Writer applicative functor (Ex. 5.3.16) inherits commutativity and idempotence from the underlying monoid.

Example 5.7.5. The subset applicative functor (Ex. 5.3.14) is commutative and idempotent.

Definition 5.7.6. The idempotent center of G over a type A consists of all values $a: GA$ that satisfy Equations (5.150) and (5.151) where, in the latter equation, $b: GB$ is an arbitrary value.

Lemma 5.7.7. nlam forms a decorated-traversable functor in the B parameter if the applicative functors are required to be commutative and idempotent. More generally, the coherence law (5.118) is satisfied provided all of the elements of type GB fall in the idempotent center of G .

COALGEBRAIC DECORATED TRAVERSABLE MONADS

Section 5.1 presented linearizable functors and monads from three perspectives, each emphasizing a different operation: `toList`, `mapReduce`, and `mconcat`. Section 5.3 strengthened the abstraction by generalizing from a monoidal fold to a distributive law over applicative functors, presented in terms of *two* operations: `dist`, which generalizes `mconcat`; and `traverse`, which generalizes `mapReduce`. Now we turn to a third perspective generalizing the `toList` operation. This leads to a radically different conception of traversable functors and DTMs emphasizing their status as *containers* of a definite *shape*.

6.1 COALGEBRAIC TRAVERSABLE FUNCTORS

6.1.1 The Batch Functor

Recall that for a linearizable monad T , $\text{toList} : T \Rightarrow \text{list}$ is required to be a monad homomorphism. This naturally raises the following question: what is the analogue of the list monad in the context of distributive laws over applicative functors? `list` is the free monoid monad—the monad mapping any set to the free monoid generated by that set. Perhaps it is not surprising then that we consider the following abstraction, which is closely related to the free applicative functor [21].

Definition 6.1.1. (🐘) *The Batch functor, written $\mathcal{B}$, is a three-argument functor recursively defined as follows:*

$$\mathcal{B}(A, B, C) \equiv C + \mathcal{B}(A, B, B \rightarrow C) \times A$$

This definition corresponds to the following constructors:

$$\text{Done} : C \rightarrow \mathcal{B} A B C$$

$$\text{Step} : \mathcal{B} A B (B \rightarrow C) \rightarrow A \rightarrow \mathcal{B} A B C$$

We use the infix notation $f \boxtimes a$ to stand for `Step f a`.

The recursive structure of $\mathcal{B}$ becomes more transparent when we unroll its definition:

$$\mathcal{B} A B C \simeq C + ((B \rightarrow C) \times A) + ((B \rightarrow B \rightarrow C) \times A \times A) + \dots$$

The chief reason for introducing $\mathcal{B}$ is that it represents a “prototypical” applicative expression. A

value of type $\mathcal{B} A B C$ has the following general form:

$$b = \text{Done mk } \underbrace{\boxtimes a_1 \boxtimes a_2 \dots \boxtimes a_n}_{\text{size } b}$$

Here, mk is a curried n -argument function of type $B \rightarrow B \dots \rightarrow B \rightarrow C$, where n is called the *size* of b . A value of this type provides a syntactical description of expressions that consist of pure and $(\boxtimes)$, written in normal form (recall Lemma 5.3.6). Compare this to a typical definition of traverse applied to a constructor C :

$$\text{traverse}_G f (C a_1 \dots a_n) = \text{pure}^G C \underbrace{\boxtimes f a_1 \boxtimes f a_2 \dots \boxtimes f a_n}_{\text{arity of } C}$$

However, $\mathcal{B}$ does not perform any effectful computation—it outlines the structure of a computation to be performed later, which motivates its name [17].

Definition 6.1.2. $\mathcal{B}$ is a three-argument functor—contravariant in its second argument—with the following definition of $\text{map}^{\mathcal{B}}$:

$$\begin{aligned} \text{map}^{\mathcal{B}}(f, g, h) &: \mathcal{B} A_1 B_1 C_1 \rightarrow \mathcal{B} A_2 B_2 C_2 \\ \text{map}^{\mathcal{B}}(f, g, h) (\text{Done } c) &= \text{Done } (h \, c) \\ \text{map}^{\mathcal{B}}(f, g, h) (k \boxtimes a) &= \text{map}^{\mathcal{B}}(f, g, (\lambda (x: B_1 \rightarrow C_1). h \circ x \circ g)) k \boxtimes (f \, a) \end{aligned}$$

Note that the type of g above is $g: B_2 \rightarrow B_1$.

This operation can be decomposed into three individual maps with the following notation:

$$\begin{aligned} \text{map}^1 &: (A_1 \rightarrow A_2) \rightarrow \mathcal{B} A_1 B C \rightarrow \mathcal{B} A_2 B C \\ \text{map}^2 &: (B_2 \rightarrow B_1) \rightarrow \mathcal{B} A B_1 C \rightarrow \mathcal{B} A B_2 C \\ \text{map}^3 &: (C_1 \rightarrow C_2) \rightarrow \mathcal{B} A B C_1 \rightarrow \mathcal{B} A B C_2 \end{aligned}$$

Note that each component of $\text{map}^{\mathcal{B}}$ commutes with the others:

$$\text{map}^i g \circ \text{map}^j f = \text{map}^j f \circ \text{map}^i g \quad \text{when } i \neq j \quad (6.1)$$

To develop the theory of $\mathcal{B}$, we introduce a family of functors, Vec , to represent a fixed number of values of a certain type.

Definition 6.1.3. () Let $n \in \mathbb{N}$ be a natural number. The type of vectors of length n is a functor

mapping a set A to its n -fold Cartesian product:

$$\text{Vec } n \ A \equiv \underbrace{A \times A \times \dots A}_{n \text{ copies}} \quad (6.2)$$

The lifted function $\text{map}^{\text{Vec } n} f$ applies f to all n components.

We write A^n to denote $\text{Vec } n \ A$. The unique A -valued vector of length 0 is written vnil . The constructor $\text{vcons}: A \rightarrow A^n \rightarrow A^{(n+1)}$ prepends an element to a vector. We can append two vectors $v_1: A^n$ and $v_2: A^m$ to yield a vector $v_1 ++ v_2: A^{(n+m)}$.

Remark 6.1.4. In Tealeaves, a vector $v: A^n$ is represented a list A paired with a proof that its length is n . This encoding is convenient for formalizing the theory described here. However, as experienced Coq users know, it also raises thorny issues involving equality between proofs. To avoid these complications, we incorporate proof irrelevance—an axiom consistent with Coq’s metatheory—when reasoning about vectors. As with functional and propositional extensionality, this axiom could be avoided by replacing propositional equality everywhere with setoid equivalence.

A value of type $\mathcal{B} \ A \ B \ C$ can be understood as a A -vector of an existentially quantified length, paired with C -valued function of the same arity with inputs of type B .

Lemma 6.1.5. $\mathcal{B}$ satisfies the following natural isomorphism:

$$\mathcal{B} \ A \ B \ C \simeq \sum_{n \in \mathbb{N}} (B^n \rightarrow C) \times A^n \quad (6.3)$$

Lemma 6.1.5 yields a convenient decomposition of a $\mathcal{B}$ value into three components. A value of the type of the RHS of (6.3) is a triple consisting of a *size*, a *make function*, and *contents* vector. The left-to-right direction of the isomorphism decomposes into three functions, the latter two having dependent types.

$$\begin{aligned} \text{size}^{\mathcal{B}}: \mathcal{B} \ A \ B \ C &\rightarrow \mathbb{N} \\ \text{make}^{\mathcal{B}}: \forall (b: \mathcal{B} \ A \ B \ C), B^{\text{size}^{\mathcal{B}} \ b} &\rightarrow C \\ \text{contents}^{\mathcal{B}}: \forall (b: \mathcal{B} \ A \ B \ C), A^{\text{size}^{\mathcal{B}} \ b} &\end{aligned}$$

Remark 6.1.6. I avoid writing double superscripts such as $B^{\text{size}^{\mathcal{B}} \ b}$, writing $B^{\text{size}^{\mathcal{B}} \ b}$ instead, but note that $\text{size}^{\mathcal{B}}$ will later be generalized to size^F where F is a traversable functor.

The advantage of having two equivalent presentations of Batch is that each is suited for different purposes. $\mathcal{B}$ is inductively defined and amenable to formal proofs using the `induction` tactic. The alternate representation is more theoretically transparent, but less convenient in proofs because of the complexity of manipulating existentially quantified types, such as vectors, in Coq.

Both presentations, and their equivalence, are formalized in Tealeaves, so that either may be used when it is more suitable.

We can begin developing the theory of $\mathcal{B}$ by considering its structure as an applicative functor.

Lemma 6.1.7. (👉) *For all types $A, B : \text{SET}$, $\mathcal{B} A B$ forms an applicative functor with the following operations:*

$$\begin{aligned} \text{pure}^{\mathcal{B}} &: \forall (C : \text{SET}), C \rightarrow \mathcal{B} A B C \\ \otimes^{\mathcal{B}} &: \forall (C, D : \text{SET}), \mathcal{B} A B C \rightarrow \mathcal{B} A B D \rightarrow \mathcal{B} A B (C \times D) \\ \text{pure}^{\mathcal{B}} c &\equiv \text{Done } c \\ b \otimes^{\mathcal{B}} (\text{Done } d) &\equiv \text{map}^3 (\lambda (c : C). (c, d)) b \\ b \otimes^{\mathcal{B}} (k \boxtimes a) &\equiv (\text{map}^3 (\lambda ((c, k) : C \times (B \rightarrow D)). \lambda (b' : B). (c, k b')) (b \otimes^{\mathcal{B}} k)) \boxtimes a \end{aligned}$$

The applicative structure is perhaps more enlightening when presented from the perspective of Lemma 6.1.5. The operation takes two triples of the following decomposed form:

$$\begin{array}{ll} n : \mathbb{N} & m : \mathbb{N} \\ \text{mk}_1 : B^n \rightarrow C & \text{mk}_1 : B^m \rightarrow D \\ \text{cont}_1 : A^n & \text{cont}_2 : A^m \end{array}$$

The multiplication appends their contents and combines their make functions in parallel:

$$(n + m, \lambda (b_1 \overset{n+m}{++} b_2). (\text{mk}_1 b_1, \text{mk}_2 b_2), \text{cont}_1 ++ \text{cont}_2)$$

Here, the pattern-matching notation $b_1 \overset{n+m}{++} b_2$ indicates the unique decomposition of a vector b of length $n + m$ into two components b_1 and b_2 of length n and m respectively, where $b = b_1 ++ b_2$.

We now introduce an operation that will play an central role in the remainder of this manuscript.

Definition 6.1.8. (👉) *The following operation accepts a function of type $f : A \rightarrow G B$ for any applicative G and collapses $\mathcal{B} A C$ into an applicative value of type $G C$:*

$$\text{runBatch} : \forall (\text{Applicative } G) (A B : \text{SET}), (A \rightarrow G B) \rightarrow \mathcal{B} A B C \rightarrow G C$$

$$\text{runBatch } f (\text{Done } c) \equiv \text{pure}^G c \tag{6.4}$$

$$\text{runBatch } f (k \boxtimes a) \equiv (\text{runBatch } f k) \otimes^G (f a) \tag{6.5}$$

If $\mathcal{B}$ describes the structure of a computation, then $\text{runBatch } f$ executes it. We can also characterize this operation through the isomorphism from Lemma 6.1.5. For this purpose we first observe that vectors are traversable in a fairly obvious way. Indeed, much as list can be regarded as the prototypical datatype for monoidally folding over a functor, vectors act as the prototypical datatype for traversable functors—a notion made precise by Theorem 6.1.28.

Lemma 6.1.9. (🚫) For any $n \in \mathbb{N}$, $\text{Vec } n$ is a traversable functor where $\text{traverse } f (a_1, a_2, \dots, a_n)$ is defined as follows:

$$\text{pure}^G (\lambda b_1 b_2 \dots b_n. (b_1, b_2, \dots, b_n)) \otimes^G (f a_1) \otimes^G (f a_2) \dots \otimes^G (f a_n)$$

This is essentially the same as traversing over a list, except the input is restricted to lists of a fixed length. Now, runBatch can be given the following alternate specification. Let b be a value of type $\mathcal{B} A B C$ and consider the decomposition of b as a triple:

$$\langle \text{size}^B b : \mathbb{N}, \text{mk} : B^{(\text{size } b)} \rightarrow C, \text{cont} : A^{(\text{size } b)} \rangle$$

Then $\text{runBatch } f b$ is equivalently defined as follows:

$$\text{runBatch}_G f b = \text{pure}^G \text{mk} \otimes^G \left(\text{traverse}^{\text{Vec}(\text{size } b)} f \text{cont} \right) \quad (6.6)$$

This operation first maps f over cont , yielding a vector $(G B)^{(\text{size } b)}$. Then G is distributed out by repeated multiplication, yielding something of type $G (B^{(\text{size } b)})$. Then, mk is mapped over G to yield something of type $G C$. A key observation is that the only G -effects generated during this computation are from applications of f to values in the contents of b . In particular, the mk function does not meaningfully participate in generating effects, being wrapped in pure^G .

The following observation should not be surprising given the apparent similarity between Equations (6.4) and (5.87), and between (6.5) and (5.88).

Lemma 6.1.10. (🚫) For all applicative functors G and functions $f : A \rightarrow G B$, $\text{runBatch } f$ is an applicative homomorphism from $\mathcal{B} A B$ to G .

Proof. The fact that $\text{runBatch } f$ satisfies (5.87) is immediate from the definition. The fact that it respects $\otimes^B$ according to (5.88) is slightly less immediate—note that $k \boxtimes a$ is not the same as $k \otimes^B a$, the latter being defined by (5.80) from the multiplication defined in Lemma 6.1.7. The core property required for the proof is that traversing over a vector distributes over appending vectors:

$$\text{traverse}^{\text{Vec}(n+m)} f (v_1 ++ v_2) = \text{pure}^G (++) \otimes^G \text{traverse}^{\text{Vec } n} f v_1 \otimes^G \text{traverse}^{\text{Vec } m} f v_2 \quad (6.7)$$

□

We make frequent use of the fact that runBatch is natural in several different senses, as a trivial verification shows.

Lemma 6.1.11. *runBatch is natural in each argument in the sense of satisfying the following equations:*

$$\text{runBatch}_G (g \circ f) = \text{runBatch}_G g \circ \text{map}^1 f \quad (6.8)$$

$$\text{runBatch}_G (g \circ f) = \text{runBatch}_G f \circ \text{map}^2 g \quad (6.9)$$

$$\text{map}^G g \circ \text{runBatch}_G f = \text{runBatch}_G f \circ \text{map}^3 g \quad (6.10)$$

$$\phi \circ \text{runBatch}_{G_1} f = \text{runBatch}_{G_2} (\phi \circ f) \quad \left(\text{all } \phi: G_1 \xrightarrow{\text{appl}} G_2 \right) \quad (6.11)$$

□

6.1.2 Parameterized Monads and Comonads

To fully tie the theory of $\mathcal{B}$ to the theory of traversable functors and ultimately DTMs, we develop supporting theory for functors of three arguments, employing the device of *parameterized monads* and *comonads*, a notion introduced by Atkey [6]. Each transformation below is required to be natural, though we omit a discussion of the precise kind of naturality conditions, which go by names like *extranaturality*, since this is secondary to our main goal, which is practical reasoning about DTMs. We use *naturality* as an umbrella term and focus on the core equational laws.

Definition 6.1.12. *A parameterized monad is a functor*

$$J: \text{SET} \rightarrow \text{SET}^{\text{op}} \rightarrow \text{SET} \rightarrow \text{SET}$$

equipped with natural transformations

$$\text{ret}^J: \forall (A, X: \text{SET}), A \rightarrow J A X X$$

$$\text{join}^J: \forall (A, B, X, C: \text{SET}), J (J A B X) X C \rightarrow J A B C$$

satisfying the following laws, which are implicitly generalized in A, B, and C:

$$\begin{array}{ccc} J A B C & \xrightarrow{\text{ret}^J} & J (J A B C) C C \\ & \searrow & \downarrow \text{join}^J \\ & & J A B C \end{array} \quad \text{join}^J \circ \text{ret}^J = \text{id} \quad (6.12)$$

$$\begin{array}{ccc} J A B C & \xrightarrow{J(\text{ret}^J)^{BC}} & J (J A B B) B C \\ & \searrow & \downarrow \text{join}^J \\ & & J A B C \end{array} \quad \text{join}^J \circ \text{map}^{J^1} \text{ret}^J = \text{id} \quad (6.13)$$

$$\begin{array}{ccc}
J(J(JABX)XY)YC & \xrightarrow{\text{join}^J} & J(JABX)XC \\
\downarrow J(\text{join}^J)YC & & \downarrow \text{join}^J \\
J(JABY)YC & \xrightarrow{\text{join}^J} & JABC
\end{array}
\quad \text{join}^J \circ \text{join}^J = \text{map}^{J^1} \text{join} \circ \text{join}^J \quad (6.14)$$

Example 6.1.13. () $\mathcal{B}$ is a parameterized monad in its first argument. The monad operations specialize to the following types:

$$\begin{aligned}
\text{ret}^{\mathcal{B}} &: \forall (A, X: \text{SET}), A \rightarrow \mathcal{B} A X X \\
\text{join}^{\mathcal{B}} &: \forall (A, B, C, X: \text{SET}), \mathcal{B}(\mathcal{B} A B X) X C \rightarrow \mathcal{B} A B C
\end{aligned}$$

The monad unit is defined as follows:

$$\text{ret}^{\mathcal{B}} a \equiv \text{id}_X \boxtimes a \quad (6.15)$$

The join operation is defined as follows:

$$\begin{aligned}
\text{join}^{\mathcal{B}} (\text{Done } c) &\equiv \text{Done } c \\
\text{join}^{\mathcal{B}} (h \boxtimes a) &\equiv (\text{join}^{\mathcal{B}} h) \circledast^{\mathcal{B}} a
\end{aligned}$$

Equivalently, $\text{join}^{\mathcal{B}}$ is defined as follows:

$$\text{join}^{\mathcal{B}} \equiv \text{runBatch}_{\mathcal{B}} \text{id}_{\mathcal{B}} \quad (6.16)$$

From the perspective of Lemma 6.1.5, ret^J and join^J have the following types:

$$\begin{aligned}
\text{ret}^J &: A \rightarrow \sum_{n \in \mathbb{N}} (X^n \rightarrow X) \times A^n \\
\text{join}^J &: \sum_{n \in \mathbb{N}} \left((X^n \rightarrow C) \times \left(\sum_{m \in \mathbb{N}} (B^m \rightarrow X) \times A^m \right)^n \right) \rightarrow \sum_{n \in \mathbb{N}} (B^n \rightarrow C) \times A^n
\end{aligned}$$

The monad unit maps $a: A$ to the triple $(1, \text{id}_X, a)$. The $\text{join}^{\mathcal{B}}$ operation is defined in an “obvious” way guided by its type. Note that the outer layer of $\mathcal{B}$ has a make function awaiting n inputs of type X , while the contents consist of as many $\mathcal{B}$ values that each yield a single X . These are flattened into a single $\mathcal{B}$ value by plugging the outputs of the inner make functions with the inputs of the outer make function.

Definition 6.1.14. A parameterized comonad is a functor

$$J: \text{SET} \rightarrow \text{SET}^{\text{op}} \rightarrow \text{SET} \rightarrow \text{SET}$$

equipped with natural transformations

$$\text{extract}^J: \forall (A, X: \text{SET}), J X X A \rightarrow A$$

$$\text{cojoin}^J: \forall (A, B, C, X: \text{SET}), J A B C \rightarrow J A X (J X B C)$$

satisfying the following laws:

$$\begin{array}{ccc} J A B C & \xrightarrow{\text{cojoin}^J} & J A A (J A B C) \\ & \searrow & \downarrow \text{extract}^J \\ & & J A B C \end{array} \quad \text{extract}^J \circ \text{cojoin}^J = \text{id} \quad (6.17)$$

$$\begin{array}{ccc} J A B C & \xrightarrow{\text{cojoin}^J} & J A B (J B B C) \\ & \searrow & \downarrow J A B (\text{extract}^J) \\ & & J A B C \end{array} \quad \text{map}^{J^3} \text{extract}^J \circ \text{cojoin}^J = \text{id} \quad (6.18)$$

$$\begin{array}{ccc} J A B C & \xrightarrow{\text{cojoin}^J} & J A Y (J Y B C) \\ \downarrow \text{cojoin}^J & & \downarrow \text{cojoin}^J \\ J A X (J X B C) & \xrightarrow{J A X (\text{cojoin}^J)} & J A X (J X Y (J Y B C)) \end{array} \quad \text{cojoin} \circ \text{cojoin} = \text{map}^{J^3} (\text{cojoin}) \circ \text{cojoin} \quad (6.19)$$

Example 6.1.15. () $\mathcal{B}$ is a parameterized comonad. The comonad operations specialize to the following types:

$$\text{extract}^{\mathcal{B}}: \forall (A, X: \text{SET}), \mathcal{B} X X A \rightarrow A$$

$$\text{cojoin}^{\mathcal{B}}: \forall (A, B, C, X: \text{SET}), \mathcal{B} A B C \rightarrow \mathcal{B} A X (\mathcal{B} X B C)$$

The *extract* function collapses $\mathcal{B}$ using the identity function $\text{id}: X \rightarrow 1 X$:

$$\text{extract}^{\mathcal{B}} \equiv \text{runBatch}_{\mathbb{1}} \text{id} \quad (6.20)$$

The *cojoin* operation can be defined by applying $\text{ret}^{\mathcal{B}}$ twice:

$$\text{double}: A \rightarrow \mathcal{B} A B (\mathcal{B} B C C)$$

$$\text{double} = \text{map}^3 \text{ret}^{\mathcal{B}} \circ \text{ret}^{\mathcal{B}}$$

Then cojoin is defined as follows:

$$\text{cojoin}^{\mathcal{B}} \equiv \text{runBatch}_{\mathcal{B}AX \circ \mathcal{B}XB} \text{ double} \quad (6.21)$$

From the perspective of Lemma 6.1.5, the $\text{extract}^{\mathcal{B}}$ operation has type

$$\sum_{n \in \mathbb{N}} (X^n \rightarrow A) \times X^n$$

and is simply a matter of applying the make function to the contents. The cojoin operation has type

$$\left(\sum_{n \in \mathbb{N}} (B^n \rightarrow C) \times A^n \right) \rightarrow \sum_{n \in \mathbb{N}} \left(X^n \rightarrow \left(\sum_{m \in \mathbb{N}} (B^m \rightarrow C) \times X^m \right) \right) \times A^n$$

and has an obvious definition that replaces $\text{make}: B^n \rightarrow C$ with

$$\lambda (x: X^n). (n, \text{make}, x)$$

Lemma 6.1.16. (👉) $\mathcal{B}$ is traversable in its first argument by the following definition:

$$\text{traverse}^1: (A \rightarrow G A') \rightarrow \mathcal{B} A B C \rightarrow G (\mathcal{B} A' B C) \quad (6.22)$$

From the perspective of Lemma 6.1.5, this operation simply traverses over the contents, then maps over the result $G A'$ to pair the A' -vector with the unchanged make function. $\square$

The traversal structure and runBatch are related by the following equations:

$$\text{runBatch}_G f = \text{map}^G \text{extract}^{\mathcal{B}} \circ \text{traverse}^{\mathcal{B}} f \quad (6.23)$$

$$\text{traverse}^1 f = \text{runBatch}_{G \circ (\mathcal{B}A'BC)} \left(\text{map}^G \text{ret}^{\mathcal{B}} \circ f \right) \quad (6.24)$$

It also follows from (6.21) and (6.24) with G specialized to $\mathcal{B}$ that $\text{cojoin}^{\mathcal{B}} = \text{traverse}^1 \text{ret}^{\mathcal{B}}$.

As with monads and their algebras (Sec. 4.2), we can also define (co-)algebras of parameterized (co-)monads [7].

Definition 6.1.17. A (parameterized) coalgebra of a parameterized comonad J is type constructor $F: \text{SET} \rightarrow \text{SET}$ equipped with an operation

$$\text{coalg}: F A \rightarrow J A X (F X)$$

subject to the following two laws:

$$\begin{array}{ccc}
 F A & \xrightarrow{\text{coalg}^F} & J A A (F A) \\
 & \searrow & \downarrow \text{extract}^J \\
 & & F A
 \end{array}
 \qquad \text{extract}^J \circ \text{coalg}^J = \text{id} \quad (6.25)$$

$$\begin{array}{ccc}
 F A & \xrightarrow{\text{coalg}^F} & J A Y (F Y) \\
 \downarrow \text{coalg}^F & & \downarrow \text{cojoin}^J \\
 J A X (F X) & \xrightarrow{J A X (\text{coalg}^F)} & J A X (J X Y (F Y))
 \end{array}
 \qquad \text{cojoin} \circ \text{coalg} = \text{map}^{J^3} (\text{coalg}^F) \circ \text{coalg}^F \quad (6.26)$$

Example 6.1.18. $\mathcal{B}$ is a parameterized coalgebra of itself, with the coalgebra laws reducing to two of the comonad laws.

6.1.3 Traversable Functors as Coalgebras

Definition 6.1.19. (🚗) A coalgebraically presented traversable functor is coalgebra of $\mathcal{B}$. Explicitly, it is a set-forming operation $F: \text{SET} \rightarrow \text{SET}$ equipped with an operation

$$\text{batch}: \forall (A B: \text{SET}), F A \rightarrow \mathcal{B} A B (F B)$$

satisfying the following equations.

$$\text{extract}^{\mathcal{B}} \circ \text{batch}^F = \text{id}_F \quad (6.27)$$

$$\text{cojoin}^{\mathcal{B}} \circ \text{batch}^F = \text{map}^3 (\text{batch}^F) \circ \text{batch}^F \quad (6.28)$$

The next several lemmas are used to prove the equivalence of the preceding definition with traversable functors.

Lemma 6.1.20. (🚗) runBatch and $\text{ret}^{\mathcal{B}}$ are related by the following equation:

$$\text{runBatch}_G f \circ \text{ret}^{\mathcal{B}} = f \quad (6.29)$$

Proof.

$$\begin{aligned}
& \text{runBatch}_G f (\text{ret}^B x) \\
= & \text{runBatch}_G f (\text{id} \boxtimes x) && \text{Unfold } \text{ret}^B \\
= & \text{pure}^G \text{id} \boxtimes^G (f x) && \text{Simplify runBatch} \\
= & \text{map}^G \text{id} (f x) && \text{Eqn. (5.81)} \\
= & f x && \text{Eqn. (4.21)}
\end{aligned}$$

□

Recall that $\text{double}: A \rightarrow \mathcal{B} A B (\mathcal{B} B C C)$ was defined in Example 6.1.15 as part of the comonad structure of $\mathcal{B}$.

Lemma 6.1.21. (🚗) *The following hold for all $g: B \rightarrow G_2 C$ and $f: A \rightarrow G_1 B$ where G_1 and G_2 are applicative functors.*

$$\text{map}^{G_1} g \circ f = \text{map}^{G_1} (\text{runBatch}_{G_2} g) \circ \text{runBatch}_{G_1} f \circ \text{double} \quad (6.30)$$

Proof.

$$\begin{aligned}
& \text{map}^{G_1} g \circ f \\
= & \text{map}^{G_1} g \circ \text{runBatch}_{G_1} f \circ \text{ret}^B && \text{Eqn. (6.29)} \\
= & \text{runBatch}_{G_1} f \circ \text{map}^3 g \circ \text{ret}^B && \text{Eqn. (6.10)} \\
= & \text{runBatch}_{G_1} f \circ \text{map}^3 (\text{runBatch}_{G_2} g \circ \text{ret}^B) \circ \text{ret}^B && \text{Eqn. (6.29)} \\
= & \text{runBatch}_{G_1} f \circ \text{map}^3 (\text{runBatch}_{G_2} g) \circ \text{map}^3 \text{ret}^B \circ \text{ret}^B && \text{Eqn. (4.22)} \\
= & \text{map}^{G_1} (\text{runBatch}_{G_2} g) \circ \text{runBatch}_{G_1} f \circ \text{map}^3 \text{ret}^B \circ \text{ret}^B && \text{Eqn. (6.10)} \\
= & \text{map}^{G_1} (\text{runBatch}_{G_2} g) \circ \text{runBatch}_{G_1} f \circ \text{double} && \text{Fold definitions}
\end{aligned}$$

□

Lemma 6.1.22. *For f, g of the same types above, the following operation is an applicative homomorphism from $(\mathcal{B} A B \circ \mathcal{B} B C)$ to $(G_1 \circ G_2)$.*

$$\text{map}^{G_1} (\text{runBatch}_{G_2} g) \circ \text{runBatch}_{G_1} f: \mathcal{B} A B (\mathcal{B} B C D) \rightarrow G_1 (G_2 D)$$

Proof. This is a special case of a more general observation that $\text{map}^{G_2} \psi \circ \phi$ is an applicative homomorphism $G_1 \circ H_1$ to $G_2 \circ H_2$ whenever $\phi: G_1 \Rightarrow G_2$ and $\psi: H_1 \Rightarrow H_2$ are homomorphisms. □

Lemma 6.1.23. *Traversable functors (Definitions 5.3.21 and 5.3.36) give rise to a coalgebraic presentation. To define batch, we traverse by the monad unit of $\mathcal{B}$, $\text{ret}^{\mathcal{B}}: A \rightarrow \mathcal{B} A B B$:*

$$\text{batch}^F \equiv \text{traverse}_{\mathcal{B}AB}^F \text{ret}^{\mathcal{B}} \quad (6.31)$$

Proof. The proofs of the coalgebras laws rely on the fact that runBatch is always an applicative homomorphism (Lemma 6.1.10) and therefore commutes with traverse^F . Equation (6.27) makes use of (indeed, corresponds to) the identity law for traversable functors:

$$\begin{aligned} & \text{extract}^{\mathcal{B}} \circ \text{batch}^F \\ = & \text{runBatch}_{\mathbb{1}} \text{id} \circ \text{traverse}_{\mathcal{B}AB}^F \text{ret}^{\mathcal{B}} && \text{Unfold definitions} \\ = & \text{traverse}_{\mathbb{1}}^F \left(\text{runBatch}_{\mathbb{1}} \text{id} \circ \text{ret}^{\mathcal{B}} \right) && \text{Eqn. (5.102)} \\ = & \text{traverse}_{\mathbb{1}}^F \text{id} && \text{Eqn. (6.29)} \\ = & \text{id}_F && \text{Eqn. (5.100)} \end{aligned}$$

The proof of (6.28) makes corresponding use of the linearity law for traversable functors:

$$\begin{aligned} & \text{cojoin}^{\mathcal{B}} \circ \text{batch}^F \\ = & \text{runBatch}_{\mathcal{B}AB \circ \mathcal{B}BC} \text{double} \circ \text{traverse}_{\mathcal{B}AB}^F \text{ret}^{\mathcal{B}} && \text{Unfold definitions} \\ = & \text{traverse}_{\mathcal{B}AB \circ \mathcal{B}BC}^F \left(\text{runBatch}_{\mathcal{B}AB \circ \mathcal{B}BC} \text{double} \circ \text{ret}^{\mathcal{B}} \right) && \text{Eqn. (5.102)} \\ = & \text{traverse}_{\mathcal{B}AB \circ \mathcal{B}BC}^F \left(\text{map}^3 \text{ret}^{\mathcal{B}} \circ \text{ret}^{\mathcal{B}} \right) && \text{Eqn. (6.29)} \\ = & \text{map}^3 \left(\text{traverse}_{\mathcal{B}AB}^F \text{ret}^{\mathcal{B}} \right) \circ \text{traverse}_{\mathcal{B}BC}^F \text{ret}^{\mathcal{B}} && \text{Eqn. (5.101)} \\ = & \text{map}^3 \text{batch}^F \circ \text{batch}^F && \text{Fold definitions} \end{aligned}$$

□

Lemma 6.1.24. *A coalgebraic presentation of traversable functors induces a presentation in terms of traverse with the following definition:*

$$\text{traverse}^F f \equiv \text{runBatch}_G f \circ \text{batch}^F \quad (6.32)$$

Proof.

Proof of (5.100):

$$\begin{aligned}
& \text{traverse}^F \text{id} \\
= & \text{runBatch}_{\mathbb{1}} \text{id} \circ \text{batch}^F && \text{Unfold definitions} \\
= & \text{extract}^B \circ \text{batch}^F && \text{Eqn. (6.20)} \\
= & \text{id}_F && \text{Eqn. (6.27)}
\end{aligned}$$

Proof of (5.101):

$$\begin{aligned}
& \text{map}^{G_1} \left(\text{traverse}_{G_2}^F g \right) \circ \text{traverse}_{G_1}^F f \\
= & \text{map}^{G_1} \left(\text{runBatch}_{G_2} g \circ \text{batch}^F \right) \circ \text{runBatch}_{G_1} f \circ \text{batch}^F && \text{Unfold definitions} \\
= & \text{map}^{G_1} \left(\text{runBatch}_{G_2} g \right) \circ \text{map}^{G_1} \text{batch}^F \circ \text{runBatch}_{G_1} f \circ \text{batch}^F && \text{Eqn. (4.22)} \\
= & \text{map}^{G_1} \left(\text{runBatch}_{G_2} g \right) \circ \text{runBatch}_{G_1} f \circ \text{map}^3 \text{batch}^F \circ \text{batch}^F && \text{Eqn. (6.10)} \\
= & \text{map}^{G_1} (\text{runBatch}_{G_2} g) \circ \text{runBatch}_{G_1} f \circ \text{cojoin}^B \circ \text{batch}^F && \text{Eqn. (6.28)} \\
= & \text{map}^{G_1} (\text{runBatch}_{G_2} g) \circ \text{runBatch}_{G_1} f \circ \text{runBatch}_{B \circ A \circ B} (\text{double}) \circ \text{batch}^F && \text{Eqn. (6.21)} \\
= & \text{runBatch}_{G_1 \circ G_2} \left(\text{map}^{G_1} (\text{runBatch}_{G_2} g) \circ \text{runBatch}_{G_1} f \circ \text{double} \right) \circ \text{batch}^F && \text{Eqn. (6.11)} \\
= & \text{runBatch}_{G_1 \circ G_2} \left(\text{map}^{G_1} g \circ f \right) \circ \text{batch}^F && \text{Eqn. (6.30)} \\
= & \text{traverse}_{G_1 \circ G_2}^F \left(\text{map}^{G_1} g \circ f \right) && \text{Fold definition}
\end{aligned}$$

Proof of (5.102): The property holds simply by (6.11). $\square$

The following theorem consists of a routine verification that the presentations of a traversable functor as an extension system and as a coalgebra uniquely determine each other. This theorem is the traversable analogue of Lemma 5.1.20, which states that linearizable functors are equivalently presented in terms of tolist and mapReduce.

Theorem 6.1.25. *Definitions 6.1.19 and 5.3.36 are equivalent.*

Just as Corollary 5.1.23 states that any mapReduce^F factorizes through tolist^F , the following states that traverse^F factorizes through B via batch^F .

Corollary 6.1.26. *For all $f: A \rightarrow G B$ where G is applicative, the following holds:*

$$\text{traverse}^F f = \text{runBatch} f \circ \text{batch}^F \quad (6.33)$$

Much like Corollary 5.1.23, this property holds irrespective of whether traverse , dist , or batch is taken as the primitive operations. The significance stems from that fact that we can reason about

traversable functors by rewriting all instances of traverse^F using Corollary 6.1.26 and performing **induction** on batch^F . This yields many important corollaries. We now turn our attention to developing some of them.

6.1.4 Traversable Functor Representation Theorem

The representation theorem for traversable functors was first proved by Bird et al. [17], who took a direct syntactical approach. The same theorem was also derived by Jaskelioff and O'Connor [43] as a corollary of a more powerful general representation theorem. The approach taken in this work employs the coalgebraic machinery developed in the latter work.

Much like the container operations (Definition 5.1.25) are generalized to arbitrary linearizable functors from primitive operations on lists, the following operations are generalized to arbitrary traversable functors.

Definition 6.1.27. (🚗) *Let F be a traversable functor. The following operations are given:*

$$\begin{aligned} \text{size}^F &: \forall (A: \text{SET}), F A \rightarrow \mathbb{N} \\ \text{make}^F &: \forall (A: \text{SET}) (t: F A) (B: \text{SET}), B^{\text{size } t} \rightarrow F B \\ \text{contents}^F &: \forall (A: \text{SET}) (t: F A), A^{\text{size } t} \end{aligned}$$

These may be defined as follows:

$$\text{size}^F \equiv \text{size}^B \circ \text{batch}^F \quad (6.34)$$

$$\text{make}^F \equiv \text{make}^B \circ \text{batch}^F \quad (6.35)$$

$$\text{contents}^F \equiv \text{contents}^B \circ \text{batch}^F \quad (6.36)$$

An important observation is that make^F is polymorphic in B . That is, given any $t: F A$, the make function has the following type:

$$\text{make}^F t: \forall (B: \text{SET}), B^{\text{size } t} \rightarrow F B$$

Moreover, the make function is an *invariant*—applying $\text{make } t$ to a vector of B values results in a value of type $F B$ with the same size and make function, but with the new vector as its contents. This invariance is subsumed by the lens laws for traversable functors, discussed below.

The following theorem is the statement of Corollary (6.1.26) from the perspective of the RHS of the natural isomorphism given by (6.3).

Theorem 6.1.28. (🚗) *Let F be a traversable functor and consider any $f: A \rightarrow G B$ where G is an*

applicative functor. Then *traverse* decomposes as follows:

$$\text{traverse}^F f t = \text{map}^G \left(\text{make}^F t \right) \left(\text{traverse}^{\text{Vec}(\text{size } t)} f \left(\text{contents}^F t \right) \right) \quad (6.37)$$

Theorem 6.1.28 states any traversal over $t : F A$ is obtained by traversing over just its *contents*, mapping a vector A^n to $G B^n$ where n is the size^F of the input. Then, the make function of type $B^n \rightarrow F B$ is mapped over the result to obtain $G (F B)$.

Remark 6.1.29. *The representation theorem as formalized in Tealeaves relates traverse^F to traversing over the contents with the opposite applicative functor, defined by multiplying effects in the opposite order, in analogy to the opposite monoid. This is because contents^F returns a list in the opposite order as would be given by *toList*. Essentially, this stems from the fact that the outermost A -value resulting from batch t value is the “last” element according to the order determined by the traversal, while the “first” element is the innermost value nearest the make function.*

Lens Decomposition

As has been revealed by work on *profunctor optics* [25], traversable functors can also be understood in terms of generalized very well behaved lenses [34]. Namely, they are very well behaved lenses that focus on multiple elements—the contents—simultaneously, where the make acts as the “put” operation. We save a more thorough development of lens-based perspective for future work, except to state the lens laws.

Lemma 6.1.30. (🚩) *Let $t : F A$ be a term of a traversable functor. The following laws are satisfied for all vectors v, v_1, v_2 of size $\text{size}^F t$ and arbitrary content types.*

$$\text{contents}^F \left(\text{make}^F t v \right) = v \quad (6.38)$$

$$\text{make}^F t \left(\text{contents}^F t \right) = t \quad (6.39)$$

$$\text{make}^F \left(\text{make}^F t v_1 \right) v_2 = \text{make}^F t v_2 \quad (6.40)$$

The lens laws are immediate to derive—(6.39) follows from (6.64) by taking G to be **I** and f to be the identity, for example. □

Remark 6.1.31. *Actually, (6.40) is not quite well-typed as written: the function $\text{make}^F \left(\text{make}^F t v_1 \right)$ in the LHS expects a vector of size $\text{size}^F \left(\text{make}^F t v_1 \right)$, while the occurrence of v_2 in the RHS has type $\text{size}^F t$. These values are provably equal, so there is no theoretical problem. However, the formalization of a lens-based perspective in Coq requires some finesse to handle this sort of type-theoretic issue, such as introducing functions that accept a proof of $n = m$ and use this to coerce a vector of length n to one of length m .*

Shapeliness

Corollary 6.1.32. *Let t_1 and t_2 be terms of type $F A$ where F is a traversable functor. Then the terms are equal if and only if they have identical contents and make functions:*

$$t_1 = t_2 \iff \text{contents } t_1 = \text{contents } t_2 \text{ and } \text{make } t_1 = \text{make } t_2. \quad (6.41)$$

Proof. Suppose $\text{contents } t_1$ is equal to $\text{contents } t_2$ and $\text{make } t_1$ is equal to $\text{make } t_2$. Then (6.39) implies the following:

$$t_1 = \text{make}^F t_1 (\text{contents}^F t_1) = \text{make}^F t_2 (\text{contents}^F t_2) = t_2$$

The direction assuming $t_1 = t_2$ is trivial. □

We can also view the above equivalence through a slightly different perspective. Instead of considering the make function of a term, which is a function with a vector-valued input, we can equivalently consider its *shape*, which is a data structure.

Definition 6.1.33. *A shape, more precisely an F -shape, is a value of type $F \mathbf{1}$. The shape of a term is the value computed by mapping all of its contents to the singleton:*

$$\text{shape}^F t \equiv \text{map}^F (\text{const } *) t \quad (6.42)$$

Lemma 6.1.34. *There is a one-to-one correspondence between shapes with length n and make functions of arity n .*

Proof. Given a shape, we simply take its make function as a term of a traversable functor. Given a make function, we simply plug in n -many copies of $*$ as its input (recall that the make function is polymorphic in its input type). The uniqueness of the correspondence is a corollary of the lens laws. □

A linearizable functor F is *shapely* (over the list functor) [45] if it satisfies the following property for all terms t, u of type $F A$:

$$\forall (t, u : F A), t = u \iff \text{shape}^F t = \text{shape}^F u \wedge \text{tolist}^F t = \text{tolist}^F u \quad (6.43)$$

Lemma 6.1.35. (🚗) *Traversable functors are shapely.*

Proof. The left-to-right direction is trivial. The right-to-left direction is effectively a different presentation of Corollary 6.1.32: if two terms have the same shape, they must have the same make function, and if they have the same value of `tolist`, then they share the same contents. Therefore they are equal. □

Lemma 5.3.30, previously presented without proof in Section 5.3, is an immediate corollary of the shapeliness of traversable functors. First, one easily verifies that mapping over a term t does not change its shape. If two maps are equal on elements of t , then they also have an equal effect on t . Because a term is determined by its shape and tolist, the two maps have an equal effect on t .

6.1.5 Zipping Terms

Now we turn our attention to an important operation on traversable functors, and hence on DTMs, which relies heavily on the presentation of traversable functors in terms of the Batch functor.

Definition 6.1.36. () Let $v_1 : A^n$ and $v_2 : B^n$ be two vectors of the same length. Their **zip** is the vector of the same length formed by pairing corresponding entries in the two vectors:

$$\text{zip}^{\text{Vec } n} v_1 v_2 : (A \times B)^n \quad (6.44)$$

That is, $\text{zip } v_1 v_2$ is defined by the following specification:

$$\text{map}^{\text{Vec } n} \text{fst} (\text{zip}^{\text{Vec } n} v_1 v_2) = v_1 \quad (6.45)$$

$$\text{map}^{\text{Vec } n} \text{snd} (\text{zip}^{\text{Vec } n} v_1 v_2) = v_2 \quad (6.46)$$

Define the zip of two vectors of the same length is a straightforward structural recursion. The decomposition of traversable functors in terms of lenses provides an elegant way to lift the same concept to syntax trees that have the same shape.

Lemma 6.1.37. () Let $t : F A$ and $u : F B$ be two terms of a traversable functor F , and suppose they have the same shape:

$$\text{shape } t = \text{shape } u$$

Then there is a term $\text{zip } t u : F (A \times B)$ which also has the same shape, and satisfies the following equations:

$$\text{map } \text{fst} (\text{zip } t u) = t \quad (6.47)$$

$$\text{map } \text{snd} (\text{zip } t u) = u \quad (6.48)$$

Moreover, **zip** is natural in both arguments in the sense that the following holds:

$$\text{map}^F (f \times g) (\text{zip } t u) = \text{zip} (\text{map}^F f t) (\text{map}^F g u) \quad (6.49)$$

Proof. By Lemma 6.1.34, terms t and u have the same size and the same make function. That is, t

and u decomposed into triples of the same length n , say

$$t \simeq (n, v_t : A^n, \text{mk} : \forall (X : \text{SET}), X^n \rightarrow F X) \quad u \simeq (n, v_u : B^n, \text{mk} : \forall (X : \text{SET}), X^n \rightarrow F X).$$

Then $\text{zip}^F t u$ is defined by applying their shared make function to the zip of their contents:

$$\text{zip}^F t u \equiv \text{mk} \left(\text{zip}^{\text{Vec } n} v_t v_u \right) : F (A \times B).$$

The naturality of the generalized zip^F operation is inherited from the naturality of mk and $\text{zip}^{\text{Vec } n}$ in both arguments. $\square$

Note that the function zip^F is parameterized by a proof its inputs have the same shape, but we do not explicitly indicate this argument in the presentation.

Lifting Relations

So far, all uses of operations like `traverse`, `binddt`, etc. have been to lift *functions* over terms. Typically, this has been used to define functions from terms to terms by specifying their action on variables. Now we consider a mechanism to lift *relations* over terms, providing a way to define n -ary relations on terms of the same shape by specifying how they relate corresponding occurrences in different terms.

Definition 6.1.38. () Let F be a traversable functor and let

$$R : A \rightarrow B \rightarrow \mathbf{Prop}$$

be a binary relation. We define an operation called the lift of R of the following type:

$$\text{lift}^F R : F A \rightarrow F B \rightarrow \mathbf{Prop}$$

The lifted relation is such that $\text{lift } R t u$ holds exactly when t and u have the same shape, and where their corresponding occurrences are related by R . This lifted relation is defined by traversing over t and interpreting R as an operation from A into the subset applicative functor (Ex. 5.3.14):

$$\text{lift}^F R t u \equiv \text{traverse}_{\text{subset } A}^F R t u \tag{6.50}$$

The lifted relation is surprisingly difficult to reason about from the first principles of `traverse`. One reason for this is that `lift` treats t and u quite differently: t is subject to a traversal, which is used to compute a predicate, while u is simply the argument to which this predicate is applied. For instance, it is difficult to prove the following property of $\text{lift } R t u$ when expressed in terms of

traverse^F , where $R^{\text{op}}: B \rightarrow A \rightarrow \mathbf{Prop}$ is defined by flipping the inputs to R :

$$\text{lift } R \ t \ u \iff \text{lift } (R^{\text{op}}) \ u \ t$$

Note that $\text{lift } R \ t \ u$ and $\text{lift } (R^{\text{op}}) \ u \ t$ are defined by traversing over different two terms, and it is not immediately apparent how to begin relating traversals over different terms. The solution is to reduce all reasoning to the “base” case, namely vectors. Just as the generalized container operations presented in Section 5.1.2 were understood by expressing them in terms of the concrete operations defined on list, lifted relations can be understood by considering concrete operations on vectors.

Lemma 6.1.39. (🐼) *For all vectors $v_1: A^n$ and $v_2: B^n$ of the same length, the following equivalence provides a specification for traversing over v_1 by R .*

$$\text{traverse}_{\text{subset}A}^{\text{Vec } n} R \ v_1 \ v_2 \iff \text{ForAll } (\lambda (a, b). R \ a \ b) (\text{zip } v_1 \ v_2) \quad (6.51)$$

Proof. This is a straightforward proof given by induction on the size n . If $n = 0$, then v_1, v_2 , and $\text{zip } v_1 \ v_2$ are all vnil and the goal is immediate. If n is $S \ m$, then $v_1 = \text{vcons } a \ w_1$ and $v_2 = \text{vcons } b \ w_2$ for some a, b, w_1 , and w_2 . Now the LHS simplifies to the following:

$$\left(\text{pure}^{\text{subset}} \text{vcons} \circledast R \ a \circledast \text{traverse } R \ w_1 \right) (\text{vcons } b \ w_2)$$

By unfolding the applicative operations for subset, we find that this proposition is true if and only if there exists some f such that $f = \text{vcons}$, some b' such that $R \ a \ b'$, and some w' such that $\text{traverse } R \ w_1 \ w' = f \ b' \ w'$ —and all of these values such that $\text{vcons } b \ w_2 = f \ b' \ w'$, which implies $b = b'$ and $w = w'$. On the other hand, the RHS simplifies to the following:

$$R \ a \ b \wedge (\text{ForAll } (\lambda (a, b). R \ a \ b) (\text{zip } w_1 \ w_2))$$

The conclusion follows immediately by the inductive hypothesis. $\square$

This characterization on the RHS of the above condition is useful because it is essentially symmetric with respect v_1 and v_2 . For example, it is straightforward to show the following property.

Lemma 6.1.40. *Zippping vectors is swap-invariant in the following sense, which holds for all vectors and binary relations:*

$$\text{ForAll } (\lambda (a, b). R \ a \ b) (\text{zip } v_1 \ v_2) \iff \text{ForAll } (\lambda (a, b). R^{\text{op}} \ a \ b) (\text{zip } v_2 \ v_1)$$

Proof. By induction on n , we observe that $\text{zip } v_1 \ v_2 = \text{map}^{\text{Vec } n} (\lambda (b, a). (a, b)) (\text{zip } v_2 \ v_1)$. The goal

is then solved using the following composition law between `traverse` and `map`

$$\text{traverse } g (\text{map } f \, t) = \text{traverse } (g \circ f)$$

combined with the observation that swapping a pair twice is the identity. $\square$

The specification for `lift R` for vectors, combined with Theorem 6.1.28, leads to the following lemma for traversable functors in general.

Theorem 6.1.41. *Let $t : F \, A$ and $u : F \, B$ be two terms where F is a traversable functor and let R be a binary relation of type $A \rightarrow B \rightarrow \mathbf{Prop}$. The following equivalence provides a specification for `lift R`.*

$$\text{lift } R \, t \, u \iff (\text{shape}^F t = \text{shape}^F u \wedge \forall (a : A) (b : B), (a, b) \in (\text{zip } t \, u) \rightarrow R \, a \, b) \quad (6.52)$$

Proof. Note that $(\text{traverse}^F R \, t)$, unlike the `zip` function, is total: it can be applied to u regardless of whether t and u have the same shape. By unfolding the statement of the representation theorem for lift^F , we obtain a clearer insight into the statement on the LHS.

$$\begin{aligned} & \text{traverse}^F R \, t \, u \\ = & \left(\text{map}^{\text{subset}} \left(\text{make}^F t \right) \left(\text{traverse}^{\text{Vec}(\text{size } t)} R \left(\text{contents}^F t \right) \right) \right) u && \text{Eqn. (6.64)} \\ = & \exists (v : B^{\text{size } t}), \left(\text{traverse}^{\text{Vec}(\text{size } t)} R \left(\text{contents}^F t \right) v \right) \wedge \text{make}^F t \, v = u && \text{Unfold map}^{\text{subset}} \end{aligned}$$

That is, `lift R t u` holds if and only if the following conditions hold:

1. There is a vector $v : B^{\text{size } t}$ of type B of the same size as t , and
2. The vector v is pointwise related to the contents of t by R , and
3. The term u decomposes as the `make` function of t applied to v .

In particular, the unrolled statement implies that if `lift R t u` holds, then t and u have the same size^F and indeed the same make^F function, since `make` functions are unique. The equivalence of these conditions with the RHS then holds fairly trivially. $\square$

Theorem 6.1.42. (🐞) *For all terms $t : F \, A$ and $u : F \, B$ of the same shape, the following equivalence provides a specification for lifting relations.*

$$\text{lift } R \, t \, u \iff \text{ForAll } (\lambda (a, b). R \, a \, b) (\text{zip } t \, u) \quad (6.53)$$

Proof. This follows from Lemma 6.1.39 and Theorem 6.1.41. $\square$

The following lemma holds by the specification above and the naturality of the `zip` operation.

Corollary 6.1.43. *lift R is natural in both arguments. That is, the following holds for all t, u, f, g :*

$$\text{lift } R \left(\text{map}^F f t \right) \left(\text{map}^F g u \right) \iff \text{lift } R \left((a, b) \mapsto R (f a, g b) \right) t u \quad (6.54)$$

All of the above theory extends cleanly to decorated traversable monads. Before presenting the details of this extension, we consider an application of the theory: instantiating Tealeaves with a variadic binding construct.

6.2 VARIADIC BINDERS

In Section 3.5, recall that we extended the syntax $\text{lam } V$ with a letin constructor that binds several definitions in its body at once. We considered this constructor with three semantics based on which binders were opened inside the list of definitions: a simple semantics, which opens no binders in the definition list; a telescoping semantics, where each definition opens one binder in subsequent definitions; and a fully mutually recursive semantics, in which all definitions are considered in a context with as many binders opened as there are definitions. The semantics of binding is reflected in the definition of binddt using the preincrement operator $(\odot)$. Under the simple semantics, we define $\text{binddt}^{\text{lam}}$ by extending the operation from Definition 5.4.26 with the following case for the letin constructor.

Example 6.2.1. *The following definition captures a simple letin semantics:*

$$\begin{aligned} \text{binddt}_G^{\text{lam}} f (\text{letin } l t) &\equiv \text{pure}^G \text{letin} \circledast \left(\text{traverse}^{\text{list}} \left(\text{binddt}_G^{\text{lam}} f \right) l \right) \\ &\quad \circledast \left(\text{binddt}_G^{\text{lam}} (f \odot \text{length } l) t \right) \end{aligned}$$

The recursive call to $\text{binddt}^{\text{lam}}$ for the body t is self-explanatory, opening length l binders in t . To perform substitution inside the list of definitions, whose type is $\text{list } (\text{lam } A)$, we begin by mapping $\text{binddt } f$ over the list without using $(\odot)$, reflecting the fact that no binders are opened in any of the entries in l . However, merely mapping this function over l results in something of type $\text{list } (G (\text{lam } B))$. We must also factor G out of list using applicative distribution—hence $\text{traverse}^{\text{list}} (\text{binddt}_G^{\text{lam}} f) l$.

After extending binddt with this new case, we must extend the proofs of the DTM axioms to the letin constructor. The proof of (5.129) is not affected, since this axiom merely posits the definition for the var case. Equations (5.130), and (5.132) are straightforward enough using a strengthened version of the induction principle inferred by Coq (see commentary below). However, the proof of the composition law (5.131) does not immediately go through. A more sophisticated approach based on the representation theorem for $\text{traverse}^{\text{list}}$ is required.

To understand the technical obstruction, consider the proof of (5.131) for lam , particularly the abs case. This proof of the DTM instance for lam is given with commentary in Appendix A,

with the composition law shown in Lemma A.4.3. The basic strategy involves induction on the structure of some $t : \text{lam } A$, where each case proceeds by rewriting the LHS and RHS into a common normal form using the equational laws governing applicative functors and the $(\odot)$ operation. The goal in each case is reduced to something of the following form:

$$\text{pure}^G \text{mk} \circledast^G a_1 \circledast^G a_2 \dots \circledast^G a_n \stackrel{?}{=} \text{pure}^G \text{mk}' \circledast^G a_1 \circledast^G a_2 \dots \circledast^G a_n$$

where mk and mk' are syntactically different expressions, while the arguments $\{a_i\}$ are syntactically identical on both sides. At this junction, the proof will be solved by `reflexivity` if and only if mk and mk' are definitionally (i.e. computationally) equal in the underlying type theory of Coq. For the $t = \text{abs } t'$ case, the equality between f and f' is the following definitional equality, which can be understood as a point-free specification of the definition of `binddt` when applied to the `abs` constructor:

$$(\text{binddt } f \circ \text{abs}) = (\text{pure } \text{abs} \circledast -) \circ \text{binddt } (f \odot 1)$$

One notable feature encoded into this definition is that regardless of the argument t to `abs`, the abstraction always opens 1 binder in its body.

Now consider the $t = \text{letin } l \ t'$ case of the induction. If precisely the same strategy is taken, as shown explicitly in Section A.5, the goal for this case is reduced to a point where the equality required between mk and mk' is the following one:

$$\begin{aligned} & \left(\left(\left(\text{binddt}_{G_2} g \right) \circ - \right) \circ \text{letin} \right) \\ & \stackrel{?}{=} \\ & \left(- \circ \text{binddt}_{G_2} (g \odot \text{length } l) \right) \circ \left(\circledast^{G_2} \right) \circ \left(\text{pure}^{G_2} \text{letin} \circledast^{G_2} - \right) \circ \text{traverse}^{\text{list}} \left(\text{binddt}_{G_2} g \right) \end{aligned} \tag{6.55}$$

The problem with this equation is that it is not true. Expressing both sides pointwise makes this slightly more clear.

$$\begin{aligned} & \lambda l'. \lambda t'. \text{pure}^{G_2} \text{letin} \circledast^{G_2} \text{traverse}^{\text{list}} \left(\text{binddt}_{G_2} g \right) l' \circledast^{G_2} \text{binddt}_{G_2} (g \odot \text{length } l') t' \\ & \stackrel{?}{=} \lambda l'. \lambda t'. \text{pure}^{G_2} \text{letin} \circledast^{G_2} \text{traverse}^{\text{list}} \left(\text{binddt}_{G_2} g \right) l' \circledast^{G_2} \text{binddt}_{G_2} (g \odot \text{length } l) t' \end{aligned}$$

The issue is that in the subexpression $(g \odot \text{length } l)$ on the RHS, $\text{length } l$ refers to the list of definitions in the *original term* `letin l t`—the object upon which we perform induction. On the LHS, the number of binders that will be opened in t' is $\text{length } l'$, where l' is the abstract argument to `letin`. This mismatch arises in the `letin` case because, unlike in the `abs` case, the number of binders opened in the body is dynamically *computed* from one of the arguments to the constructor. The DTM composition law (5.131) requires that successive invocations of `binddt` read the same values for binders at the same locations in the term—reflected in the two appearances of w on the RHS

of the generalized Kleisli composition (5.133). On the LHS for the $\text{letin } l \ t$ case, $\text{binddt } f$ computes length l and opens this many binders in the body. The next call, $\text{binddt } g$, also computes the length of the list—but this list has just changed, since $\text{binddt } f \ (\text{letin } l \ t')$ traverses over l by $\text{binddt } f$. It is not immediately obvious that this traversal does not change the length of the list, and the standard rewriting strategy leaves the goal in an unsolvable state.

It might seem that what is needed is a proof that the length of l is preserved by the traversal. Actually, the situation is more subtle: a naïve statement of the proposition yields the following ill-typed proposition:

$$\text{length} \left(\text{traverse}^{\text{list}} f \ l \right) \stackrel{?}{=} \text{length } l$$

This is ill-typed because $\text{traverse}^{\text{list}} f$ does not return a list but something of type $G \ (\text{list } B)$. The appearance of G obstructs the application of length to the list. In fact, consider the case where G is the option applicative (Ex. 5.3.12). In this case, one possible outcome of $\text{traverse}^{\text{list}} f \ l$ is None ! It is hardly clear in what sense such an outcome preserves the length of l . Or consider the case in which G is list itself (Ex. 5.3.13). In this case, $\text{traverse}^{\text{list}} f \ l$ yields not just a list but a *list* of lists—which of these should length measure? Clearly, a new approach is required to prove the composition law for letin . We return to this in Section 6.2.2 after clearing a few orthogonal implementation details concerning variadic syntax.

6.2.1 Tealeaves Implementation

There are two caveats we must give at this point, which are not essential to the theory but to the implementation. Both are related to the appearance of something of type $\text{list } (A)$ in the inductive definition of lam itself. This is a nested inductive datatype [15], which introduces some additional complexity. First, the subexpression $\text{traverse}^{\text{list}} (\text{binddt } f)$ does not apply binddt to a direct subterm, so this definition is rejected because it is not structurally recursive. The user must resort to a tactic such as expanding the definition of $\text{traverse}^{\text{list}}$ inline in the definition of binddt . After the fact, the user can prove equality with the expression shown in Example 6.2.1 and proceed as normal.

A second caveat related to the nested inductive structure of lam extended with letin is that the default induction principle inferred by Coq is too weak. To prove the DTM laws, we must manually prove a stronger induction principle. Specifically, we derive a principle with the following statement of the inductive hypothesis for the letin case, where P is the inductive predicate:

$$(\forall (u : \text{lam } A), u \in l \implies P u) \implies P t \implies P (\text{letin } l \ t) \quad (6.56)$$

This stronger induction allows the proofs of (5.130) and (5.132) to go through fairly straightforwardly, so we concentrate on the composition law, which as shown above is not as straightforward. For a pragmatic tutorial about nested inductive datatypes in Coq, see [23], Chapter 3.

6.2.2 The Representation Theorem for list

To prove that traversing over a list of definitions to perform substitution does not change the shape of the list, we specialize the representation theorem to list. When specialized to the list type, the $\text{size}^{\text{list}}$ operation in Definition 6.1.27 is equal to the normal length of the list, while the other operations specialize to the following types:

$$\begin{aligned} \text{make}^{\text{list}} &: \forall (A : \text{SET}) (l : \text{list } A) (B : \text{SET}), B^{\text{length } l} \rightarrow \text{list } B \\ \text{contents}^{\text{list}} &: \forall (A : \text{SET}) (l : \text{list } A), A^{\text{length } l} \end{aligned}$$

Recalling that a vector is implemented as a list paired with a proof of its length, the $\text{contents}^{\text{list}}$ operation simply pairs any list l with a proof that l has length $\text{length } l$ (of course). The $\text{make}^{\text{list}} l$ operation accepts any list l' of B -values with same length the same length as l and returns l' . Effectively, it represents the identity function with inputs restricted to lists of length $\text{length } l$. Theorem 6.1.28 specializes to the following:

$$\text{traverse}^{\text{list}} f l = \text{map}^G \left(\text{make}^{\text{list}} l \right) \left(\text{traverse}^{\text{Vec}(\text{length } l)} f \left(\text{contents}^{\text{list}} l \right) \right) \quad (6.57)$$

Effectively, this decomposition of $\text{traverse}^{\text{list}}$ can be thought of as proceeding in steps. First, the length of l is “frozen” by coercing that list into a vector, whose length is baked into its very type. We then traverse over this vector, yielding a vector whose type is wrapped in G . Then, we map over G to “unfreeze” the vector, turning it back into a list by discarding the static information about its length. To push the proof through, before applying the unfreezing operation

$$\text{map}^G \left(\text{make}^{\text{list}} l \right),$$

we use this operation as a witness to the subsequent application of $\text{binddt}_{G_2} g$ that the list under G , so to speak, has a particular length. We now return to the flawed proof discussed above, and demonstrate the pivotal rewriting steps push the proof through. Below, (6.64) is invoked, followed by (4.22), with the net effect that letin becomes pre-composed with a witness to the fact that the argument to letin has length l .

$$\begin{aligned} & \text{map}^{G_1} \left(\text{traverse}^{\text{list}} \left(\text{binddt}_{G_2} g \right) \right) \left(\text{traverse}^{\text{list}} \left(\text{binddt } f \right) l \right) \\ = & \text{map}^{G_1} \left(\text{traverse}^{\text{list}} \left(\text{binddt}_{G_2} g \right) \right) \left(\text{map}^{G_1} \left(\text{make } l \right) \left(\text{traverse}^{\text{Vec}} \left(\text{binddt } f \right) \left(\text{contents } l \right) \right) \right) \\ = & \text{map}^{G_1} \left(\text{traverse}^{\text{list}} \left(\text{binddt}_{G_2} g \right) \circ \text{make } l \right) \left(\text{traverse}^{\text{Vec}} \left(\text{binddt } f \right) \left(\text{contents } l \right) \right) \end{aligned}$$

After this step, the difficulty in Section 6.2 is overcome. By strategically inserting the steps shown above into the proof of Lemma A.5.1, but otherwise leaving the proof unchanged, the goal

is reduced to a point where the final equation that must be demonstrated, rather than the false equation (6.55), is instead the following true one:

$$\begin{aligned} & \left(\left(\left(\text{binddt}_{G_2} g \right) \circ - \right) \circ \text{letin} \circ \text{make}^{\text{list}} l \right) \\ & \quad \stackrel{?}{=} \\ & \left(- \circ \text{binddt}_{G_2} (g \odot \text{length } l) \right) \circ \left(\circledast^{G_2} \right) \circ \left(\text{pure}^{G_2} \text{letin} \circledast^{G_2} - \right) \circ \text{traverse}^{\text{list}} \left(\text{binddt}_{G_2} g \right) \circ \text{make}^{\text{list}} l \end{aligned} \quad (6.58)$$

Or, expressed pointwise, where v is a *vector* of type $\text{Vec}(\text{length } l) B$:

$$\begin{aligned} & \text{pure}^G \text{letin} \circledast \left(\text{traverse}^{\text{list}} \left(\text{binddt}_G^{\text{lam}} f \right) \left(\text{make}^{\text{list}} l v \right) \right) \\ & \quad \circledast \left(\text{binddt}_G^{\text{lam}} \left(f \odot \text{length} \left(\text{make}^{\text{list}} l v \right) \right) t \right) \\ = & \text{pure}^G \text{letin} \circledast \left(\text{traverse}^{\text{list}} \left(\text{binddt}_G^{\text{lam}} f \right) \left(\text{make}^{\text{list}} l v \right) \right) \\ & \quad \circledast \left(\text{binddt}_G^{\text{lam}} (f \odot \text{length } l) \right) t \end{aligned}$$

Thus, we must only verify the trivial equality $\text{length } l = \text{length}(\text{make}^{\text{list}} l v)$. The proof goes through and we can reason about variadic syntax as a DTM.

6.3 COALGEBRAIC DECORATED TRAVERSABLE MONADS

We now extend the coalgebraic presentation of traversable functors to decorated traversable functors, traversable monads, and DTMs. The coalgebraic theory of DTMs is a topic of ongoing formalization efforts and likely to be of some interest to non-syntactic applications.

6.3.1 Coalgebraic Decorated Traversable Functors

Let E be any set. We consider traversable functors decorated by E , leaving the E parameter itself implicit in this section. This abstraction is rather straightforward: we simply consider the $\mathcal{B}$ datatype composed with the environment comonad in its first type parameter. In other words,

$$\mathcal{D} A B C = \mathcal{B} (E \times A) B C$$

We consider the following generalization of the double operation, originally defined in Example 6.1.15 as part of the cojoin operation of the $\mathcal{B}$ comonad.

$$\begin{aligned} & \text{doubleDec}: E \times A \rightarrow \mathcal{B} (E \times A) B (\mathcal{B} (E \times B) C C) \\ & \text{doubleDec}(e, a) \equiv \text{map}^3 \left(\text{ret}^{\mathcal{B}} (e, -) \right) \left(\text{ret}^{\mathcal{B}} (e, a) \right) \end{aligned}$$

The presentation of $\mathcal{B}$ as a comonad has the following generalization that incorporates an additional E annotation on its contents.

Lemma 6.3.1. $\mathcal{D}$ forms a parameterized comonad with the following definitions:

$$\begin{aligned} \text{extract}^{\mathcal{D}} &: \mathcal{B}(E \times X) X A \rightarrow A \\ \text{cojoin}^{\mathcal{D}} &: \mathcal{B}(E \times A) B C \rightarrow \mathcal{B}(E \times A) X (\mathcal{B}(E \times X) B C) \\ \text{extract}^{\mathcal{D}} &\equiv \text{runBatch}_{\mathbb{I}} \text{extract}^{E \times} \\ \text{cojoin}^{\mathcal{D}} &\equiv \text{runBatch}_{\mathcal{D}AX \circ \mathcal{D}XB} \text{doubleDec} \end{aligned}$$

Definition 6.3.2. (🚗) A coalgebraically presented decorated traversable functor is coalgebra of $\mathcal{D}$. Explicitly, it is a set-forming operation $F: \text{SET} \rightarrow \text{SET}$ equipped with an operation

$$\text{batchd}: \forall (A, B: \text{SET}), F A \rightarrow \mathcal{B}(E \times A) B (F B)$$

satisfying the following equations:

$$\text{extract}^{\mathcal{D}} \circ \text{batchd}^F = \text{id}_F \tag{6.59}$$

$$\text{cojoin}^{\mathcal{D}} \circ \text{batchd}^F = \text{map}^3 \left(\text{batchd}^F \right) \circ \text{batchd}^F \tag{6.60}$$

Lemma 6.3.3. (🚗) Every instance of Definition 5.4.7 gives rise to an instance of Definition 6.3.2 as follows:

$$\text{batchd}^F \equiv \text{mapdt}^F \text{ret}^B \tag{6.61}$$

□

Lemma 6.3.4. (🚗) Every instance of Definition 6.3.2 gives rise to an instance of Definition 5.4.7 as follows:

$$\text{mapdt}^F f \equiv \text{runBatch}_G f \circ \text{batchd}^F \tag{6.62}$$

□

These definitions witness an equivalence between Definition 5.4.7 and Def. 6.3.2. □

Definition 6.3.5. The following operation enumerates the contents of a decorated traversable functor, where each element is paired with its contextual value given by the decoration:

$$\begin{aligned} \text{contentsd}^F &: \forall (A: \text{SET}) (t: F A), (E \times A)^{\text{size } t} \\ \text{contentsd}^F &\equiv \text{contents}^B \circ \text{batchd}^F \end{aligned} \tag{6.63}$$

Theorem 6.3.6. (🔗) *Let F be a decorated traversable functor and consider any $f: E \times A \rightarrow G B$ where G is an applicative functor. Then the following holds:*

$$\text{mapdt}_G^F f t = \text{map}_G^G \left(\text{make}_F^F t \right) \left(\text{traverse}_G^{\text{Vec}(\text{size } t)} f \left(\text{contentsd}_F^F t \right) \right) \quad (6.64)$$

□

6.3.2 Coalgebraic Traversable Monads

The coalgebraic presentation of traversable monads has a somewhat circular aspect to it because we cannot define the comonad without first stipulating that T forms a coalgebra—though we cannot properly call it a “coalgebra” before showing that its laws give rise to a comonad. We previously saw a situation like this when defining the Z comonad in Section 4.6, where Z is defined in terms of what would later be shown to form a Z -coalgebra.

“Coalgebra” should be read somewhat loosely here—the structure considered is richer than a mere parameterized comonad. For example, it is also equipped with a ret operation. The formalization and investigation of coalgebraic (decorated) traversable monads is one of the frontiers of this work.

For any type constructor T , the following datatype generalizes $\mathcal{B}$ datatype to insert T into its second parameter.

$$\mathcal{B}^M A B C = \mathcal{B} A (T B) C$$

Definition 6.3.7. (🔗) *A coalgebraically presented traversable monad $T: \text{SET} \rightarrow \text{SET}$ is a set-forming operation paired with operations of the following types:*

$$\begin{aligned} \text{ret}^T &: \forall (A: \text{SET}), A \rightarrow T A \\ \text{batchm} &: \forall (A B: \text{SET}), T A \rightarrow \mathcal{B} A (T B) (T B) \end{aligned}$$

These must satisfy the following equations, where the operations extractm^B and cojoinm^B will be defined below in terms of batchm itself.

$$\text{batchm}^T \circ \text{ret}^T = \text{ret}^B \quad (6.65)$$

$$\text{extractm}^B \circ \text{batchm}^T = \text{id}_T \quad (6.66)$$

$$\text{cojoinm}^B \circ \text{batchm}^T = \text{map}^3 \left(\text{batchm}^T \right) \circ \text{batchm}^T \quad (6.67)$$

As mentioned, these equations make reference to two operations that we have not defined yet. Suppose we have operations ret^T and batchm . We begin by considering the following generalization of the double operation.

Definition 6.3.8. (🚗) *The following operation generalizes `double` to a traversable monad.*

$$\begin{aligned}\text{doublem} &: A \rightarrow \mathcal{B}^M A B \left(\mathcal{B}^M B C (TC) \right) \\ \text{doublem} &: A \rightarrow \mathcal{B} A (TB) (\mathcal{B} B (TC) (TC)) \\ \text{doublem} &\equiv \text{map}^3 \text{batchm} \circ \text{ret}^B\end{aligned}$$

Definition 6.3.9. (🚗) *We define the following operations:*

$$\begin{aligned}\text{extractm}^B &: \mathcal{B} X (TX) A \rightarrow A \\ \text{cojoinm}^B &: \mathcal{B} A (TB) C \rightarrow \mathcal{B} A (TX) (\mathcal{B} X (TB) C) \\ \text{extract}^B &\equiv \text{runBatch}_{\mathbb{1}} \text{ret}^T \\ \text{cojoinm}^B &\equiv \text{runBatch}_{\mathcal{B}A(TX) \circ \mathcal{B}(TX)C} \text{doublem}\end{aligned}$$

Lemma 6.3.10. (🚗) *Every instance of Definition 5.3.45 gives rise to an instance of Definition 6.3.7.*

$$\text{batchm}^T \equiv \text{bindt}_{\mathcal{B}A(TB)}^T \text{ret}^B \quad (6.68)$$

□

Lemma 6.3.11. (🚗) *Every instance of Definition 6.3.7 gives rise to an instance of Definition 5.3.45.*

$$\text{bindt}^T f \equiv \text{runBatch}_G f \circ \text{batchm}^T \quad (6.69)$$

□

Lemma 6.3.17 can be regarded as one way of presenting the representation theorem for traversable monads. In particular, we can use equation to rewriting all expressions like $\text{bindt}^T f t$ to an expressions involving $\text{batchm}^T t$, which offers a concrete datatype over which we may perform induction.

Theorem 6.3.12. *Definitions 5.3.45 and 6.3.7 are equivalent.* □

6.3.3 Coalgebraic Decorated Traversable Monads

The coalgebraic presentation of decorated traversable monads is essentially the combination of the preceding abstractions. We state the definitions and lemmas without elaboration. For any monoid W and type constructor T , the following datatype generalizes $\mathcal{B}$ datatype to annotate its first parameter with W and wrap its second parameter with T :

$$\mathcal{B}^{\text{DM}} A B C = \mathcal{B} (W \times A) (TB) C$$

Definition 6.3.13. (🎮) A coalgebraically presented *decorated traversable monad* $T: \text{SET} \rightarrow \text{SET}$ is a set-forming operation paired with operations of the following types:

$$\text{ret}^T: \forall (A: \text{SET}), A \rightarrow T A$$

$$\text{batchdm}: \forall (A, B: \text{SET}), T A \rightarrow \mathcal{B} (W \times A) (TB) (TB)$$

These must satisfy the following equations, where the operations extractdm^B and cojoindm^B will be defined below in terms of batchdm .

$$\text{batchdm}^T \circ \text{ret}^T = \text{ret}^B \circ \text{ret}^{W \times} \quad (6.70)$$

$$\text{extractdm}^B \circ \text{batchdm}^T = \text{id}_T \quad (6.71)$$

$$\text{cojoindm}^B \circ \text{batchdm}^T = \text{map}^3 \left(\text{batchdm}^T \right) \circ \text{batchdm}^T \quad (6.72)$$

Definition 6.3.14. (🎮) The following operation generalizes `double` to a decorated traversable monad. Here, $\text{incr } w$ is the operation that maps (w', b) to $(w \cdot w', b)$.

$$\text{doubledm}: A \rightarrow \mathcal{B} (W \times A) (TB) (\mathcal{B} (W \times B) (TC) (TC))$$

$$\text{doubledm} (w, a) \equiv \text{map}^3 \left(\text{map}^1 (\text{incr } w) \circ \text{batchdm} \right) \left(\text{ret}^B (w, a) \right)$$

Definition 6.3.15. (🎮) We define the following operations:

$$\text{extractdm}^B: \mathcal{B} (W \times X) (TX) A \rightarrow A$$

$$\text{cojoindm}^B: \mathcal{B} (W \times A) (TB) C \rightarrow \mathcal{B} (W \times A) (TX) (\mathcal{B} (W \times X) (TB) C)$$

$$\text{extractdm}^B \mathcal{B} \equiv \text{runBatch}_{\mathbb{I}} \left(\text{ret}^T \circ \text{extract}^{W \times} \right)$$

$$\text{cojoindm}^B \equiv \text{runBatch}_{\mathcal{B}A(TX) \circ \mathcal{B}(TX)C} \text{doublem}$$

Lemma 6.3.16. (🎮) Every instance of Definition 5.3.45 gives rise to an instance of Definition 6.3.7.

$$\text{batchdm}^T \equiv \text{binddt}_{\mathcal{B}(W \times A)(TB)}^T \text{ret}^B \quad (6.73)$$

□

Lemma 6.3.17. (🎮) Every instance of Definition 6.3.7 gives rise to an instance of Definition 5.3.45.

$$\text{binddt}^T \text{runBatch}_G f \circ \text{batchdm}^T \quad (6.74)$$

□

Following a pattern established in previous sections, the equation in Lemma 6.3.17 is highlighted as an invariant of DTMs, and not merely a definition specific to a particular presentation.

Theorem 6.3.18. *Every decorated traversable monad, regardless of how it is defined, satisfies the following equation:*

$$\text{binddt}^T \text{runBatch}_G f \circ \text{batchdm}^T \quad (6.75)$$

□ *This can equivalently be expressed as follows:*

$$\text{binddt}^F t = \text{map}^G \left(\text{join}^T \circ \text{make}^F t \right) \left(\text{traverse}^{\text{Vec}(\text{size } t)} f \left(\text{contentsd}^T t \right) \right) \quad (6.76)$$

Theorem 6.3.18 can be regarded as a representation theorem for decorated traversable monads. We can use equation to rewrite all expressions involving binddt_f^T to ones involving $\text{batchdm}^T t$, then perform induction over this datatype.

Theorem 6.3.19. *Definitions 5.3.45 and 6.3.7 are equivalent.* □

6.4 MUTUALLY RECURSIVE BINDERS

The variadic syntax considered Section 6.2 did not feature binders opened within the list of definitions. In this section, we formalize the two semantics that open binders within this list: the “telescoping” semantics and the mutually recursive semantics. All three semantics share an underlying inductive datatype of datatype of terms. Indeed, they have identical categorical structure except for the decoration, which varies to reflect their distinct binding policies. This results in different definitions of binddt , which are structurally identical except that they refer to different instances of a decorated-traversable structure on list.

6.4.1 Telescoping Semantics

The first possibility we consider is that each item in the list introduces a bound index within all subsequent items, so that later definitions can refer to earlier ones. To obtain this semantics, the context of the element at index i the list should be the length $i - 1$, or the length of the prefix of that element. This corresponds to the prefix decoration on list, which was first presented as Example 4.3.16.

Example 6.4.1. *The following definition captures a telescoping letin semantics on list.*

$$\begin{aligned} \text{mapdt}^{\text{tele}} f \text{ nil} &\equiv \text{pure}^G \text{ nil} \\ \text{mapdt}^{\text{tele}} f (\text{cons } a \, l) &\equiv \text{pure}^G \text{ cons } \circledast f(0, a) \circledast \text{mapdt}^{\text{tele}} (f \circ 1) l \end{aligned}$$

This definition states that the head element of the list has no binders opened, but all elements in the tail see one additional binder opened, corresponding to the head element. Recursively, an element at index i in l sees $i - 1$ binders opened.

Definition 6.4.2. *The following definition captures a telescoping letin semantics for the lambda calculus extended with a variadic letin constructor:*

$$\begin{aligned} \text{binddt}_G^{\text{lam}} f (\text{letin } l \ t) \quad \equiv \quad & \text{pure}^G \text{letin} \circledast \left(\text{mapdt}^{\text{tele}} \left((i, u) \mapsto \left(\text{binddt}_G^{\text{lam}} (f \odot i) \right) u \right) \right) \\ & \circledast \left(\text{binddt}_G^{\text{lam}} (f \odot \text{length } l) \ t \right) \end{aligned}$$

In this definition, substitute inside the element u in l with $\text{binddt } f$ as usual, except we also examine its context, the index i , and pass this information to f using $(\odot)$. This definition would be *identitcal* to Example 6.2.1 if, rather than the index decoration, we considered the constant decoration (Ex. 4.3.15) by 0, in which case i would be always be zero and the call to mapdt over the list would reduce to $\text{traverse}^{\text{list}}$.

To prove that this syntax forms a DTM, we can apply essentially the same trick used in Section 6.2 except that, rather than invoking the representation theorem for traversable functors, we invoke the representation theorem for decorated-traversable functors (Thm. 6.3.6).

6.4.2 Mutually Recursive Semantics

The final possibility is that all items in l are within the scope of all other items. The results in the constructor often known as letrec . To obtain this semantics, each element in l must see $\text{length } l$ binders in scope, which corresponds to the length decoration originally presented in Example 4.3.17. This is captured in the following definition.

Example 6.4.3. *The following definition captures a letrec semantics for list.*

$$\text{mapdt}^{\text{rec}} f \ l \quad \equiv \quad \text{traverse}^{\text{list}} (\lambda a. f (\text{length } l, a)) \ l$$

We could also have defined this operation by structural recursion on the list, as in Example 6.4.1, but the above definition is slightly more compositional by reusing the definition $\text{traverse}^{\text{list}}$. We now lift this function over terms with the following definition, which merely swaps out $\text{mapdt}^{\text{tele}}$ for $\text{mapdt}^{\text{rec}}$.

Definition 6.4.4. *The following definition captures a recursive letin semantics for the lambda calculus extended with a variadic letin constructor:*

$$\begin{aligned} \text{binddt}_G^{\text{lam}} f (\text{letin } l \ t) \quad \equiv \quad & \text{pure}^G \text{letin} \circledast \left(\text{mapdt}^{\text{rec}} \left((i, u) \mapsto \left(\text{binddt}_G^{\text{lam}} (f \odot i) \right) u \right) \right) \\ & \circledast \left(\text{binddt}_G^{\text{lam}} (f \odot \text{length } l) \ t \right) \end{aligned}$$

6.5 ALPHA EQUIVALENCE

We can now finally prove that the translation in Section 5.6.2 between the nominal and locally nameless representations is correct up to alpha equivalence. First, we must define α -equivalence following the same strategy as always: we consider a localized operation first, then lift this operation over terms. Recall that an occurrence v in context l is free precisely when $v \notin l$. Also recall that if $v \in l$, the predicate $\text{bound } l \mid l_{\text{pre}} v l_{\text{post}}$ holds for a unique choice of l_{pre} and l_{post} and means that $l = l_{\text{pre}} ++ [v] ++ l_{\text{post}}$ and $v \notin l_{\text{post}}$.

Definition 6.5.1. *In context l , we say v resolves to index n if $\text{bound } l \mid l_{\text{pre}} v l_{\text{post}}$ holds for some l_{pre} and l_{post} and n is equal to $\text{length } l_{\text{post}} - 1$.*

Effectively, Definition 6.5.1 translates nominal variables to de Bruijn indices. This is not the only way to define α -equivalence, but it works. We now define that equivalence.

Definition 6.5.2. *The localized α -equivalence relation has the following type:*

$$\alpha_{\text{loc}} : (\text{list atom} \times \text{atom}) \times (\text{list atom} \times \text{atom}) \rightarrow \mathbf{Prop}$$

$$\alpha_{\text{loc}}(l_1, v_1)(l_2, v_2) = \begin{cases} \text{True} & \text{if } v_1 \notin l_1, v_2 \notin l_2, \text{ and } v_1 = v_2 \\ \text{True} & \text{if } v_1 \in l_1, v_2 \in l_2, \text{ and } v_1 \text{ and } v_2 \text{ resolve to the same } n \\ \text{False} & \text{else} \end{cases} \quad (6.77)$$

This choice to use “resolves to” in this definition is somewhat arbitrary. For instance, we could have measured the size of l_{pre} in Definition 6.5.1 and required this value to be the same for v_1 and v_2 , which would effectively be the same as translating named variables to de Bruijn levels (de Bruijn indices that index from the head of the list instead of the tail). Additionally, it would have been quite sensible to require l_1 and l_2 to be the same size in the definition of α_{loc} , because a priori these lists have no relationship to each other. However, this does not add anything: we only use α_{loc} to define α -equivalence, and this relation inherently requires the terms to have the same shape. In this case, the lists will necessarily have the same length, so there is no need to check this property in α_{loc} .

To lift this kind of context-sensitive relation over terms, we need a version of lift that incorporates the decoration of a decorated traversable functor. We give this definition now.

Definition 6.5.3. () *Let F be a decorated (by E) traversable functor and let*

$$R : E \times A \rightarrow E \times B \rightarrow \mathbf{Prop}$$

be a binary relation. The decorated lifting function is defined as follows:

$$\begin{aligned} \text{lift}^F R &: F A \rightarrow F B \rightarrow \mathbf{Prop} \\ \text{lift}^F R t u &\equiv \text{lift}^F R \left(\text{dec}^F t \right) \left(\text{dec}^F u \right) \end{aligned} \quad (6.78)$$

Note that this is equivalent to $\text{mapd}^F_{\text{subset}} R t \left(\text{dec}^F u \right)$ but the definition given is more symmetrical in its treatment of t and u .

To reason about propositions such as lift^F , with or without incorporating dec^F , the following lemma will be useful:

Lemma 6.5.4. *For all relations R , we have the following equivalences*

$$\text{lift} R t t \iff \text{ForAll } (a \mapsto R a a) t \quad (6.79)$$

$$\text{lift}^F R t t \iff \text{ForAllDec } ((e, a) \mapsto R (e, a) (e, a)) t \quad (6.80)$$

That is, lifted relations relate a term to itself precisely when it relates all elements to themselves.

Proof. Follows from Theorem 6.1.42. □

Note that in this lemma, R does not have to be reflexive in general. However, it is easy to see that $\text{lift} R$ inherits reflexivity, symmetry, and transitivity from R .

6.5.1 Reasoning About Relations

To solve goals of the form $\text{lift}^F R t u$, one strategy is to reduce them to goals of the form $\text{lift}^F R' t t$ for some R' and then invoke Lemma 6.5.4. Such an approach, which uses the naturality properties of lift (Corollary 6.1.43), works when t and u are formed by applying different functions to the same underlying term. Precisely this strategy will be used to show that the translation between nominal and locally nameless syntax is correct up to α -equivalence. In fact, this is one way of understanding properties such as Lemma 5.4.12, whose statement we recall:

$$\forall (e : E) (a : A), (e, a) \in_d t \implies f(e, a) = g(e, a) \iff \text{mapd}^F f t = \text{mapd}^F g t \quad (6.81)$$

Let Eq be the curried equality relation, in other words let $\text{Eq } t u$ be defined $t = u$. It is easy to show the following:

$$t = u \iff \text{lift Eq } t u$$

This holds because Eq is precisely $\text{ret}^{\text{subset}}$ for the subset monad (Ex. 4.2.6). This is also equal to $\text{pure}^{\text{subset}}$ the subset applicative (Ex. 5.3.14). It is straightforward to show that pure^G is an

applicative homomorphism from the identity functor to G for any applicative. By (5.102), we have

$$\text{traverse}^F \text{pure}^G t = \text{pure}^G t$$

for any G and t . As a result, we have the following:

$$\text{lift Eq } t u \iff \text{traverse}^F \text{pure}^{\text{subset}} t u \iff \text{pure}^{\text{subset}} t u \iff t = u$$

Therefore, the RHS of (6.81) is equivalent to the following:

$$\text{ForAllDec Eq } (\text{mapd } f \, t, \text{mapd } g \, t)$$

On the other hand, by Lemma 5.1.33, the LHS of (6.81) is equivalent to the following:

$$\text{ForAllDec } ((e, a) \mapsto \text{Eq } (f(e, a)) (g(e, a))) \, t \, u$$

By expressing mapd as map composed with dec , the equivalence of the two expressions above follows from Corollary 6.1.43, which allows moving map from the arguments of a lifted relation into the relation itself.

6.5.2 Correctness of the Translation

To, verify that the nominal/locally nameless translation is correct, it suffices to verify its correctness locally by the reasoning shown above.

Theorem 6.5.5. *If we use the translation in Section 5.6.2 to convert α -equivalent terms to locally nameless terms, then back again, the result is α -equivalent to the starting term.*

Proof. Both directions of the translation clearly leave free variables alone. Thus, it suffices to reason about what happens to a bound variable. We consider an arbitrary bound occurrence in some context, and show that after translating in both directions, we end up with a new context and variable that resolves to the same location in the context.

Let $(l, v) \in_d t$ be an occurrence in t and suppose $\text{bound } l \, l_{\text{pre}} \, v \, l_{\text{post}}$. When t is translated to a locally nameless term, the resulting context is a list of units with the same length as l . Denote this context by l_u . For its part, v is mapped to the de Bruijn index $\text{toLN}_{\text{loc}}(l, v)$, which is $\text{Bd}(\text{length } l_{\text{post}})$. After translating this term back to a nominal representation, the context l_u is mapped to $\text{hmap historyToName } l_u$. Because we know $\text{length } l_{\text{post}} < \text{length } l_u$, $\text{Bd}(\text{length } l_{\text{post}})$ is mapped to

$$\text{bdToName} \Gamma (\text{replicate } * (\text{length } l - (\text{length } l_{\text{post}} + 1)))$$

where Γ is the set of free variables, which do not play a role here. By arithmetic, the expression

above is precisely

$$\text{bdToName } \Gamma(l', *)$$

where l' is a list of units with the same length as l_{pre} . Note that this is the same name as that assigned by bdToName to the abstraction to which the original v was bound. Thus, this variable *would* continue to refer to the same location its context, *if* we can guarantee that the translated context corresponding to l_{post} does not include this same name. But this follows from the decomposition given in Lemma 4.6.32, which shows that each element of the list is unique. Thus, this new occurrence variable satisfies bound $l' l'_{\text{pre}} l'_{\text{post}}$ for lists of the same size as the original values of l , l_{pre} , and l_{post} . Therefore it resolves to the same value n . In conclusion, the translation respects local alpha equivalence, so by extension it respects α -equivalence. $\square$

EVALUATION

In this chapter we present the implementation and evaluation of Tealeaves, the formal Roqc library that underlies this work. Tealeaves was designed with two main goals:

1. To formalize the category-theoretic foundation of decorated traversable monads (DTMs)
2. To provide a practical, reusable library for reasoning about syntax in Coq formalizations

We refer to the parts of Tealeaves focus on general theory as its *core*. The core defines key category-theoretic typeclasses such as DTMs, formalizes their properties and relationships (e.g. Corollary 5.1.43, Lemma 5.4.24), and proves the equivalence of the three presentations of DTMs. Although the core accounts for approximately 90% of the codebase by lines of code (LOC), this chapter focuses on the second goal: to serve as a practical metatheory library.

The remaining parts 10% of the codebase resides in a set of *backend* modules that implement concrete representations of variable binding. These currently include the nominal, locally nameless, and de Bruijn representations, with future extensions anticipated. Each backend defines operations and predicates such as substitution, free-variable enumeration, or (local) closure. Backends are implemented parametrically over an abstract instance T of the DTM typeclass, with the backend specifying the variable representation and the monoid with respect to which T is decorated. The properties of these operations are derived from the DTM laws, usually from high-level theorems established in the core rather than the DTM axioms directly.

We evaluate the practicality of the backends with respect to the following criteria:

1. What syntactic operations can be formalized and what properties can be proved?
2. What classes of syntax are supported by this framework?
3. How much user effort is required to instantiate Tealeaves with a particular syntax?

Each of these criteria will be looked at in turn.

7.1 EXPRESSIVENESS OF BACKENDS

In this section we examine the range of syntactic operations and properties that can be defined generically over an arbitrary DTM. As a baseline, we have sought to replicate the functionality of two existing Roqc libraries: Autosubst [64], which implements the confluent rewriting strategy for de Bruijn indices developed by Schäfer et al. [63]; and LNgen [9], which implements the locally nameless approach advocated by Aydemir et al. [10]. While we defer a fuller discussion of these

tools to Chapter 8, we use them here as benchmarks for evaluating the expressiveness of the DTM abstraction.

Note that there is also a second-generation version of Autosubst, Autosubst 2 [69]. Our evaluation focuses on the original Autosubst, which remains a widely used reference point.

de Bruijn Backend

Our de Bruijn indices backend (🐘) is, in essence, a faithful reimplementation of Autosubst. As shown in Section 4.4, the equational axioms used to instantiate Autosubst are derivable from the laws of a monad decorated by the additive monoid $\langle \mathbb{N}, +, 0 \rangle$. This choice of monoid reflects the key structural property that to resolve de Bruijn indices, one must know the number of abstractions in scope at each occurrence—a value that increases additively as we recurse under binders.

To give a sense of how a backend is implemented, to a first approximation the de Bruijn indices backend begins with the following invocation after importing the relevant modules from Tealeaves’ core:

```
Section de_bruijn_theory.
  Context {DecoratedTraversableMonad nat T}.
  Definition inst (sub: nat * nat -> T nat): T nat -> T nat := bindd ...
  ...
End de_bruijn_theory.
```

Inside the `de_bruijn_theory` section, we assume an arbitrary DTM called `T`, which is decorated by the type `nat` of natural numbers. The combinator `bindd` and friends are used to define functions over values of type `T nat`, corresponding to syntax trees with `nat`-valued leaves. Once this section is closed, functions such as `inst` will be polymorphic over any instance of `DecoratedTraversableMonad nat`.

The code shown above is slightly simplified. The actual setup we must specify that the monoid assumed of `nat` is given by addition. We work over an instance of `DecoratedTraversableRightModule` instead of a plain `DecoratedTraversableMonad` (see Section 5.4.6). Additionally, we explicitly specify that operations such as `bindd` must be accompanied by appropriate compatibility typeclass instances; as a result, the backend does not assume that `bindd` is actually defined in terms of `binddt`, only that the two operations are related appropriately.

Tealeaves’ de Bruijn indices backend is strictly more expressive than Autosubst due to the additional structure provided by traversability. For example, the backend can define operations in terms of `mapdReduce` for an arbitrary monoid M :

$$\text{mapdReduce}_M^T: (\mathbb{N} \times \mathbb{N} \rightarrow M) \rightarrow T \mathbb{N} \rightarrow M$$

For example, with this operation we define the property of being *closed* by taking M to be the monoid of propositions under conjunction. We can also measure how far away a term is from being closed, so to speak, by specializing M to the lattice of natural numbers under max and applying `mapdReduce` to the operation $(d, n) \mapsto n + 1 - d$. (This specific function was implemented for the locally nameless backend (🚗), but the idea is the same.)

Simplification

There are two main caveats to using Tealeaves in place of Autosubst. The first concerns simplification. As discussed in Section 3.2, Tealeaves defines operations in terms of the recursion combinator `binddt`, whereas Autosubst implements them by structural recursion on syntax. As a result, using standard simplification tactics like `simpl` on Tealeaves-defined operations reveals internal details of `binddt`, which is generally undesirable. For example, simplifying the de Bruijn substitution operation reveals an expression involving `pure`¹ and `⊗`¹, operations from the applicative structure of the identity functor (Ex. 5.3.18)—a consequence of the fact that `bindd` is defined as a special case of `binddt` (Lem. 5.4.23).

To address this, Tealeaves provides a tactic called `simplify_binddt` (🚗) which performed targeted unfolding of `binddt`-based operations. Building on this low-level primitive, the de Bruijn backend implements a tactic called `simplify_db` (🚗), which mimics the Autosubst tactic `asimpl` by following a confluent rewriting strategy [63] to normalize expressions involving de Bruijn operations.

Instantiation

The second caveat of Tealeaves is that it requires a more elaborate instantiation process than Autosubst, as will be discussed below. The difference is not necessarily dramatic, as both frameworks require the user to prove four equations (except for nominal substitution, see below). In the case of Autosubst, the four equations are (2.10) and (2.12)–(2.14). For Tealeaves, these are the axioms of DTMs ((5.129)–(5.132)).

7.1.1 Locally Nameless Backend

Our locally nameless backend (🚗) is, in essence, a reimplementaion of LNgen. Its structure parallels that of the de Bruijn indices backend: we assume an arbitrary DTM $\mathbf{T}$ decorated by $\langle \mathbb{N}, +, 0 \rangle$, except that the type parameter of $\mathbf{T}$ is specialized to the type `LN` of locally nameless variables, a disjoint union of de Bruijn indices and atoms.

In Section 2.3, Figure 2.1, we listed nine representative lemmas drawn from Figure 4 of the technical report by Aydemir and Weirich [9], along with references to our corresponding proofs for an arbitrary DTM. In fact, our backend formalizes all 20 of lemmas highlighted in that report,

omitting only three lemmas (e.g. `lc-unique`) particular to their implementation of local closure.

A notable aspect of locally nameless is that, unlike the properties proved by Autosubst, many of the fundamental properties are not stated as unconditional equations but assume side conditions (e.g. variable freshness) or state non-equational relationships like subset inclusions. Nonetheless, we have shown that all of these results can be derived from the unconditional equational axioms of DTMs. This is made possible by the traversability of DTMs: the axioms of traversable functors are quantified over an arbitrary applicative functor, giving them a second order character (c.f. [43]). By instantiating these laws to concrete applicatives such as `list` or $\mathcal{B}$ (Sec. 6.1.1), we can reason about concrete data structures and carry out inductive proofs using the sorts of techniques presented in Chapter 6.

7.1.2 Translations

Because our de Bruijn and locally nameless backends share a common abstraction, differing only in the type parameter of `T`, we are able to formally relate the two representations in a way that standalone syntax utilities cannot. Interestingly, for reasons explained in Section 5.6.1, translating between de Bruijn indices and locally nameless is inherently a partial function in a setting where syntax is not intrinsically scoped, such as ours. Far from being a flaw, for our intended applications, this is a feature. Consider, for example, that a real-world compiler does not know a priori that syntax written by a programmer only mentions variables drawn from a particular subset. Instead, it must validate this sort of property or otherwise throw an error.

Once again, the key enabler of this expressiveness is the traversability of the DTM. By instantiating the traversal to the `option` monad, we obtain a notion of partial function that can be reasoned about in a principled way. This applicative could easily be replaced with a more informative one, e.g. an error monad tracking information about *which* occurrence caused an error, reflecting the generality of the DTM abstraction.

7.1.3 Nominal Substitution

Tealeaves is, to our knowledge, the first syntax metatheory library that provides a generic implementation of *nominal* capture-avoiding substitution, the way it is defined in textbooks about the lambda calculus (e.g. [30]). Supporting this form of substitution required extending the DTM abstraction to support operations that manipulate binders in addition to variables. As exhibited in Section 4.6 and Section 5.7, this extended abstraction is nontrivial and remains the subject of ongoing investigation and formalization.

The current implementation of binder freshening, described in Section 4.6, is far from optimal from a runtime perspective. The primary issue is that the same computations must be performed repeatedly at each binder occurrence to compute the set of names to avoid during renaming. In

a real-world implementation, this value would be maintained in the form of *state* that is updated at each site, rather than computed from scratch at each site. This inefficiency reflects the localized nature of reasoning with DTMs: all operations are defined, and proofs reasoned, in terms of individual binder and variable occurrences in isolation. Thus, at each binder, we must use the binding context to reconstruct the state of the avoid set.

In future work, it may be possible to give a more natural implementation of binder freshening by, for example, threading the state monad through the computation. We observed in Section 5.7 that syntax is only decorated-traversable in the binder parameter if the applicative is commutative-idempotent, so whether this sort of stateful operation can be faithfully reproduced in the DTM framework is not yet clear.

7.2 EXPRESSIVENESS OF DTMS

We now turn our attention to the question of which classes of syntax can be used with DTMs. We have already discussed the fact that DTMs, with the appropriate extension, can represent syntax with named binders. The other innovation of this work is to support syntax with binders that binder a variable number of definitions simultaneously, possibly mutually recursively, as demonstrated in Section 6.2 and Section 6.4. We gave three examples corresponding to different semantics for a *letin* constructor binding a list of variables in a body: a “simple” semantics where definitions are not mutually visible to each other, a telescoping semantics where each definition is visible to all subsequent entries, and a recursive semantics where every definition is visible to every other definition, including itself.

Note that we have not implemented an example to fully demonstrate and exploit this expressiveness at the level of a complete language formalization. To give a semantics to a syntax with variadic syntax, one would like a substitution operation that can, for example, substitute several entries at once. Our locally nameless backend currently implements substitution of one variable at a time. Our de Bruijn indices backend defines a parallel substitution to replace several variables at once, but at this time we have not used this to reason about substitution used to give semantics to mutually recursive binding constructs; at this time we have only validated the non-obvious fact that such constructs form a legitimate DTM. In Chapter 3, Table 3.5, we demonstrated the behavior of the level operation on examples of a variadic syntax, which has the expected behavior. We defer a fuller exploration of variadic binding to future work.

We have not formalized an example of a nominal syntax with variadic binding. The question of how to adapt the techniques of Sections 6.2 and 6.4 to prove the DTM instance for a syntax with nominal variadic binding remains unexplored.

7.3 COST OF INSTANTIATION

We now consider the relative difficulty of proving the DTM typeclass instance. We focus on ordinary, not parameterized, DTMs—the process of instantiating the parameterized typeclasses for use with nominal syntax has not been extensively studied. The process of instantiating the ordinary typeclasses, both the traditionally category-theoretic (Definition 5.4.17) and extension system (Definition 4.3.46) presentations, has received more attention.

Tealeaves provides a tactic, `derive_dtm`, that applies a rewriting strategy discussed in Appendix A to prove the DTM axioms for Definition 5.4.19. This tactic is analogous to the `derive` tactic used by Autosubst. The `derive_dtm` tactic is based on heuristics based on a general expectation of how `binddt` is defined. In the case of the lambda calculus, this tactic is entirely sufficient to prove the full set of laws. For other kinds of syntax, such as syntax with variadic and mutually recursive binders, the tactic does not fully succeed and user must write at least part of the proof by hand. There are generally two reasons, often coinciding, why manual intervention is required from the user:

1. The syntax is a nested inductive datatype, requiring a custom induction principle manually defined by the user
2. The binding policy is not constant (e.g. lambda abstraction that always binds a single variable in its body) but performs computation (e.g. computing the length of a list of definitions) and it is genuinely non-obvious, at least at the level of formal proof, that the composition law is satisfied

In the cases considered thus far, the difficulty is concentrated in a small number of pivotal steps. In the variadic examples considered in Section 6.2 and Section 6.4, the pivotal step involves invoking the representation theorem for traversable or decorated-traversable functors to isolate the make function of a substructure (the list of definitions) from its contents. The make function, being pure, is composed with the constructor to act as a witness that the shape of the substructure does not change. Finally, the user manually shows that the decoration only depends on the shape of the substructure, not its contents, which pushes the proof through. By combining these key steps with a judicious use of lower-level tactics—the building blocks used to implement `derive_dtm`—the proofs of the variadic instances require fewer than 50 lines.

The greatest practical challenge of instantiating the DTM typeclass is not per se the “real” difficulty of proving the axioms, once the basic technique is understood, but rather than abstract category-theoretic nature of the laws and the domain-specific knowledge required. Tealeaves mostly shields users from the deep categorical abstractions, since Tealeaves users presumably more interested in using the syntactic lemmas exported by backends, rather than directly reasoning about DTMs, which is the responsibility of the backends themselves. Even the development of

the backends does not typically required deep categorical knowledge, as Tealeaves’ core exports high-level properties such as Corollary 5.1.43 whose statement does not self-evidently require any particular experience with category theory. Consider that the development of locally nameless metatheory in Section 5.2 and Section 5.5 did not directly involve category-theoretic reasoning. Indeed, I believe the localized reasoning exhibited in these proofs is very close to what most programming language metatheorists would offer if asked to give prove the same syntactic theorems on a blackboard.

The core abstractions of this work—monads, decorated functors (closely related to comonads), and traversable functors—are more or less familiar to Haskell programmers, and therefore I expect the degree of abstraction is not prohibitively high for a wide class of users. Nonetheless, a detailed investigation of the DTM laws from the point of view of fully automating the DTM axioms would be highly useful. Fortunately, this work has shown that DTMs are highly *compositional*. Any decorated traversable signature endofunctor gives rise to a freely generated monad automatically (Theorem 5.4.18), while decorated-traversable functors are closed under products, coproducts, and other basic constructions. Therefore, I am optimistic that the DTM laws can be fully automated for a wide class of syntactic datatypes.

7.4 LIMITATIONS OF THE CURRENT FRAMEWORK

Because traversable functors are equivalent to finitary containers in $\mathbf{SET}$, this work is limited to finitary syntax, at least in $\mathbf{SET}$ -like categories. For instance, any DTM supports a tolist operation that extracts a finite list of elements. In future work, it may be profitable to modify this notion of traversability with a more general notion that supports infinitary traversals [13].

Because all of the underlying category theoretic axioms are stated up to propositional equality in Coq, Tealeaves as it stands does not support syntax where the underlying set of terms is subject to a quotient relation, such a process calculus whose syntax features constructors assumed to be commutative at the level of the syntax itself. Such limitations might be removed by weakening the assumptions made about the distributive law—we have already shown in Section 5.7 in some instances for nominal syntax, the applicative functor must be commutative and idempotent in order to obtain the coherence law.

RELATED WORK

The literature on representations of syntax is vast. We group existing approaches, somewhat arbitrarily, into several broad categories. Each section below discusses connections between our work and prior and ongoing research, with an emphasis on work that aligns closely with ours, beginning with those that emphasize the monadic nature of syntax. In particular, intrinsically-scoped and presheaf-theoretic approaches are nearly a point of contact between related work and decorated traversable monads (DTMs). We present partial results relating these approaches to DTMs in Chapter 9. Readers interested in bridging our results to that of others should consult that chapter for suggestions on where to start.

8.1 BINDING, SUBSTITUTION, AND RECURSION

John Power [62] has made a distinction between a theory of *variable binding* and a theory of (capture-avoiding) *substitution*. In that sense, the present work is about neither. Rather, I have developed the theory of a specific *recursion principle*, the *binddt* combinator, underlying the kinds of datatypes used to define syntactic structures inductively. As the representation theorems in Chapter 6 indicate, ours is the theory of operations that manipulate the concrete data in container-like datatypes, including their concretely-defined contexts, but otherwise do not scrutinize the grammatical shape of their inputs. This recursion principle can be used to define and reason about operations up to and including capture-avoiding nominal substitution, but these operations are not correct-by-design; their correctness is established externally, such as by relating them to other operations.

It may be fairly said that this work is also not a theory of syntax, per se. For instance, we have not defined a notion of *binding signature*—an extended notion of algebraic signature incorporating information about which constructors are binders in which arguments. The analogous notion here is that of a decoration of type $\Sigma \Rightarrow \Sigma \circ W^\times$ on the signature functor, where W is an arbitrary monoid. Except for satisfying the laws of decorated traversable functors, this natural transformation is entirely arbitrary. This provides the freedom to define operations that bind a dynamic number of arguments as in Sections 6.2 and 6.4.

With a few exceptions (namely Theorem 5.4.18 and its dependencies), the theory contained in Chapters 4–6 makes no assumptions about how T is presented, e.g. that T is a coproduct of products or freely generated. For this reason, DTMs may be useful for applications other than syntax metatheory. As noted in Section 7.2, the assumption of traversability limits this work to finitary syntax, at least in SET-like categories. Weakening the assumptions made about the distributive law over applicative functors, such as by restricting to a specific class of them (e.g. the

commutative idempotent ones), or adopting ideas about infinitary traversals [13], may be used to lift this restriction.

8.2 MONADIC SYNTAX

8.2.1 Monadic de Bruijn Indices

Bellegarde and Hook [14] were apparently the first to “define substitution once and use the monadic structure of the definition to instantiate it on different abstract syntax structures.” Specifically, [14] provides a generic implementation of substitution for de Bruijn indices using a category called FUNSEQ, defined below. Although they did not provide a set of equations to *prove theorems* about their abstraction, it can be recognized as a special case of ours: namely, a monad decorated by the monoid of natural numbers under addition.

Definition 8.2.1. *The category FUNSEQ has sets as objects, where morphisms of type $A \rightarrow B$ are infinite sequences of functions $(f_0, f_1, \dots)$ where each f_i is an ordinary function $A \rightarrow B$. Composition of morphisms is pointwise:*

$$(g_i)_{i \in \mathbb{N}} \circ (f_i)_{i \in \mathbb{N}} = (g_i \circ f_i)_{i \in \mathbb{N}}$$

Identities are constant sequences of the identity function.

Lemma 8.2.2. *FUNSEQ is equivalent to the category of co-Kleisli arrows of the $\mathbb{N}^\times$ comonad (Thm. 4.3.12).*

Proof. An infinite sequence of functions of type $A \rightarrow B$ is equivalent to a single function of type $\mathbb{N} \times A \rightarrow B$. Composition in FUNSEQ is precisely co-Kleisli composition:

$$g \star f \equiv g \circ \text{cobind}^{\mathbb{N}^\times} f \equiv \lambda n a. (g(n, (f(n, a))))$$

Infinite sequences of the identity function correspond to $\text{extract}^{\mathbb{N}}$. The map operation defined in [14] for the lambda calculus as an endofunctor on FUNSEQ corresponds precisely to our map^{lam} . $\square$

The authors of [14] define a combinator called “extend with policy” (in our terminology, “bind with policy”) as follows:

$$\text{bwp}_{A,B}^T : ((A \rightarrow TB) \rightarrow (A \rightarrow TB)) \rightarrow (A \rightarrow TB) \rightarrow TA \rightarrow TB$$

$$\begin{aligned} \text{bwp } P f \text{ (var } x) &\equiv f x \\ \text{bwp } P f \text{ (abs } t) &\equiv \text{abs } (\text{bwp } P (P f) t) \\ \text{bwp } P f \text{ (app } t_1 t_2) &\equiv \text{app } (\text{bwp } P f t_1) (\text{bwp } P f t_2) \end{aligned}$$

The “map with policy” operation mwp has a similar definition. In these operations, P is a *policy function* used to update the argument to bwp when passing under a binder. Their de Bruijn operations are defined as follows:

$$\text{rename } \rho \equiv \text{mwp } (\uparrow_{\text{ren}}) \rho \quad (8.1)$$

$$\text{inst } \sigma \equiv \text{bwp } (\uparrow) \sigma \quad (8.2)$$

The “with policy” operations have a simple relation to our mapd and bindd :

$$\text{mwp } P f = \text{mapd } (\lambda(d, a). P^d f a) \quad (8.3)$$

$$\text{bwp } P f = \text{bindd } (\lambda(d, a). P^d f a) \quad (8.4)$$

This is precisely how we defined the de Bruijn operations in Section 4.4, except that we considered more transparent specifications of $(\uparrow_{\text{ren}}^d)$ and $(\uparrow^d)$ (Lems. 4.4.5 and 4.4.7). We can also recover the mapd/bindd operations from the “with policy” operations, albeit in somewhat tortuous fashion:

$$\text{mapd } f = \text{mwp } (\lambda f. f \odot 1) f \circ \text{mwp id } (\lambda a. (0, a)) \quad (8.5)$$

$$\text{bindd } f = \text{bwp } (\lambda f. f \odot 1) f \circ \text{mwp id } (\lambda a. (0, a)) \quad (8.6)$$

I suspect many uses of “policy functions” to modify an argument during recursion can be seen as an implicit manifestation of decorated functors; bringing out this structure shifts the perspective to a more *localized* one, where all of the interesting computation and reasoning takes place at the base case of recursion (e.g. the var constructor of lam).

The view taken by [14] of lam as a functor of type $\text{FUNSEQ} \rightarrow \text{FUNSEQ}$ is matched by our view of lam as an ordinary functor of type $\text{SET} \rightarrow \text{SET}$ that is additionally equipped with a suitable co-action of the environment comonad. However, the authors did consider an isolated dec operation or an axiomatization of mwp/bwp to support generic reasoning about these operations. Therefore, Equations (4.98)–(4.100) and the monad-decoration coherence laws (Equations (4.89) and (4.90)) do not make an appearance. Instead, the majority of [14] discusses how their operations might be automatically processed into more efficient definitions that “exploit the ‘algebras’ implemented in computer hardware.” In future work, the same consideration may be given to the operations defined in this work—the nominal substitution defined in Section 4.6 is particularly inefficient and may greatly benefit from such an analysis.

The authors of [14] did not consider non-substitution such as level or $\mathbf{cl}$ (discussed in Sec. 3.2.1) that treat syntax as a container of variables. Thus, they did not consider the sort of theory developed in Chapters 5 and 6, accurately noting “There is [...] no need to calculate sets of free and bound variables when performing substitution.” More broadly, the literature’s emphasis on de Bruijn indices, and the non-essential nature of aggregation to define de Bruijn substitution, might

```

Inductive lam (V : Type) :=
| var : V -> lam V
| abs : lam (option V) -> lam V
| app : lam V -> lam V -> lam V.

```

Figure 8.1: Intrinsically scoped definition of lam

explain why discussion of aggregation-like operations in the generic syntax metatheory literature is scant. A contribution of this work is to highlight this useful structure and demonstrate the ease with which it is incorporated into the bigger picture, provided the coherence law (5.118) is satisfied.

8.2.2 Intrinsically Scoped Monads

Subsequent work [16, 5] has emphasized well-scoped de Bruijn indices using nested datatypes [15]. The syntax considered in this line of work is isomorphic to the definition shown in Figure 8.1. In this syntax, V is a base set of *free* variables. In the *abs* case, the type parameter changes from V to $\text{option } V$, or $\mathbf{1} + V$, where the newly introduced element corresponds to the variable bound to the abstraction. Therefore, V can be specialized to provide more precise information about the occurrences in a term. For example, a term of type $\text{lam } \emptyset$, where $\emptyset$ is the empty set, corresponds to a term without free variables.

The benefit of a well-scoped representation is clear: using types to reflect static information about a term or program is the *raison d'être* of types in the first place. However, this approach is antithetical to our particular goal to reason about raw abstract syntax trees. For example, if an AST is the result of parsing externally-provided source code, then there can be no static guarantees about the occurrences in the tree. From our perspective, a well-scoped syntax tree might instead be the final output of a processing phase that walks over a raw syntax tree to produce either a well-scoped output or an error message for the user. We consider exactly this sort of application in Chapter 9 with a version of *binddt* modified to work with well-scoped syntax.

8.3 WELL TYPED SYNTAX

McBride [54] considered well-scoped, well-typed renamings and substitution for the simply-typed lambda calculus (note that the sense of “traversal” in that work is not the same as a distributive law over applicative functors [55]). Subsequent work [3, 4] extended the approach further, to include type- and scope-safe semantic operations. Our work does not give any consideration to semantics; there is no particular reason to think any semantic domain would retain the structure of a DTM. Instead, our work has focused entirely on syntactic operations and conversion between

different variable binding representations, fully embracing the sorts of low-level implementation details that so much related work abstracts (for good reason).

8.4 PRESHEAF THEORETIC SYNTAX

The presheaf-theoretic model of syntax developed by Fiore, Plotkin, and Turi [31] represents the collection of terms not as a flat set but as a (covariant) *presheaf*. Their work considers a category $\mathbb{F}$, a skeleton of the category of finite sets and functions. That is, this category has one finite set $\bar{n} = \{0, 1, \dots, n-1\}$ for each natural number $n \in \mathbb{N}$ and includes all functions $\bar{n} \rightarrow \bar{m}$ as morphisms. Here, the set $\bar{n}$ represents a generic variable binding context with n elements. A presheaf $A \in \text{SET}^{\mathbb{F}}$ is simply a functor from $\mathbb{F}$ to SET ; that is, a $\mathbb{N}$ -indexed family of sets such that each $\rho: \bar{n} \rightarrow \bar{m}$ induces a map $A(\rho): A(n) \rightarrow A(m)$ satisfying the functor laws.

One presheaf considered by [31] is the family Λ_l where $\Lambda_l(n)$ consists of all de Bruijn lambda terms that are closed when placed into a context with n free variables (their paper focuses on de Bruijn *levels* rather than indices, but the difference does not concern this discussion). $\Lambda_l(0)$ is essentially like the set $\text{lam } \emptyset$ considered above, for instance. In our framework, $\Lambda_l(n)$ is in bijection with the set of terms of type $\text{lam } \mathbb{N}$ paired with an external proof witnessing $\text{cl}_{\text{at}} n t$. Recall from Chapter 3 that this proposition states no de Bruijn index in t exceeds $d-1$ by more than n , where d is the number of binders the occurrence is underneath and we subtract one to account for indexing from 0. Thus, the first conceptual difference between our work and theirs, besides the fact we incorporate a distributive law over applicative functors, is that we formalize claims about the scope of a term externally, without reflecting this information in the type of the term itself.

Subsequent work [32] extended the presheaf-theoretic approach to second order and dependently-sorted abstract syntax. Most recently, Fiore and Szamozvancev have developed an Agda framework for intrinsically well-scoped, well-typed second order abstract syntax [33]. An advantage of this approach is that the derived substitution operation is inherently type preserving, obviating the need for a separate proof that the type system satisfies a substitution lemma. By contrast, in our formalization of simply-typed lambda calculus in Chapter 7, the type system and its substitution lemma are both established entirely independently of the syntax itself. The role of the Tealeaves backend in such a situation is to supply the definition of substitution on raw terms and to help reason about this operation. Tealeaves does not directly offer assistance for reasoning about type systems except in this capacity; indeed, the fact that `binddt` inspects the data in a syntax tree, but does not inspect the shape or its individual constructors, runs somewhat counter to the idea of a type system, which is often associated with a unique typing rule for each constructor in the syntax. Of course, users of Tealeaves can still define such operations by structural recursion on terms independently of Tealeaves. This line of inquiry closely resembles the presheaf-theoretic formalization of syntax.

8.5 NOMINAL LOGIC

Nominal logic [60] is a version of first-order logic extended with a first-class notion of variables names (called *atoms*), variable binding, freshness constraints, and a permutation action on terms. The expression $(a\ b) \cdot t$ denotes the term obtained by swapping names a and b everywhere in t . These operations are axiomatized to support reasoning about syntax with binding. The key advantage of this approach is that all properties expressible in nominal logic are invariant under name swapping—mirroring the intuition that bound variable names are irrelevant. As a result, nominal logic supports structurally recursive definitions and induction principles that respect α -equivalence by construction, eliminating the need for ad-hoc reasoning about renaming or variable capture. In some respects, nominal logic can be seen as a rigorous foundation justifying the Barendregt convention and the informal style of reasoning often seen on paper.

Our axiomatization of nominal substitution, by contrast, fully embraces concrete reasoning about renaming and variable capture—precisely the sort of reasoning that nominal logic helps users avoid. Variable swapping is a special case of renaming and easily expressed in our framework. Concrete reasoning about the properties of swapping is also possible; it is trivial to show that swapping is idempotent, for example, because swapping is idempotent *locally*. This raises the intriguing possibility of *deriving* the rules of nominal logic as a kind of abstraction layer on top of a fully explicit implementation of nominal substitution using Tealeaves. While I have not pursued this line of investigation, it would likely be useful to do so. Doing so would allow users to reason about syntax with variable binding as easily as with nominal logic, with the benefit of an underlying concrete implementation of substitution and the possibility of translating between representations, perhaps to formally relate two formalization efforts that share an underlying syntax but a different representation of variables.

8.5.1 Other Related Work

Explicit Substitutions

Our emphasis on rewriting and low-level details of capture-avoiding substitution has much in common with work on explicit substitutions [1, 27]. The explicit substitution calculus underlies Autosubst [64, 69]. We proved in Section 4.4 that the equational theory used by Autosubst is a special case of ours, deriving their axioms from the laws of monads decorated by $\langle \mathbb{N}, +, 0 \rangle$. However, the theory of DTMs is not per se an explicit substitution calculus, and we have not given any consideration to properties such as confluence. The theory of binddt is independent of the grammar of any particular syntax, unlike the $\lambda\sigma$ -calculus which is specialized to the grammar of the λ -calculus.

Lenses and Optics

Our work exploits the nature of traversable functors as a kind of very-well behaved lens [34] focusing on several targets. Recent work has led to a rich categorical theory of generalized lenses, particularly in the unifying framework of *profunctor optics* [19, 25]. The (generalized) lens laws were derived from the axioms of traversable functors in Chapter 6, but from the coalgebraic approach rather than the profunctorial one. While I found the lens laws difficult to work with in Coq due to the type-theoretic issues, the presentation of traversable functors as a kind of generalized optic more clearly illustrates than any other presentation the fact that the traverse operation focuses on the contents of data structures without distributing their shapes. Decorated traversable functors might also be seen as generalized optics, with the additional feature that each data item under focus is annotated with a static value specific to its location in the structure; this value is unaffected by the put operation, which only replaces the data themselves. While I have not investigated how the monadic structure can be incorporated into the optical perspective, doing so would likely be of interest.

Future Work

9.1 AUTOMATIC INSTANTIATION

The `derive_dtm` tactic is not complete. It can be made complete. As shown in Section 6.2 and Section 6.4, in some cases this proof depends on deep theory concerning the representation theorem for traversable and decorated-traversable functors. While the technique of exploiting the representation theorem can be automated, these cases also required specific insight into the decoration itself. Namely, in required showing that the length of a list depends only on its shape, but not its contents. This particular case was obvious: the length of a shape precisely corresponds to its shape, and so the creative insight required was not deep. However, other examples might be more complex.

One can imagine a system much like Ott [65], where a user expresses their syntax, including the binding policy, from a domain-specific language used explicitly for the purpose. By constraining the kinds of syntax that can be expressed, there is hope of defining a complete instantiation procedure. This system would accept a specification and generate a Coq module containing a DTM instance for the user’s syntax. By importing this automatically generated module, and the appropriate Tealeaves backend(s), there is hope for a workflow where users can obtain a fully certified implementation of capture-avoiding substitution for the syntax of realistic programming languages as quickly as they can specify the shape and binding policy of their syntax. Because Tealeaves provides the ability to convert between representations, a user can reason about their language using a convenient internal representation of their choice, while also having a nominal representation available so they can read and write programs conveniently, without introducing the possibility that the internal and external representations introduce a discrepancy.

9.2 INTRINSICALLY SCOPED DTMS

Our emphasis on raw, extrinsically scoped syntax is evident in the fact that the type parameter $V : \mathbf{Type}$ is thought of as flat *set* of variables that does not vary with the binding context, as it does in the above definition. To modify our approach to the well-scoped setting, we can make V dependent on the binding context of an occurrence. For instance, Figure 8.1 depicts a version of `lam` modified from the version in Chapter 3 so that the variable set is instead a *family* of sets parameterized by $\mathbb{N}$, realized in Coq as a function of type $\mathbb{N} \rightarrow \mathbf{Type}$.

The appearance of $(\odot 1)$ can be thought of as passing information to V about the binding context, exactly as it is used in the definition of `binddtlam` (Def. 5.4.26). This usage is closely related to the *context extension* operation of Fiore et al. [31], discussed below. `binddt` can be modified fairly

```

Inductive lam (V : nat -> Type) : Type :=
| var : V 0 -> lam V
| abs : lam (V ⊙ 1) -> lam V
| app : lam V -> lam V -> lam V.

```

Figure 9.1: Intrinsically scoped definition of lam in the style of Tealeaves

straightforwardly to the scoped version of lam. Weaving the extra dependency of the variable type parameter on the W through the axioms of binddt is not conceptually difficult; Equations (5.129)–(5.132) do not fundamentally change if A and B depend on the context. We provide the definition below, although we have not investigated the practical usage of this abstraction.

Definition 9.2.1. *Let W be any set. A W -indexed family is a family of sets indexed by W .*

Definition 9.2.2. *Let $\langle W, e_M, \cdot \rangle$ be a monoid. An W -scoped DTM is a type constructor*

$$T : (W \rightarrow \text{SET}) \rightarrow \text{SET}$$

paired with operations of the following polymorphic types, where parameters A, B are W -indexed families and G is any applicative functor:

$$\begin{aligned} \text{ret}^T &: A\ 0 \rightarrow T\ A \\ \text{binddt}_G^T &: (\forall (w : W), A\ w \rightarrow G\ (T\ (B\ \odot\ w))) \rightarrow T\ A \rightarrow G\ (T\ B) \end{aligned}$$

This operation is subject to straightforward generalizations of (5.129)–(5.132).

The fact that binddt can be carried over to an intrinsically scoped setting is promising, as this is the operation in terms of which other syntactic operations is defined. However, the theoretical decomposition of lam into individual components—an ordinary monad, a decorated functor, and a traversable functor, each related to the others by coherence laws—no longer applies if the type parameter is a type family. For example, T as presented is not a monad (it is not an endofunctor), and the concept of codifying the binding policy as an extrinsic operation like dec^{lam} no longer makes sense, since the binding policy is already reflected in the nested inductive structure of lam.

While it would be worthwhile to explore, I have not formalized Definition 9.2.2 in Coq due to the type-theoretic burden of this new dependency of the variable set on W . Expressions such as $(n + 0)$ and n are not equal computationally in Coq; they are merely equal propositionally. Therefore, terms of different types such as $V\ (n + 0)$ and $V\ n$ cannot be compared for equality natively—we must instead reach for a solution such as heterogeneous/John Major equality [53] or explicit type coercions. This same issue is why multisorted DTMs have been implemented in

terms of a compromise abstraction presented in Chapter C, which involves fewer type coercions than would be required for monads on the category of sort-indexed type families. I believe this speaks more to the underlying type theory of Coq than the DTM framework; it may be that in proof assistants more favorable to dependently-typed programming, or with improvements to the same in Coq, an intrinsically scoped version of Tealeaves would be quite practical.

In future work, one can imagine formalizing this sort of processing step using an operation like `binddt` whose input is a raw syntax tree, but which produces a well-scoped syntax tree as an output. Of course, the output would have to be wrapped a datatype such as `option` to reflect the inherent possibility of failure.

9.2.1 DTMs and Refinement Types

An inherent limitation of the first-order approach is that, a priori, one does not have any particular insight into the variables in a particular term. For example, again the translation between locally nameless and de Bruijn terms. Rather than viewing this as a partial function, and then proving *post-hoc* that the function is total for a certain class of inputs, we consider one way to obtain a total function without fundamentally modifying the DTM abstraction. For instance, if $t : T\ LN$ is a locally nameless term and we have $\mathbf{lc}\ t$. By Theorem 5.2.4, we know the following holds:

$$\forall (d : \mathbb{N}) (v : LN), (d, v) \in_d t \implies \mathbf{lc}_{\text{loc}} (d, v) \quad (9.1)$$

Now, we might expect to be able to use $\mathbf{lc}\ t$ to map over t by an operation of the following type, which is provided, besides a context variable occurrence, a proof that this occurrence satisfies $\mathbf{lc}_{\text{loc}}$:

$$\forall (d : \mathbb{N}) (v : LN), \mathbf{lc}_{\text{loc}} (d, v) \rightarrow T\ LN$$

Presumably, there ought to be a way to define a restricted version of `binddt` that only operates on locally closed terms, and allows lifting functions that are only defined on inputs satisfying $\mathbf{lc}_{\text{loc}}$:

$$\text{binddt}^{\mathbf{lc}} : ((\forall (d : \mathbb{N}) (v : LN), \mathbf{lc}_{\text{loc}} (d, v) \rightarrow T\ LN) \rightarrow (\exists (t : T\ LN), \mathbf{lc}\ t) \rightarrow T\ LN$$

Let us consider a different way of saying the same thing, by packaging the inputs to this function as an existential type:

$$\text{binddt}^{\mathbf{lc}} : ((\exists (d : \mathbb{N}) (v : LN), \mathbf{lc}_{\text{loc}} (d, v)) \rightarrow T\ LN) \rightarrow (\exists (t : T\ LN), \mathbf{lc}\ t) \rightarrow T\ LN$$

We cannot immediately define this function using `binddt`, which only provides a mechanism to lift functions of this type:

$$\text{binddt}^{\text{lam}}: (\mathbb{N} \times \text{LN} \rightarrow T \text{LN}) \rightarrow T \text{LN} \rightarrow T \text{LN}$$

Note that the difference between this type and the prior one is simply that the latter function does not receive a proof that its input satisfies $\text{lc}_{\text{loc}}(d, v)$, even if we know that all occurrences in t satisfy this property. Therefore, we must define our operation in a way that implements some sort of default behavior for inputs that *do not* satisfy lc_{loc} . Afterwards, we can use properties such as the following to reason about this function when it so happens to be applied to a locally nameless term.

$$\text{lc } t \implies (\forall (d: \mathbb{N}) (v: \text{LN}), \text{lc}_{\text{loc}}(d, v) \implies f(d, v) = g(d, v)) \implies \text{binddt } ft = \text{binddt } g t$$

This sort of strategy was effectively how we proved the translation: we define a function that can fail, and then we prove that it does not fail for some input.

I have argued that “raw” nature of the Tealeaves methodology is a *feature*, making it conceptually well-suited for a range of real-world circumstances. For example, a real-world compiler, I suspect, would implement a definition of substitution that is close to the ones considered in this work. For example, a compiler might indeed throw an error in the process of converting a raw term to a de Bruijn term. The strategy of “define now, prove later” might not be satisfying for all purposes. In this case, another strategy is possible.

Lemma 9.2.3. *The following function can be defined for any traversable functor F .*

$$\text{addWitness}: \forall (t: F A), F (\exists (a: A), a \in t)$$

This operation satisfies the following property, where π_1 projects out the value a and discards the “witness” term proving $a \in t$:

$$\forall (t: F A), \text{map}^F \pi_1 (\text{addWitness } t) = t$$

Proof. In informal practice, defining an operation such as this one is so straightforward that it would hardly be worth mentioning, much less in the form of a lemma. However, defining this operation in Tealeaves is very tricky, but possible. To do this, we convert t to `runBatch id (batch t)` using (5.100) and (6.33). We likewise express $a \in t$ in a similar form, using the fact that $a \in t$ can be expressed as a traversal w.r.t. the constant applicative functor of propositions under disjunction. Then, this function can be defined by a conceptually straightforward structural recursion on `batch t`. $\square$

A similar strategy can be used to take any term t and “push” a predicate of the form $\text{ForAll } P \ t$ into the leaves—we simply express $\text{ForAll } P \ t$ in terms of batch , as above. However, the operation above is already sufficient. By precomposing map with the addWitness , we can derive operations such as the following one:

$$\begin{aligned} \text{map_with_witness} &: \forall (t: F \ A), (\forall (a: A), a \in t \rightarrow B) \rightarrow F \ B \\ \text{map_with_witness } t \ f &= \text{map}^F (\text{apply } f) (\text{addWitness } t) \end{aligned}$$

where $\text{apply } f (a, \text{pf}) = f \ a \ \text{pf}$. Using the characterization of ForAll from Lemma 5.1.33, we can use map_with_witness to define an operation of the following type:

$$\begin{aligned} \text{localize_Forall} &: \forall (t: F \ A), \text{ForAll } P \ t \rightarrow F (\exists (a: A), P \ a) \\ \text{localize_Forall} &= \text{map_with_witness } t \ \forall (a: A), a \in t \rightarrow (\exists (a: A), P \ a) \end{aligned}$$

The binding context can also be incorporated by applying the above strategy to a decorated traversable functor, using batchd in place of batch . This yields the following functions:

$$\begin{aligned} \text{addWitnessd} &: \forall (t: T \ A), \exists (t': T (\exists (w: W) (a: A), (w, a) \in_d t)), \text{map}^F \pi_1 t' = t \\ \text{pushForalld} &: \forall (t: T \ A), \text{ForAllDec } P \ t \rightarrow \exists (t': T (\exists (w: W) (a: A), P (w, a))), \text{map}^F \pi_1 t' = t \end{aligned}$$

The operations defined this way are difficult to reason about, as they are all defined in terms of batch and its variants. Therefore, the practical usability of this approach requires further work. However, it would seem to provide an avenue to relate this work to well-scoped approaches: we represent a well-scoped term as a term paired with a predicate stating (for instance) that all of its free variables belong to a given set. By pushing this predicate into the leaves, we can reason about total functions that depend on this information for totality, without the same type-theoretic machinery as intrinsically scoped DTMs as presented in Section 9.2.

CONCLUDING REMARKS

This dissertation has developed a unified, category-theoretic foundation for reasoning about conventional first-order abstract syntax with variable binding through the abstraction of decorated traversable monads (DTMs).

We have examined DTMs from three quite different perspectives, each offering something different. The traditionally category-theoretic understanding of DTMs as monoids in a particular category of structured endofunctors decomposes the theory into a set of primitive building blocks, shown in Figures 5.1 and 5.2. Besides being familiar to category theorists, these building blocks reveal something *topological* about syntactic reasoning by way of string diagrammatic reasoning. The presentation of DTMs in terms of the `binddt` combinator illustrates that the theory is fundamentally about a class of datatypes that support a structured form of recursion that does not depend on the grammatical specifics of the underlying signature of the language. The coalgebraic perspective decomposes DTMs into their abstract shape and their concrete data. This perspective is *essential* as an avenue for deriving reasoning principles whose truth seems intuitively clear, but whose formal proof would otherwise remain elusive. When taken together, these perspectives and their equivalence yields a very rich framework. The richness of the framework might even be considered surprising because the entire framework can be reduced to just four (if Definition 5.4.17 is used) or three (if Definition 6.3.13 is used) equations.

By isolating the algebraic structure common to a wide range of syntactic operations—such as substitution, renaming, and free-variable enumeration, and even the construction of relations—the DTM framework enables generic, reusable definitions and proofs across multiple binding representations, including de Bruijn indices, locally nameless, and nominal syntax. The accompanying implementation in Tealeaves demonstrates that this theory is not only elegant, but practical: it provides a library of generic metatheoretic infrastructure that can be instantiated with minimal effort, yielding reasoning tools for formal language definitions. Moreover, the theory handles advanced features such as variadic and mutually recursive binders, and supports translation between representations. All of this exists within a clean category-theoretic framework, exposing underlying principles and providing some indication of how the raw approach to syntax relates to the rich body of existing theory, which has typically focused on more intrinsic notions of syntax.

In conclusion, this work advances the state of the art in formalized reasoning about syntax, in support of a vision of formal metatheory that is frictionless and elegant, supporting both rapid prototyping and formal verification, without relying on heavy metaprogramming or introducing gaps between paper and proof assistant. It is my hope that this brings us one step closer to a world where compilers are routinely formally verified, and we can be sure that programs actually do what we intend for them to do.

APPENDIX A

Proof of DTM Instance for lam

In this section we prove the DTM axioms (5.129)–(5.132) for $\text{binddt}^{\text{lam}}$ (Definition 5.4.26). We illustrate these proofs from the perspective of automating them, and thus make the rewriting details very explicit. Before proving the axioms we lay out some of the supporting equations used for rewriting, which are derived from the properties of decorated and applicative functors.

A.1 EXTENDED APPLICATIVE FUNCTOR LAWS

The axioms of applicative functors (Definition 5.3.5) are already enough to rewrite expressions formed from pure and $(\otimes)$ to a normal form (Lemma 5.3.6). We now extend these rewriting laws to incorporate map . This is not theoretically necessary— map can be expressed in terms of pure and $(\otimes)$ by Equation (5.81)—but these shortcuts compress the proofs significantly.

Remark A.1.1. We write $(_ \circ _)$ to indicate the composition function $\lambda g. \lambda f. (g \circ f)$.

Lemma A.1.2. $\text{pure}^G : \mathbb{1} \Rightarrow G$ is a natural transformation:

$$\text{map}^G g \left(\text{pure}^G a \right) = \text{pure}^G (g a) \quad (\text{A.1})$$

Proof. This follows from (5.81) and (5.76).

$$\text{map}^G g \left(\text{pure}^G a \right) = \text{pure}^G g \otimes^G \text{pure}^G a = \text{pure}^G (g a)$$

□

Lemma A.1.3. map distributes over $(\otimes)$ as follows:

$$\text{map}^G g \left(f \otimes^G a \right) = \left(\text{map}^G (g \circ -) f \right) \otimes^G a \quad (\text{A.2})$$

Proof.

$$\begin{aligned} & \text{map}^G g \left(f \otimes^G a \right) \\ = & \text{pure}^G g \otimes^G \left(f \otimes^G a \right) && \text{Eqn. (5.81)} \\ = & \text{pure} (_ \circ _) \otimes^G \text{pure}^G g \otimes^G f \otimes^G a && \text{Eqn. (5.77)} \\ = & \text{pure}^G (g \circ -) \otimes^G f \otimes^G a && \text{Eqn. (5.76)} \\ = & \left(\text{map}^G (g \circ -) f \right) \otimes^G a && \text{Eqn. (5.81)} \end{aligned}$$

□

Lemma A.1.4. *map can be moved from left-to-right across $(\otimes)$ as follows:*

$$f \otimes^G \text{map}^G g a = \left(\text{map}^G (- \circ g) f \right) \otimes^G a \quad (\text{A.3})$$

Proof. The proof makes use of the following equality, which holds by definition:

$$\begin{aligned} (- \circ g) &= (\text{evalAt } g) \circ (- \circ -) \\ &= ((\text{evalAt } g) \circ (- \circ -)) f \\ &= (\text{evalAt } g) (f \circ -) \\ &= f \circ g \end{aligned} \quad (\text{A.4})$$

With this identity we prove (A.3):

$$\begin{aligned} & f \otimes^G (\text{map}^G g a) \\ = & f \otimes^G (\text{pure}^G g \otimes^G a) && \text{Eqn. (5.81)} \\ = & \text{pure}^G (- \circ -) \otimes^G f \otimes^G \text{pure}^G g \otimes^G a && \text{Eqn. (5.77)} \\ = & \text{pure}^G (\text{evalAt } g) \otimes^G (\text{pure}^G (- \circ -) \otimes^G f) \otimes^G a && \text{Eqn. (5.78)} \\ = & \text{pure}^G (- \circ -) \otimes^G \text{pure}^G (\text{evalAt } g) \otimes^G \text{pure}^G (- \circ -) \otimes^G f \otimes^G a && \text{Eqn. (5.77)} \\ = & \text{pure}^G ((\text{evalAt } g) \circ -) \otimes^G \text{pure}^G (- \circ -) \otimes^G f \otimes^G a && \text{Eqn. (5.76)} \\ = & \text{pure}^G ((\text{evalAt } g) \circ (- \circ -)) \otimes^G f \otimes^G a && \text{Eqn. (5.76)} \\ = & \text{pure}^G (- \circ g) \otimes^G f \otimes^G a && \text{Eqn. (A.4)} \\ = & (\text{map}^G (- \circ g) f) \otimes^G a && \text{Eqn. (5.81)} \end{aligned}$$

□

Lemma A.1.4 allows moving map from the RHS to the LHS of $(\otimes)$, while Lemma A.1.3 pushes map from the outside of an $(\otimes)$ to underneath it, and Lemma A.1.2 merges map into pure. Taking these rewriting steps as primitive remove the need to unfold map while normalizing an applicative expression—the verbosity of the proofs above demonstrates the space savings. Combining (A.1), (A.2), and (A.3) with Equations (5.75)–(5.78) allows rewriting all expressions into ones of the following canonical form [55]:

$$\text{pure}^G f \otimes^G a_1 \otimes^G a_2 \dots \otimes^G a_n$$

A.1.1 Composition of Applicative Functors

We introduce one other time-saving measure by re-expressing (5.85) in terms of `map` to obtain the following equation.

$$f \circledast^{G_1 \circ G_2} a \equiv \text{map}^{G_1} \left(\circledast^{G_2} \right) f \circledast^{G_1} a \quad (\text{A.5})$$

The following two equations are used in the course of proving the composition law (5.131) for constructors other than `var`. These rewrites handle the first argument of the constructor on the RHS of the goal.

Lemma A.1.5.

$$\text{pure}^{G_1} f \circledast^{G_1 \circ G_2} a = \text{pure}^{G_1} \left(f \circledast^{G_2} - \right) \circledast^{G_1} a \quad (\text{A.6})$$

$$\text{pure}^{G_1} f \circledast^{G_1} \text{map}^{G_1} g a = \text{pure}^{G_1} (f \circ g) \circledast^{G_1} a \quad (\text{A.7})$$

Proof.

Proof of (A.6):

$$\begin{aligned} & \text{pure}^{G_1} f \circledast^{G_1 \circ G_2} a \\ = & \text{map}^{G_1} (\circledast) \left(\text{pure}^{G_1} f \right) \circledast^{G_1} a && \text{Unfold } \circledast^{G_1 \circ G_2} \text{ by (A.5)} \\ = & \text{pure}^{G_1} \left(f \circledast^{G_1} - \right) \circledast^{G_1} a && \text{Merge map into pure by (A.1)} \end{aligned}$$

Proof of (A.7):

$$\begin{aligned} & \text{pure}^{G_1} f \circledast^{G_1} \text{map}^{G_1} g a \\ = & \text{map}^{G_1} (- \circ g) \left(\text{pure}^{G_1} f \right) \circledast^{G_1} a && \text{Move map to the left of } (\circledast) \text{ using (A.3)} \\ = & \text{pure}^{G_1} (f \circ g) \circledast^{G_1} a && \text{Merge map into pure by (A.1)} \end{aligned}$$

□

We also give corresponding lemmas for the second argument of a two-argument constructor. These are used in the proof of the composition law (5.131) for the `app` case.

$$\text{pure}^{G_1} f \circledast^{G_1} a_1 \circledast^{G_1 \circ G_2} a_2 = \text{pure}^{G_1} \left(\left(\circledast^{G_2} \right) \circ f \right) \circledast^{G_1} a_1 \circledast^{G_1} a_2 \quad (\text{A.8})$$

$$\text{pure}^{G_1} f \circledast^{G_1} a_1 \circledast^{G_1} \text{map}^{G_1} g a_2 = \text{pure}^{G_1} ((- \circ g) \circ f) \circledast^{G_1} a_1 \circledast^{G_1} a_2 \quad (\text{A.9})$$

Proof of (A.8):

$$\begin{aligned}
& \text{pure}^{G_1} f \circledast^{G_1} a_1 \circledast^{G_1 \circ G_2} a_2 \\
= & \text{map}^{G_1} (\circledast) \left(\text{pure}^{G_1} f \circledast^{G_1} a_1 \right) \circledast^{G_1} a_2 && \text{Unfold } \circledast^{G_1 \circ G_2} \text{ by (A.5)} \\
= & \text{map}^{G_1} ((\circledast) \circ -) \left(\text{pure}^{G_1} f \right) \circledast^{G_1} a_1 \circledast^{G_1} a_2 && \text{Move map under } (\circledast) \text{ by (A.2)} \\
= & \text{pure}^{G_1} ((\circledast) \circ f) \circledast^{G_1} a_1 \circledast^{G_1} a_2 && \text{Merge map into pure by (A.1)}
\end{aligned}$$

Proof of (A.9):

$$\begin{aligned}
& \text{pure}^{G_1} f \circledast^{G_1} a_1 \circledast^{G_1} \text{map}^{G_1} g a_2 \\
= & \text{map}^{G_1} (- \circ g) \left(\text{pure}^{G_1} f \circledast^{G_1} a_1 \right) \text{map}^{G_1} g a_2 && \text{Move map to the left of } (\circledast) \text{ using (A.3)} \\
= & \text{map}^{G_1} ((- \circ g) \circ -) \left(\text{pure}^{G_1} f \right) \circledast^{G_1} a_1 \text{map}^{G_1} g a_2 && \text{Move map under } (\circledast) \text{ by (A.2)} \\
= & \text{pure}^{G_1} ((- \circ g) \circ f) \circledast^{G_1} a_1 \circledast^{G_1} a_2 && \text{Merge map into pure by (A.1)}
\end{aligned}$$

A.2 PREINCREMENT LAWS

We incorporate the following basic equations into our general rewriting strategy.

Lemma A.2.1. *Preincrement satisfies the following equalities:*

$$\text{extract} \odot w = \text{extract} \tag{A.10}$$

$$(g \circ f) \odot w = g \circ (f \odot w) \tag{A.11}$$

$$f \odot e_W = f \tag{A.12}$$

□

A.3 MAKE FUNCTION DEFINITIONAL EQUALITIES

The proof of the composition law in Lemma A.4.3 makes use of two somewhat cryptic, albeit trivial equations. The first gives a point-free specification for the behavior of `binddt` when applied to the `abs` constructor and is stated as follows:

$$\left(\text{binddt}_{G_1} f \circ \text{abs} \right) = \left(\text{pure}^{G_1} \text{abs} \circledast^{G_1} - \right) \circ \text{binddt}_{G_1} (f \odot 1) \tag{A.13}$$

Both sides above are a point-free expression of the following:

$$\lambda t. \left(\text{pure}^{G_1} \text{abs} \circledast^{G_1} \left(\text{binddt}_{G_1} (f \odot 1) t \right) \right)$$

We also have a corresponding equality for app case:

$$\begin{aligned} & \left(\left(\text{binddt}_{G_2} g \right) \circ - \right) \circ \text{app} = \\ & \left(- \circ \text{binddt}_{G_2} g \right) \circ \left(\circledast^{G_2} \right) \circ \left(\left(\text{pure}^{G_2} \text{app} \right) \circledast^{G_2} - \right) \circ \left(\text{binddt}_{G_2} g \right) \end{aligned} \quad (\text{A.14})$$

Both sides of this expression are equal to the following:

$$\lambda t_1. \lambda t_2. \text{pure}^{G_2} \text{app} \circledast^{G_2} \text{binddt}_{G_2} g t_1 \circledast^{G_2} \text{binddt}_{G_2} g t_2$$

Both (A.13) and (A.14) hold by definition in Coq and are proved with `reflexivity`. The different expressions occur because of the basic strategy used in the proof of (5.131), as discussed in Section 6.2: the LHS and RHS of the goal are repeatedly rewritten to something of the form

$$\text{pure}^G f \circledast^G t_1 \circledast^G t_2 \dots \circledast^G t_n$$

using the laws of applicative functors, the equations of Lemmas A.1.2–A.1.4, and the inductive hypotheses. When this strategy is followed during the course of proving (5.131), for the LHS of the abs case, the function “ f ” shown above is precisely $\text{binddt} \circ \text{abs}$. When this strategy is applied to the right side, the expressions are identical, except that the function is expressed in this form:

$$\left(\text{pure}^{G_1} \text{abs} \circledast^{G_1} - \right) \circ \text{binddt}_{G_1} (f \odot 1).$$

The proof step is then complete by `reflexivity`, since these are simply two ways of writing the same expression. Similarly, repeatedly applying the proof strategy to the LHS and RHS side of the goal in the app case leads to the expressions on the LHS and RHS of (A.14).

A.4 DTM PROOFS FOR LAMBDA TERMS

We separate each law into its own lemma. The proof of (5.129) is trivial: It stipulates the definition of binddt on the var case, and it can be proved in Coq using `reflexivity`. The proofs of (5.130) and (5.132) are straightforward, while (5.131) is notably more complex.

Lemma A.4.1 (Proof of (5.129) for lam).

$$\text{binddt}_G^{\text{lam}} f \circ \text{ret}^{\text{lam}} = f \circ \text{ret}^{\mathbb{N} \times}$$

Proof. This equality is simply the definition of binddt on the var case.

$$\begin{aligned}
& \left(\text{binddt}_G f \circ \text{ret}^{\text{lam}} \right) v \\
= & \text{binddt}_G f (\text{var } v) \\
= & f (0, v) \\
= & \left(f \circ \text{ret}^{\mathbb{N}^\times} \right) v
\end{aligned}$$

This is a constraint on the definition of $\text{binddt}^{\text{lam}}$ on the var case and holds by **reflexivity**. $\square$

Lemma A.4.2 (Proof of (5.130) for lam).

$$\text{binddt}_{\mathbb{I}}^{\text{lam}} \left(\text{ret}^{\text{lam}} \circ \text{extract}^{\mathbb{N}^\times} \right) = \text{id}_{\text{lam}}$$

Proof.

var case:

$$\begin{aligned}
& \text{binddt}_{\mathbb{I}} \left(\text{ret}^{\text{lam}} \circ \text{extract}^{\mathbb{N}^\times} \right) (\text{var } v) \\
= & \left(\text{ret}^{\text{lam}} \circ \text{extract}^{\mathbb{N}^\times} \right) (0, v) \\
= & \text{var } v
\end{aligned}$$

abs case:

$$\begin{aligned}
& \text{binddt}_{\mathbb{I}} \left(\text{ret}^{\text{lam}} \circ \text{extract}^{\mathbb{N}^\times} \right) (\text{abs } t) && \text{Reduce } \text{binddt}^{\text{lam}} \text{ for abs} \\
= & (\text{pure}^{\mathbb{I}} \text{ abs}) \circledast^{\mathbb{I}} \left(\text{binddt}_{\mathbb{I}} \left(\left(\text{ret}^{\text{lam}} \circ \text{extract}^{\mathbb{N}^\times} \right) \odot 1 \right) t \right) && \text{Unfold } \text{pure}^{\mathbb{I}} \text{ and } \circledast^{\mathbb{I}} \\
= & \text{abs} \left(\text{binddt}_{\mathbb{I}} \left(\left(\text{ret}^{\text{lam}} \circ \text{extract}^{\mathbb{N}^\times} \right) \odot 1 \right) t \right) && \text{Eqns. (A.11) and (A.10)} \\
= & \text{abs} \left(\text{binddt}_{\mathbb{I}} \left(\text{ret}^{\text{lam}} \circ \text{extract}^{\mathbb{N}^\times} \right) t \right) && \text{I.H.} \\
= & \text{abs } t
\end{aligned}$$

app case:

$$\begin{aligned}
& \text{binddt}_{\mathbb{I}} \left(\text{ret}^{\text{lam}} \circ \text{extract}^{\mathbb{N}^\times} \right) (\text{app } t_1 t_2) \\
& \quad \text{Reduce binddt}^{\text{lam}} \text{ for app} \\
= & \quad (\text{pure}^{\mathbb{I}} \text{ app}) \circledast^{\mathbb{I}} \left(\text{binddt}_{\mathbb{I}} \left(\text{ret}^{\text{lam}} \circ \text{extract}^{\mathbb{N}^\times} \right) t_1 \right) \circledast^{\mathbb{I}} \left(\text{binddt}_{\mathbb{I}} \left(\text{ret}^{\text{lam}} \circ \text{extract}^{\mathbb{N}^\times} \right) t_2 \right) \\
& \quad \text{Unfold pure}^{\mathbb{I}} \text{ and } \circledast^{\mathbb{I}} \\
= & \quad \text{app} \left(\text{binddt}_{\mathbb{I}} \left(\text{ret}^{\text{lam}} \circ \text{extract}^{\mathbb{N}^\times} \right) t_1 \right) \left(\text{binddt}_{\mathbb{I}} \left(\text{ret}^{\text{lam}} \circ \text{extract}^{\mathbb{N}^\times} \right) t_2 \right) \\
& \quad \text{I.H.s} \\
= & \quad \text{app } t_1 t_2
\end{aligned}$$

□

Lemma A.4.3 (Proof of (5.131) for lam).

$$\text{map}^{G_1} \left(\text{binddt}_{G_2}^{\text{lam}} g \right) \circ \left(\text{binddt}_{G_1}^{\text{lam}} f \right) = \text{binddt}_{G_1 \circ G_2}^{\text{lam}} (g \star f)$$

Proof.

var case:

$$\begin{aligned}
& \text{map}^{G_1} \left(\text{binddt}_{G_2} g \right) \left(\text{binddt}_{G_1} f (\text{var } v) \right) \\
& \quad \text{Reduce binddt}^{\text{lam}} \text{ for var} \\
= & \quad \text{map}^{G_1} \left(\text{binddt}_{G_2} g \right) (f (0, \text{var } v)) \\
& \quad \text{Eqn. (A.12)} \\
= & \quad \text{map}^{G_1} \left(\text{binddt}_{G_2} (g \odot 0) \right) (f (0, \text{var } v)) \\
& \quad \text{Fold def. of Kleisli composition} \\
= & \quad (g \star f) (0, v) \\
& \quad \text{Fold binddt}^{\text{lam}} \text{ for var} \\
= & \quad \text{binddt}_{G_1 \circ G_2} (g \star f) (\text{var } v)
\end{aligned}$$

abs case (LHS):

$$\begin{aligned}
& \text{map}^{G_1} \left(\text{binddt}_{G_2} g \right) \left(\text{binddt}_{G_1} f (\text{abs } t) \right) \\
& \quad \text{Reduce binddt}^{\text{lam}} \text{ for abs} \\
= & \quad \text{map}^{G_1} \left(\text{binddt}_{G_2} g \right) \left(\text{pure}^{G_1} \text{ abs } \circledast^{G_1} \text{binddt} (f \odot 1) t \right) \\
& \quad \text{Push map under } (\circledast) \text{ by Eqn. (A.2)}
\end{aligned}$$

$$\begin{aligned}
&= \left(\text{map}^{G_1} \left(\left(\text{binddt}_{G_2} g \right) \circ - \right) \left(\text{pure}^{G_1} \text{abs} \right) \right) \otimes^{G_1} \text{binddt} (f \odot 1) t \\
&\quad \text{Commute map and pure by naturality (A.1)} \\
&= \text{pure}^{G_1} \left(\text{binddt}_{G_2} g \circ \text{abs} \right) \otimes^{G_1} \text{binddt} (f \odot 1) t
\end{aligned}$$

abs case (RHS):

$$\begin{aligned}
&= \text{binddt}_{G_1 \circ G_2} (g \star f) (\text{abs } t) \\
&\quad \text{Reduce binddt}^{\text{lam}} \text{ for abs} \\
&= \text{pure}^{G_1 \circ G_2} \text{abs} \otimes^{G_1 \circ G_2} \text{binddt}_{G_1 \circ G_2} ((g \star f) \odot 1) t \\
&\quad \text{Unfold pure}^{G_1 \circ G_2} \text{ by (5.84)} \\
&= \text{pure}^{G_1} \left(\text{pure}^{G_2} \text{abs} \right) \otimes^{G_1 \circ G_2} \text{binddt}_{G_1 \circ G_2} ((g \star f) \odot 1) t \\
&\quad \text{Unfold } \otimes^{G_1 \circ G_2} \text{ by (A.6)} \\
&= \text{pure}^{G_1} \left(\text{pure}^{G_2} \text{abs} \otimes^{G_2} - \right) \otimes^{G_1} \text{binddt}_{G_1 \circ G_2} ((g \star f) \odot 1) t \\
&\quad \text{Distribute } (- \odot 1) \text{ past } (\star) \text{ by (5.142)} \\
&= \text{pure}^{G_1} \left(\text{pure}^{G_2} \text{abs} \otimes^{G_2} - \right) \otimes^{G_1} \text{binddt}_{G_1 \circ G_2} (g \odot 1 \star f \odot 1) t \\
&\quad \text{I.H. for } t \\
&= \text{pure}^{G_1} \left(\text{pure}^{G_2} \text{abs} \otimes^{G_2} - \right) \otimes^{G_1} \left(\text{map}^{G_1} \left(\text{binddt}_{G_2} (g \odot 1) \right) \left(\text{binddt}_{G_1} (f \odot 1) t \right) \right) \\
&\quad \text{Move binddt } (g \odot 1) \text{ to the left of } \otimes \text{ by (A.7)} \\
&= \text{pure}^{G_1} \left(\left(\text{pure}^{G_2} \text{abs} \otimes^{G_2} - \right) \circ \text{binddt}_{G_2} (g \odot 1) \right) \otimes^{G_1} \text{binddt}_{G_1} (f \odot 1) t
\end{aligned}$$

After the LHS and RHS have both been rewritten following the above procedures, we are left with the following goal:

$$\begin{aligned}
&\text{pure}^{G_1} \left(\text{binddt}_{G_2} g \circ \text{abs} \right) \otimes^{G_1} \text{binddt} (f \odot 1) t \\
&= \text{pure}^{G_1} \left(\left(\text{pure}^{G_2} \text{abs} \otimes^{G_2} - \right) \circ \text{binddt}_{G_2} (g \odot 1) \right) \otimes^{G_1} \text{binddt}_{G_1} (f \odot 1) t
\end{aligned}$$

At this point the goal is solved with **reflexivity**, which succeeds because there is a definitional equality between the make functions:

$$\text{binddt}_{G_2} g \circ \text{abs} = \left(\text{pure}^{G_2} \text{abs} \otimes^{G_2} - \right) \circ \text{binddt}_{G_2} (g \odot 1)$$

app case (LHS):

$$\begin{aligned}
&\text{map}^{G_1} \left(\text{binddt}_{G_2} g \right) \left(\text{binddt}_{G_1} f (\text{app } t_1 t_2) \right) \\
&\quad \text{Reduce binddt} \\
&= \text{map}^{G_1} \left(\text{binddt}_{G_2} g \right) \left(\text{pure}^{G_1} \text{app} \otimes^{G_1} \text{binddt}_{G_1} f t_1 \otimes^{G_2} \text{binddt}_{G_1} f t_2 \right) \\
&\quad \text{Push map under } (\otimes) \text{ by Eqn. (A.2)}
\end{aligned}$$

$$\begin{aligned}
&= \left(\text{map}^{G_1} \left(\left(\text{binddt}_{G_2} g \right) \circ - \right) \left(\text{pure}^{G_1} \text{app} \circledast^{G_1} \text{binddt}_{G_1} f t_1 \right) \right) \circledast^{G_2} \text{binddt}_{G_1} f t_2 \\
&\quad \text{Push map under } (\circledast) \text{ by Eqn. (A.2)} \\
&= \left(\text{map}^{G_1} \left(\left(\left(\text{binddt}_{G_2} g \right) \circ - \right) \circ - \right) \left(\text{pure}^{G_1} \text{app} \right) \circledast^{G_1} \text{binddt}_{G_1} f t_1 \circledast^{G_2} \text{binddt}_{G_1} f t_2 \right) \\
&\quad \text{Merge map and pure by (A.1)} \\
&= \text{pure}^{G_1} \left(\left(\left(\text{binddt}_{G_2} g \right) \circ - \right) \circ \text{app} \right) \circledast^{G_1} \text{binddt}_{G_1} f t_1 \circledast^{G_2} \text{binddt}_{G_1} f t_2
\end{aligned}$$

app case (RHS):

$$\begin{aligned}
&\mathbf{B}_{G_1 \circ G_2} (g \star f) (\text{app } t_1 t_2) \\
&\quad \text{Reduce } \mathbf{B} \\
&= \mathbf{P}^{G_1 \circ G_2} \text{app} \circledast^{G_1 \circ G_2} \left(\mathbf{B}_{G_1 \circ G_2} (g \star f) t_1 \right) \circledast^{G_1 \circ G_2} \left(\mathbf{B}_{G_1 \circ G_2} (g \star f) t_2 \right) \\
&\quad \text{Unfold } \mathbf{P}^{G_1 \circ G_2} \text{ by (5.84)} \\
&= \mathbf{P}^{G_1} \left(\mathbf{P}^{G_2} \text{app} \right) \circledast^{G_1 \circ G_2} \left(\mathbf{B}_{G_1 \circ G_2} (g \star f) t_1 \right) \circledast^{G_1 \circ G_2} \left(\mathbf{B}_{G_1 \circ G_2} (g \star f) t_2 \right) \\
&\quad \text{Unfold inner } \circledast^{G_1 \circ G_2} \text{ by (A.6)} \\
&= \mathbf{P}^{G_1} \left(\mathbf{P}^{G_2} \text{app} \circledast^{G_1} - \right) \circledast^{G_1} \left(\mathbf{B}_{G_1 \circ G_2} (g \star f) t_1 \right) \circledast^{G_1 \circ G_2} \left(\mathbf{B}_{G_1 \circ G_2} (g \star f) t_2 \right) \\
&\quad \text{I.H. for } t_1 \\
&= \mathbf{P}^{G_1} \left(\mathbf{P}^{G_2} \text{app} \circledast^{G_1} - \right) \circledast^{G_1} \mathbf{M} \left(\mathbf{B}_{G_2} g \right) \left(\mathbf{B}_{G_1} f t_1 \right) \circledast^{G_1 \circ G_2} \left(\mathbf{B}_{G_1 \circ G_2} (g \star f) t_2 \right) \\
&\quad \text{Move } \mathbf{B}_{G_2} g \text{ to the left of } \circledast \text{ by (A.7)} \\
&= \mathbf{P}^{G_1} \left(\left(\left(\mathbf{P}^{G_2} \text{app} \circledast^{G_1} - \right) \circ \mathbf{B}_{G_2} g \right) \circledast^{G_1} \mathbf{B}_{G_1} f t_1 \right) \circledast^{G_1 \circ G_2} \left(\mathbf{B}_{G_1 \circ G_2} (g \star f) t_2 \right) \\
&\quad \text{Unfold outer } \circledast^{G_1 \circ G_2} \text{ by (A.8)} \\
&= \mathbf{P}^{G_1} \left(((\circledast) \circ \left(\mathbf{P}^{G_2} \text{app} \circledast^{G_1} - \right) \circ \mathbf{B}_{G_2} g) \circledast^{G_1} \mathbf{B}_{G_1} f t_1 \right) \circledast^{G_1} \left(\mathbf{B}_{G_1 \circ G_2} (g \star f) t_2 \right) \\
&\quad \text{I.H. for } t_2 \\
&= \mathbf{P}^{G_1} \left(((\circledast) \circ \left(\mathbf{P}^{G_2} \text{app} \circledast^{G_1} - \right) \circ \mathbf{B}_{G_2} g) \circledast^{G_1} \mathbf{B}_{G_1} f t_1 \right) \circledast^{G_1} \mathbf{M} \left(\mathbf{B}_{G_2} g \right) \left(\mathbf{B}_{G_1} f t_1 \right) \\
&\quad \text{Move } \mathbf{B}_{G_2} g \text{ into } \mathbf{P} \text{ by (A.9)} \\
&= \mathbf{P}^{G_1} \left(\left(\left(- \circ \mathbf{B}_{G_2} g \right) \circ ((\circledast) \circ \left(\mathbf{P}^{G_2} \text{app} \circledast^{G_1} - \right) \circ \mathbf{B}_{G_2} g) \right) \circledast^{G_1} \mathbf{B}_{G_1} f t_1 \right) \circledast^{G_1} \mathbf{B}_{G_1} f t_1
\end{aligned}$$

After the LHS and RHS have both been rewritten following the above procedures, we are left

with the following goal:

$$\begin{aligned}
& \mathbf{P}^{G_1} \left(\left(\left(\mathbf{B}_{G_2} g \right) \circ - \right) \circ \text{app} \right) \overset{G_1}{*} \mathbf{B}_{G_1} f t_1 \overset{G_2}{*} \mathbf{B}_{G_1} f t_2 \\
= & \mathbf{P}^{G_1} \left(\left(- \circ \mathbf{B}_{G_2} g \right) \circ (\otimes) \circ \left(\left(\mathbf{P}^{G_2} \text{app} \right) \overset{G_1}{*} - \right) \circ \mathbf{B}_{G_2} g \right) \overset{G_1}{*} \mathbf{B}_{G_1} f t_1 \overset{G_1}{*} \mathbf{B}_{G_1} f t_1
\end{aligned}$$

At this point the goal is solved with **reflexivity**, which succeeds because there is a definitional equality between the make functions:

$$\left(\left(\mathbf{B}_{G_2} g \right) \circ - \right) \circ \text{app} = \left(- \circ \mathbf{B}_{G_2} g \right) \circ (\otimes) \circ \left(\left(\mathbf{P}^{G_2} \text{app} \right) \overset{G_1}{*} - \right) \circ \left(\mathbf{B}_{G_2} g \right)$$

□

Lemma A.4.4 (Proof of (5.132) for lam). *This induction is proved by induction where the goal is universally quantified over all f . This generality is necessary for the abs case where the argument to binddt is of the form $f \odot 1$.*

$$\phi \circ \text{binddt}_{G_1}^{\text{lam}} f = \text{binddt}_{G_2}^{\text{lam}} (\phi \circ f)$$

Proof.

var case:

$$\begin{aligned}
& \phi \left(\text{binddt}_{G_1} f (\text{var } v) \right) \\
= & \phi (f (0, v)) && \text{Reduce binddt}^{\text{lam}} \text{ for var} \\
= & \text{binddt}_{G_2} (\phi \circ f) (\text{var } v) && \text{Fold binddt}^{\text{lam}} \text{ for var}
\end{aligned}$$

abs case:

$$\begin{aligned}
& \phi \left(\text{binddt}_{G_1} f (\text{abs } t) \right) \\
= & \phi \left(\text{pure}^{G_1} \text{abs} \overset{G_1}{*} \text{binddt}_{G_1} (f \odot 1) t \right) && \text{Reduce binddt}^{\text{lam}} \text{ for abs} \\
= & \phi \left(\text{pure}^{G_1} \text{abs} \right) \overset{G_2}{*} \phi \left(\text{binddt}_{G_1} (f \odot 1) t \right) && \text{Eqn. (5.88)} \\
& && \text{Eqn. (5.87)}
\end{aligned}$$

$$\begin{aligned}
&= \left(\text{pure}^{G_2} \text{abs} \right) \circledast^{G_2} \phi \left(\text{binddt}_{G_1} (f \odot 1) t \right) && \text{I.H.} \\
&= \left(\text{pure}^{G_2} \text{abs} \right) \circledast^{G_2} \left(\text{binddt}_{G_2} (\phi \circ (f \odot 1)) t \right) && \text{Eqn. (A.11)} \\
&= \left(\text{pure}^{G_2} \text{abs} \right) \circledast^{G_2} \left(\text{binddt}_{G_2} ((\phi \circ f) \odot 1) t \right) && \text{Fold binddt}^{\text{lam}} \text{ for abs} \\
&= \text{binddt}_{G_2} (\phi \circ f) (\text{abs } t)
\end{aligned}$$

app case:

$$\begin{aligned}
&\phi (\text{binddt } f (\text{app } t_1 t_2)) && \text{Reduce binddt}^{\text{lam}} \text{ for app} \\
&= \phi \left(\text{pure}^{G_1} \text{app} \circledast^{G_1} \text{binddt } f t_1 \circledast^{G_1} \text{binddt } f t_2 \right) && \text{Eqn (5.88)} \\
&= \phi \left(\text{pure}^{G_1} \text{app} \circledast^{G_1} \text{binddt } f t_1 \right) \circledast^{G_2} \phi (\text{binddt } f t_2) && \text{Eqn (5.88)} \\
&= \phi \left(\text{pure}^{G_1} \text{app} \right) \circledast^{G_2} \phi (\text{binddt } f t_1) \circledast^{G_2} \phi (\text{binddt } f t_2) && \text{Eqn. (5.87)} \\
&= \left(\text{pure}^{G_2} \text{app} \right) \circledast^{G_2} \phi (\text{binddt } f t_1) \circledast^{G_2} \phi (\text{binddt } f t_2) && \text{I.H.s} \\
&= \left(\text{pure}^{G_2} \text{app} \right) \circledast^{G_2} (\text{binddt}(\phi \circ f) t_1) \circledast^{G_2} (\text{binddt}(\phi \circ f) t_2) && \text{Fold binddt}^{\text{lam}} \text{ for app} \\
&= \text{binddt} (\phi \circ f) (\text{app } t_1 t_2)
\end{aligned}$$

Theorem A.4.5. $\text{binddt}^{\text{lam}}$ forms a decorated traversable monad.

Proof. Proved in Lemmas A.4.1–A.4.4. □

□

A.5 DTM PROOFS FOR VARIADIC SYNTAX

The following proof depicts a naïve attempt to prove the DTM composition law for the lambda calculus extended with a letin constructor that binds a whole list of variables at the same time. This proof, which is examined in Section 6.2 leaves the goal in an unsolvable state. We show how the unsolvable goal arises below before discussing its resolution.

Lemma A.5.1. *The extension of $\text{binddt}^{\text{lam}}$ to letin satisfies the composition law (5.131).*

Proof.

letin **case** (LHS):

$$\begin{aligned}
& \mathbf{M}^{G_1} \left(\mathbf{B}_{G_2} g \right) \left(\mathbf{B}_{G_1} f (\text{letin } l \ t) \right) \\
& \hspace{25em} \text{Reduce } \mathbf{B} \\
& = \mathbf{M}^{G_1} \left(\mathbf{B}_{G_2} g \right) \left(\mathbf{P}^{G_1} \text{letin } \overset{G_1}{*} \mathbf{T}^{\text{list}} \left(\mathbf{B}_{G_1} f \right) l \overset{G_1}{*} \mathbf{B}_{G_1} (f \odot \mathbf{len} \ l) \ t \right) \\
& \hspace{25em} \text{Push } \mathbf{M} \text{ under } (*) \text{ by Eqn. (A.2)} \\
& = \left(\mathbf{M}^{G_1} \left(\mathbf{B}_{G_2} g \circ - \right) \left(\mathbf{P}^{G_1} \text{letin } \overset{G_1}{*} \mathbf{T}^{\text{list}} \left(\mathbf{B}_{G_1} f \right) l \right) \right) \overset{G_1}{*} \mathbf{B}_{G_1} (f \odot \mathbf{len} \ l) \ t \\
& \hspace{25em} \text{Push } \mathbf{M} \text{ under } (*) \text{ by Eqn. (A.2)} \\
& = \left(\mathbf{M}^{G_1} \left(\mathbf{B}_{G_2} g \circ - \right) \circ - \right) \left(\mathbf{P}^{G_1} \text{letin} \right) \overset{G_1}{*} \mathbf{T}^{\text{list}} \left(\mathbf{B}_{G_1} f \right) l \overset{G_1}{*} \mathbf{B}_{G_1} (f \odot \mathbf{len} \ l) \ t \\
& \hspace{25em} \text{Merge } \mathbf{M} \text{ and } \mathbf{P} \text{ by (A.1)} \\
& = \mathbf{P}^{G_1} \left(\left(\mathbf{B}_{G_2} g \circ - \right) \circ \text{letin} \right) \overset{G_1}{*} \mathbf{T}^{\text{list}} \left(\mathbf{B}_{G_1} f \right) l \overset{G_1}{*} \mathbf{B}_{G_1} (f \odot \mathbf{len} \ l) \ t
\end{aligned}$$

letin **case** (RHS):

$$\begin{aligned}
& \mathbf{B}_{G_1 \circ G_2} (g \star f) (\text{letin } l \ t) \\
& \hspace{25em} \text{Reduce } \mathbf{B} \\
& = \mathbf{P}^{G_1 \circ G_2} \text{letin } \overset{G_1 \circ G_2}{*} \mathbf{T}^{\text{list}} \left(\mathbf{B}_{G_1 \circ G_2} (g \star f) \right) l \overset{G_1 \circ G_2}{*} \left(\mathbf{B}_{G_1 \circ G_2} ((g \star f) \odot \mathbf{len} \ l) \ t \right) \\
& \hspace{25em} \text{Unfold } \mathbf{P}^{G_1 \circ G_2} \text{ by (5.84)} \\
& = \mathbf{P}^{G_1} \left(\mathbf{P}^{G_2} \text{letin} \right) \overset{G_1 \circ G_2}{*} \mathbf{T}^{\text{list}} \left(\mathbf{B}_{G_1 \circ G_2} ((g \star f) \odot \mathbf{len} \ l) \right) l \overset{G_1 \circ G_2}{*} \left(\mathbf{B}_{G_1 \circ G_2} ((g \star f) \odot \mathbf{len} \ l) \ t \right) \\
& \hspace{25em} \text{Unfold inner } \overset{G_1 \circ G_2}{*} \text{ by (A.6)} \\
& = \mathbf{P}^{G_1} \left(\mathbf{P}^{G_2} \text{letin } \overset{G_2}{*} - \right) \overset{G_1}{*} \mathbf{T}^{\text{list}} \left(\mathbf{B}_{G_1 \circ G_2} ((g \star f) \odot \mathbf{len} \ l) \right) l \overset{G_1 \circ G_2}{*} \left(\mathbf{B}_{G_1 \circ G_2} ((g \star f) \odot \mathbf{len} \ l) \ t \right) \\
& \hspace{25em} \text{I.H. for } l
\end{aligned}$$

$$\begin{aligned}
&= \mathbf{P}^{G_1} \left(\mathbf{P}^{G_2} \text{letin } \overset{G_2}{*} - \right) \overset{G_1}{*} \mathbf{T}^{\text{list}} \left(\mathbf{M}^{G_1} \left(\mathbf{B}_{G_2} g \right) \circ \mathbf{B}_{G_1} f \right) l \overset{G_1 \circ G_2}{*} \left(\mathbf{B}_{G_1 \circ G_2} ((g \star f) \odot \mathbf{len } l) t \right) \\
&\quad \text{Factor out } \mathbf{T}(\mathbf{B} g) \text{ by Eqn. (5.101)} \\
&= \mathbf{P}^{G_1} \left(\mathbf{P}^{G_2} \text{letin } \overset{G_2}{*} - \right) \overset{G_1}{*} \mathbf{M}^{G_1} \left(\mathbf{T}^{\text{list}} \left(\mathbf{B}_{G_2} g \right) \right) \left(\mathbf{T}^{\text{list}} \left(\mathbf{B}_{G_1} f \right) l \right) \overset{G_1 \circ G_2}{*} \left(\mathbf{B}_{G_1 \circ G_2} ((g \star f) \odot \mathbf{len } l) t \right) \\
&\quad \text{Move } \mathbf{T}(\mathbf{B}_{G_2} g) \text{ to the left of } (*) \text{ by (A.7)} \\
&= \mathbf{P}^{G_1} \left(\left(\mathbf{P}^{G_2} \text{letin } \overset{G_2}{*} - \right) \circ \mathbf{T}^{\text{list}} \left(\mathbf{B}_{G_2} g \right) \right) \overset{G_1}{*} \mathbf{B}_{G_1} f l \overset{G_1 \circ G_2}{*} \left(\mathbf{B}_{G_1 \circ G_2} ((g \star f) \odot \mathbf{len } l) t \right) \\
&\quad \text{Unfold outer } \overset{G_1 \circ G_2}{*} \text{ by (A.8)} \\
&= \mathbf{P}^{G_1} \left(\left(\overset{G_2}{*} \right) \circ \left(\mathbf{P}^{G_2} \text{letin } \overset{G_2}{*} - \right) \circ \mathbf{T}^{\text{list}} \left(\mathbf{B}_{G_2} g \right) \right) \overset{G_1}{*} \mathbf{B}_{G_1} f l \overset{G_1}{*} \left(\mathbf{B}_{G_1 \circ G_2} ((g \star f) \odot \mathbf{len } l) t \right) \\
&\quad \text{Distribute } \odot \text{ over } \star \text{ by (5.142)} \\
&= \mathbf{P}^{G_1} \left(\left(\overset{G_2}{*} \right) \circ \left(\mathbf{P}^{G_2} \text{letin } \overset{G_2}{*} - \right) \circ \mathbf{T}^{\text{list}} \left(\mathbf{B}_{G_2} g \right) \right) \overset{G_1}{*} \mathbf{B}_{G_1} f l \overset{G_1}{*} \left(\mathbf{B}_{G_1 \circ G_2} (g \odot \mathbf{len } l \star f \odot \mathbf{len } l) t \right) \\
&\quad \text{I.H. for } t \\
&= \mathbf{P}^{G_1} \left(\left(\overset{G_2}{*} \right) \circ \left(\mathbf{P}^{G_2} \text{letin } \overset{G_2}{*} - \right) \circ \mathbf{T}^{\text{list}} \left(\mathbf{B}_{G_2} g \right) \right) \overset{G_1}{*} \mathbf{B}_{G_1} f l \overset{G_1}{*} \mathbf{M}^{G_1} \left(\mathbf{B}_{G_2} (g \odot \mathbf{len } l) \right) \left(\mathbf{B}_{G_1} (f \odot \mathbf{len } l) t \right) \\
&\quad \text{Move } \mathbf{B}_{G_2} g \text{ into } \mathbf{P} \text{ by (A.9)} \\
&= \mathbf{P}^{G_1} \left(\left(- \circ \mathbf{B}_{G_2} (g \odot \mathbf{len } l) \right) \circ \left(\overset{G_2}{*} \right) \circ \left(\mathbf{P}^{G_2} \text{letin } \overset{G_2}{*} - \right) \circ \mathbf{T}^{\text{list}} \left(\mathbf{B}_{G_2} g \right) \right) \overset{G_1}{*} \mathbf{B}_{G_1} f l \overset{G_1}{*} \mathbf{B}_{G_1} f l
\end{aligned}$$

At this point, the goal has been reduced to the following equation, as shown also in (6.55):

$$\begin{aligned}
&\left(\left(\left(\text{binddt}_{G_2} g \right) \circ - \right) \circ \text{letin} \right) \\
&\quad \stackrel{?}{=} \\
&\left(- \circ \text{binddt}_{G_2} (g \odot \text{length } l) \right) \circ \left(\overset{G_2}{*} \right) \circ \left(\text{pure}^{G_2} \text{letin } \overset{G_2}{*} - \right) \circ \text{traverse}^{\text{list}} \left(\text{binddt}_{G_2} g \right)
\end{aligned} \tag{A.15}$$

As discussed in Section 6.2, this equation is **not true**—the RHS has baked in the expression $\text{length } l$, referring to the list in the original expression $\text{letin } l t$. However, the LHS will compute the length of its input list, which a priori may have no relation to l .

To resolve this mismatch, we modify the above proof as follows. Before the step (A.7) is

invoked, we invoke the representation theorem to rewrite the subexpression

$$\text{map}^{G_1} \left(\text{traverse}^{\text{list}} \left(\text{binddt}_{G_2} g \right) \right) \left(\text{traverse}^{\text{list}} \left(\text{binddt}_{G_1} f \right) l \right)$$

to the following one:

$$\text{map}^{G_1} \left(\text{traverse}^{\text{list}} \left(\text{binddt}_{G_2} g \right) \circ \text{make } l \right) \left(\text{traverse}^{\text{vec}} \left(\text{binddt } f \right) \left(\text{contents } l \right) \right)$$

At this point, the proof proceeds unmodified from the step that invokes (A.7). As discussed in Section 6.2.2, this minor modification reduces the goal to the following true equation:

$$\begin{aligned} & \left(\left(\left(\text{binddt}_{G_2} g \right) \circ - \right) \circ \text{letin} \circ \text{make}^{\text{list}} l \right) \\ & = \\ & \left(- \circ \text{binddt}_{G_2} (g \odot \text{length } l) \right) \circ \left(\textcircled{*}^{G_2} \right) \circ \left(\text{pure}^{G_2} \text{letin} \textcircled{*}^{G_2} - \right) \circ \text{traverse}^{\text{list}} \left(\text{binddt}_{G_2} g \right) \circ \text{make}^{\text{list}} l \end{aligned} \tag{A.16}$$

APPENDIX B

Monoidal Category Instances

B.1 CATEGORY OF DECORATED FUNCTORS

Lemma B.1.1 (Identity Decorated Functor). *Definition 4.3.31 is a valid decorated functor.*

Proof. We validate $\text{dec}^{\mathbb{1}}$ defines a valid decoration on $\mathbb{1}$.

Proof of (4.76): $\text{map}^{\mathbb{1}} \text{extr}^{W^\times} \circ \text{dec}^{\mathbb{1}} = \text{id}_{\mathbb{1}}$

$$\begin{aligned}
 & \text{Diagram 1: } \mathbb{1} \text{ (dashed line) } \rightarrow \text{Black dot} \xrightarrow{\text{dashed}} \mathbb{1} \text{ (dashed line)} \\
 & \quad \text{Black dot} \xrightarrow{\text{solid}} \text{Pink dot} \\
 & = \text{Diagram 2: } \text{Pink dot} \text{ --- } \text{Pink dot} \quad \text{Unfold } \text{dec}^{\mathbb{1}} \\
 & = \text{Diagram 3: } \text{Pink dot} \text{ --- } \text{Pink dot} \quad \text{Baton law (Eqn. (4.17))}
 \end{aligned}$$

Proof of (4.77): $\text{dec}^{\mathbb{1}} \circ \text{dec}^{\mathbb{1}} = \text{map}^{\mathbb{1}} \text{dup}^{W^\times} \circ \text{dec}^{\mathbb{1}}$

$$\begin{aligned}
 & \text{Diagram 1: } \mathbb{1} \text{ (dashed line) } \rightarrow \text{Black dot} \xrightarrow{\text{dashed}} \text{Black dot} \xrightarrow{\text{dashed}} \mathbb{1} \text{ (dashed line)} \\
 & \quad \text{Black dot} \xrightarrow{\text{solid}} \text{Pink dot} \xrightarrow{\text{solid}} W^\times \\
 & \quad \text{Black dot} \xrightarrow{\text{solid}} \text{Pink dot} \xrightarrow{\text{solid}} W^\times \\
 & = \text{Diagram 2: } \text{Pink dot} \text{ --- } W^\times \\
 & \quad \text{Pink dot} \text{ --- } W^\times \quad \text{Unfold } \text{dec}^{\mathbb{1}} \\
 & = \text{Diagram 3: } \text{Pink dot} \text{ --- } \text{Pink dot} \xrightarrow{\text{solid}} W^\times \\
 & \quad \text{Pink dot} \xrightarrow{\text{solid}} W^\times \quad \text{Cap law (Eqn. (4.18))} \\
 & = \text{Diagram 4: } \mathbb{1} \text{ (dashed line) } \rightarrow \text{Black dot} \xrightarrow{\text{dashed}} \mathbb{1} \text{ (dashed line)} \\
 & \quad \text{Black dot} \xrightarrow{\text{solid}} \text{Pink dot} \xrightarrow{\text{solid}} W^\times \\
 & \quad \text{Black dot} \xrightarrow{\text{solid}} \text{Pink dot} \xrightarrow{\text{solid}} W^\times \quad \text{Fold } \text{dec}^{\mathbb{1}}
 \end{aligned}$$

□

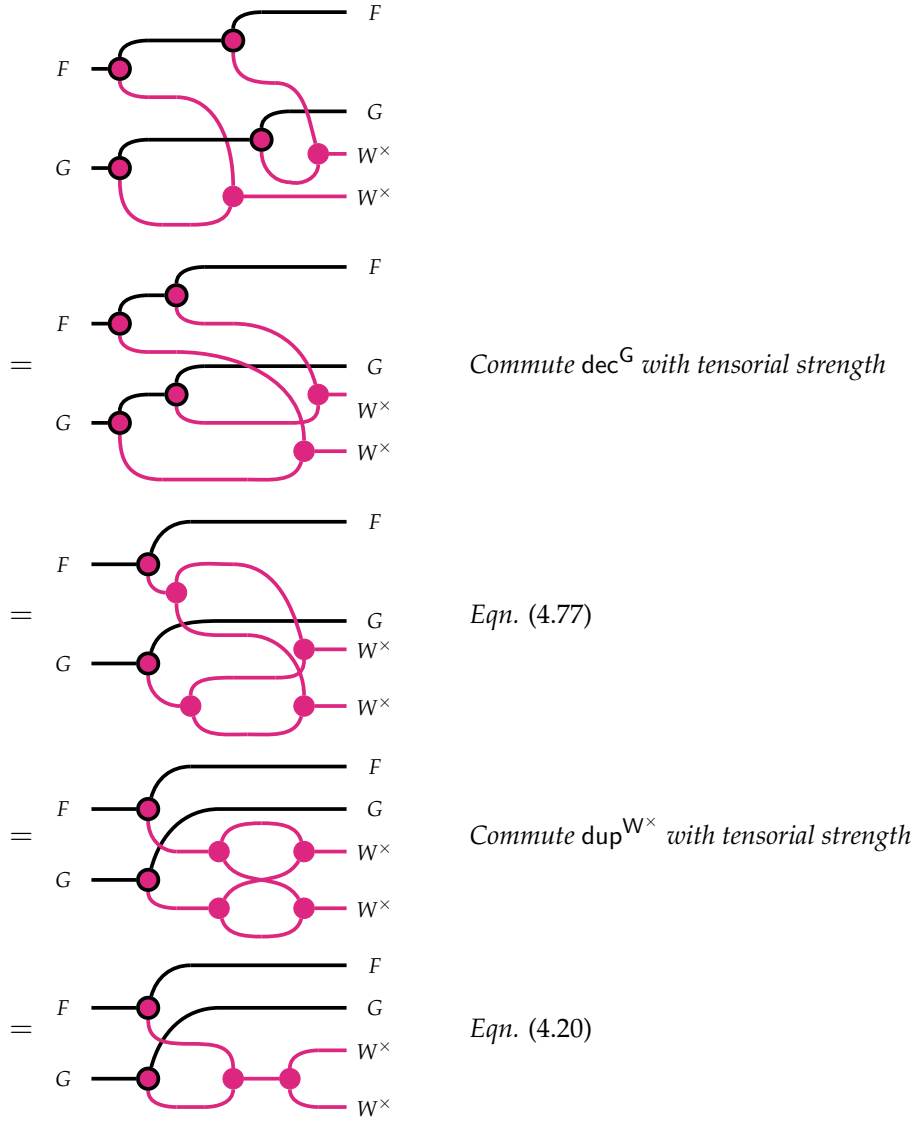
Lemma B.1.2 (Composition of Decorated Functors). *Definition 4.3.33 is a valid decorated functor.*

Proof.

Proof of (4.76): $\text{map}^{F \circ G} \text{extr}^{W^\times} \circ \text{dec}^{F \circ G} = \text{id}_{F \circ G}$

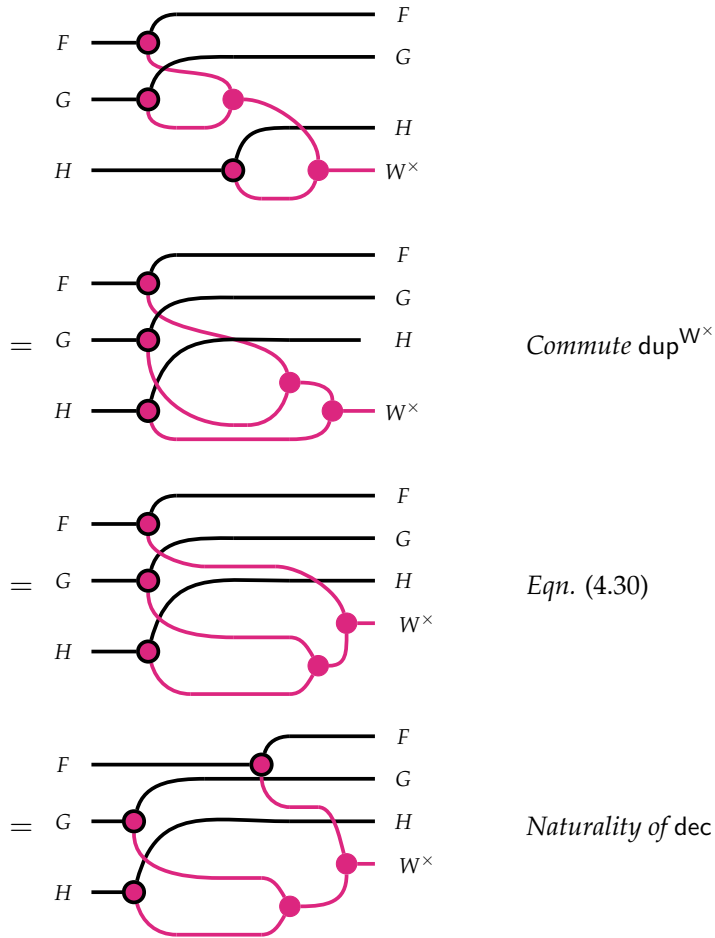
$$\begin{aligned}
 & \begin{array}{c} F \\ \text{---} \bullet \\ \text{---} \bullet \\ G \end{array} \begin{array}{c} F \\ \text{---} \\ \text{---} \\ G \end{array} \\
 &= \begin{array}{c} F \\ \text{---} \bullet \\ \text{---} \bullet \\ G \end{array} \begin{array}{c} F \\ \text{---} \\ \text{---} \\ G \end{array} \quad \text{Cup law (Eqn. (4.89))} \\
 &= \begin{array}{c} F \\ \text{---} \bullet \\ \text{---} \bullet \\ G \end{array} \begin{array}{c} F \\ \text{---} \\ \text{---} \\ G \end{array} \quad \text{Commute counit with tensorial strength} \\
 &= \begin{array}{c} F \\ \text{---} \\ \text{---} \\ G \end{array} \begin{array}{c} F \\ \text{---} \\ \text{---} \\ G \end{array} \quad \text{Counit law, twice (Eqn. (4.76))}
 \end{aligned}$$

Proof of (4.77): $\text{dec}^{F \circ G} \circ \text{dec}^{F \circ G} = \text{map}^{F \circ G} \text{dup}^{W^\times} \circ \text{dec}^{F \circ G}$



□

Proof of associativity: We prove that the decoration of $(F \circ G) \circ H$ is the same as that of $F \circ (G \circ H)$.



□

B.2 CATEGORY OF TRAVERSABLE FUNCTORS

Lemma B.2.1 (Composition of Traversable Functors). *Definition 5.3.34 is a valid traversable functor.*

Proof.

Proof of (5.89):

Eqn. (5.89)

Proof of (5.90):

Naturality of $\text{dist}_G^{F_1}$

Eqn. (5.90)

□

APPENDIX C

Multisorted Tealeaves

In Section 5.4.6, we considered the syntax of first order logic, which involves one type of variable, namely term variables, but two grammatical categories, namely formulas and terms. The set of formulas, which were not associated with their own type of variables but contained *term variables*, did not themselves form a monad but instead a module of the term monad. Here, we consider the syntax of the polymorphic lambda calculus. Expressions are either SystemF types τ (which may be called object-level types to distinguish metatheory-level types, i.e. Coq types) or terms t .

$$\begin{aligned}\tau &::= \text{tvar } X \mid \text{arr } \tau \mid \text{forall } X \tau \\ t &::= \text{var } v \mid \text{abs } t \mid \text{app } t t \mid \text{tabs } X t \mid \text{tapp } t \tau\end{aligned}$$

Here, term variables v are drawn from a set V_{tm} , while type variables X are drawn from a set V_{ty} . The set of terms is term $V_{\text{tm}} V_{\text{ty}}$, while object types are typ $V_{\text{tm}} V_{\text{ty}}$. Note that this gives a collection of two sets each parameterized by the same two sets. For this situation, we generalize from the category of sets and functions to the category of *collections* of sets and collections of functions.

Let K be a collection of sorts.

Definition C.0.1. *The category KSET^K of K -indexed sets, where K is a discrete category, is defined as follows:*

- *Objects are families of sets indexed by K , i.e., functors of type $K \rightarrow \text{SET}$. Concretely, an object is a collection $\{X_k\}_{k \in K}$ where each X_k is a set. We write X as an abbreviation for $\{X_k\}_{k \in K}$.*
- *Morphisms are natural transformations between such functors. Explicitly, a morphism*

$$\{f_k\}_{k \in K} : \{X_k\}_{k \in K} \rightarrow \{Y_k\}_{k \in K}$$

consists of a function $f_k : X_k \rightarrow Y_k$ for each $k \in K$. We write f to abbreviate $\{f_k\}_{k \in K}$.

- *The identity on an object $\{X_k\}_{k \in K}$ is the family of identity functions $\{\text{id}_{X_k}\}_{k \in K}$.*
- *Composition is defined pointwise: given morphisms $\{f_k\}_{k \in K}$ and $\{g_k\}_{k \in K}$, their composition is $\{g_k \circ f_k\}_{k \in K}$.*

For short, we call a K -indexed collection of sets a K -set. Similarly, an indexed collection of functions is an K -morphism.

Example C.0.2. *Acting as the index set, let $\mathcal{S} \equiv \{\text{tm}, \text{ty}\}$ be a two-element set codifying the sorts of SystemF. An $\mathcal{S}$ -index collection can be expressed as a tuple $\mathcal{V} = (\mathcal{V}_{\text{tm}}, \mathcal{V}_{\text{ty}})$. We consider two such collections.*

- The $\mathcal{S}$ -set of variables V defined $(V_{\text{tm}}, V_{\text{ty}})$.
- The $\mathcal{S}$ -set of SystemF expressions $(\text{term } V_{\text{tm}} V_{\text{ty}}, \text{typ } V_{\text{tm}} V_{\text{ty}})$.

A $\mathcal{S}$ -morphism is a pair of functions. For example, a global renaming operation ρ which maps all possible variables in SystemF to new names can be represented as a $\mathcal{S}$ -morphism from V to V :

$$(\rho_{\text{tm}} : V_{\text{tm}} \rightarrow V_{\text{tm}}, \rho_{\text{ty}} : V_{\text{ty}} \rightarrow V_{\text{ty}})$$

term and typ can be described as forming a K -functor. This notion is obtained by a technically straightforward generalization of Definition 4.1.46.

Definition C.0.3. The category $\text{END}_{\text{Set}}^K$ of endofunctors on KSET^K is defined as follows:

- **Objects:** Functors of type $F : \text{KSET}^K \rightarrow \text{KSET}^K$. Explicitly, a functor consists of a map F from a K -indexed collection of sets X to another collection $F X$. And for each morphism $f : X \rightarrow Y$ in KSET^K , there is a morphism

$$(\text{map}^F f) : F X \rightarrow F Y$$

That is, for each $k \in K$ there is a function

$$(\text{map}^F f)_{k \in K} : (F X)_k \rightarrow (F Y)_k.$$

- **Morphisms:** Natural transformations between such endofunctors. Given two functors $F, G : \text{KSET}^K \rightarrow \text{KSET}^K$, a morphism $\eta : F \rightarrow G$ is ...
- **Identities:** The identity on F is the identity natural transformation.
- **Composition:** Given by vertical composition of natural transformations: if $\eta : F \rightarrow G$ and $\theta : G \rightarrow H$, then their composition is given by $(\theta \circ \eta)_X = \theta_X \circ \eta_X$.

Example C.0.4. Define a functor $\text{Sysf} : \text{END}_{\text{Set}}^K$ that maps a pair $X = (X_{\text{tm}}, X_{\text{ty}})$ to the pair

$$\text{Sysf } X = (\text{term } X_{\text{tm}} X_{\text{ty}}, \text{typ } X_{\text{tm}} X_{\text{ty}}).$$

Given a $\mathcal{S}$ -morphism

$$f = (f_{\text{tm}} : X_{\text{tm}} \rightarrow Y_{\text{tm}}, f_{\text{ty}} : X_{\text{ty}} \rightarrow Y_{\text{ty}})$$

we can define a $\mathcal{S}$ -morphism $\text{Sysf } X \rightarrow \text{Sysf } Y$ by a pair of functions

$$\begin{aligned} \text{map}^{\text{typ}} f (\text{tvar } X) &\equiv \text{tvar } (f_{\text{ty}} X) \\ \text{map}^{\text{typ}} f (\text{arr } \tau_1 \tau_2) &\equiv \text{arr } (\text{map}^{\text{typ}} f \tau_1) (\text{map}^{\text{typ}} f \tau_2) \\ \text{map}^{\text{typ}} f (\text{forall } X \tau) &\equiv \text{forall } (f_{\text{ty}} X) (\text{map}^{\text{typ}} f \tau) \end{aligned}$$

while the other is defined

$$\begin{aligned}
\text{map}^{\text{term}} f (\text{var } v) &\equiv \text{tvar } (f_{\text{tm}} v) \\
\text{map}^{\text{term}} f (\text{abs } t) &\equiv \text{abs } (\text{map}^{\text{term}} f t) \\
\text{map}^{\text{term}} f (\text{app } t_1 t_2) &\equiv \text{app } (\text{map}^{\text{term}} f t_1) (\text{map}^{\text{term}} f t_2) \\
\text{map}^{\text{term}} f (\text{tabs } X t) &\equiv \text{tabs } (f_{\text{ty}} X) (\text{map}^{\text{term}} f t) \\
\text{map}^{\text{term}} f (\text{tapp } t \tau) &\equiv \text{tapp } (\text{map}^{\text{term}} f t) (\text{map}^{\text{typ}} f \tau)
\end{aligned}$$

Note that in the last case, these map^{term} is defined from map^{typ} . It happens that types do not contain term variables, and so map^{typ} does not depend on map^{term} . For more general $\mathcal{S}$ -sorted languages, these operations are defined mutually recursively, with both definitions mentioning the other.

The monoidal structure on SET (Example 4.1.5) easily generalizes to KSET^K .

Definition C.0.5. KSET^K is a monoidal category, where the monoidal structure is given by pointwise Cartesian product. Specifically:

- **Monoidal Unit:** The unit object $\mathbf{1}^K$ is the functor that assigns to each $k \in K$ the singleton set $\mathbf{1}$.

$$\mathbf{1}_k^K = \mathbf{1} = \{*\}.$$

- **Monoidal Product:** Given two objects $\{X_k\}_{k \in K}$ and $\{Y_k\}_{k \in K}$, their monoidal product is the pointwise Cartesian product:

$$(X \otimes Y)_k \equiv X_k \times Y_k.$$

The product of morphisms is also pointwise: given $\{f_k\}_{k \in K}$ and $\{g_k\}_{k \in K}$, their tensor product is defined as:

$$(f \otimes g)_k \equiv f_k \times g_k.$$

- **Associativity and Unitality:** The associativity and unitality constraints are inherited pointwise from the Cartesian product in SET , ensuring that KSET^K forms a strict monoidal category.

Similarly, the monoidal structure of END_{Set} (Definition 4.1.49) extends to $\text{END}_{\text{Set}}^K$.

Definition C.0.6. The category $\text{END}_{\text{Set}}^K$ of endofunctors on KSET^K forms a monoidal category under composition. Specifically:

- **Monoidal Unit:** The unit object is the identity functor mapping K -sets and K -morphisms to themselves.
- **Monoidal Product:** The tensor product is given by functor composition:

$$((F \otimes G)(X))_k \equiv F(G(X))_k$$

Given natural transformations $\eta : F \rightarrow F'$ and $\theta : G \rightarrow G'$, their tensor product is defined by:

$$(\eta \otimes \theta)_X \equiv \eta_{G(X)} \circ F(\theta_X).$$

- **Associativity and unitality:** The associativity and unitality constraints are inherited from functor composition, ensuring that $\text{END}_{\text{Set}}^K$ forms a strict monoidal category.

C.0.1 Multisorted DTMs as Extension Systems

We re-interpret the syntax such that X and v are drawn from the *same* underlying set of variables. In Coq, the syntax of System F would be defined as follows.

```
Inductive typ (V: Type): Type :=
| ty_c : base_typ -> typ
| ty_v : V -> typ
| ty_ar : typ -> typ -> typ
| ty_univ : typ -> typ.

Inductive term (V: Type): Type :=
| tm_var : V -> term
| tm_abs : typ -> term -> term
| tm_app : term -> term -> term
| tm_tab : term -> term
| tm_tap : term -> typ -> term.
```

Then we *redefine* $\text{SysF} : \text{SET} \rightarrow \text{KSET}^K$ as an operation mapping a *single* set V to the $\mathcal{S}$ -set $(\text{term } V, \text{typ } V)$. Note that with these single-parameter definitions, we can no longer consider sets like

$$\text{term } (\text{term } V_{\text{tm}} V_{\text{ty}}) (\text{typ } V_{\text{tm}} V_{\text{tm}})$$

and, it follows, we cannot express the type of join. However, we can still express a meaningful bind operation of the following type

$$(\forall (s: \mathcal{S}), A \rightarrow \text{Sysf}_s B) \rightarrow \text{term } A \rightarrow \text{term } B$$

Note that up to isomorphism, the above type is equal to

$$(A \rightarrow \text{typ } B) \rightarrow (A \rightarrow \text{typ } B) \rightarrow \text{term } A \rightarrow \text{term } B$$

The following definitions—which admittedly seem circuitous at first blush—represent a favor-

able compromise between practicality and theoretical purity.

Definition C.0.7 (Multisorted Decorated Traversable Premodule). A K -sorted decorated traversable T -premodule consists of an operation $T : \text{SET} \rightarrow \text{KSET}^K$ mapping any set V to a K -indexed collection of sets $\{T_k(V)\}_{k \in K}$, and a type constructor $U : \text{SET} \rightarrow \text{SET}$ equipped with a set of operations of the following types.

$$\begin{aligned} \text{mret}^T \quad (A : \mathbf{Type}) & : \quad \forall (k : K), A \rightarrow T_k A \\ \text{mbinddt}^{T_k} \quad (\text{Applicative } G) \quad (A B : \mathbf{Type}) & : \quad (\forall (k : K), W \times A \rightarrow G(T_k B)) \rightarrow (T_k A \rightarrow G(T_k B)) \\ \text{mbinddt}^U \quad (\text{Applicative } G) \quad (A B : \mathbf{Type}) & : \quad (\forall (k : K), W \times A \rightarrow G(T_k B)) \rightarrow (U A \rightarrow G(U B)) \end{aligned}$$

subject to the following laws.

$$\text{mbinddt}_\perp^U \left(\text{mret}^U \circ_k \text{allk extract}^{W \times} \right) = \text{id}_U \quad (\text{C.1})$$

$$\text{map}^{G_1} \left(\text{mbinddt}_{G_2}^U g \right) \circ \left(\text{mbinddt}_{G_1}^U f \right) = \text{mbinddt}_{G_1 \circ G_2}^U (g \star f) \quad (\text{C.2})$$

$$\phi \circ \text{mbinddt}_{G_1}^U f = \text{binddt}_{G_2}^U ((\lambda k. \phi_{T_k B}) \circ_k f) \quad \left(\text{all } \phi : G_1 \xrightarrow{\text{appl}} G_2 \right) \quad (\text{C.3})$$

In (C.2), the generalized Kleisli composition operation of functions

$$f : \forall (k : K), W \times A \rightarrow G_1(T_k B) \quad \text{and} \quad g : \forall (k : K), W \times B \rightarrow G_2(T_k C)$$

is a function $(g \star f) : \forall (k : K), W \times A \rightarrow G_1(G_2(T_k C))$ defined as follows.

$$(g \star f)(k, w, a) \equiv \text{map}^{G_1} \left(\text{binddt}_{G_2}^{T_k} (g \odot_k w) \right) (f(k, w, a)) \quad (\text{C.4})$$

Definition C.0.8 (Multisorted Decorated Traversable Monad). A K -sorted DTM consists of an operation $T : \text{SET} \rightarrow \text{KSET}^K$ and a family of operations

$$\begin{aligned} \text{mret}^T \quad (A : \mathbf{Type}) & : \quad \forall (k : K), A \rightarrow T_k A \\ \text{mbinddt}^{T_k} \quad (\text{Applicative } G) \quad (A B : \mathbf{Type}) & : \quad (\forall (k : K), W \times A \rightarrow G(T_k B)) \rightarrow (T_k A \rightarrow G(T_k B)) \end{aligned}$$

such that for all $k \in K$, mbinddt^{T_k} forms a T -premodule (i.e. with T_k acting in place of U above) and the following condition holds.

$$\forall (a : A), \text{mbinddt}_G^T f \left(\text{mret}^T(k, a) \right) = f(k, e_W, a) \quad (\text{C.5})$$

Definition C.0.9 (Multisorted Decorated Traversable Module). A K -sorted decorated traversable T -module consists of an K -sorted decorated traversable monad T and an operation mbinddt^U such that mbinddt^U forms a pre-module.

The difference between Definitions C.0.7 and C.0.9 is simply that in the latter case, T is known to form a decorated traversable monad, meaning each $\text{mbinddt}^{\top k}$ satisfies Equations (C.1)–(C.3) and (C.5).

BIBLIOGRAPHY

- [1] M. Abadi, L. Cardelli, P.-L. Curien, and J.-J. Lévy. Explicit substitutions. *Journal of Functional Programming*, 1(4):375–416, 1991. doi: 10.1017/S0956796800000186.
- [2] Samson Abramsky. No-cloning in categorical quantum mechanics. *Semantic Techniques in Quantum Computation*, 10 2009. doi: 10.1017/CBO9781139193313.002.
- [3] Guillaume Allais, James Chapman, Conor McBride, and James McKinna. Type-and-scope safe programs and their proofs. In *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs, CPP 2017*, page 195–207, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450347051. doi: 10.1145/3018610.3018613. URL <https://doi.org/10.1145/3018610.3018613>.
- [4] Guillaume Allais, Robert Atkey, James Chapman, Conor McBride, and James McKinna. A type- and scope-safe universe of syntaxes with binding: their semantics and proofs. *Journal of Functional Programming*, 31:e22, 2021. doi: 10.1017/S0956796820000076.
- [5] Thorsten Altenkirch and Bernhard Reus. Monadic presentations of lambda terms using generalized inductive types. In *Proceedings of the 13th International Workshop and 8th Annual Conference of the EACSL on Computer Science Logic, CSL ’99*, page 453–468, Berlin, Heidelberg, 1999. Springer-Verlag. ISBN 3540665366. doi: 10.1007/3-540-48168-0_32.
- [6] Robert Atkey. Parameterised notions of computation. In *Proceedings of the 2006 International Conference on Mathematically Structured Functional Programming, MSFP’06*, page 5, Swindon, GBR, 2006. BCS Learning & Development Ltd.
- [7] Robert Atkey. Algebras for parameterised monads. In Alexander Kurz, Marina Lenisa, and Andrzej Tarlecki, editors, *Algebra and Coalgebra in Computer Science*, pages 3–17, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg. ISBN 978-3-642-03741-2.
- [8] Steve Awodey. *Category Theory*. Oxford University Press, Inc., USA, 2nd edition, 2010. ISBN 0199237182.
- [9] Brian Aydemir and Stephanie Weirich. LNgens: Tool Support for Locally Nameless Representations. Technical report, University of Pennsylvania, Department of Computer and Information Science, 06 2010. URL <https://repository.upenn.edu/handle/20.500.14332/7902>.
- [10] Brian Aydemir, Arthur Charguéraud, Benjamin C. Pierce, Randy Pollack, and Stephanie Weirich. Engineering formal metatheory. In *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL ’08*, page 3–15, New York, NY, USA, 2008. Association for Computing Machinery. ISBN 9781595936899. doi: 10.1145/1328438.1328443. URL <https://doi.org/10.1145/1328438.1328443>.
- [11] Brian E. Aydemir, Aaron Bohannon, Matthew Fairbairn, J. Nathan Foster, Benjamin C.

- Pierce, Peter Sewell, Dimitrios Vytiniotis, Geoffrey Washburn, Stephanie Weirich, and Steve Zdancewic. Mechanized metatheory for the masses: The poplmark challenge. In *Proceedings of the 18th International Conference on Theorem Proving in Higher Order Logics*, TPHOLs'05, page 50–65, Berlin, Heidelberg, 2005. Springer-Verlag. ISBN 3540283722. doi: 10.1007/11541868_4. URL https://doi.org/10.1007/11541868_4.
- [12] Tørris Koløen Bakke. Hopf algebras and monoidal categories. Master’s thesis, University of Tromsø, 2007. URL <https://munin.uit.no/bitstream/handle/10037/1084/finalthesis.pdf>.
- [13] Gershom Bazerman. A totally predictable outcome: an investigation of traversals of infinite structures. In *Proceedings of the 15th ACM SIGPLAN International Haskell Symposium*, Haskell 2022, page 39–53, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450394383. doi: 10.1145/3546189.3549915. URL <https://doi.org/10.1145/3546189.3549915>.
- [14] Françoise Bellegarde and James Hook. Substitution: A formal methods case study using monads and transformations. *Sci. Comput. Program.*, 23(2–3):287–311, December 1994. ISSN 0167-6423. doi: 10.1016/0167-6423(94)00022-0. URL [https://doi.org/10.1016/0167-6423\(94\)00022-0](https://doi.org/10.1016/0167-6423(94)00022-0).
- [15] Richard Bird and Lambert Meertens. Nested datatypes. volume 1422, pages 52–67, 06 1998. ISBN 978-3-540-64591-7. doi: 10.1007/BFb0054285.
- [16] Richard Bird and Ross Paterson. de bruijn notation as a nested datatype. *Journal of Functional Programming*, 9:77 – 91, 1999. doi: 10.1017/S0956796899003366.
- [17] Richard Bird, Jeremy Gibbons, Stefan Mehner, Janis Voigtländer, and Tom Schrijvers. Understanding idiomatic traversals backwards and forwards. In *Proceedings of the 2013 ACM SIGPLAN Symposium on Haskell*, Haskell ’13, page 25–36, New York, NY, USA, 2013. Association for Computing Machinery. ISBN 9781450323833. doi: 10.1145/2503778.2503781. URL <https://doi.org/10.1145/2503778.2503781>.
- [18] Gabriella Böhm, Stefaan Caenepeel, and Kris Janssen. Weak bialgebras and monoidal categories. *Communications in Algebra*, 39:4584–4607, 2011. ISSN 0092-7872.
- [19] Guillaume Boisseau and Jeremy Gibbons. What you needa know about yoneda: profunctor optics and the yoneda lemma (functional pearl). *Proc. ACM Program. Lang.*, 2(ICFP), July 2018. doi: 10.1145/3236779. URL <https://doi.org/10.1145/3236779>.
- [20] Ana Bove, Peter Dybjer, and Ulf Norell. A brief overview of agda – a functional language with dependent types. In Stefan Berghofer, Tobias Nipkow, Christian Urban, and Makarius Wenzel, editors, *Theorem Proving in Higher Order Logics*, pages 73–78, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg. ISBN 978-3-642-03359-9.
- [21] Paolo Capriotti and Ambrus Kaposi. Free applicative functors. *Electronic Proceedings in*

Theoretical Computer Science, 153, 03 2014. doi: 10.4204/EPTCS.153.2.

- [22] Arthur Charguéraud. The locally nameless representation. *J. Autom. Reason.*, 49(3):363–408, 2012. doi: 10.1007/s10817-011-9225-2. URL <https://doi.org/10.1007/s10817-011-9225-2>.
- [23] Adam Chlipala. *Certified Programming with Dependent Types: A Pragmatic Introduction to the Coq Proof Assistant*. The MIT Press, 2013. ISBN 0262026651.
- [24] Alonzo Church. A set of postulates for the foundation of logic. *Annals of Mathematics*, 33(2): 346–366, 1932. ISSN 0003486X. URL <http://www.jstor.org/stable/1968337>.
- [25] Bryce Clarke, Derek Elkins, Jeremy Gibbons, Fosco Loregian, Bartosz Milewski, Emily Pillmore, and Mario Román. Profunctor optics, a categorical update. *Compositionality*, 6:1, 02 2024. doi: 10.32408/compositionality-6-1.
- [26] Roy L. Crole. Alpha equivalence equalities. *Theoretical Computer Science*, 433:1–19, 2012. ISSN 0304-3975. doi: <https://doi.org/10.1016/j.tcs.2012.01.030>. URL <https://www.sciencedirect.com/science/article/pii/S0304397512000667>.
- [27] Pierre-Louis Curien, Thérèse Hardin, and Jean-Jacques Lévy. Confluence properties of weak and strong calculi of explicit substitutions. *J. ACM*, 43(2):362–397, March 1996. ISSN 0004-5411. doi: 10.1145/226643.226675. URL <https://doi.org/10.1145/226643.226675>.
- [28] Lawrence Dunn, Val Tannen, and Steve Zdancewic. Syntax monads for the working formal metatheorist. *ArXiv*, abs/2312.08897, 2023. URL <https://api.semanticscholar.org/CorpusID:266210127>.
- [29] Lawrence Dunn, Val Tannen, and Steve Zdancewic. Tealeaves: Structured monads for generic first-order abstract syntax infrastructure. In *International Conference on Interactive Theorem Proving*, 2023. URL <https://api.semanticscholar.org/CorpusID:260169665>.
- [30] E. Engeler. H. p. barendregt. the lambda calculus. its syntax and semantics. studies in logic and foundations of mathematics, vol. 103. north-holland publishing company, amsterdam, new york, and oxford, 1981, xiv + 615 pp. *Journal of Symbolic Logic*, 49(1):301–303, 1984. doi: 10.2307/2274112.
- [31] M. Fiore, G. Plotkin, and D. Turi. Abstract syntax and variable binding. In *Proceedings. 14th Symposium on Logic in Computer Science (Cat. No. PR00158)*, pages 193–202, 1999. doi: 10.1109/LICS.1999.782615.
- [32] Marcelo Fiore. Second-order and dependently-sorted abstract syntax. In *Proceedings of the 2008 23rd Annual IEEE Symposium on Logic in Computer Science, LICS '08*, page 57–68, USA, 2008. IEEE Computer Society. ISBN 9780769531830. doi: 10.1109/LICS.2008.38. URL <https://doi.org/10.1109/LICS.2008.38>.

- [33] Marcelo Fiore and Dmitrij Szamozvancev. Formal metatheory of second-order abstract syntax. *Proc. ACM Program. Lang.*, 6(POPL), jan 2022. doi: 10.1145/3498715. URL <https://doi.org/10.1145/3498715>.
- [34] Sebastian Fischer, Zhenjiang Hu, and Hugo Pacheco. A clear picture of lens laws. In Ralf Hinze and Janis Voigtländer, editors, *Mathematics of Program Construction*, pages 215–223, Cham, 2015. Springer International Publishing. ISBN 978-3-319-19797-5.
- [35] Jeremy Gibbons. Datatype-generic programming. In Roland Backhouse, Jeremy Gibbons, Ralf Hinze, and Johan Jeuring, editors, *Datatype-Generic Programming*, pages 1–71, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg. ISBN 978-3-540-76786-2.
- [36] Jeremy Gibbons and Bruno Oliveira. The essence of the iterator pattern. *J. Funct. Program.*, 19:377–402, 07 2009. doi: 10.1017/S0956796809007291.
- [37] Jean-Yves Girard, Paul Taylor, and Yves Lafont. *Proofs and types*. Cambridge University Press, USA, 1989. ISBN 0521371813.
- [38] J. A. Goguen, J. W. Thatcher, E. G. Wagner, and J. B. Wright. Initial algebra semantics and continuous algebras. *J. ACM*, 24(1):68–95, January 1977. ISSN 0004-5411. doi: 10.1145/321992.321997. URL <https://doi.org/10.1145/321992.321997>.
- [39] Yuri Gurevich and Saharon Shelah. Fixed-point extensions of first-order logic. *Annals of Pure and Applied Logic*, 32(nil):265–280, 1986. doi: 10.1016/0168-0072(86)90055-2. URL [https://doi.org/10.1016/0168-0072\(86\)90055-2](https://doi.org/10.1016/0168-0072(86)90055-2).
- [40] J. Roger Hindley and Jonathan P. Seldin. *Lambda-Calculus and Combinators: An Introduction*. Cambridge University Press, 2 edition, 2008.
- [41] Martin Hyland and John Power. The category theoretic understanding of universal algebra: Lawvere theories and monads. *Electronic Notes in Theoretical Computer Science*, 172:437–458, 2007. ISSN 1571-0661. doi: <https://doi.org/10.1016/j.entcs.2007.02.019>. URL <https://www.sciencedirect.com/science/article/pii/S1571066107000874>. Computation, Meaning, and Logic: Articles dedicated to Gordon Plotkin.
- [42] James Iry. A Brief, Incomplete, and Mostly Wrong History of Programming Languages. <https://james-iry.blogspot.com/2009/05/brief-incomplete-and-mostly-wrong.html>, May 2009. Accessed: 2025-03-25.
- [43] Mauro Jaskelioff and Russell O’Connor. A representation theorem for second-order functionals. *J. Funct. Program.*, 25, 02 2014. doi: 10.1017/S0956796815000088.
- [44] Mauro Jaskelioff and Ondrej Rypacek. An investigation of the laws of traversals. *Electronic Proceedings in Theoretical Computer Science*, 76, 02 2012. doi: 10.4204/EPTCS.76.5.

- [45] C. Barry Jay and J. R. B. Cockett. Shapely types and shape polymorphism. In Donald Sannella, editor, *Programming Languages and Systems — ESOP '94*, pages 302–316, Berlin, Heidelberg, 1994. Springer Berlin Heidelberg. ISBN 978-3-540-48376-2.
- [46] Joachim Lambek. A fixpoint theorem for complete categories. *Mathematische Zeitschrift*, 103: 151–161, 1968. URL <https://api.semanticscholar.org/CorpusID:122552525>.
- [47] G Lee, BCDS Oliveira, S Cho, and K Yi. Gmeta: A generic formal metatheory framework for first-order representations. 2012. doi: 10.1007/978-3-642-28869-2_22. URL <http://hdl.handle.net/10722/188496>.
- [48] Xavier Leroy. Formal verification of a realistic compiler. *Commun. ACM*, 52(7):107–115, July 2009. ISSN 0001-0782. doi: 10.1145/1538788.1538814. URL <https://doi.org/10.1145/1538788.1538814>.
- [49] Xavier Leroy. Well-founded recursion done right. In *CoqPL 2024: The Tenth International Workshop on Coq for Programming Languages*, London, United Kingdom, January 2024. ACM.
- [50] N.G. Leveson and C.S. Turner. An investigation of the therac-25 accidents. *Computer*, 26(7): 18–41, 1993. doi: 10.1109/MC.1993.274940.
- [51] Saunders MacLane. *Categories for the Working Mathematician*. Springer-Verlag, New York, 1971. Graduate Texts in Mathematics, Vol. 5.
- [52] Conor McBride. Elimination with a motive. In *Selected Papers from the International Workshop on Types for Proofs and Programs, TYPES '00*, page 197–216, Berlin, Heidelberg, 2000. Springer-Verlag. ISBN 3540432876.
- [53] Conor McBride. Dependently typed functional programs and their proofs. 2000. URL <https://api.semanticscholar.org/CorpusID:8333926>.
- [54] Conor McBride. Type-preserving renaming and substitution. Unpublished note, 2005. URL <http://strictlypositive.org/ren-sub.pdf>.
- [55] Conor McBride and Ross Paterson. Applicative programming with effects. *J. Funct. Program.*, 18(1):1–13, January 2008. ISSN 0956-7968. doi: 10.1017/S0956796807006326. URL <https://doi.org/10.1017/S0956796807006326>.
- [56] J. McCarthy. Towards a mathematical science of computation. In Timothy R. Colburn, James H. Fetzer, and Terry L. Rankin, editors, *Program Verification: Fundamental Issues in Computer Science*, pages 35–56. Springer Netherlands, Dordrecht, 1993. ISBN 978-94-011-1793-7. doi: 10.1007/978-94-011-1793-7_2. URL https://doi.org/10.1007/978-94-011-1793-7_2.
- [57] John McCarthy. A basis for a mathematical theory of computation¹). In P. Braffort and D. Hirschberg, editors, *Computer Programming and Formal Systems*, volume 35 of *Studies in*

- Logic and the Foundations of Mathematics*, pages 33–70. Elsevier, 1963. doi: [https://doi.org/10.1016/S0049-237X\(08\)72018-4](https://doi.org/10.1016/S0049-237X(08)72018-4). URL <https://www.sciencedirect.com/science/article/pii/S0049237X08720184>.
- [58] John McCarthy and James Painter. Correctness of a compiler for arithmetic expressions. In *Proc. Sympos. Appl. Math., Vol. XIX*, pages 33–41. Amer. Math. Soc., Providence, RI, 1967.
- [59] Ross Paterson. Constructing applicative functors. In Jeremy Gibbons and Pablo Nogueira, editors, *Mathematics of Program Construction*, pages 300–323, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg. ISBN 978-3-642-31113-0.
- [60] Andrew M. Pitts. Nominal logic, a first order theory of names and binding. *Information and Computation*, 186(2):165–193, 2003. ISSN 0890-5401. doi: [https://doi.org/10.1016/S0890-5401\(03\)00138-X](https://doi.org/10.1016/S0890-5401(03)00138-X). URL <https://www.sciencedirect.com/science/article/pii/S089054010300138X>. Theoretical Aspects of Computer Software (TACS 2001).
- [61] Pollack, Randy. Reasoning about languages with binding: Can we do it yet? URL: https://web.archive.org/web/20101122040606/http://homepages.inf.ed.ac.uk/rpollack/export/bindingChallenge_slides.pdf, 2 2006.
- [62] John Power. Abstract syntax: Substitution and binders: Invited address. *Electronic Notes in Theoretical Computer Science*, 173:3–16, 2007. ISSN 1571-0661. doi: <https://doi.org/10.1016/j.entcs.2007.02.024>. URL <https://www.sciencedirect.com/science/article/pii/S1571066107000965>. Proceedings of the 23rd Conference on the Mathematical Foundations of Programming Semantics (MFPS XXIII).
- [63] Steven Schäfer, Gert Smolka, and Tobias Tebbi. Completeness and decidability of de bruijn substitution algebra in coq. In *Proceedings of the 2015 Conference on Certified Programs and Proofs, CPP '15*, page 67–73, New York, NY, USA, 2015. Association for Computing Machinery. ISBN 9781450332965. doi: 10.1145/2676724.2693163. URL <https://doi.org/10.1145/2676724.2693163>.
- [64] Steven Schäfer, Tobias Tebbi, and Gert Smolka. Autosubst: Reasoning with de bruijn terms and parallel substitutions. In Xingyuan Zhang and Christian Urban, editors, *Interactive Theorem Proving - 6th International Conference, ITP 2015, Nanjing, China, August 24-27, 2015*, LNAI. Springer-Verlag, Aug 2015. doi: 10.1007/978-3-319-22102-1_24.
- [65] Peter Sewell, Francesco Zappa Nardelli, Scott Owens, Gilles Peskine, Thomas Ridge, Susmit Sarkar, and Rok Strniša. Ott: Effective tool support for the working semanticist. In *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming, ICFP '07*, page 1–12, New York, NY, USA, 2007. Association for Computing Machinery. ISBN 9781595938152. doi: 10.1145/1291151.1291155. URL <https://doi.org/10.1145/1291151.1291155>.
- [66] Mike Shulman. You could have invented de bruijn indices, August 2021. URL https://golem.ph.utexas.edu/category/2021/08/you_could_have_invented_de_bru.html. Blog post

on The n-Category Café.

- [67] Matthieu Sozeau and Nicolas Oury. First-class type classes. In *Proceedings of the 21st International Conference on Theorem Proving in Higher Order Logics*, TPHOLs '08, page 278–293, Berlin, Heidelberg, 2008. Springer-Verlag. ISBN 9783540710653. doi: 10.1007/978-3-540-71067-7_23. URL https://doi.org/10.1007/978-3-540-71067-7_23.
- [68] Bas Spitters and Eelis Weegen. Type classes for mathematics in type theory. *Mathematical Structures in Computer Science*, 21, 02 2011. doi: 10.1017/S0960129511000119. URL <http://dx.doi.org/10.1017/S0960129511000119>.
- [69] Kathrin Stark, Steven Schäfer, and Jonas Kaiser. Autosubst 2: Reasoning with multi-sorted de bruijn terms and vector substitutions. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP 2019, page 166–180, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450362221. doi: 10.1145/3293880.3294101. URL <https://doi.org/10.1145/3293880.3294101>.
- [70] Philip Wadler. The essence of functional programming. In *Proceedings of the 19th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '92, page 1–14, New York, NY, USA, 1992. Association for Computing Machinery. ISBN 0897914538. doi: 10.1145/143165.143169. URL <https://doi.org/10.1145/143165.143169>.
- [71] W. Eric Wong, Vidroha Debroy, Adithya Surampudi, HyeonJeong Kim, and Michael F. Siok. Recent catastrophic accidents: Investigating how software was responsible. In *2010 Fourth International Conference on Secure Software Integration and Reliability Improvement*, pages 14–22, 2010. doi: 10.1109/SSIRI.2010.38.